

ALGORITMI FUNDAMENTALI

Ghid practic pentru curs și laborator

perspectivă C++

Lect. Dr. Alexandra BĂICOIANU, Asist. Drd. Luciana MAJERCSIK

Universitatea Transilvania din Brașov, Facultatea de Matematică și
Informatică

Cuprins

Prefață	3
1 Analiza eficienței algoritmilor. Complexități	5
1.1 Analiza eficienței algoritmilor. Complexități	5
1.1.1 Aspecte teoretice	5
1.1.2 Complexități - Probleme rezolvate	17
1.1.3 Complexități - Probleme propuse	20
2 Operații elementare. Procesarea numerelor	22
2.1 Operații elementare - Aspecte teoretice	22
2.1.1 Tipuri de date	22
2.1.2 Variabile	23
2.1.3 Funcții	27
2.2 Procesarea numerelor - probleme rezolvate	31
2.3 Procesarea numerelor - probleme propuse	38
3 Tablouri unidimensionale	40
3.1 Aspecte teoretice	40
3.1.1 Cerințe preliminare pentru prelucrarea tablourilor unidimensionale	41
3.1.2 Reprezentarea tablourilor unidimensionale. Alocare dinamică și alocare statică	41
3.1.3 Reprezentarea tablourilor unidimensionale în Standard Template Library (C++)	47
3.2 Tablouri unidimensionale - probleme rezolvate	53
3.3 Tablouri unidimensionale - probleme propuse	63
4 Tablouri bidimensionale	65
4.1 Aspecte teoretice	65
4.1.1 Cerințe preliminare pentru prelucrarea tablourilor bidimensionale	66
4.1.2 Reprezentarea tablourilor bidimensionale. Alocare dinamică și alocare statică	66

4.1.3	Reprezentarea tablourilor bidimensionale folosind Standard Template Library (C++)	73
4.2	Tablouri bidimensionale - probleme rezolvate	75
4.3	Tablouri bidimensionale - probleme propuse	93
5	Algoritmi de căutare și sortare	96
5.1	Tipuri de algoritmi de căutare. Aspecte teoretice	96
5.1.1	Căutare secvențială	96
5.1.2	Căutare binară	97
5.1.3	Alte tipuri de căutări	99
5.2	Tipuri de algoritmi de sortare. Aspecte teoretice	107
5.2.1	Sortarea rapidă - Quick Sort	108
5.2.2	Sortare prin interclasare - Merge Sort	111
5.2.3	Sortarea în Standard Template Library	112
5.2.4	Ghid pentru alegerea unui algoritm potrivit de sortare	121
5.3	Algoritmi de căutare și sortare - probleme rezolvate	123
5.4	Algoritmi de căutare și sortare - probleme propuse	130
6	Algoritmi Divide et Impera	132
6.1	Aspecte teoretice	132
6.1.1	Studii de caz pe diverse probleme	133
6.2	Algoritmi Divide et Impera - Probleme rezolvate	150
6.3	Algoritmi Divide et Impera - Probleme propuse	157
7	Algoritmi Greedy	160
7.1	Aspecte teoretice	160
7.2	Algoritmi Greedy - Probleme rezolvate	163
7.3	Algoritmi Greedy - Probleme propuse	170
8	Algoritmi de Programare Dinamică	173
8.0.1	Aspecte teoretice	173
8.0.2	Algoritmi de Programare Dinamică - Probleme rezolvate	175
8.0.3	Algoritmi de Programare Dinamică - Probleme propuse	195
9	Diverse probleme practice	201

Prefață

În culegere sunt descrise o serie de tehnici de programare, care sunt ilustrate cu exemple de probleme rezolvate în limbajul de programare C++. Exemplele sunt alese cu grijă, fiind prezentate în ordinea gradului lor de dificultate. Pentru fiecare problemă aleasă, se prezintă pe scurt, în limbaj natural, metoda de rezolvare, după care este prezentată implementarea în limbajul C++. Astfel, dorința noastră este ca această culegere să devină un instrument atât pentru fixarea sintaxei limbajului, cât și pentru însușirea unor principii corecte în algoritmică și programare.

Rolul acestei lucrări este acela de a contribui la aprofundarea diferitelor metode folosite în programare și de a încerca să ofere o inițiere, o bază sau o motivație celor care își doresc să facă progrese în acest domeniu. Lucrarea se adresează unui cerc mai larg de cititori, dar, în primul rând este concepută în scopul de a oferi un suport de curs și de laborator studenților Facultății de Matematică și Informatică. Pentru parcurgerea acestei cărți, cititorul nu trebuie să consulte categoric materiale suplimentare, dar, cu certitudine trebuie să stăpânească foarte bine noțiunile de informatică predate în învățământul liceal. Astfel, cartea poate fi utilă chiar și elevilor de liceu cu aptitudini deosebite în studiul algoritmicii și programării.

Lucrarea este structurată pe mai multe capitole.

Primul capitol, *Analiza eficienței algoritmilor*. *Complexități*, prezintă, pe scurt, modul în care se poate măsura eficiența unui algoritm. De asemenea, sunt descrise notațiile uzuale de măsurare a complexității algoritmilor și modul în care aceste complexități pot fi deduse. O serie de exerciții rezolvate și propuse se găsesc la finalul capitolului, iar rolul lor este de a aprofunda noțiunile din această secțiune.

Capitolul al doilea, *Operații elementare*. *Procesare numerelor* se ocupă de probleme introductive legate de procesarea numerelor, dar tratăm și câteva elemente specifice limbajului C++, și anume tipuri de date primitive în C++, definirea variabilelor, dar și concepte esențiale legate de definirea unui subprogram.

Capitolul al treilea, *Tablouri unidimensionale*, introduce tipul tablou și modalități diferite de reprezentare ale acestuia în limbajul C++, folosind fie alocarea statică, fie cea dinamică, dar și o perspectivă modernă cu containerele din Standard Template Library (STL). Informațiile expuse în acest capitol pot ajuta atât la formarea unei baze în prelucrarea structurilor unidimensionale, dar este un capitol potrivit și celor care au deja bazele formate, dar doresc să înțeleagă o direcție către structurile de date moderne din C++.

În capitolul patru, *Tablouri bidimensionale*, se introduc tablourile cu mai multe dimensiuni. Ca și în capitolul precedent, sunt urmărite aspectele cele mai importante legate de modalitățile de reprezentare a acestora în limbajul C++ și de prelucrarea acestor tablouri în diferite situații practice. O serie de exerciții

rezolvate și propuse se pot studia la final de capitol.

Capitolul cinci, *Algoritmi de căutare și sortare*, se ocupă cu prezentarea unor metode de căutare și sortare, criteriul de selecție a metodelor descrise fiind complexitatea scăzută a acestora. În acest capitol se insistă și asupra modului în care este acoperit subiectul căutării și sortării din punct de vedere C++ (algoritmi specifici utilizați și implementați în STL).

Capitolul șase, *Algoritmi Divide et Impera*, se ocupă cu descrierea aspectelor teoretice legate de metoda de programare Divide et Impera și cu exemplificarea calculului complexității pentru algoritmi de acest tip. Capitolul conține și o serie de probleme rezolvate folosind aceasta tehnică.

Capitolul șapte, *Algoritmi Greedy*, se ocupă cu exemplificarea situațiilor în care se poate aplica aceasta tehnică de programare. În acest capitol se discută atât avantajele cât și a dezavantajele acestei strategii, și sunt prezentate o serie de probleme care urmăresc o cât mai bună înțelegere a situațiilor în care poate fi folosită.

Capitolul opt, *Algoritmi de Programarea Dinamică*, se ocupă cu descrierea acestor metode de programare, cu accent pe diferența dintre Greedy și Programare dinamică, dar și insistând pe diverse probleme practice rezolvabile prin această tehnică.

Ultimul capitol, *Diverse probleme practice* însumează o serie de probleme recapitulative, probleme tradiționale care ajută la fixarea și exersarea informațiilor tratate în toate capitolele din această carte.

Lucrarea se încheie cu o *Bibliografie* în care sunt incluse principalele cărți și materiale care au servit autoarelor ca sursă de documentare.

Capitolul 1

Analiza eficienței algoritmilor. Complexități

1.1 Analiza eficienței algoritmilor. Complexități

1.1.1 Aspecte teoretice

Pentru a rezolva o problemă cu ajutorul calculatorului avem nevoie de un algoritm. Vom considera că un algoritm este compus dintr-o succesiune finită de operații sau procese. Aplicarea unui algoritm presupune executarea acestora într-o anumită ordine. Algoritmii pe care îi descriem trebuie să respecte câteva restricții:

- să fie cât mai generali, adică să rezolve o clasă specifică de probleme de același tip;
- să fie corect, de cele mai multe ori corectitudinea fiind demonstrată pe cazul general, dar și pe cazuri particulare;
- să se încheie într-un anumit timp finit (număr finit de pași indiferent de datele de intrare);
- să asigure unicitatea rezultatelor, adică pentru același set de date de intrare să obținem același rezultat indiferent de context;
- în plus, un algoritm ar trebui să fie flexibil, eficient, lizibil și modular;

Studiul unei probleme cuprinde mai multe etape:

1. Formularea clară a problemei și a cerințelor;
2. Modelarea formală a problemei;

3. Clarificarea sau alegerea algoritmilor necesari;
4. Implementarea algoritmilor într-un limbaj de programare;
5. Testarea funcționalității;
6. Mentenanța rezolvării.

Așadar, studiul algoritmilor cuprinde mai multe aspecte: stabilirea contextului practic pentru algoritmul în cauză, elaborarea algoritmului, implementarea lui într-un limbaj de programare, validarea, analiza și testarea programelor.

În această secțiune vom discuta despre conceptul de eficiența a unui algoritm. Rolul ei practic este important în condițiile în care ne interesează să comparăm mai mulți algoritmi care rezolvă aceeași problemă. Evident, vom compara numai algoritmi despre care știm că sunt corecți. Putem compara doi algoritmi în raport cu cantitatea de memorie necesară și viteza de lucru, deci timpul necesar rezolvării problemei.

Timpul necesar execuției unui program depinde de numărul operațiilor ce trebuie executate, iar numărul operațiilor efectuate depinde de datele de intrare, care se schimbă de la o execuție la alta. Există însă un cel mai rău caz, pentru acele date de intrare pentru care numărul operațiilor efectuate este maxim. În acest caz vorbim despre complexitate în cel mai rău caz. De asemenea, putem vorbi despre numărul mediu de operații efectuate într-o execuție. Dacă numărul execuțiilor posibile este finit atunci acest număr mediu este egal cu numărul operațiilor efectuate în toate execuțiile, împărțit la numărul execuțiilor. Sunt însă foarte puține programe cu această proprietate. Pentru aproape toate programele, cel puțin teoretic, numărul excepțiilor posibile este infinit.

Scopul principal al analizei eficienței algoritmilor este acela de a determina modul în care timpul de execuție crește odată cu creșterea dimensiunii problemei.

În general, analiza unui algoritm se desfășoară în două etape:

- analiza apriorică: constă în aprecierea din punct de vedere temporal a operațiilor care se utilizează și a costului lor relativ. Aceasta conduce la stabilirea expresiei unei funcții care mărginește timpul de execuție al algoritmului;
- testul ulterior: constă în stabilirea unui număr suficient de seturi de date inițiale care să acopere practic toate posibilitățile de comportament ale algoritmului. Această etapă se finalizează prin culegerea unor date în baza cărora pot fi elaborate o serie de statistici referitoare la consumul de timp specific execuției algoritmului în cauză (profilul algoritmului).

Prezentăm un scurt **Q&A** cu observații legate de analiza eficienței unui algoritm:

- În ce constă analiza eficienței algoritmilor?

Analiza eficienței algoritmilor înseamnă estimarea volumului de resurse de calcul (spațiu de memorie, timp de execuție) necesare execuției algoritmilor. Analiza eficienței este utilă pentru a compara algoritmi între ei și pentru a obține informații privind resursele de calcul necesare pentru execuția lor.

Avantajul analizei teoretice constă în faptul că nu depinde de calculatorul folosit, de limbajul de programare ales sau de programator. Astfel se poate evita programarea și rularea unui algoritm care se dovedește a fi (la final) ineficient. Din motive absolut practice, un algoritm nu poate fi testat pe calculator pentru cazuri oricât de mari. Avantajul oferit de analiza teoretică este acela că ea permite studiul eficienței algoritmului pentru cazuri de orice mărime.

- Cum se poate măsura eficiența algoritmilor?

Pentru estimarea timpului de execuție este necesar să se stabilească un model de calcul și o unitate de măsură a timpului de execuție (timpul necesar execuției unei prelucrări elementare - prelucrări de bază: asignare, operații aritmetice, comparații, operații logice). Timpul de execuție al algoritmului ¹ = numărul de operații elementare. Estimarea timpului de execuție are ca scop determinarea dependenței dintre numărul de operații elementare executate și dimensiunea problemei executate.

- Analiza în cel mai favorabil și cel mai defavorabil caz?

Analiza în cazul cel mai favorabil furnizează o margine inferioară pentru timpul de execuție și permite identificarea algoritmilor ineficienți (dacă un algoritm are un cost ridicat chiar și în cel mai favorabil caz atunci el nu reprezintă o soluție acceptabilă). Analiza în cazul cel mai defavorabil furnizează cel mai mare timp de execuție în raport cu toate datele de intrare de dimensiune n (reprezintă o margine superioară a timpului de execuție). Este important de precizat faptul că marginea superioară a timpului de execuție este mai importantă decât marginea inferioară.

- Analiza în cazul mediu?

De cele mai multe ori, cazul cel mai favorabil sau cazul cel mai defavorabil sunt cazuri particulare/exceptii. De aceea, este necesară o evaluare pe cazul mediu. Scopul acestui tip de analiză este să furnizeze informații privind comportarea algoritmului în cazul unor date de intrare arbitrare. Analiza în cazul mediu se bazează pe cunoașterea distribuției de probabilitate a datelor de intrare. Timpul mediu de execuție este valoarea medie (în sens

¹Notat în secțiunile viitoare, dar și în documentația de pe platforma de elearning cu $T(n)$

statistic) a timpilor de execuție corespunzători diferitelor instanțe ale datelor de intrare.

- Care sunt etapele analizei eficienței?
 - Identificarea dimensiunii problemei;
 - Stabilirea operației dominante (cea mai costisitoare operație din algoritm);
 - Determinarea numărului de execuții ale operației dominante;
 - Dacă numărul de execuții ale operației dominante depinde de proprietățile datelor de intrare, atunci se analizează cel mai favorabil caz, cel mai defavorabil caz și cazul mediu;
 - Se stabilește ordinul (clasa) de complexitate.

După cum precizam anterior, obiectivul central al analizei eficienței algoritmilor este să determine modul în care timpul de execuție al algoritmului crește odată ce se mărește și dimensiunea problemei. Pentru a obține această informație nu este necesar să se cunoască expresia detaliată a timpului de execuție, dar este suficient să se identifice ordinul de creștere al timpului de execuție (în raport cu dimensiunea problemei) și clasa de eficiență (complexitatea) căreia îi aparține algoritmul.

Prezentăm un scurt **Q&A** cu observații legate de analiza asimptotică:

- Ce înseamnă ordinul de creștere (a timpului de execuție)?

În expresia timpului de execuție există în general un termen semnificativ mai mare decât ceilalți termeni atunci când dimensiunea problemei crește. Acest termen este denumit termen dominant și el impune comportamentul algoritmului în cazul în care dimensiunea problemei crește.

$$T_1(n) = a * n + b, T_2(n) = a \log n + b$$

$$T_3(n) = a * n^2 + b * n + c, T_4(n) = a^n + b * n + c, a > 1$$

Termenii dominanți în expresiile specificate mai sus sunt: $a * n, a \log n, a * n^2, a^n$.

Să evaluăm ce se întâmplă cu acest termen dominant atunci când dimensiunea problemei crește de k ori:

$$T'_1(k * n) = k * T_1(n), T'_2(k * n) = T_2(n) + a \log k$$

$$T'_3(k * n) = k^2 * T_3(n), T'_4(k * n) = T_4(n)^k, a > 1$$

Ordinul de creștere expune modalitatea în care crește termenul dominant al timpului de execuție în raport cu dimensiunea problemei; în cazurile noastre

(în ordine): liniar, logaritm, pătratic, exponențial. Atunci când comparăm doi algoritmi, cel cu ordinul de creștere mai mic este considerat a fi mai eficient (comparația se realizează pentru dimensiuni mari ale dimensiunii problemei (cazul asimptotic)), vezi tabelul 1.1.

Tabelul 1.1: Timpul de execuție în raport cu dimensiunea problemei

n	$\log_2 n$	$n * \log_2 n$	n^2	2^n	$n!$
10	3.3	33	10^2	1024	$36288 * 100$
10^2	6.6	664	10^4	10^{30}	10^{157}
10^4	13	132877	10^8	10^{3010}	10^{35659}

Ordinele de creștere a doi timpi de execuție $T_1(n)$ și $T_2(n)$ pot fi *comparate* prin calculul limitei raportului, cu $n \rightarrow \infty$. Sunt 3 cazuri valabile aici: limita este 0 și vom conclud că $T_1(n)$ are un ordin de creștere mai mic decât $T_2(n)$; limita este o constantă finită strict pozitivă, c ($c > 0$) și se poate conclud că $T_1(n)$ și $T_2(n)$ au același ordin de creștere; limita este infinită și se poate conclud că $T_1(n)$ are un ordin de creștere mai mare decât $T_2(n)$.

- La ce se referă analiza asimptotică?

Analiza timpilor de execuție pentru dimensiuni mici ale problemei nu permite diferențierea algoritmilor eficienți de cei ineficienți. Diferențele dintre ordinele de creștere devin relevante pe parcursul creșterii dimensiunii problemei. De aceea, analiza asimptotică are ca obiectiv studiul proprietăților timpului de execuție atunci când dimensiunea problemei tinde către infinit. Există trei notații folosite pentru diverse clase de eficiență: O (Omicron), Ω (Omega) și Θ (Theta).

1. *Notația O (Omicron)* desemnează marginea asimptotică superioară a unei funcții.

Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei. Spunem că funcțiile sunt în relația:

$$f(n) = O(g(n))$$

dacă și numai dacă

$$\exists n_0 \in \mathbb{N}, \exists c > 0 \mid 0 \leq f(n) \leq c * g(n), \forall n \geq n_0$$

Notația se folosește între funcțiile f și g în contextul în care $f(n)$ are un ordin de creștere cel mult egal cu cel al lui $g(n)$ (o altă notație utilizată este $f(n) \in O(g(n))$). Altfel spus, pentru valori mari ale lui

n , $f(n)$ este mărginită superior de $g(n)$ multiplicată cu o constantă pozitivă, vezi Figura 1.1², prima secvență. Se poate observa faptul că, pentru valori mai mici ale lui n , comportamentul nu este important.

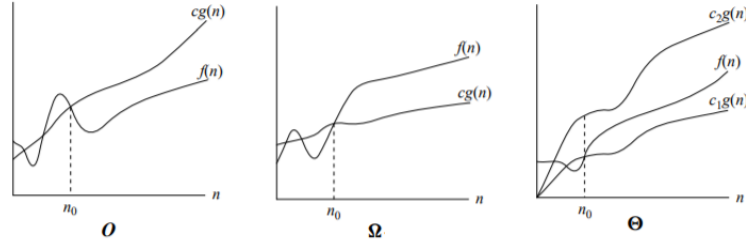


Figura 1.1: Ordine de creștere asimptotică

Să considerăm un exemplu cu $f(n) = n^4 + 2n^2 + 10n + 500$ și $g(n) = n^5$. Pe baza definiției de mai sus, $f(n) = O(g(n))$ dacă găsim constantele pozitive c și n_0 , astfel încât $0 \leq n^4 + 2n^2 + 10n + 500 \leq c * n^5, \forall n \geq n_0$. Inegalitatea nu trebuie să fie adevărată pentru toate valorile posibile ale lui n . Pot exista valori mici pentru care inegalitatea să nu fie adevărată, dar trebuie să arătăm că există o limită n_0 de la care inegalitatea este adevărată pentru orice valoarea mai mare ca n_0 . Există mai multe posibilități de a alege pentru c , noi alegem $c = 2$. Astfel avem rezultatele din tabelul 1.2.

Tabelul 1.2: Exemplu notația O

n	$f(n)$	$2 * g(n)$
1	513	2
3	629	486
4	828	2048
5	1225	6250
7	3069	33614

Pe baza valorilor din tabelul 1.2, putem concluda că pentru orice $n_0 \geq 4$, vom avea $0 \leq n^4 + 2n^2 + 10n + 500 \leq 2 * n^5$, deci avem $f(n) = O(n^5)$.

Alte exemple:

$$3n^2 - 100n + 6 = O(n^2), 3n^2 > 3n^2 - 100n + 6$$

$$3n^2 - 100n + 6 = O(n^3), 0.01n^3 > 3n^2 - 100n + 6$$

²<https://cdn.manesht.ir/19908/Introduction%20to%20Algorithms.pdf>

Deoarece notația O este folosită pentru a descrie o margine superioară, ea este utilizată pentru a mărgini cazul cel mai nefavorabil de execuție al unui algoritm. Prin implicație, notația pune în evidență o margine superioară a complexității algoritmului în aceeași măsură pentru orice intrare.

Sunt de precizat câteva proprietăți pentru notația O :

- (a) $f(n) \in O(f(n))$ (reflexivitate)
 - (b) $f(n) \in O(g(n)), g(n) \in O(h(n)) \Rightarrow f(n) \in O(h(n))$ (tranzitivitate)
 - (c) Dacă $T(n) = a_d * n^d + a_{d-1} * n^{d-1} + \dots + a_1 * n + a_0$ atunci $T(n) \in O(n^k)$ pentru orice $k \geq d$. (funcții polinomiale)
 - (d) Dacă pentru cazul cel mai nefavorabil obținem $T(n) \leq g(n)$, atunci $T(n) \in O(g(n))$.
2. Notația Ω (*Omega*) desemnează marginea asimptotică inferioară a unei funcții.

Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei. Spunem că funcțiile sunt în relația:

$$f(n) = \Omega(g(n))$$

dacă și numai dacă

$$\exists n_0 \in \mathbb{N}, \exists c > 0 \mid f(n) \geq c * g(n) \geq 0, \forall n \geq n_0$$

Notația se folosește între funcțiile f și g , în contextul în care $f(n)$ are un ordin de creștere cel puțin la fel de mare ca al lui $g(n)$. Altfel spus, pentru valori mari ale lui n , funcția $f(n)$ este mărginită inferior de $g(n)$ multiplicată eventual cu o constantă pozitivă, vezi Figura 1.1, a doua secvență. Ca și în cazul notației O , se poate observa că, pentru valori mai mici ale lui n , comportamentul nu este important.

Luăm un alt exemplu, funcțiile $f(n) = n^4 + 2n^2 + 10n + 500$ și $g(n) = n^2$. Conform definiției, dacă găsim constante pozitive pentru care să se îndeplinească relația, atunci $f(n) = \Omega(g(n))$. Putem alege $c = 50$ și obținem valorile din tabelul 1.3.

Pe baza valorilor din tabelul 1.3, putem concluda că pentru orice $n_0 \geq 7$, vom avea $0 \leq 50 * n^2 \leq n^4 + 2n^2 + 10n + 500$, deci avem $f(n) = \Omega(n^2)$.

Alte exemple:

$$T(n) = 3n + 3 \Rightarrow T(n) \in \Omega(n), 3n \leq 3n + 3, c = 3, n_0 = 1, g(n) = n$$

$$5 \leq T(n) \leq 3n + 2 \Rightarrow T(n) \in \Omega(1), c = 5, n_0 = 1, g(n) = 1$$

Tabelul 1.3: Exemplu notația Ω

n	$f(n)$	$50 * g(n)$
1	513	50
3	629	450
4	828	800
5	1225	1250
7	3069	2450
10	10800	5000

Și în cazul complexității de tip $\Omega(n)$ se păstrează proprietățile următoare: reflexivitate, tranzitivitate și funcții polinomiale. Observăm că pentru o funcție $f(n)$ pot exista mai multe funcții $g(n)$, astfel încât $f(n) = \Omega(g(n))$. Este important ca atunci când folosim notația $\Omega(n)$ să încercăm să găsim cea mai mare valoare posibilă pentru $g(n)$, astfel încât să avem o idee mai clară despre cea mai bună margine inferioară. Deci, vom spune că $T(n) = 3 * n + 4$ aparține lui $\Omega(n)$ și $T(n) = 2$ aparține lui $\Omega(1)$.

3. Notația Θ (Theta)

Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei. Spunem că funcțiile sunt în relația:

$$f(n) = \Theta(g(n))$$

dacă și numai dacă

$$\exists n_0 \in \mathbb{N}, \exists c_1, c_2 > 0 \mid c_1 * g(n) \leq f(n) \leq c_2 * g(n), \forall n \geq n_0$$

Notația se folosește între funcțiile f și g în contextul în care $f(n)$ are același ordin de creștere ca al lui $g(n)$. Altfel spus, pentru valori suficient de mari ale lui n , funcția $f(n)$ este mărginită atât superior, cât și inferior de $g(n)$ multiplicată cu niște constante pozitive, vezi Figura 1.1, a treia secvență. Idem notațiile anterioare, se poate observa că pentru valori mai mici ale lui n , comportamentul nu este important.

Luăm un alt exemplu, funcțiile $f(n) = n^4 + 2n^2 + 10n + 500$ și $g(n) = n^4$. Conform definiției, dacă găsim constante pozitive pentru care să se îndeplinească relația, atunci $f(n) = \Theta(g(n))$. Trebuie să găsim constantele pozitive c_1, c_2 și n_0 astfel încât funcția să fie delimitată corect. Astfel, dacă alegem valorile $c_1 = 0.3$ și $c_2 = 5$, atunci obținem valorile din tabelul 1.4.

Pe baza valorilor din tabelul 1.4, putem conclud că pentru orice $n_0 \geq 4$, vom avea $0.3 * n^4 \leq n^4 + 2n^2 + 10n + 500 \leq 5 * n^4$, deci avem $f(n) = \Theta(n^4)$. Așadar, $\Theta(g(n))$ se referă la valoarea exactă.

Tabelul 1.4: Exemplu notația Θ

n	$0.3 * n^4$	$f(n)$	$5 * n^4$
1	0.3	513	5
2	4.8	544	80
3	24.3	629	405
4	76.8	828	1280
5	187.5	1225	3125

Am observat că pentru o funcție $f(n)$ pot exista mai multe funcții $g(n)$ astfel încât $f(n) = O(g(n))$ sau $f(n) = \Omega(g(n))$, dar există o singură funcție pentru care $f(n) = \Theta(g(n))$. Atunci când acest lucru este posibil, vom utiliza notația Θ pentru complexitatea unui algoritm.

Sunt de precizat câteva proprietăți pentru notația Θ :

- (a) $f(n) \in \Theta(f(n))$ (reflexivitate)
- (b) $f(n) \in \Theta(g(n)), g(n) \in \Theta(h(n)) \Rightarrow f(n) \in \Theta(h(n))$ (tranzitivitate)
- (c) $f(n) \in \Theta(g(n)) \Rightarrow g(n) \in \Theta(f(n))$ (simetrie)
- (d) Dacă $T(n) = a_d * n^d + a_{d-1} * n^{d-1} + \dots + a_1 * n + a_0$ atunci $T(n) \in O(n^k)$ pentru orice $k \geq d$. (funcții polinomiale)
- (e) Dacă $f(n) = c * g(n)$, atunci $\Theta(f(n)) = \Theta(g(n)), c > 0$
Cazuri particulare: $\Theta(c) = \Theta(1), \Theta(\log_a h(n)) = \Theta(\log_b h(n))$
 $\forall a, b > 1$
- (f) Dacă $f(n) = f_1(n) + f_2(n) + \dots + f_m(n)$ și $f_i(n) \leq f_{i+1}(n), \forall i = 1, m$, atunci $\Theta(f(n)) = \Theta(\max(f_1(n), f_2(n), \dots, f_m(n)))$
- (g) $\Theta(g(n)) \subset O(g(n))$
De exemplu: $f(n) = 10 * n * \lg n + 5, g(n) = n^2, f(n) \leq g(n), \forall n \geq 36 \Rightarrow f(n) \in O(g(n))$, dar nu există constante c și n_0 astfel încât $c * n^2 \leq 10 * n * \lg n + 5$ pentru $n \geq n_0$.
- (h) $\Theta(g(n)) \subset \Omega(g(n))$
De exemplu: $f(n) = 10 * n * \lg n + 5, g(n) = n, f(n) \geq 10 * g(n), \forall n \geq 1 \Rightarrow f(n) \in \Omega(g(n))$, dar nu există constante c și n_0 astfel încât $10 * n * \lg n + 5 \leq c * n$ pentru orice $n \geq n_0$.
- (i) $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

- Cum arată clasificarea algoritmilor folosind notația O ?

$$O(1) \subset O(\log n) \subset O(\log^k n) \subset O(n) \subset O(n^2) \subset \dots \subset O(n^{k+1}) \subset O(2^n)$$

- Câteva complexități des întâlnite în practică?

$T(n) = O(1)$ - complexitate constantă, $T(n) = O(\log_2 n)$ - complexitate logaritmică, $T(n) = O(n)$ - complexitate liniară, $T(n) = O(n^2)$ - complexitate pătratică, $T(n) = O(n^k)$ - complexitate polinomială, $T(n) = O(2^n)$ - complexitate exponențială.

Se poate observa în Figura 1.2 ³ care este suita de valori pe complexitatea O .

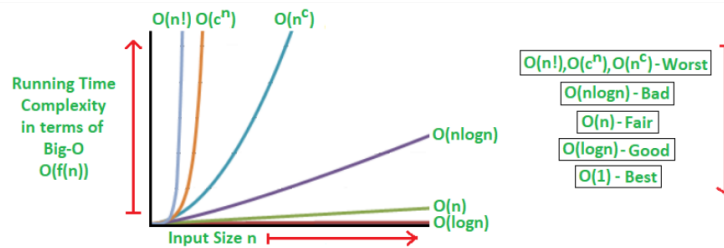


Figura 1.2: Reprezentarea diverselor complexități

- La ce se referă complexitatea de spațiu?

Complexitatea de spațiu măsoară spațiul de memorie de care are nevoie un algoritm pentru a stoca variabilele folosite pe parcursul algoritmului.

- Ce înseamnă $\log n$?

Convențional, $\log$ înseamnă $\log_{10}$. În acest ghid s-a folosit de fiecare dată cu semnificația $\log_2$, însă pentru notația O este nesemnificativă diferența deoarece $\log_2(x) = \log_k(x) / \log_k(2)$.

În același context, avem câteva relații care pot ajuta în identificarea/calcularea unor complexități: $\log 2^n = n$, $\log x^y = y \log x$, $\log xy = \log x + \log y$, $2^{\log n} = n$, $(2^x)^y = 2^{xy}$, $2^x * 2^y = 2^{x+y}$.

Exemple: $2^n = O(3^n)$, dar nu invers: $2^n < c * 3^n$ este adevărat pentru oricare n când $c = 1$, deci $2^n = O(3^n)$. Presupunem că $3^n = O(2^n)$, aceasta implică $3^n < c * 2^n$ pentru un anumit c și orice n . Deci $3^n / 2^n < c$, $\forall n$, însă se poate vedea că limita ($n \rightarrow \infty$) este ∞ , deci se poate alege oricând un n care să ne contrazică presupunerea.

- Cum calculăm complexitatea pentru funcții recursive?

În cazul algoritmilor recursivi, pentru estimarea timpului de execuție se stabilește relația de recurență care exprimă legătura dintre timpul de execuție

³<https://www.geeksforgeeks.org/analysis-algorithms-big-o-analysis/>

corespunzător problemei inițiale și timpul de execuție corespunzător problemei reduse (de dimensiune mai mică). Estimarea timpului de execuție se obține prin rezolvarea relației de recurență.

Plecăm de la ipoteza că un algoritm recursiv arată ca în Figura 1.3, unde $\langle SOME \rangle$ este prelucrarea cazului particular care are costul c_0 , iar h este o funcție descrescătoare și există k astfel încât $h^k(n) = h(h(\dots(h(n))\dots)) = n_0$, iar costul lui $h(n)$ este c .

```
functieRecursiva (n)
  IF n = n0
    <SOME>
  ELSE
    functieRecursiva(h(n))
  ENDIF
```

Figura 1.3: Model de funcție recursivă

Cu aceste ipoteze, relația de recurență pentru timpul de execuție este:

$$T(n) = \begin{cases} c_0 & n = n_0 \\ T(h(n)) + c & n > n_0 \end{cases}$$

Exemplu: Calcul pentru $n!$, unde relația de recurență este:

$$n! = \begin{cases} 1 & n = 1 \\ (n-1)! * n & n > 1 \end{cases}$$

Dimensiunea problemei este n , iar operația dominantă este înmulțirea. Relația de recurență este: $T(n) = T(n-1) + 1, n > 1$.

Sunt mai multe metode pentru determinarea relațiilor de recurență:

- *Metoda iterației (substituției directe)* în care se execută primii pași, se intuiește forma generală, iar apoi se demonstrează prin inducție matematică faptul că forma este corectă.

Raportat la exemplul nostru: $T(1) = 0, T(2) = 1, T(3) = 2, \dots, T(n) = n - 1$.

- *Metoda substituției inverse*, unde se pornește de la cazul $T(n)$ și se înlocuiește $T(h(n))$ cu membrul drept al relației corespunzătoare, apoi se înlocuiește $T(h(h(n)))$ etc. până se ajunge la cazul particular; sau se înmulțesc egalitățile cu factori care să permită eliminarea tuturor termenilor de forma $T(h(n))$ cu excepția lui $T(n)$, iar apoi se efectuează calculele și se obține $T(n)$.

Raportat la exemplul nostru: $T(n) = T(n-1) + 1$, $T(n-1) = T(n-2) + 1$, ..., $T(2) = T(1) + 1$, $T(1) = 0$ adunăm tot și obținem $T(n) = n - 1$.

- *Teorema master* care este o metodă directă pentru a obține soluția. Teorema funcționează doar pentru recurențe care sunt (sau pot fi transformate la recurențe) de forma: $T(n) = a * T(n/b) + f(n)$, unde $a \geq 1, b > 1$.

Există 3 cazuri:

- * Dacă $f(n) = \Theta(n^c)$, cu $c < \log_b a$ atunci $T(n) = \Theta(n^{\log_b a})$
- * Dacă $f(n) = \Theta(n^c)$, cu $c = \log_b a$ atunci $T(n) = \Theta(n^c \log n)$
- * Dacă $f(n) = \Theta(n^c)$, cu $c > \log_b a$ atunci $T(n) = \Theta(f(n))$

Aceasta poate fi aplicată în analiza algoritmilor bazați pe tehnica divide et impera. De asemenea, aceasta evită rezolvarea explicită a relației de recurență corespunzătoare timpului de execuție. În multe aplicații, etapele de divizare (reducere) și de combinare sunt de complexitate polinomială. Spre deosebire de variantele de rezolvare explicită a relației de recurență, teorema furnizează doar ordinul de complexitate, nu și constantele ce intervin în estimarea timpului de execuție. Sunt de precizat următoarele probleme în care se poate utiliza teorema: Merge-Sort, căutare binară.

1.1.2 Complexități - Probleme rezolvate

1. Care este complexitatea pentru metoda `fun()`?

Secvență de cod 1.1: Cod C++ pentru funcția `fun()`

```

1  int fun(int n)
2  {
3      int count = 0;
4      for (int i = 0; i < n; i++)
5          for (int j = i; j > 0; j--)
6              count = count + 1;
7      return count;
8  }
```

- A. $\Theta(n)$
 B. $\Theta(n^2)$
 C. $\Theta(n * \log n)$
 D. $\Theta(n * \log n * \log n)$

Explicație: $\Theta(0 + 1 + 2 + 3 + \dots + n - 1) = \Theta(n * (n - 1) / 2) = \Theta(n^2)$

2. Care dintre următoarele NU este $O(n^2)$?

- A. $15^{10} * n + 120999$
 B. $n^{1.98}$
 C. $n^3 / \sqrt{n}$
 D. $2^{20} * n$

Explicație: Ordinul de creștere pentru punctul C este $n^{2.5}$ care este mai mare ca n^2 .

3. Care dintre următoarele variante arată ordinea crescătoare a complexităților asimptotice ale funcțiilor f_1, f_2, f_3, f_4 ? $f_1(n) = 2^n$, $f_2(n) = n^{3/2}$, $f_3(n) = n \log n$, $f_4(n) = n^{\log n}$

- A. f_3, f_2, f_4, f_1
 B. f_3, f_2, f_1, f_4
 C. f_2, f_3, f_1, f_4
 D. f_2, f_3, f_4, f_1

Explicație: f_3 este logaritmică, deci față de exponențiale și polinomială, este prima. Dintre cele rămase, $n^{3/2}$ este următoarea. Pentru compararea funcțiilor f_1 și f_4 putem să le transformăm pe ambele în log, de aici ordinul de creștere pentru $\log(f_1(n))$ este $\Theta(n)$ iar pentru $\log(f_4(n))$ este $\Theta(\log n * \log n) \Rightarrow f_1(n)$ crește mai rapid ca $f_4(n)$.

4. Dacă $n > 0$, care dintre secvențele următoare este cea mai eficientă (ignorând corpul ciclului repetitiv)?

- A. for (i = 0; i < n; i++)
- B. for (i = 0; i < n; i += 2)
- C. for (i = 1; i < n; i *= 2)
- D. for (i = n; i > -1; i /= 2)

Explicație: Pentru prima secvență avem $O(n)$, pentru a doua avem $O(n/2)$, echivalentă cu $O(n)$ în notație asimptotică. Pentru a treia secvență avem $O(\log n)$, iar a patra secvență nu se termină.

5. Fie funcțiile n^k și c^n . Care este relația asimptotică corectă? Presupunem $k \geq 1$, iar $c > 1$, ambele constante.

- A. n^k este $O(c^n)$
- B. n^k este $\Omega(c^n)$
- C. n^k este $\Theta(c^n)$
- D. niciuna dintre variantele de mai sus

Explicație: n^k este o funcție polinomială, iar c^n este una exponențială. Cunoaștem faptul că polinomialele cresc întotdeauna mai puțin ca exponențialele. Dacă spunem că n^k este $O(c^n)$ atunci spunem că n^k este o margine superioară pentru c^n , ceea ce este adevărat, luând în calcul ce am spus mai sus despre funcțiile polinomiale și exponențiale.

6. Pentru fiecare secvență, să se stabilească valoarea de adevăr.

- A. $3 * n^2 + 10 * n * \log n = O(n * \log n)$
- B. $3 * n^2 + 10 * n * \log n = \Omega(n^2)$
- C. $3 * n^2 + 10 * n * \log n = \Theta(n^2)$
- D. $n * \log n + n/2 = O(n)$
- E. $10 * \sqrt{n} + \log n = O(n)$
- F. $\sqrt{n} + \log n = O(\log n)$
- G. $\sqrt{n} + \log n = \Theta(\log n)$

H. $\sqrt{n} + \log n = \Theta(n)$

I. $2 * \sqrt{n} + \log n = \Theta(\sqrt{n})$

J. $\sqrt{n} + \log n = \Omega(1)$

K. $\sqrt{n} + \log n = \Omega(\log n)$

L. $\sqrt{n} + \log n = \Omega(n)$

Explicație: A. Fals, din cauza faptului că n^2 (termenul dominant) crește mai rapid asimptotic față de $n * \log n$;

B. Adevărat, cum avem același termen dominant și cresc la fel;

C. Adevărat, idem explicația de la [B.];

D. Fals, cum $n * \log n$ (termenul dominant) nu este asimptotic superior mărginit de n ;

E. Adevărat;

F, G, H. Fals;

I, J, K. Adevărat.

1.1.3 Complexități - Probleme propuse

1. Pentru fiecare dintre cazurile din tabelul 1.5, să se precizeze relația validă dintre $f(n)$ și $g(n)$:
 - i1. $f(n) \in \Theta(g(n))$
 - i2. $f(n) \in O(g(n))$
 - i3. $f(n) \in \Omega(g(n))$

Tabelul 1.5: Timp de execuție în raport cu dimensiunea problemei

Variantă	$f(n)$	$g(n)$
A.	$100 * n + \log n$	$n + (\log n)^2$
B.	$\log n$	$\log n^2$
C.	$\frac{n^2}{\log n}$	$n * (\log n)^2$
D.	$(\log n)^{\log n}$	$\frac{n}{\log n}$
E.	$\sqrt{n}$	$(\log n)^5$
F.	$n * 2^n$	3^n
G.	$2^{\sqrt{\log n}}$	$\sqrt{n}$

2. Se consideră un algoritm pentru calculul produsului dintre o matrice A de ordin $m \times n$ și un vector x de lungime n : $y[i] = A[i, 1] * x[1] + A[i, 2] * x[2] + \dots + A[i, n] * x[n]$ pentru $i = 1 \dots m$, vezi Figura 1.4 Numărul de operații de

```

produsMatriceVector(A[1..m, 1..n], x[1..n])
FOR i ← 1, m DO
  y[i] ← 0
  FOR j ← 1, n DO
    y[i] ← y[i] + A[i, j] * x[j]
  ENDFOR
ENDFOR
RETURN y[1..m]

```

Figura 1.4: Pseudocod pentru produs matrice cu vector

înmulțire efectuate în cazul algoritmului este:

- A. m
- B. n
- C. $m + n$
- D. $m * n$
- E. $(m - 1) * (n - 1)$

3. Presupunând că dimensiunea problemei este n iar operația dominantă este adunarea, care este ordinul de complexitate al algoritmului din Figura 1.5?

```
algorithm(n)
if n=0 then
    rez ← 0
else
    rez ← alg(n DIV 10)+n MOD 10
endif
return rez
```

Figura 1.5: Pseudocod pentru `algorithm(n)`

- A. Complexitate $O(n)$
- B. Complexitate $\Theta(n)$
- C. Complexitate $O(\log n)$
- D. Complexitate $\Theta(\log n)$
- E. Complexitate $\Theta(1)$

Capitolul 2

Operații elementare. Procesarea numerelor

2.1 Operații elementare - Aspecte teoretice

Acest capitol prezintă câteva dintre elementele de bază ale limbajului C++, care vor fi folosite ulterior în contextul elaborării algoritmilor. Capitolul nu urmărește o trecere exhaustivă în revistă a facilităților oferite de limbajul C++, ci doar punerea în evidență a unor aspecte considerate importante.

Pentru că orice program/algoritm lucrează cu date și execută prelucrări asupra acestor date, unul dintre subiectele atinse va fi modul în care se păstrează și se folosesc acestea, ca și variabile în programele noastre.

Scrierea modulară a aplicațiilor este un alt aspect pe care vom insista în cele ce urmează. Aceasta se realizează cu ajutorul subprogramelor/ funcțiilor și este extrem de utilă deoarece permite divizarea logică a programelor mari și complexe în bucăți mici, mai ușor de înțeles și de programat. Acesta este motivul pentru care vom relua o parte dintre noțiunile teoretice legate de declararea/definirea funcțiilor, apelul acestora și transmiterea parametrilor către ele în momentul apelului.

2.1.1 Tipuri de date

Valorile variabilelor cu care lucrează orice program sunt stocate în memoria calculatorului. Programul accesează zona de memorie corespunzătoare prin referirea la numele variabilei. El trebuie să știe care este tipul de date păstrat în variabilă, pentru că în funcție de asta compilatorul alocă o zonă mai mare sau mai mică de memorie. Nu este același lucru din punct de vedere al reprezentării, stocarea unui număr întreg cu stocarea unui număr mare în virgulă mobilă.

În plus, deși toate datele sunt reprezentate folosind cifre unu și zero, compilatorul le interpretează diferit, în funcție de tipul de date cu care știe că lucrează.

Tipurile de date primitive sunt tipuri de bază recunoscute direct de limbaj. Acestea pot fi clasificate în principal astfel:

- *Tipuri pentru numere întregi.* Acestea pot stoca o valoare din mulțimea numerelor întregi. Există într-o gamă largă de dimensiuni și pot fi precedate de diverși specificatori de conversie, ca de exemplu **signed/unsigned** în funcție de capacitatea de a suporta sau nu valori negative, sau **short/long**. Tipul de bază este **int**.
- *Tipuri pentru numere reale:* Pot reprezenta valori reale cu grade precizii diferite. De obicei în raport cu acest criteriu se decide care dintre ele va fi folosit. Tipurile reale sunt: **float**, **double**, **long double**.
- *Tipuri caracter:* Acestea pot reprezenta un singur caracter. Tipul de bază este **char** care se reprezintă pe un octet. Dar există și alte tipuri de caractere mai largi.
- *Tipul boolean:* Tipul boolean poate reprezenta date care iau numai una din cele două valori: **true(1)** sau **false(0)**.

2.1.2 Variabile

1. Identificatori

În limbajul C++, numele variabilelor, ale funcțiilor și ale altor diverse obiecte definite de către utilizator sunt numite *identificatori*. Un identificator valid este o secvență de una sau mai multe litere, cifre sau caractere de subliniere "_". Când alegem un astfel de identificator trebuie să ținem cont de următoarele reguli:

- Nu pot face parte dintr-un identificator spațiile, semnele de punctuație sau simbolurile;
- Identificatorii încep întotdeauna cu o literă. Ei pot începe și cu un caracter de subliniere "_";
- Identificatorii nu pot începe cu o cifră;
- Limbajul C++ este *case sensitive*. Acest lucru înseamnă că se va face diferența între un identificator scris cu litere mici și unul cu același nume, dar în care măcar o literă a fost schimbată din literă mică în literă mare.
- Limbajul C++ utilizează o serie de cuvinte cheie pentru a identifica descrierile de date și operațiile. Asta înseamnă că identificatorii aleși de un programator nu pot folosi aceste cuvinte cheie. Cuvintele cheie rezervate standard care nu pot fi folosite ca identificatori sunt:

alignas, alignof, and, and_eq, asm, auto, bitand, bitor, bool, break, case, catch, char, char16_t, char32_t, class, compl, const, constexpr, const_cast, continue, decltype, default, delete, do, double, dynamic_cast, else, enum, explicit, export, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, noexcept, not, not_eq, nullptr, operator, or, or_eq, private, protected, public, register, reinterpret_cast, return, short, signed, sizeof, static, static_assert, static_cast, struct, switch, template, this, thread_local, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t, while, xor, xor_eq

2. Declararea variabilelor

În C++ este necesar ca fiecare variabilă să fie declarată de tipul considerat corespunzător, înainte de prima sa utilizare. Această declarație îi spune compilatorului care este dimensiunea pe care să o aloce în memorie pentru variabilă declarată și este o modalitate de a-i transmite compilatorului cum trebuie interpretate valorile sale. Sintaxa C++ pentru declararea unei variabile este următoarea:

tip nume_var

unde *tip* este tipul de date al variabilei.

Pentru o serie de variabile de același tip, declararea se poate face într-o singură linie de cod, separând numele variabilelor prin virgulă. De exemplu:

float a, b, c;

Unde se declară variabilele

Variabilele se pot declara în trei locuri: în interiorul funcțiilor, în cadrul definiției parametrilor unei funcții sau în afara oricărei funcții. În situațiile amintite mai sus avem de a face cu variabile locale, parametri formali și variabile globale.

Variabile locale

Variabilele care sunt declarate în interiorul unei funcții (metode) sunt numite *variabile locale*. Variabilele locale nu sunt accesibile decât instrucțiunilor care sunt în interiorul metodei în care sunt declarate aceste variabile. Cu alte cuvinte, ele sunt "cunoscute" doar în interiorul propriului lor bloc de instrucțiuni.

Ele există numai cât timp se execută metoda în care sunt declarate. Asta înseamnă că o variabilă locală este creată la începerea execuției metodei și este distrusă la încheierea ei.

În C++, declararea variabilelor locale poate fi făcută în orice punct la programului, înainte de prima lor utilizare.

Deoarece variabilele locale sunt create și distruse la fiecare intrare/ieșire din corpul metodei în care au fost declarate, conținutul lor se pierde odată cu ieșirea din metodă. Totuși, dacă este nevoie, compilatorul poate fi făcut să păstreze aceste valori, folosind specificatorul de conversie **static**.

Parametri formali

Dacă o funcție are nevoie de argumente, ea trebuie să declare variabilele pe care le acceptă ca valori ale argumentelor. Aceste variabile sunt denumite *parametri formali* ai funcției. Ele se comportă ca și variabilele locale din acea funcție.

Aceste variabile primesc la apelul metodei valoarea argumentelor transmise funcției, dar ele pot fi folosite ca orice altă variabilă locală.

Variabile globale

Spre deosebire de variabilele locale, *variabilele globale* sunt cunoscute în întreg programul și pot fi folosite de către orice secvență de cod. De asemenea ele își vor păstra valoarea pe parcursul întregii execuții a programului. Variabilele globale se creează prin declarare în afara oricărei funcții și de preferat la începutul programului.

Stocarea variabilelor globale este făcută de către compilator într-o anumită zonă de memorie destinată acestui scop. Variabilele globale sunt utile atunci când mai multe metode ale aceluiași program ar folosi aceleași date. Cu toate acestea, utilizarea lor ar trebui evitată atunci când nu este (absolut) necesar. Printre dezavantajele folosirii variabilelor globale am putea aminti:

- Ele ocupă spațiu în memorie tot timpul cât este executat programul, nu numai atunci când este nevoie de ele;
- Utilizarea unei variabile globale în locul uneia locale va face funcția mai puțin generală deoarece se bazează pe ceva ce trebuie definit în afara ei;
- Folosirea unui număr mare de variabile globale, poate fi o sursă de erori în program. O posibilă eroare poate fi schimbarea accidentală a valorii variabilei în altă parte, unde este ea utilizată.

3. Inițializarea variabilelor

Momentul în care unei variabile i se atribuie o valoare pentru prima dată se numește inițializarea variabilei. Aceasta poate fi făcută chiar în momentul declarării variabilei sau ulterior. În C++, există trei variante de a inițializa variabilele. Toate trei sunt echivalente.

- Prima dintre ele, este inițializarea în stilul limbajului C. Ea are următoarea formă:

$$\text{tip nume_var} = \text{valoare_initiala}$$

unde *identifier* reprezintă numele variabilei iar valoarea de după semnul `=` este valoarea care se atribuie la inițializare.

Exemplu: Declararea unei variabile de tip `int`, numită *v*, și inițializarea ei în momentul declarării, se face astfel:

```
int v = 15
```

- A doua metodă este cunoscută ca inițializare cu ajutorul constructorului (ea a fost introdusă în limbajul C++). Ea se face scriind după numele variabilei, valoarea inițială între paranteze (`.`).

```
tip nume_var(valoare_initia);
```

Exemplu:

```
int v(15);
```

- A treia metodă este cunoscută ca inițializare uniformă. Ea este similară inițializării cu ajutorul constructorului, dar folosește acolade `{}` în locul parantezelor. Această metodă a fost introdusă la revizuirea standardului C++, în anul 2011). Așa cum am precizat, forma ei este:

```
tip nume_var{valoare_initia};
```

Exemplu:

```
int v{15};
```

Deducerea tipului unei variabile

Atunci când o nouă variabilă este inițializată, compilatorul poate identifica automat tipul de dată al variabilei, în funcție de valoarea cu care s-a făcut inițializarea. Pentru aceasta este suficient să folosim **auto** ca specificator de tip pentru acea variabilă.

Exemplu:

```
float v = 15.5;
```

```
auto s = v;
```

```
cout << s;
```

În această secvență de cod, variabila *s* este declarată ca având tip **auto**. Prin urmare, tipul variabilei *s* este tipul valorii folosite la inițializarea lui *s*.

În cazul variabilelor care nu sunt inițializate, deducerea tipului variabilei se poate face folosind specificatorul **decltype**.

Exemplu:

```
float v = 0.5;
decltype(v) s;
```

Astfel, în a doua linie de cod a secvenței anterioare, variabila *s* este declarată ca având același tip cu variabila *v*.

Cu toate că **auto** și **decltype** sunt facilități puternice ale limbajului C++, se recomandă ca acestea să nu fie folosite în mod abuziv. Este recomandat ca deducerea tipului unei variabile să se folosească numai atunci când nu se poate obține altfel, sau atunci când folosirea ei îmbunătățește claritatea codului.

2.1.3 Funcții

1. Forma generală a unei funcții

Funcțiile sunt secvențe de cod, de dimensiuni relativ reduse, care îndeplinesc sarcini individuale bine definite, și care permit scrierea modulară a programelor. O funcție (metodă) este identificată printr-un nume și va fi executată numai dacă va fi apelată dintr-un alt punct al programului.

Există o serie de avantaje în scrierea algoritmilor de rezolvare a unor probleme folosind funcții:

- Problema se descompune în probleme mai simple, care vor fi mai ușor de rezolvat;
- Metodele, odată scrise, pot fi folosite și în alte programe (reutilizarea codului) ;
- Erorile sunt mai ușor de depistat și de corectat;
- Programele sunt mai ușor de urmărit dacă sunt scrise modular (folosind metode);

Scrierea și conceperea funcțiilor trebuie să aibă la bază următoarele principii: *claritate, lizibilitate, ușurință în întreținere, reutilizabilitate*.

2. Definirea/declararea și apelul unei funcții

În limbajul C++ se folosesc **declarații** și **definiții** de funcții.

Caracteristicile definitorii ale unei funcții în C++ sunt: numele, parametrii de apel și valoarea returnată de funcție. Sintaxa standard pentru **declararea** unei funcții este:

tip_returnat nume_functie(lista_de_parametri_formali)

Această declarație poartă numele de antet al funcției. Odată declarată, o funcție trebuie **definită**, în sensul că trebuie descrise instrucțiunile pe care aceasta trebuie să le execute.

Cea mai utilizată sintaxă pentru **definirea** unei funcții este:

tip_returnat nume_functie(lista_de_parametri_formali)
{corpul funcției}

În secvențele anterioare:

- **tip_returnat** este tipul valorii returnate de funcție. El poate fi orice tip standard sau definit de utilizator. Dacă funcția nu trebuie să returneze nicio valoare, înaintea numelui funcției se scrie *void*. Acesta este un tip special ce reprezintă absența unei valori.
- **nume_functie** este identificatorul cu care funcția poate fi apelată;
- **lista_de_parametri_formali** este o listă separată prin virgulă de nume de variabile și de tipurile asociate lor. Scopul parametrilor este de a permite transmiterea de argumente către funcție atunci când aceasta este apelată. O funcție poate să nu aibă parametri, caz în care lista este vidă. Totuși, parantezele sunt necesare chiar dacă nu există parametri.
- **corpul funcției** este un bloc de instrucțiuni, cuprinse între acolade { }, care precizează ceea ce trebuie să facă funcția;

Limbajul C++ permite separarea declarației unei funcții de definiția acesteia. Pentru ca funcția să poată fi folosită este obligatorie doar declararea acesteia înainte de codul sursă care o apelează. Definiția poate să apară ulterior în fișierul sursă, sau chiar într-un alt fișier sursă sau bibliotecă.

Apelul unei funcții este modul în care secvența de instrucțiuni corespunzătoare este pusă în execuție. El poate fi făcut din funcția principală, **main()**, dintr-o altă funcție sau chiar din ea însăși prin autoapelare (recursivitate). Instrucțiune de apel a funcției este de forma:

nume_functie(lista_de_parametri_actuali)

În momentul apelului unei funcții, valorile parametrilor actuali se atribuie parametrilor formali corespunzători. În cazul în care tipul unui parametru

actual diferă de tipul parametrului formal corespunzător, acesta va fi convertit spre tipul parametrului formal, dacă este posibil, altfel compilatorul va genera eroare.

La întâlnirea unui apel de funcție, controlul execuției programului este transferat primei instrucțiuni din funcție, urmând a se executa corpul funcției. Revenirea dintr-o funcție se face fie după execuția ultimei instrucțiuni din corpul funcției, fie la întâlnirea unei instrucțiuni **return**. La revenirea din funcție, controlul este redat funcției apelante.

O altă posibilitate de a apela o funcție este aceea în care apelul funcției constituie operandul unei expresii. Acest lucru este însă posibil numai în cazul în care funcția returnează o valoare, care va fi folosită în evaluarea expresiei.

3. Argumente transmise prin valoare și prin referință

În general, funcțiile comunică între ele prin transmiterea de argumente (parametri). Limbajul C++ permite următoarele moduri de transmitere a parametrilor către funcțiile apelate:

- (a) Transfer prin **valoare**;
- (b) Transfer prin **referință**.

Transmiterea prin valoare

În cazul transmiterii unui argument prin valoare, ceea ce se transmite este valoarea pe care o are argumentul în momentul apelului funcției. Această valoare se *copiază* în parametrul formal al metodei și orice modificare din cadrul funcției va opera asupra acestui parametru și nu asupra variabilei transmise ca argument, deoarece în apel nu a fost transmisă variabila efectiv ci doar valoarea ei în acel moment.

La sfârșitul execuției funcției, parametrii formali ai metodei sunt distruși din cauză că sunt variabile locale, și astfel se pierde orice modificare efectuată. Pentru a nu pierde modificările făcute se folosește instrucțiunea **return**, care poate întoarce, la terminarea funcției noua valoare a variabilei. O situație specială ar fi aceea în care funcția modifică mai multe variabile și se dorește ca rezultatul lor să fie disponibil și la terminarea execuției funcției.

Secvență de cod 2.1: Cod C++ Exemplu de cod pentru transmiterea parametrilor prin valoare

```
1
2 #include <iostream>
3 using namespace std;
4
5 // transmiterea parametrilor in functia patrat se face prin valoare
6 int patrat(int x)
7 {
8     x = x * x;
9     return x;
10 }
11
12 int main()
13 {
14     int a = 5;
15
16     cout << patrat(a) << " " << a;
17 }
```

În exemplul anterior, valoarea argumentului funcției **patrat()**, care este 5, este copiată în parametrul x . Când are loc atribuirea $x = x * x$, este modificată doar variabila locală x . Variabila a folosită în apelul funcției **patrat()** are în continuare valoarea 5. De aceea valorile afișate vor fi 25 și 5.

Atragem atenția asupra faptului că, în contextul în care parametrii funcției sunt de tip pointer, și aceștia sunt transmiși funcțiilor la fel ca orice altă valoare. Aceasta pentru că un pointer nu este nimic altceva decât o variabilă care conține o adresă de memorie.

Transmiterea prin referință

În cazul transmiterii unui argument prin referință, nu se mai face o copie a valorii sale, ci se transmite funcției un pointer către argument (adresa acestuia). În acest caz, numele parametrului formal este de fapt un **alias** al argumentului transmis prin referință și orice modificare asupra acestui parametru în funcție se reflectă asupra variabilei transmise ca argument. În C++ referințele sunt indicate de simbolul ampersand (&) după tipul parametrilor, așa cum se poate vedea în exemplul de la jos

Secvență de cod 2.2: Cod C++ Exemplu de cod pentru transmiterea parametrilor prin referință

```
1
2 #include <iostream>
3 using namespace std;
4
5 // transmiterea parametrului in functia patrat
6 // se face prin referinta
7 void patrat(int &x)
8 {
9     x = x * x;
10 }
11
12 int main()
13 {
14     int a = 5;
15     patrat(a);
16     cout << a;
17 }
```

În secvența anterioară x devine un alias al argumentului a transmis în apelul funcției și orice schimbare asupra lui x în interiorul funcției devine schimbare a variabilei a în afara funcției. Astfel valoarea afișată pentru variabila a va fi 25.

2.2 Procesarea numerelor - probleme rezolvate

În secțiunea care urmează propunem o serie de probleme cu numere:

1. Să se citească numere naturale până la întâlnirea lui 0 și să se afișeze acele numere care sunt palindroame.
2. Să se afișeze primii n termeni din șirul lui Fibonacci (fără a utiliza funcții pentru calculul lor).
3. Se citesc de la tastatură numere naturale cel mult până se introduce -1. Să se verifice dacă sunt în progresie aritmetică. Numerele nu se rețin într-un vector.
4. Să se verifice dacă un număr întreg x , cu cel puțin trei cifre, este "zâmbăreț" (cifrele lui privite de la stânga la dreapta sunt în ordine strict descrescătoare și de la un punct încolo sunt în ordine strict crescătoare).

Exemplu: Numărul 64324789 este "zâmbăreț". Numărul 4311254 nu este "zâmbăreț".

5. Să se determine reprezentarea în baza 2 a unui număr natural n .

Rezolvarea unei probleme constituie un proces complex, care conține mai multe etape.

- Analiza problemei în scopul stabilirii *datelor de intrare*, precum și a rezultatelor pe care trebuie să le obținem prin rezolvarea problemei (*date de ieșire*);
- Elaborarea unui algoritm de rezolvare a problemei (ideal ar fi prin crearea de diverși subalgoritmi, în care fiecare să execute o singură acțiune - KISS principle ¹);
- Implementarea algoritmului într-un limbaj de programare, în cazul nostru descrierea în limbajul C++;

¹<https://www.interaction-design.org/literature/article/kiss-keep-it-simple-stupid-a-design-principle>

- Verificarea corectitudinii algoritmului propus. Un prim pas constă în testarea programului pe diverse seturi de date de test. Seturile de date de test trebuie elaborate cu atenție, astfel încât să acopere, pe cât posibil, toate variantele de execuție a algoritmului, inclusiv situații de excepție, și să verifice dacă fiecare subproblema a problemei date este rezolvată corect (pe cât posibil, testarea se face separat pe fiecare subprogram). Un pas mai avansat în demonstrarea corectitudinii este utilizarea unor metode formale de demonstrare a corectitudinii algoritmului, o etapa mai consistentă, care necesită o modelare matematică complexă);
- Analiza complexității algoritmului, vezi capitol de "Analiza eficienței algoritmilor. Complexități".

Acestea sunt etapele pe care le vom urmări de fiecare dată în rezolvarea problemelor.

Pentru problema 1 avem:

- Date de intrare: Un șir de numere naturale, citite rând pe rând de la tastatură.
- Date de ieșire: Se afișează în consolă numerele care verifică condiția problemei.
- Algoritm de rezolvare: Pentru un numar natural n citit se construiește oglinditul său, care este de fapt numărul citit de la dreapta la stânga. Dacă numărul coincide cu oglinditul său, atunci acesta este palindrom.
- Algoritmul scris în C++ are la bază o metodă care verifică dacă un număr dat este palindrom. În metoda main() se citesc numere naturale până la întâlnirea lui 0. Pentru fiecare număr citit se verifică dacă este palindrom, și dacă da, atunci se afișează numărul. Altfel se citește un altul nou.

Secvență de cod 2.3: Cod C++ pentru afisarea numerelor care sunt palindroame, problema 1

```

1  #include <iostream>
2  using namespace std;
3
4  bool estePalindrom(int numar)
5  {
6      //declaram doua variabile: una pentru a pastra numarul initial,
7      //alta pentru a construi numarul oglindit
8      int numarInitial = numar, oglindit = 0;
9
10     while (numar)
11     {
12         //luam cifrele de la dreapta la stanga si le
13         //concatenam pentru a forma imaginea in oglinda
14         int cifra = numar % 10;
15         oglindit = oglindit * 10 + cifra;
16         numar /= 10;
17     }
18
19     if (numarInitial == oglindit)
```

```

20         return true;
21     else
22         return false;
23 }
24
25 int main()
26 {
27     int numar;
28     do
29     {
30         cout << "\n Introduceți un număr ";
31         cin >> numar;
32         if (numar != 0 && estePalindrom(numar))
33         {
34             cout << "\n numărul " << numar << " este palindrom";
35         }
36     } while (numar);
37
38 }

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității: Metoda care verifică dacă un număr este palindrom are complexitate constantă, deoarece numărul de cifre al unui `int` este maxim 11. Aceasta înseamnă că în acest caz, complexitatea algoritmului este dată de numărul de întregi citați până la întâlnirea lui 0. Dacă acesta este m atunci complexitatea algoritmului este $O(m)$.

Pentru problema 2 avem:

- Date de intrare: Un număr natural n .
- Date de ieșire: Primii n termeni ai șirului lui Fibonacci.
- Algoritm de rezolvare: Termenii șirului lui Fibonacci sunt dați de relația de recurență:

$$Fib_n = Fib_{n-1} + Fib_{n-2}; \quad Fib_0 = 0; \quad Fib_1 = 1$$

Asta înseamnă că pentru simpla lor afișare ne sunt suficiente variabilele *penultim*, *ultim* și *curent* care să rețină ultimii doi termeni ai șirului pe baza cărora să îl calculăm pe cel curent. Metoda parcurge un număr de iterații egal cu numărul transmis ca parametru algoritmului, afișând la fiecare iterație termenul curent al șirului.

- Algoritmul scris în C++ conține metoda care face calculul termenilor șirului, conform relației de recurență și afișarea termenului curent. Metoda este apelată din funcția `main()` pentru o valoare n citită de la tastatură.

Secvență de cod 2.4: Cod C++ pentru afișarea termenilor șirului lui Fibonacci, problema 2

```

1
2 #include <iostream>
3 using namespace std;
4
5 void afisareFibonacci (int n)
6 {

```

```

7      int penultim = 0, ultim = 1;
8      int curent;
9
10     cout << "primii" << n << " termeni ai sirului sunt: ";
11     for (int i = 1; i <= n; i++)
12     {
13         // afisam primul termen
14         if (i == 1)
15         {
16             cout << penultim << " ";
17             continue;
18         }
19
20         // afisam al doilea termen
21         if (i == 2)
22         {
23             cout << ultim << " ";
24             continue;
25         }
26
27         curent = ultim + penultim;
28         cout << curent << " ";
29
30         penultim = ultim;
31         ultim = curent;
32     }
33 }
34
35 int main()
36 {
37     int numar;
38     cout << "\n Introduceti numarul de termeni din sirul lui Fibonacci pe care
39         ↪ vreti sa ii afisati ";
40     cin >> numar;
41     afisareFibonacci(numar);
42 }

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității: Numarul de operații efectuate pentru calculul termenilor sirului până la al n -ulea termen din sir este proportional cu n , ceea ce înseamnă că avem complexitate de timp $O(n)$. Complexitatea de spațiu este constantă, deoarece nu se folosesc structuri auxiliare pentru memorarea termenilor șirului, ci doar variabilele `textitpenultim`, `textitultim` și `textitcurent`.

Pentru problema 3 avem:

- Date de intrare: Un șir de numere naturale, citite rând pe rând de la tastatură.
- Date de ieșire: Validarea faptului că numerele introduse au fost sau nu în progresie aritmetică.
- Algoritm de rezolvare: Se citesc numerele unul după altul. După citirea primelor două se inițializează variabila `ultim` și se determină diferența dintre ele, care reprezintă rația progresiei aritmetice. Se continuă citirea fie până când se întâlnește valoarea -1, fie până când diferența ultimilor doi termeni este diferită de rație. La fiecare iteratie se actualizează valoarea curentă și cea de dinaintea ei.

- Algoritmul scris în C++ conține metoda care verifică faptul că secvența de numere introdusă este progresie sau nu și metoda main () care conține numai apelul acestei funcții.

Secvență de cod 2.5: Cod C++ pentru problema 3

```

1  #include <iostream>
2  using namespace std;
3
4  void verificareProgresie()
5  {
6      // marcam variabilele ultim si ratie ca neinitializate
7      int ultim = INT_MAX;
8      int ratie = INT_MAX;
9      int curent;
10
11      do
12      {
13          cout << "\n Introduceti un numar ";
14          cin >> curent;
15
16          // daca ultim este inca neinitializat inseamna ca
17          // numarul citit este primul numar
18          if (ultim == INT_MAX)
19          {
20              if (curent == -1)
21              {
22                  cout << "\n nu au fost introduse suficiente numere";
23              }
24              else
25              {
26                  ultim = curent;
27              }
28          }
29          // daca ratie este inca neinitializata inseamna ca
30          // numarul citit este al doilea numar
31          else if (ratie == INT_MAX)
32          {
33              if (curent == -1)
34              {
35                  cout << "\n nu au fost introduse suficiente numere";
36              }
37              else {
38                  ratie = curent - ultim;
39                  ultim = curent;
40              }
41          }
42          // daca se pastreaza diferenta intre termeni consecutivi
43          // se continua citirea
44          else if (curent - ultim == ratie)
45          {
46              ultim = curent;
47          }
48          // citirea se face pana cand se introduce -1
49          else if (curent == -1)
50          {
51              cout << "\n numerele introduse au fost in progresie aritmetica";
52          }
53          // altfel, opreste citirea numerelor.
54          else
55          {
56              cout << "\n numerele introduse nu au fost in progresie aritmetica";
57              cout << "\n nu are sens sa continuati citirea";
58              curent = -1;
59          }
60      } while (curent != -1);
61  }
62
63  int main()
64  {
65      verificareProgresie();
66  }

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.

- Analiza complexității: Numarul de operații este proportional cu numarul de valori citite până la încălcarea condiției ca diferența dintre oricare două numere introduse consecutiv să fie constantă, adică $O(nr_citite)$. Complexitatea de spațiu este constantă pentru ca nu reținem decât ultimii doi termeni ai sirului, și într-o altă variabilă rația progresiei aritmetice (diferența dintre primii doi termeni)

Pentru problema 4 avem:

- Date de intrare: Un număr întreg cu cel puțin trei cifre.
- Date de ieșire: Rezultatul validării faptului că numărul are sau nu forma dorită.
- Algoritm de rezolvare: Se parcurge numărul de la dreapta la stânga. Vrem să verificăm dacă, mergând de la dreapta la stânga, cifrele numărului descresc până la un punct, pentru ca apoi să crească până se ajunge la prima cifră a numărului. Pentru aceasta se consideră perechi de forma (ultima cifră, penultima cifră) care se compară. La fiecare iteratie, din număr se mai taie câte o cifră de la final.
- Algoritmul scris în C++ constă în metoda de validare, scrisă conform precizărilor anterioare și metoda main() în care se regăsește apelul metodei de validare. Codul este completat cu comentarii pentru o mai bună înțelegere a pașilor urmați.

Secvență de cod 2.6: Cod C++ pentru problema 4

```

1  #include <iostream>
2  using namespace std;
3  bool numarZambaret(long long int numar)
4  {
5      const long long int numarOriginal = numar;
6
7      // daca numarul nu are cel puțin trei cifre
8      // nu putem verifica condiția din problema
9      if (numar < 100)
10     {
11         return false;
12     }
13
14     // parcurgem cifrele de la dreapta la stanga
15     // pana cand ele nu mai sunt in ordine descrescatoare
16     bool descrescator = true;
17     while (descrescator)
18     {
19         int ultimaCifra = numar % 10;
20         int penultimaCifra = (numar / 10) % 10;
21
22         descrescator = (numar >= 10 && ultimaCifra > penultimaCifra);
23
24         if (descrescator) {
25             numar /= 10;
26         }
27     }
28
29     // daca numarul are o singura cifra ramasa inseamna ca
30     // lipste din stanga partea descrescatoare
31     // daca numarul este nealterat atunci ultimele doua cifre
32     // nu sunt in ordinea corecta

```

```

33     if (numar < 10 || numar == numarOriginal)
34     {
35         return false;
36     }
37
38     //daca numarul are mai mult de doua cifre
39     //verificam daca acestea sunt in ordine crescatoare
40     bool crescator = true;
41     while (crescator)
42     {
43         int ultimaCifra = numar % 10;
44         int penultimaCifra = (numar / 10) % 10;
45
46         crescator = (numar >= 10 && ultimaCifra < penultimaCifra);
47
48         if (crescator) {
49             numar /= 10;
50         }
51     }
52
53     // daca se iese din while cu numar mai mare sau egal decat zece
54     // conditia ca primele cifre sa fi fost in ordine descrescatoare
55     // nu a fost verificata
56     return (numar < 10);
57 }
58
59 int main()
60 {
61     long long int numar;
62     cout << "Introduceti numarul ";
63     cin >> numar;
64     if (numarZambaret(numar)) {
65         cout << "\n Numarul " << numar << " este zambaret";
66     }
67     else {
68         cout << "\n Numarul " << numar << " nu este zambaret";
69     }
70 }

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității: în cel mai rau caz, cifrele numărului se parcurg toate (de la dreapta la stânga). Aceasta înseamnă o complexitate $O(nr_cifre_x)$

Pentru problema 5 avem:

- Date de intrare: Un număr natural n .
- Date de ieșire: Un număr în baza 2.
- Algoritm de rezolvare: Se împarte numarul n , în mod repetat la 2. Restul împărțirii este folosit la construcția reprezentării în baza 2. Câțul împărțirii îi ia locul numărului n la fiecare iterație, până când acesta devine 0.
- Algoritmul scris în C++ constă în metoda de transformare în baza 2, scrisă conform precizărilor anterioare și metoda main() în care se regăsește apelul metodei de conversie.

Secvență de cod 2.7: Cod C++ pentru problema 5

```

1 #include <iostream>
2 using namespace std;
3
4 long long int convertToBinary(int number)
5 {

```

```

6      long long int binaryNumber = 0;
7      long long int power = 1;
8
9      while (number)
10     {
11         int bit = number % 2;
12         binaryNumber = bit * power + binaryNumber;
13         power *= 10;
14         number /= 2;
15     }
16     return binaryNumber;
17 }
18
19 int main()
20 {
21     int numar;
22     cout << "Introduceți numărul ";
23     cin >> numar;
24     cout << "\n Reprezentarea în baza 2 a numărului " << numar << " este: ";
25     cout << "\n " << convertToBinary(numar);
26 }

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității: Dacă numărul n dat este încadrat între două puteri consecutive ale lui 2,

$$2^{p-1} \leq n < 2^p$$

complexitatea algoritmului este $O(p)$.

2.3 Procesarea numerelor - probleme propuse

Propunem, pentru rezolvare, următoarele probleme cu numere:

1. Se citesc n numere întregi. Să se calculeze suma numerelor pare și produsul celor impare, fără a reține valorile într-un vector.
2. Se citesc de la tastatură n numere naturale. Să se verifice câte dintre acestea au exact 3 cifre nenule.
3. Să se verifice dacă un număr natural n , dat, este prim.
4. Să se afișeze toate numerele prime din intervalul $[p, q]$ cu p și q numere naturale, $p < q$.
5. Să se calculeze cel mai mare divizor comun a două numere naturale (cmdc).
6. Să se determine toate numerele întregi mai mici decât un număr n dat, care sunt egale cu suma pătratelor cifrelor lor.
7. Să se determine cel mai mare număr natural $k \geq 0$ pentru care $k! \leq n$, pentru n număr natural dat.

8. Fie n un număr natural citit de la tastatură. Scrieți un program care calculează și afișează cifra de control a lui n . Cifra de control a unui număr natural se obține calculând suma cifrelor numărului, apoi suma cifrelor sumei etc, până se obține o singură cifră.
9. Să se verifice dacă cifrele unui număr sunt în ordine crescătoare.
10. Se citește un număr întreg x . Să se afișeze numărul lui de divizori naturali.
11. Se citește un număr natural nenul n . Să se afișeze suma puterilor factorilor din descompunerea lui n în factori primi.
12. Se citește un număr natural n . Să se afișeze cel mai mare divizor comun al cifrelor lui.
13. Să se calculeze ultima cifră a numărului 2^x , pentru x număr natural, fără a calcula efectiv puterea.
Exemplu: Ultima cifră a numărului 2^{259} este 8.
14. Se citește un număr întreg n . Să se descompună în factori primi.
15. Să se citească pe rând n numere întregi și să se determine primele două minime (fără a reține valorile într-un vector).
16. Să se șteargă toate cifrele unui număr întreg n care sunt precedate de o cifră identică.
17. Să se introducă între oricare două cifre egale ale unui număr n prima cifră a numărului, numai dacă aceasta nu este egală cu cele două cifre.
18. Să se verifice dacă cifrele unui număr $x \geq 100$ sunt în progresie aritmetică.
19. Să se verifice dacă un număr $x \geq 100$ are cel puțin trei cifre alăturate consecutive (fie în ordine crescătoare, fie descrescătoare).
20. Să se afișeze cel mai mare număr care se poate forma din cifrele unui număr dat x , luate o singură dată.
21. Să se afișeze cel mai mic număr care se poate forma cu cifrele unui număr dat x , fiecare cifră repetându-se de câte ori se repetă și în x .
22. Numere prietene: Două numere a și b se numesc "prietene" dacă a este egal cu suma divizorilor lui b (fără b) și b este egal cu suma divizorilor lui a (fără a). Scrieți un program care să determine primele n perechi (a, b) de numere prietene, cu $a < b$.

Exemplu:

Capitolul 3

Tablouri unidimensionale

În capitolele anterioare am studiat diverse elemente despre algoritmi: ce este un algoritm, ce proprietăți trebuie să aibă, cum sunt descriși, ce tipuri de date putem utiliza, dar și despre eficiența acestora. După cum am observat, sunt multe probleme care se pot rezolva (doar) cu numere și prelucrare de numere. Însă, este idealist să credem că rezolvarea unor probleme mai complexe, se rezumă doar la asta. Cu siguranță, intervine problema prelucrării mai multor numere, sau problema reținerii lor pentru o prelucrare ulterioară.

Metodele de rezolvare pentru o problema dată sunt foarte diverse, însă există situații în care apare necesitatea utilizării unor structuri de date unidimensionale: mulțimi, secvențe, vectori, șiruri, stive, cozi, liste, etc. Aceste structuri de date sunt colecții de date/containere organizate după diverse reguli.

Despre aceste elemente se va discuta în acest capitol, accentul fiind pus pe implementarea lor în limbajul C++. Vom introduce succint în acest capitol câteva elemente despre Standard Template Library (STL) și vom prezenta cele trei componente cheie ale acestei biblioteci: containerele (structuri de date sub formă de template-uri), iteratori și algoritmi. Deoarece STL cuprinde foarte multe clase (reprezentând diverse containere, fiecare cu proprietăți diverse), vom prezenta doar o parte dintre acestea, însoțite de câteva exemple explicate integral.

3.1 Aspecte teoretice

Vectorii sau tablourile unidimensionale sunt structuri de date bine definite și organizate în memorie (dispuse contiguu într-un bloc de memorie). Cu ajutorul acestora, se pot păstra în memorie și accesa ulterior mai multe variabile, fără a fi nevoie de reținerea explicită a fiecăreia dintre ele. Cu alte cuvinte, un vector reține sub același nume mai multe valori de același tip. Fiecare valoare poate fi accesată folosind poziția sa în vector (pozițiile sunt numere naturale cuprinse între 0 și dimensiunea maximă a vectorului - 1). Elementele pot fi accesate individual

prin indici sau ca un tot unitar. Toate elementele au un predecesor (excepție primul) și un succesor (excepție ultimul). Tablourile unidimensionale se pot utiliza pentru a reține: șiruri (mai multe elemente omogene), mulțimi (mai multe elemente omogene, dar care nu se repetă), secvențe (mai multe elemente, dar în care contează ordinea), etc.

3.1.1 Cerințe preliminare pentru prelucrarea tablourilor unidimensionale

Considerăm că pentru a înțelege corect prelucrarea tablourilor unidimensionale, este nevoie de acoperirea totală a elementelor legate de prelucrarea numerelor.

În secțiunile următoare ne vom orienta către următoarele aspecte legate de tablouri unidimensionale:

- cum declarăm un vector;
- cum inițializăm un vector;
- cum memorăm un vector;
- cum accesăm elementele unui vector;
- cum citim un vector;
- cum afișăm un vector;
- cum efectuăm diverse prelucrări asupra elementelor unui vector.

3.1.2 Reprezentarea tablourilor unidimensionale. Alocare dinamică și alocare statică

Ne punem problema utilizării într-un program C/C++ a unui număr foarte mare de date de un anume tip, de exemplu `double`, date care trebuie prelucrate rapid și care, în consecință, trebuie să se afle în memorie în momentul rulării. Să presupunem că folosim tablouri cu dimensiuni adecvate volumului de date.

Utilizarea tablourilor este foarte simplă dar, dacă volumul de date prelucrat este cu adevărat mare, vom avea un neajuns major: nu pot fi declarate tablouri de dimensiuni variabile, deci alocarea unui tablou are un caracter static, adică este făcută la compilarea programului, prin urmare numărul de elemente trebuie desemnat printr-o constantă. Programatorul trebuie să estimeze o dimensiune maximă, acoperitoare pentru toate situațiile în care va fi folosit respectivul tablou, și să-l declare în consecință. Apare astfel o pierdere de spațiu de memorie. De exemplu, dacă declarăm tabloul `double mat[100]` și la rulare îl încărcăm cu o matrice de 10 elemente, folosim numai 10% din spațiul rezervat prin program.

Pierderea este evidentă, iar o variantă mai bună ar putea fi alocarea dinamică a memoriei, adică alocarea în timpul rulării programului.

În secțiunea imediat viitoare, vom renunța să declarăm static tablouri, în locul lor vom folosi pointeri pentru a reține adresele locațiilor de memorie obținute de la sistemul de calcul prin operații de alocare dinamică. Însă, pentru a încuraja la cât mai mult exercițiu, ceea ce ne dorim este să prezentăm ambele perspective: alocare dinamică și alocare statică.

Alocare statică

Memoria statică este cea în care variabilele sunt plasate de compilator. Toate variabilele pe care le declarăm într-un program ajung în zona statică. Pentru acest tip de alocare, memoria este alocată la compilare în segmentul de date din cadrul programului și nu se mai poate modifica în cursul execuției. Ca și neajuns major, putem preciza faptul că oricând (pe parcursul aplicației) dacă realizăm că avem nevoie de mai mult/mai puțin spațiu, nu putem să intervenim.

În această secțiune vom exemplifica toate operațiile elementare pe tablouri unidimensionale utilizând alocarea statică.

- Declararea unui tablou unidimensional

Sintaxa este:

```
tip_elemente_vector nume_vector[dimensiune_max_vector];
```

- *tip_elemente_vector*: poate să fie oricare tip primitiv din C++, orice container, sau oricare tip introdus de către utilizator (oricare clasă/structură bine definită);
- *nume_vector*: trebuie să respecte regulile unui identicator în C++¹. Pentru o mai bună înțelegere exprimăm o expresie regulată: $[a-zA-Z_][a-zA-Z0-9_]*$, adică un șir de caractere care începe cu literă sau `_`, continuă cu oricâte litere, cifre sau `_`;
- *dimensiune_max_vector*: numărul maxim de componente (o constantă întreagă); compilatorul C++ alocă un spațiu de memorie egal cu numărul maxim de elemente ale tabloului, rezervând octeți în funcție de tipul de bază al fiecărui tablou.

Exemple:

Secvență de cod 3.1: Cod C++ demo pentru declararea unui vector

```
1  int sirInt[100]; // tipul elementelor vectorului este int iar dimensiunea maxima 100
2  double sirDouble[50];
3  char sir_de_caractere[1000];
```

¹Vezi identicatori <https://en.cppreference.com/w/cpp/language/identifiers>

Observați faptul că în declarațiile de mai sus am folosit două notații diferite: `camelCase` și `snake_case`, ambele variante fiind corecte și acceptate în scrierea de cod C++.

- Inițializarea unui tablou unidimensional

Tablourile, ca și variabilele simple pot fi inițializate. Tablourile globale se inițializează prin definițiile lor la compilare, iar tablourile locale se inițializează prin declarațiile lor la execuție. Inițializarea se poate face prin introducerea datelor de la tastatură/fișier, prin atribuire a unei valori date, prin utilizarea unei constante de tip tablou.

Secvență de cod 3.2: Cod C++ pentru demo declararea unui vector

```

1 //Versiune citire de la tastatura
2 int vector[100], dimension;
3 cin >> dimension;
4 for (int index = 0; index < dimension; ++index)
5     cin >> vector[index];
6
7 //Versiune egalare cu o constanta data
8 int vector[100], dimension;
9 cin >> dimension;
10 for (int index = 0; index < dimension; ++index)
11     vector[index] = 0;
12
13 //Versiune prin utilizarea unei constante de tip tablou
14 int vector[5] = { 1, 2, 3, 4, 5 };
15 int vectorTwo[10] = { 1, 2, 3 };
16
17 /* vectorTwo[0] = 1, vectorTwo[1] = 2, vectorTwo[2] = 3;
18    celelalte componente ale tabloului vectorTwo[3], ..., vectorTwo[9] au
19    ↳ valoarea 0 daca tabloul este declarat global
20    sau o valoare initiala default (imprevizibila) daca tabloul este declarat
21    ↳ local. */

```

- Memorarea unui vector

Necesarul de memorie pentru memorarea elementelor unui tablou unidimensional este direct proporțional cu tipul și cu dimensiunea lui, numărul total de octeți fiind *`sizeof(tip_elemente_vector)*dimensiune`*. Elementele unui tablou sunt memorate într-o zonă de memorie contiguă în ordinea indicilor.

Secvență de cod 3.3: Cod C++ demo pentru memorarea unui vector

```

1 int main()
2 {
3     int vector[10];
4     for (int index = 0; index < 10; ++index)
5     {
6         vector[index] = index + 1;
7         std::cout << &vector[index] << "\t" << *(vector + index) << std::endl;
8     }
9     return 0;
10 }

```

Ieșirea pentru demo-ul de memorare este în Figura 3.1. Observați următoarele:

- `&` este operator de referențiere;
- `*` este operator de dereferențiere;

- $\&(*a)$ este același lucru pentru un pointer;
- $\&a$ pentru un pointer înseamnă că cerem ADRESA unde este "alocat"/"declarat" pointerul;
- $*a$ înseamnă că dorim să accesăm VALOAREA locației spre care indică pointerul;
- a este echivalent cu $\&a[0]$, $*a$ este echivalent cu $a[0]$;
- $a+1$ este echivalent cu $\&a[1]$, $*(a+1)$ este echivalent cu $a[1]$;
- $a+i$ este echivalent cu $\&a[i]$, $*(a+i)$ este echivalent cu $a[i]$;

006FFB58	1
006FFB5C	2
006FFB60	3
006FFB64	4
006FFB68	5
006FFB6C	6
006FFB70	7
006FFB74	8
006FFB78	9
006FFB7C	10

Figura 3.1: Output pentru memorarea unui tablou unidimensional

- Accesarea elementelor unui tablou unidimensional

Accesul la fiecare element al tabloului se face prin numele acestuia urmat între paranteze drepte, de indicele său (adică poziția pe care o ocupă în tablou). În limbajul C++, indicii tablourilor se numerotează încep de la valoarea 0 și se termină cu limita superioară - 1 (dimensiune - 1).

- Citirea elementelor unui tablou unidimensional

Orice vector este caracterizat prin dimensiunea sa (de tip întreg), plus mulțimea elementelor sale. Citirea se poate face în diverse moduri, citirea de la tastatura sau dintr-un fișier de intrare.

Secvență de cod 3.4: Cod C++ demo pentru citirea unui tablou unidimensional

```

1  void readVector(int &lenght, int vector[])
2  {
3      std::cout << "Introduceți lungimea vectorului: ";
4      std::cin >> lenght;
5      std::cout << "Introduceți elementele vectorului: ";
6      for (int i = 0; i < lenght; i++)
7          std::cin >> vector[i];
8  }
9  void readVectorFile(std::ifstream& f, int &lenght, int vector[])
10 {
11     f >> lenght;
12     for (int index = 0; index < lenght; index++)
13         f >> vector[index];
14 }
```

- Afișarea elementelor tabloului este similară cu citirea (iar pentru afișarea într-un fișier este necesar `std::ofstream`)

Secvență de cod 3.5: Cod C++ demo pentru afișarea elementelor unui tablou unidimensional

```

1      void writeVector(int lenght, int vector[])
2      {
3          std::cout << "[";
4          for (int i = 0; i < lenght - 1; i++)
5              std::cout << vector[i] << ", ";
6          std::cout << vector[i] << "]\n";
7      }
```

Alocare dinamică

În această secțiune, vom renunța să declarăm static tablouri. În locul acestui tip de alocare, vom folosi pointeri pentru a reține adresele locațiilor de memorie obținute de la sistemul de calcul prin operații de alocare dinamică.

În cazul alocării dinamice a tablourilor se garantează că elementele succesive au locații succesive și, prin urmare, parcurgerea tablourilor alocate dinamic poate fi făcută tot cu operatorul de indexare, exact ca în cazul alocării statice, utilizând identitatea dintre expresia $p[i]$ și $*(p+i)$. Zona de memorie dinamică elimină limitările de la alocarea statică, prin faptul că ne permite să mărim sau să micșorăm dimensiunea blocurilor de memorie. În schimb apare o altă limitare, și anume că nu putem declara variabile care să fie plasate în zona de memorie dinamică. Toate variabilele pe care le declarăm în program vor ajunge în zona statică. Pentru a obține acces la memoria dinamică trebuie să folosim anumite funcții standard.

În această secțiune dorim doar să oferim câteva informații de bază legate de alocarea dinamică, acest tip de alocare al memoriei nefiind un scop în sine al acestei secțiuni. De aceea, vom comenta succint sintaxa și vom exprima câteva exemple ca și argumentare.

În continuare, vom exemplifica toate operațiile pe tablouri unidimensionale utilizând alocarea dinamică.

- Declararea unui tablou unidimensional

Diferența esențială dintre pointeri și tablouri:

- `double* variable1` –> declară o variabilă pointer de tip `double*`, variabilă alocată pe 8 octeți (bytes);
- `double variable2[10]` –> declară un tablou static al cărui nume, *variable2*, are două semnificații: prima, `variable2` este identificatorul unei date de tip `double[10]` care ocupă 80 de octeți de memorie, și, a doua, `variable2` este o constantă de tip `double*`, cu valoarea egală cu adresa primului element al tabloului.

Secvență de cod 3.6: Cod C++ demo pentru declararea unui tablou

```

1      double variable2[10];
2      // prima semnificatie este folosita in instructiunea
3      std::cout << sizeof(variable2) << std::endl; //80
4      //iar a doua
5      std::cout << variable2 << std::endl; //00CFFC7C
6      //care este echivalent\u a cu
7      std::cout << &variable2[0] << std::endl; //00CFFC7C

```

Atribuirea $variable1 = variable2$ este corectă, adresa primului element al tabloului $variable2$ este scrisă în cei 8 octeți de memorie alocați lui $variable1$, în timp ce atribuirea $variable2 = variable1$ nu este permisă: în locația de 80 de octeți ai lui $variable2$ pot fi scrise numai valorile celor 10 elemente ale sale, și nu o adresă.

- Alocarea dinamică/Eliberare memoriei unui tablou unidimensional/Accesarea elementelor unui tablou unidimensional

În limbajul C alocarea dinamică a memoriei poate fi făcută numai cu funcții din biblioteci specializate în gestionarea memoriei (așa cum am precizat și anterior), cele mai utilizate fiind funcțiile `malloc()`, `calloc()`, `free()`, `realloc()`, toate din `<malloc.h>`.

În C++ au fost introduși, special pentru alocarea dinamică, operatorii `new` și `delete`, care fac parte integrantă din limbaj (nu necesită apelarea vreunei biblioteci). Operatorul `new` poate fi utilizat în două moduri: pentru a rezerva spațiu de memorie pentru un singur obiect, caz în care `new` întoarce adresa obiectului alocat, sau pentru a rezerva spațiu pentru un tablou cu un număr variabil de obiecte (dar bine determinat în momentul evaluării expresiei, la rulare), caz în care `new` întoarce adresa primului element din tablou. Un tablou alocat astfel este numit tablou dinamic.

Secvență de cod 3.7: Cod C++ demo pentru alocarea si dealocarea unui tablou

```

1  #include <iostream>
2  int* allocate(int dimension) {
3      int* pointer;
4      pointer = new int[dimension];
5      for (int index = 0; index < dimension; ++index) {
6          pointer[index] = 100 * (index + 1);
7      }
8      return pointer;
9  }
10 void deAllocate(int* pointer) {
11     delete[] pointer;
12 }
13 int main() {
14     int dimension = 5;
15     int* vector;
16     vector = allocate(dimension);
17     for (int index = 0; index < dimension; index++)
18         std::cout << vector[index] << std::endl;
19     deAllocate(vector);
20     for (int index = 0; index < dimension; ++index)
21         std::cout << vector[index] << std::endl;
22     return 0;
23 }

```

Trebuie observat faptul că eliberarea, în cursul rulării programului, a locațiilor de memorie alocate dinamic și care nu mai sunt folosite în procesul de calcul este în totalitate în sarcina programatorului. Ca regulă generală, spațiul alocat cu `new` în interiorul unei funcții și care este utilizat numai în corpul acelei funcții, fără ca adresa sa să fie transmisă în vreun fel funcției apelante, trebuie dealocat cu `delete` înainte încheierii funcției, altfel la terminarea apelului adresa acestui spațiu se pierde definitiv iar memoria rămâne ocupată inutil până la finalul execuției programului.

De asemenea, pentru un lucru corect cu pointerii, este necesară cunoașterea unor elemente specifice despre aritmetica pointerilor ².

Citirea și afișarea componentelor unui tablou alocat dinamic se face absolut asemănător cu varianta statică. În secțiunea următoare este prezentată o a treia variantă mai modernă de a lucra cu tablouri/șiruri de elemente, folosind containere generice.

3.1.3 Reprezentarea tablourilor unidimensionale în Standard Template Library (C++)

Structurile de date sunt colecții de date sau containeri organizate după diverse reguli. În programarea orientată pe obiecte, structurile de date sunt obiecte care conțin colecții de obiecte de un anumit tip considerat.

Presupunând că dorim implementarea unei colecții de tip *MyArray* care ar reprezenta un tablou de obiecte de tip `int`, această clasă (sau structură) ar conține două atribute:

1. dimensiunea array-ului dorit
2. mulțimea/secvența de elemente de tip `int` (reținută static sau dinamic)

Folosind template-uri, am putea extinde acest tip de dată la *MyArray*<*T*> astfel încât să putem implementa *MyArray*<*char*>, *MyArray*<*int*>, *MyArray*<*double*>, *MyArray*<*Point*> sau, în general, tablouri de orice tip de dată.

În mod similar putem proceda pentru implementarea structurilor de date de tip stivă, coadă sau pentru alte tipuri de structuri de date. STL este o bibliotecă de clase template [1] care conține, între altele, și implementări ale structurilor de date. În C și C++ obișnuim să accesăm elementele unui tablou folosind pointeri.

În C++ STL, accesăm elementele containerilor prin obiecte iterator care arată ca pointerii, dar se comportă mai inteligent. Clasele iterator sunt proiectate să poată fi folosite generic pentru orice container. Containerii implementează operații primitive, iar algoritmii STL sunt implementați independent de containeri.

²<https://www.cs.swarthmore.edu/~richardw/classes/cs31/s18/offsite/pointer.html>

Containerii STL sunt unul dintre cele mai bune exemple de reutilizare a codului și de folosire a claselor template. Implementările structurilor de date care fac legăturile între obiecte prin pointeri pot conduce la probleme serioase legate de gestiunea dinamică a memoriei, erori care nu pot fi detectate la compilare și care sunt uneori foarte dificil de rezolvat. Unul dintre marile avantaje ale containerilor STL este că rezolvă aceste neajunsuri. STL evită folosirea moștenirii și a funcțiilor virtuale din considerente de performanță. De asemenea, STL evită folosirea operatorilor `new` și `delete` în favoarea alocatorilor pentru a permite o mai mare varietate de metode de control al alocării și dealocării memoriei. În cazul în care avem curiozitate de a vedea ce se află (din punct de vedere structuri de date) în STL³, vom avea de interpretat Figura 3.2.

În secțiunile imediat viitoare vom utiliza câteva dintre aceste structuri de date, de aceea credem ca este necesar studierea (măcar a) structurilor de tip vector și stack (LIFO stivă).

Secvență de cod 3.8: Cod C++ pentru demo vector (STL)

```

1  #include <iostream>
2  #include <vector>
3
4  void printVersion1(const std::vector<int>& toPrint)
5  {
6      for (const int& elem: toPrint)
7      {
8          std::cout << elem << " ";
9      }
10 }
11
12 void printVersion2(const std::vector<int>& toPrint)
13 {
14     for (auto it = toPrint.begin(); it < toPrint.end(); ++it)
15     {
16         std::cout << *it << " ";
17     }
18 }
19
20
21 int main()
22 {
23     {
24         std::vector<int> vec;
25
26         vec.push_back(5);
27         vec.push_back(10);
28         vec.push_back(15);
29
30         //printVersion3
31         std::cout << "Elements: ";
32         for (int index = 0; index < vec.size(); ++index)
33             std::cout << vec.at(index) << " ";
34         std::cout << std::endl;
35
36         std::cout << "First element: " << vec.front() << std::endl;
37         std::cout << "Last element: " << vec.back() << std::endl;
38
39         vec.pop_back();
40
41         std::cout << "Elements: ";
42         for (int index = 0; index < vec.size(); ++index)
43             std::cout << vec.at(index) << " ";
44         std::cout << std::endl;
45
46         for (int index = 0; index < vec.size(); ++index)
47             vec[index] = 0;

```

³<http://www.cplusplus.com/reference/stl/>

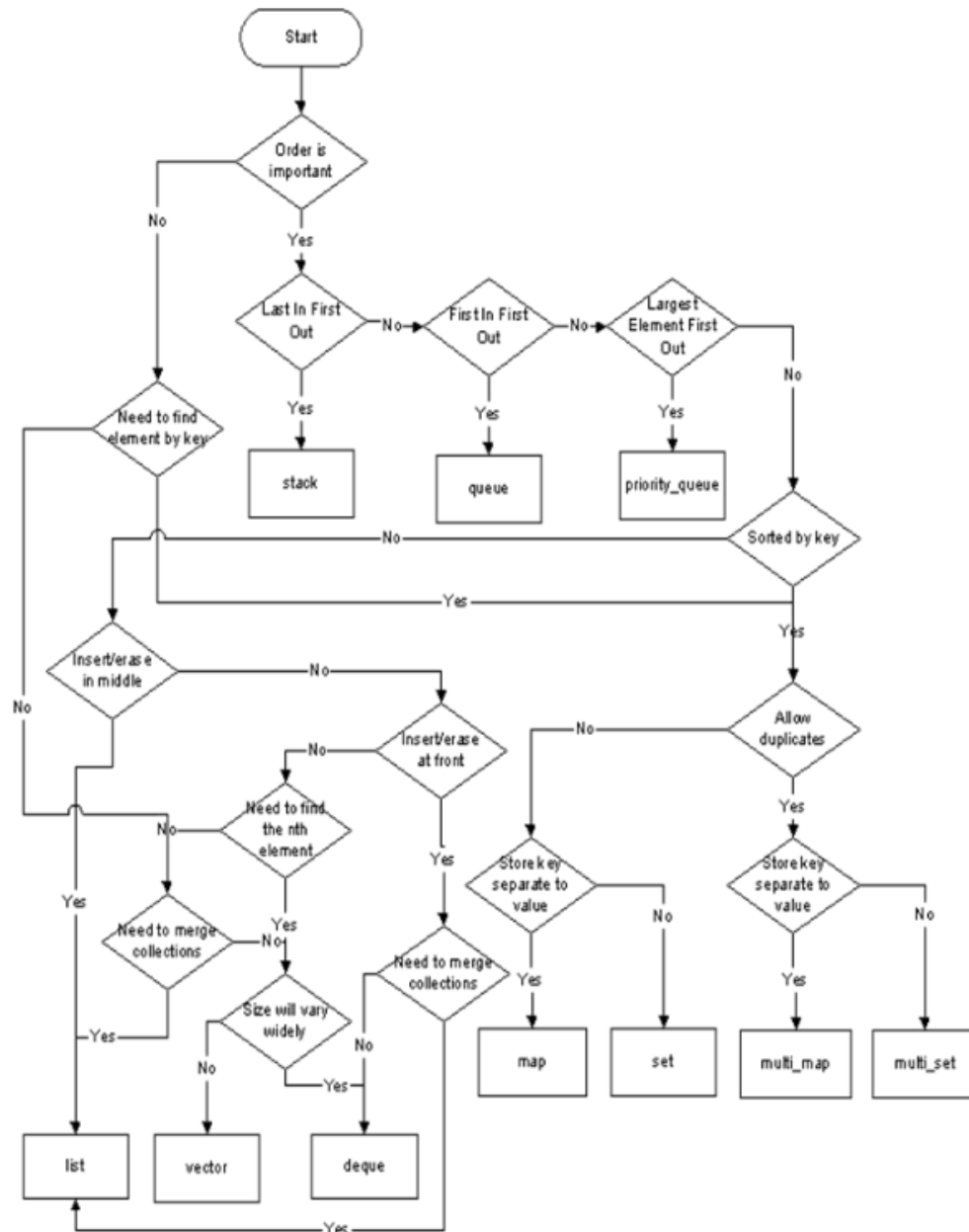


Figura 3.2: Recomandări pentru alegerea container-ului potrivit (preluată "C++ Container choice")

```

48         std::cout << "Elements: ";
49         printVersion1(vec);
50         std::cout << std::endl;
51
52         vec.clear();
53
54         std::cout << "Size: " << vec.size() << std::endl;
55     }
56
57     {
58         std::vector<int> vec(5);
59         vec[0] = 1;
60
61         std::cout << "Size: " << vec.size() << std::endl;
62         std::cout << "Capacity: " << vec.capacity() << std::endl;
63
64         vec.reserve(50);
65         std::cout << "Size: " << vec.size() << std::endl;
66         std::cout << "Capacity: " << vec.capacity() << std::endl;
67     }
68
69     {
70
71         std::vector<int> vec{ 5, 10, 15 };
72
73         vec.erase(vec.begin() + 1);
74
75         std::cout << "Elements: ";
76         printVersion2(vec);
77         std::cout << std::endl;
78
79         vec.insert(vec.begin() + 1, 10);
80
81         vec.erase(vec.end() - 2);
82
83         std::cout << "Elements: ";
84         for (int index = 0; index < vec.size(); ++index)
85             std::cout << vec.at(index) << " ";
86         std::cout << std::endl;
87
88         std::cout << "First element: " << *vec.begin() << std::endl;
89         std::cout << "Last element: " << *(vec.end() - 1) << std::endl;
90
91         vec.resize(1);
92
93         std::cout << "Size: " << vec.size() << std::endl;
94         std::cout << "Elements: ";
95         for (int index = 0; index < vec.size(); ++index)
96             std::cout << vec.at(index) << " ";
97         std::cout << std::endl;
98     }
99
100     return 0;
101 }
102

```

Secvență de cod 3.9: Cod C++ pentru demo stack (STL)

```

1  #include<iostream>
2  #include<stack>
3
4  int main()
5  {
6      std::stack<int> one, two;
7
8      one.push(1);
9      one.push(23);
10     one.push(42);
11     one.push(13);
12
13     two.push(10);
14     two.push(5);
15     two.push(15);
16     two.push(25);
17
18     one.swap(two);
19
20     for (; !one.empty(); one.pop())
21     {

```

```

22         std::cout << "Size= " << one.size() << std::endl;
23         std::cout << "Top " << one.top() << std::endl << std::endl;
24     }
25     std::cout << std::endl;
26     return 0;
27 }
28

```

După cum am menționat și anterior, aceste containere sunt diverse, iar alegerea potrivită a lor vine din identificarea corectă a problemelor, dar și din cunoașterea necesității care există. Este util să știm care e implementarea fiecărui container (chiar și sumar) pentru a alege secvența potrivită în funcție de operațiile pe care le avem de făcut, dar și în funcție de frecvența lor. De exemplu:

- *vector* are cel mai rapid acces indexat; pe de altă parte adăugarea unui element poate duce la copierea întregului conținut dacă nu mai există memorie rezervată la coadă (adica `v.capacity() == v.size()`).
- *List* este rapidă pentru inserări de elemente în interior, dar este lentă pentru accesul la acestea.
- *Deque* este mai eficientă decât *vector* dacă se fac multe operații *push_back* dar este ceva mai lentă pentru accesări indexate uzuale.

În plus, mai există și o diferență de interfață. Atât *deque*, cât și *list* oferă față de *vector* operațiile *push_front* și *pop_front*, deoarece în cazul vectorilor, ele ar fi avut complexitatea $O(\text{dimension})$ dacă se dorea păstrarea rapidității accesului indexat⁴⁵⁶.

Informații generale de observat:

- Toate containerele oferă funcționalități (metode) pentru:
 - accesare elemente (ex. `[]`);
 - gestiune capacitate (ex. `size()`);
 - modificare elemente (ex. `insert`, `clear`);
 - iterator (`begin()`, `end()`);
 - alte operații (ex. `find`, `sort`, etc).
- Decizia în alegerea containerului potrivit pentru o problemă concretă se bazează pe:
 - funcționalitățile oferite de container;

⁴<https://alyssaq.github.io/stl-complexities/>

⁵<http://john-ahlgren.blogspot.com/2013/10/stl-container-performance.html>

⁶<https://www.topcoder.com/community/competitive-programming/tutorials/power-up-c-with-the-standard-template-library-part-1/>

- eficiența operațiilor (complexitate).
- Container (clase template) de forma:
 - Container de tip secvență (Sequence containers): `vector<T>`, `deque<T>`, `list<T>`;
 - Adaptor de containere (Container adaptors): `stack<T, ContainerT>`, `queue<T, ContainerT>`, `priority_queue<T, ContainerT, CompareT>`;
 - Container asociativ (Associative containers): `set<T, CompareT>`, `multiset<T, CompareT>`, `map<KeyT, ValueT, CompareT>`, `multimap<KeyT, ValueT, CompareT>`, `bitset<T>`.
- Dintre cele mai utilizate containere precizăm:
 - Vector (Dynamic Array): elementele sunt stocate secvențial în zone continue de memorie, are performanțe bune la: accesare elemente individuale de pe o poziție dată (timp constant), iterare elemente în orice ordine (timp liniar), adăugare/ștergere elemente de la sfârșit (timp amortizat constant);
 - Deque (double ended queue)/Coadă dublă (completă): elementele sunt stocate în blocuri de memorie (chunks of storage), elementele se pot adăuga/șterge eficient de la ambele capete;
 - List: implementat ca și listă dublă înlănțuită, are performanțe bune la: ștergere/adăugare de elemente pe orice poziție (timp constant), mutarea de elemente sau secvențe de elemente în liste sau chiar și între liste diferite (timp constant), iterare de elemente în ordine (timp liniar);
 - Vector vs Deque: accesul la elemente de pe orice poziție este mai eficient la vector, inserare/ștergerea elementelor de pe orice poziție este mai eficient la Deque (dar nu e timp constant), pentru liste mari Vector alocă zone mari de memorie, Deque alocă multe zone mai mici de memorie - Deque este mai eficient în gestiunea memoriei.

3.2 Tablouri unidimensionale - probleme rezolvate

În secțiunea care urmează vom utiliza câteva dintre metodele realizate în capitolul anterior, și anume, vom presupune că pentru prelucrarea tablourilor unidimensionale avem următoarea interfață predefinită:

Secvență de cod 3.10: Cod C++ Intefată de lucru cu tablouri unidimensionale

```

1      void readVector(int&, int []);
2      void readVectorFile(std::ifstream&, int&, int []);
3      void writeVector(const int&, int []);
4      int* allocate(const int&);
5      void deAllocate(int*);
6      void printVersion1(const std::vector<int>&);
7      void printVersion2(const std::vector<int>&);

```

La această interfață vom adauga diferite alte operații considerate esențiale pentru lucrul cu vectorii.

Problemele pe care le propunem în această secțiune sunt:

1. Se dă un tablou unidimensional de numere întregi, de dimensiune dată. Să se testeze dacă vectorul este sortat crescător.
2. Se dă un tablou unidimensional de numere întregi, de dimensiune dată. Să se elimine elementele din vector care au ultima cifra egală cu 0.
3. Se dă un tablou unidimensional de numere întregi, de dimensiune dată. Să se afișeze pentru fiecare element din vector cifra maximă.
4. Se dă un tablou unidimensional de numere întregi, de dimensiune dată. Să se determine primele două maxime dintr-un vector *dintr-o singură parcurgere*.
5. Se dau două tablouri unidimensionale de numere întregi, de dimensiune dată (aceeași). Să se verifice dacă cei doi vectori sunt diferiți (nici un element din primul vector nu se regăsește în al doilea vector).

Așa cum am amintit și în capitolul anterior, în rezolvarea problemelor vom parcurge următoarele etape:

- Analiza problemei în scopul stabilirii *datelor de intrare*, precum și a rezultatelor pe care trebuie să le obținem prin rezolvarea problemei (*date de ieșire*);
- Elaborarea unui algoritm de rezolvare a problemei (prin crearea, dacă este nevoie, a unor subalgoritmi);
- Implementarea algoritmului în limbajul C++;
- Verificarea corectitudinii algoritmului propus;

- Analiza complexității algoritmului (vezi capitol de "Analiza eficienței algoritmilor. Complexități").

Pentru problema 1 avem:

- Date de intrare: vectorul de numere întregi + dimensiunea acestuia;
- Date de ieșire: un mesaj corespunzător cerinței date ("DA" pentru respectarea condiției impuse, "NU" pentru nerespectarea condiției);
- Algoritm de rezolvare: pentru rezolvarea acestei probleme, prelucram elementele vectorului, astfel încât, dacă găsim un element de pe o poziție "index" mai mare decât un element de pe oricare poziție de tipul "index + ceva", atunci vectorul NU respectă condiția. În caz contrar, condiția este respectată.
- Algoritmul scris în C++ implică utilizarea mai multor funcții, printre care: una pentru citirea a elementelor vectorului (oricare din interfața precizată la începutul acestei secțiuni), una pentru afișarea elementelor unui vector, la care adăugă o funcție care face testarea cerută.

Secvență de cod 3.11: Cod C++ Metoda pentru verificarea condiției impuse de problema 1

```

1  /*
2  verificare a condiției ca un vector este sortat crescator
3  */
4  bool verificareVectorCrescator(int vector[], int nr_elem)
5  {
6      for (int index = 0; index < nr_elem - 1; index++)
7          if (vector[index] > vector[index + 1])
8              return false;
9      return true;
10 }
```

Secvență de cod 3.12: Cod C++ Continut pentru main(), problema 1

```

1  citire(vector, nr_elem);
2  afisareVector(vector, nr_elem);
3  if (verificareVectorCrescator(vector, nr_elem))
4      std::cout << "DA" << std::endl;
5  else
6      std::cout << "NU" << std::endl;
```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou unidimensional, adică $O(nr_elem)$, iar din punct de vedere al timpului de execuție, se parcurg toate elementele vectorului, până când se întâlnește condiția impusă. Astfel, în cazul cel mai favorabil, cazul în care deja primele două elemente NU respectă condiția, atunci algoritmul face o singură comparație, adică, $\Theta(1)$, însă pentru cazul cel mai defavorabil, se obține complexitate $\Theta(nr_elem)$, așadar medie a algoritmului este $O(nr_elem)$.

Pentru problema 2 avem:

- Date de intrare: vectorul de numere întregi + dimensiunea acestuia;
- Date de ieșire: un vector actualizat, din care s-au scos elementele cerute;
- Algoritm de rezolvare: căutăm toate elementele cu ultima cifră 0 și le eliminăm, mutând elementele mai la stânga cu o poziție;
- Algoritmul scris în C++ implică utilizarea mai multor funcții, dar punem accent pe eliminarea de elemente dintr-un vector, *eliminareElementVector*, care primește ca și parametrii vectorul, numărul de elemente, dar și poziția pe care dorim să o scoatem, iar apoi, plecând de la indexul egal cu poziția transmisă, shiftăm toate elementele la stânga, decrementând numărul de elemente al vectorului dat. O altă metodă utilizată este *eliminareElementeCuUltimaCifra0* care, plecând de la vectorul citit, se impune condiția ca ultima cifră să fie 0, după care se apelează metoda de eliminare, conform poziției determinate.

Secvență de cod 3.13: Cod C++ Cele doua metode pentru utilizate pentru problema 2

```

1  /*
2   * eliminare a elementului de pe pozitie din vector
3   */
4  void eliminareElementVector(int vector[], int& nr_elem, int pozitie)
5  {
6      for (int index = pozitie; index < nr_elem - 1; index++)
7          vector[index] = vector[index + 1];
8      nr_elem--;
9  }
10
11 /*
12 * eliminare a elementelor cu ultima cifra 0 din vector
13 */
14 void eliminareElementeCuUltimaCifra0(int vector[], int& nr_elem)
15 {
16     for (int index = 0; index < nr_elem; index++)
17         if (vector[index] % 10 == 0)
18             eliminareElementVector(vector, nr_elem, index);
19 }
```

Secvență de cod 3.14: Cod C++ Continut pentru main(), problema 2

```

1  citire(vector, nr_elem);
2  std::cout << "Afisare elemente vector inainte: " << std::endl;
3  afisareVector(vector, nr_elem);
4  eliminareElementeCuUltimaCifra0(vector, nr_elem);
5  std::cout << "Afisare elemente vector dupa modificari: " << std::endl;
6  afisareVector(vector, nr_elem);
```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității: Din punct de vedere al spațiului de memorie, se utilizează un tablou unidimensional, adică $O(nr_elem)$, iar din punct de vedere al timpului de execuție, se parcurg elementele, se caută elementele dorite (căutare executată în $O(nr_elem)$ pentru situația în care șirul NU

este sortat), iar apoi se șterg elementele dorite (ștergere care consumă tot $O(nr_elem)$, pentru că implică shiftarea tuturor elementelor).

Pentru problema 3 vom insista asupra a două variante de rezolvare, și anume:

Secvență de cod 3.15: Cod C++ pentru problema 3, Variantă 1

```

1  /*
2  pentru fiecare element din vector, se afiseaza cifra maxima din el
3  */
4
5  void afisareCifraMaximaVersion1(int vector[], int nr_elem)
6  {
7      int maxim;
8      for (int index = 0; index < nr_elem; index++)
9      {
10         maxim = 0;
11         while (vector[index] != 0)
12         {
13             if (maxim < vector[index] % 10)
14                 maxim = vector[index] % 10;
15             vector[index] /= 10;
16         }
17         std::cout << "Cifra maxima a elementului de pe pozitia " << index << " este: " << maxim << std::endl;
18     }
19 }
20 
```

Secvență de cod 3.16: Cod C++ pentru problema 3, Variantă 2

```

1  int getMaximumDigit(int number)
2  {
3      int maximum = 0;
4      while (number)
5      {
6          if (number % 10 > maximum)
7              maximum = number % 10;
8          number /= 10;
9      }
10
11     return maximum;
12 }
13
14 /*
15 pentru fiecare element din vector, se afiseaza cifra maxima din el
16 */
17
18 void afisareCifraMaximaVersion2(int vector[], int nr_elem)
19 {
20     for (int index = 0; index < nr_elem; index++)
21     {
22         std::cout << "Cifra maxima a elementului de pe pozitia " << index << " este: "
23             << getMaximumDigit(vector[index]) << std::endl;
24     }
25 }

```

Câteva observații:

- datele de intrare pentru ambele variante sunt: vectorul de numere întregi + dimensiunea acestuia;
- datele de ieșire: cifra maximă asociată fiecăruia dintre numerele din vector;
- Varianta 1: calculăm cifra maximă pentru fiecare element și apoi o afișăm - totul într-o singură funcție;

- Varianta 2: se scrie întâi o funcție care determină cifra maximă a unui număr (metodă care se poate reutiliza în oricare alt context), apoi se folosește această metodă în contextul cerut de problema noastră;
- Varianta 2 este mai generică decât varianta 1, pentru că permite reutilizarea codului pentru cifra maximă;
- ambele variante expuse au complexitate $O(nr_elem)$.

Pentru problema 4 oferim o rezolvare eficientă, adică vom rezolva cerința printr-o singură parcurgere a vectorului dat. Astfel, rezolvarea constă în a parcurge elementele vectorului dat și, prin comparații succesive, salvăm valoarea primului maxim, apoi, prin comparația acestuia cu un *secondMax*, putem afla și al doilea maxim din șir. Varianta clasică de rezolvare este ca aflarea acestor două maxime să se facă prin pdublă parcurgere a vectorului dat. Complexitatea variantei expuse este $O(nr_elem)$.

Secvență de cod 3.17: Cod C++ pentru problema 4

```

1  /*
2  afisare dintr-o singura parcurgere a primelor 2 elemente cele mai mari din vector
3  */
4
5  void printPrimeleDouaMaxime(int vector[50], int dimension)
6  {
7      int firstMax = 0, secondMax = 0;
8      for (int index = 0; index < dimension; index++)
9      {
10         if (vector[index] > firstMax)
11         {
12             secondMax = firstMax;
13             firstMax = vector[index];
14         }
15         else if (vector[index] > secondMax)
16             secondMax = vector[index];
17     }
18
19     std::cout << "Primul maxim este " << firstMax << std::endl;
20     std::cout << "Al doilea maxim este " << secondMax << std::endl;
21 }
```

Pentru problema 5 oferim o rezolvare care conține o metodă pentru a testa dacă doi vectori sunt sau nu diferiți, o metoda cu tip returnat *true* sau *false*, deci tip *bool*.

Secvență de cod 3.18: Cod C++ pentru problema 5

```

1  /*
2  verificare daca doi vectori sunt diferiti
3  */
4
5  bool vectoriDiferiti(int firstVector[50], int firstDimension, int secondVector[50], int
6      ↪ secondDimension)
7  {
8      for (int indexV1 = 0; indexV1 < firstDimension; ++indexV1)
9      {
10         for (int indexV2 = 0; indexV2 < secondDimension; indexV2++)
11             if (firstVector[indexV1] == secondVector[indexV2])
12                 return false;
13     }
14     return true;
15 }
```

Pentru toate problemele rezolvate mai sus, [1](#), [2](#), [3](#), [4](#), [5](#), am respectat câteva reguli de scriere de cod, de aceea, oferim mai jos câteva sfaturi practice de scriere de cod curat (**Clean Code tips**):

- Keep it simple, stupid (KISS)
- Don't repeat yourself (DRY)
- Be precise
- Only one responsibility
- Describe the intention
- Use explanatory names
- Avoid abbreviation
- Use names from the domain language
- Delete commented & unused code
- Keep functions/classes short
- Avoid magic numbers
- Simplify complex conditional expressions

De observat faptul că, comentariile și formatul codului pot face codul mai ușor de citit și de înțeles. De asemenea, o denumire corectă (vis a vis de contextul în care se folosește variabila respectivă), poate să ofere o claritate maximă a codului pe care îl scriem. Nu în ultimul rând, scrierea de cod reutilizabil/generic este o obligație pentru orice programator, nu o opțiune.

Pentru a exersa suplimentar lucrul cu vectorii, propunem încă **2 probleme** cu un specific diferit decât cele anterior date, pentru care oferim câteva explicații și cod C++ aferent:

- Se citesc dintr-un fișier valori reale ce reprezintă viteza unei mașini la fiecare secundă (în m/s).

Observați că între secundele x și $x+1$ viteza rămâne constantă și are valoarea de la secunda $x + 1$. Un exemplu în Figura [3.3](#).

- Să se calculeze distanța parcursă de mașină între secunde consecutive (de la 0 la 1, de la 1 la 2, etc);
- Să se calculeze distanța totală parcursă;
- Să se calculeze schimbarea vitezei ($\dot{v}$) la fiecare secundă;

Viteza	0	0	1	2	4	5	5	5	3	4	3	2	1	1	0	0	0
Distanța	0	0	1	3	7	12	17	22	25	29	32	34	35	36	36	36	36
Schimbarea vitezei Δv	0	0	1	1	2	1	0	0	-2	1	-1	-1	0	-1	0	0	0

Figura 3.3: Exemplu pentru problema 3.2

- Determinați Δv maxim a mașinii;
- Determinați Δv minim a mașinii.

Am ales ca această rezolvare să o oferim în două variante, una cu array, iar una folosind STL.

Secvență de cod 3.19: Cod C++ pentru problema 3.2, Varianta 1 - cu array

```

1  #include <fstream>
2  #include <iostream>
3
4  void subCars() {
5      std::ifstream fin("cars.in");
6      double vCurenta = 0, vAnterioara = 0, distParcursa[60], distTot = 0;
7      double deltaMin = 999999, deltaMax = -999999;
8      distParcursa[0] = 0;
9      unsigned int sec = 1;
10     while (!fin.eof()) {
11         std::cout << vCurenta << std::endl;
12         vAnterioara = vCurenta;
13         fin >> vCurenta;
14         distParcursa[sec] = distParcursa[sec - 1] + vCurenta;
15         distTot += vCurenta;
16         sec++;
17         if (vAnterioara - vCurenta > deltaMax)
18             deltaMax = vAnterioara - vCurenta;
19         if (vAnterioara - vCurenta < deltaMin)
20             deltaMin = vAnterioara - vCurenta;
21     }
22     std::cout << std::endl;
23     for (int i = 0; i < sec; i++)
24         std::cout << distParcursa[i] << std::endl;
25     std::cout << std::endl;
26     std::cout << "Distanța totală parcursă este de: " << distTot << " metri,
        ↳ diferența minimă dintre viteze este: " << deltaMin << " și
        ↳ diferența maximă este: " << deltaMax << std::endl;
27     fin.close();
28 }
29
30 int main() {
31     subCars();
32     return 0;
33 }
```

Pentru această rezolvare, vă invităm să realizați următoarele modificări:

- alocări dinamice în locul celor statice;
- folosirea unor constante predefinite, în loc de inițializările de forma $\text{deltaMin} = 999999, \text{deltaMax} = -999999$ ⁷;
- fiecare dintre cerințele problemei să se modeleze în metode separate/unitare/reutilizabile;

⁷http://www.cplusplus.com/reference/limits/numeric_limits/

- afișarea rezultatelor să se facă în consolă, dar tabelar (vezi exemplul oferit) folosind manipulatori specifici⁸.

Dorim să rezolvăm o parte din neajunsurile rezolvării cu array, de aceea, vă propunem o variantă de rezolvare a problemei 3.2 cu STL.

Secvență de cod 3.20: Cod C++ pentru problema 3.2, Varianta 2 - cu STL

```

1  #include <iostream>
2  #include <vector>
3  #include <fstream>
4  #include <algorithm>
5  #include <numeric>
6
7  void printDistanceAfterXSeconds(std::vector<double>& speedValues, int
    ↳ numberOfSeconds)
8  {
9      std::vector<double> distances(numberOfSeconds);
10     std::partial_sum(speedValues.begin(), speedValues.begin() + numberOfSeconds,
    ↳ distances.begin());
11     std::cout << "Distance after " << numberOfSeconds << " seconds: " <<
    ↳ distances[numberOfSeconds - 1] << "\n";
12 }
13
14 void printMinMaxAcceleration(std::vector<double>& speedValues)
15 {
16     std::vector<double> accelerationValues(speedValues.size());
17
18     std::adjacent_difference(speedValues.begin(), speedValues.end(),
    ↳ accelerationValues.begin());
19
20     double maxAcceleration = (*std::max_element(accelerationValues.begin(),
    ↳ accelerationValues.end()));
21     double minAcceleration = (*std::min_element(accelerationValues.begin(),
    ↳ accelerationValues.end()));
22     std::cout << "Maximum acceleration " << maxAcceleration << "\n";
23     std::cout << "Minimum acceleration " << minAcceleration << "\n";
24 }
25
26 int main()
27 {
28     std::vector<double> speedValues;
29
30     std::ifstream file("values.txt");
31     double currentValue;
32
33     while (!file.eof())
34     {
35         file >> currentValue;
36         speedValues.push_back(currentValue);
37     }
38
39     printDistanceAfterXSeconds(speedValues, 5);
40
41     printMinMaxAcceleration(speedValues);
42
43     file.close();
44     return 0;
45 }

```

Pentru această rezolvare, vă invităm să rezolvați câteva provocări:

- Ce se folosește din STL în rezolvarea de mai sus și ce proprietăți cunoașteți despre containerul utilizat?
- Se poate utiliza și alt container? Care? Se modifică rezolvarea?
- Ce se folosește în rezolvare din biblioteca "algorithm"? Dar din "numeric"?

⁸<http://www.cplusplus.com/reference/library/manipulators/>

- Ce etape de optimizare (la nivelul codului) puteți realiza?
- Să presupunem ca ați avut un exercițiu complicat la matematică, iar rezultatul final este o ecuație pe mai multe rânduri care conține paranteze rotunde, paranteze pătrate și acolade. Nu are nimeni timp să verifice dacă parantezele sunt în ordinea bună sau dacă sunt la fel de multe deschise cât și închise. Implementați o aplicație care verifică această situație pentru noi.

Exemplu de fișiere de intrare: Fișiere de test:

```
1-2+[3-2]*4/(2*3)-[(1-5
)+2]
```

aftersch00ldgjk3534and23a[ding956(fridayNIGht(dfh)6)e]dog-nelu[tea(rugat)iulia(sa)[idai(si)ei(osuta)dojda]mi(sa)sia]tigari]si(maine)tida

Soluția pentru această problemă constă în a testa diverse situații de caractere pe care le putem avea în input. Pentru acesta, am scris câteva metode pentru fiecare situație pe care doream să o acoperim:

- dacă un caracter este sau nu paranteză deschisă (de orice tip: rotundă, dreaptă, acoladă)
- dacă un caracter este sau nu paranteză închisă (de orice tip)
- verificare dacă un caracter este sau nu un tip de paranteză
- verificare dacă avem pereche de (paranteză închisă, paranteză deschisă)

Pentru a testa toate aceste metode, vom utiliza un container de tip stivă de caractere, care, în funcție de situația în care ne aflăm, se populează cu diverse caractere (la fiecare evaluare a input-ului, dacă avem o paranteză deschisă, atunci punem acel caracter în stivă, prin metoda `push()`, iar în caz contrar efectuăm `pop()`). La final, verificăm dacă stiva rămâne sau nu vidă, cu o metodă specifică containerului.

Secvență de cod 3.21: Cod C++ pentru problema 3.2

```
1 #include <iostream>
2 #include <stack>
3 #include <fstream>
4
5 //verify if a character is a opened parenthesis
6 bool isOpenParanthesis(char c)
7 {
8     if (c == '(' || c == '[' || c == '{')
9         return true;
10    return false;
11 }
12
13 //verify if a character is a closed parenthesis
14 bool isCloseParanthesis(char c)
15 {
16     if (c == ')' || c == ']' || c == '}')
```

```

17         return true;
18     return false;
19 }
20
21 //verify if a character is a paranthesis
22 bool isAParanthesis(char c)
23 {
24     if (isOpenParanthesis(c) || isCloseParanthesis(c))
25         return true;
26     return false;
27 }
28
29 //verify if two paranthesis are oposite
30 bool areOpositeParanthesis(char open, char close)
31 {
32     if (open == '{' && close == '}')
33         return true;
34     if (open == '[' && close == ']')
35         return true;
36     if (open == '(' && close == ')')
37         return true;
38     return false;
39 }
40
41 int main()
42 {
43     std::ifstream file("Exercise.txt");
44     std::stack<char> paranthesisStack;
45
46     char currentChar;
47
48     //break it if something went wrong
49     bool canDoIt = true;
50     for (int i = 0; file >> currentChar && canDoIt; i++)
51     {
52
53         //do stuff only if the character is a paranthesis
54         if (isAParanthesis(currentChar))
55         {
56             //push elements in the stack if a paranthesis is opened
57             if (isOpenParanthesis(currentChar))
58                 paranthesisStack.push(currentChar);
59
60             //pop elements from the stack if a paranthesis is closed
61             if (isCloseParanthesis(currentChar))
62             {
63
64                 //check if the stack is empty or
65                 //if the paranthesis are in the wrong order
66                 if (paranthesisStack.size() == 0 ||
67                     !areOpositeParanthesis(paranthesisStack.top(),
68                                             currentChar))
69                 {
60                     canDoIt = false;
61                     break;
62                 }
63                 paranthesisStack.pop();
64             }
65         }
66     }
67
68     //check if something went wrong or if the stack is not empty
69     if (canDoIt == false || paranthesisStack.size() > 0)
70         std::cout << "Ai gresit!" << std::endl;
71     else
72         std::cout << "Merge!" << std::endl;
73
74     file.close();
75     return 0;
76 }

```

3.3 Tablouri unidimensionale - probleme propuse

Fie următoarele probleme cu tablouri unidimensionale:

1. Se dă un vector cu n elemente. Să se insereze după fiecare număr prim valoarea minimă din șir la momentul respectiv (până în numărul prim detectat). Să se afișeze la final șirul modificat.

Exemplu: 12 34 11 6 5 8 4 3, atunci se afișează 12 34 11 12 6 5 6 8 4 3 4.

2. Să se verifice dacă există dubluri într-un vector. (Să se determine ce elemente apar de mai multe ori și de câte ori apare fiecare și să se afișeze aceste informații).

Exemplu: 10 3 4 3 5 8 10 3 504 293 21 se afișează "3 a fost găsit de 3 ori, 10 a fost găsit de 2 ori".

3. Un număr natural se reține într-un vector (număr mare), astfel încât fiecare componentă a vectorului conține câte o cifră a numărului. Să se înmulțească numărul cu un număr între 1 și 9.

4. Se citește un număr natural n și n medii (numere reale cu două zecimale cu valori cuprinse între 1 și 10). Să se afișeze o statistică a mediilor astfel: numărul de medii cuprinse în intervalul $[1,2]$, numărul de medii cuprinse în intervalul $(2,3]$, ..., numărul de medii cuprinse în intervalul $(9,10]$.

Hint: Se pot utiliza vectori de frecvență.

5. Să se scrie două variante de implementare (una iterativă, una recursivă) pentru o funcție care are ca parametri un polinom (dat prin grad și prin coeficienții lui), dar și o valoare reală. Funcția returnează valoarea polinomului dat în punctul precizat.

Hint 1: Pentru rezolvarea cu ambele variante, se ține cont de schema lui Horner. De asemenea, pentru a reține coeficienții polinomului, folosim un tablou unidimensional, de dimensiune $\text{grad}+1$.

Hint 2: Pentru metoda recursivă, se poate lua în calcul faptul că, pentru a calcula valoarea, luăm coeficientul de pe poziția dorită, pe care îl înmulțim cu puterea valorii (în care calculăm) la grad, unde grad trebuie să descrească la fiecare etapă.

Hint 3: Pentru rezolvare, se poate utiliza și o structură care reține un obiect de tip polinom, structură de forma:

Secvență de cod 3.22: Definire structură Polinom

```

1      struct Polinom
2      {
3          double coef[101];
4          int dim;
5      };

```


6. Să se modeleze un sistem bancar care să ofere următoarele servicii clienților săi:

- Deschiderea unui cont nou; la deschidere, clientul poate depune o sumă inițială de bani; clientul e înștiințat de numărul noului său cont. Observație: Banca poate avea maxim 100 de conturi deschise.
- Interogare sold: afișarea sumei dintr-un cont. Observație: contul trebuie să fie deschis; se afișează și numărul contului.
- Depunere: adăugarea unei valori la un cont și afișarea noii sume din cont. Observație: contul trebuie să fie deschis; se afișează și numărul contului.
- Retragere: scăderea unei valori dintr-un cont și afișarea noii sume din cont. Observație: contul trebuie să fie deschis și suma retrasă trebuie să fie mai mare decât cea din cont; se afișează și numărul contului.
- Închiderea contului: numai dacă un cont e deschis.

Funcții adiționale ale băncii:

- (a) Afișarea tuturor conturilor și a sumelor din ele;
- (b) Închiderea tuturor conturilor (și ieșirea din program).

Observații de implementare:

- Funcție separată pentru fiecare acțiune;
- Fiecare funcție se apelează conform literei tastate;
- În `main()` avem un ciclu `while` în care putem realiza operațiile bancare mai sus definite prin tastarea literelor corespunzătoare.

Pașii de urmat:

- (a) Deschidere cont nou cu o sumă inițială (O)
- (b) Afișarea tuturor conturilor + sumelor din bancă (P)
- (c) Interogare sold (B) - se cere numărul contului
- (d) Depunere (D) - se cere numărul contului și suma
- (e) Retragere (W) - se cere numărul contului și suma
- (f) Închidere cont (C) - se cere numărul contului
- (g) Închiderea tuturor conturilor (E)

Pentru toate problemele de mai sus să se țină cont de principiile de cod curat enunțate în această secțiune. De asemenea, încurajăm folosirea containerelor, plus a structurilor, `struct` (care permit introducerea obiectelor).

Capitolul 4

Tablouri bidimensionale

În capitolul anterior am văzut că vectorii sau tablourile unidimensionale au o gamă largă de aplicații în scrierea algoritmilor. Pentru a descrie variantele posibile de a lucra cu ei în contextul limbajului C++, au fost prezentate, în acel capitol, o serie de modalități de a declara, de a inițializa și de a prelucra acest tip de date. Capitolul de față își propune să meargă cu un pas mai departe și să aducă în discuție tablourile bidimensionale sau matricele.

4.1 Aspecte teoretice

O matrice reprezintă o structură de date standard, care se folosește pentru a păstra un set de date de același tip, organizate pe linii și coloane. Ea poate fi văzută ca extinderea la două dimensiuni a unui vector, de unde provine și denumirea alternativă de tablou bidimensional. O altă posibilă perspectivă este aceea de a privi o matrice ca un vector cu elemente de tip vector, perspectivă care poate facilita înțelegerea mecanismului alocării dinamice în cazul matricelor. Ca și în cazul vectorilor, o serie de aspecte legate de lucrul cu matrice vor fi discutate în continuare. Pentru moment însă, vom enumera câteva dintre situațiile în care folosirea unui tablou bidimensional (matrice) poate fi extrem de utilă:

- Rezolvarea sistemelor de ecuații liniare sau a unor sisteme de ecuații diferențiale (cu aplicații în fizică), rezolvarea unor probleme de grafică pe calculator.
- Pentru prelucrarea sau stocarea unei imagini alb negru se poate folosi o matrice în care, pentru fiecare pixel al imaginii, (i, j) , reținem nuanța sa de alb. Aceasta este un număr întreg în intervalul $[0, 255]$. Se pot stoca și imagini color, dar atunci în fiecare celulă a matricei se păstrează trei atribute (eventual sub forma unei structuri), care reprezintă nuanțele de roșu, verde și albastru.

- Un alt context în care matricele sunt folosite este simularea jocurilor. Poate fi vorba de un joc de sudoku, reversi, binaris, sau de un joc de șah, și exemplele pot continua.
- O altă situație în care matricele pot juca un rol important este aceea în care vrem să modelăm relații între elementele unei mulțimi. Amintim în acest context cazul rețelelor de socializare, caz în care matricele ar putea modela relațiile de prietenie. În cazul reprezentării unei hărți, ele pot modela drumurile dintre orașe. Acest tip de matrice este cunoscută, în teoria grafurilor, ca *matrice de adiacență*.
- Matricele pot fi însă folosite într-o serie de alte situații practice în care dispunem de date care sunt aranjate într-o zonă de formă dreptunghiulară, care este împărțită pe linii și coloane. Exemplu: un sistem de senzori dispuși la un etaj al unei clădiri, un sistem de sprinklere într-o grădină, etc.

4.1.1 Cerințe preliminare pentru prelucrarea tablourilor bidimensionale

În această secțiune vom discuta următoarele aspecte legate de tablourile bidimensionale:

- cum declarăm un tablou bidimensional;
- cum inițializăm un tablou bidimensional;
- cum memorăm un tablou bidimensional;
- cum accesăm elementele unui tablou bidimensional;
- cum citim un tablou bidimensional;
- cum afișăm un tablou bidimensional;
- cum efectuăm diverse prelucrări asupra elementelor unui tablou bidimensional.

4.1.2 Reprezentarea tablourilor bidimensionale. Alocare dinamică și alocare statică

Așa cum s-a discutat și în cazul vectorilor, există în C++ două posibilități de a declara matricele, și anume în variantă statică și în variantă dinamică. În cazul declarării statice, numărul maxim de linii și coloane ale matricei trebuie estimat anterior declarării acesteia și odată ce compilatorul a stabilit locația și dimensiunea zonei de memorie ocupată de matrice, aceasta nu mai poate fi modificată (nici nu poate fi mărită dacă este nevoie, dar nici nu poate fi micșorată dacă se

dovedește a fi prea mare). Alocarea dinamică elimină această limitare prin faptul că dimensiunile matricei pot fi stabilite la momentul execuției programului. Ambele tipuri de alocări sunt discutate în secțiunile următoare.

Alocare statică

Mecanismul declarării statice a matricelor în C++ este unul ușor de utilizat și prezintă avantajul că alocarea zonelor de memorie corespunzătoare și apoi ștergerea lor se face în mod automat, în momentul părăsirii secvenței de cod în care acestea au fost declarate ca și variabile.

În această secțiune vom exemplifica toate operațiile pe tablouri bidimensionale folosind alocarea statică.

- Declararea unui tablou bidimensional

Sintaxa este:

```
tip_elem_matrice nume_matrice[nr_maxim_linii][nr_maxim_col];
```

unde:

- *tip_elem_matrice*: poate fi orice tip primitiv din C++, orice container, sau orice tip introdus de către utilizator (oricare clasă/structură);
- *nume_matrice*: trebuie să respecte regulile unui identificator C++ ;
- *nr_maxim_linii*/*nr_maxim_col* : reprezintă numărul maxim de linii, respectiv coloane pe care le poate avea matricea. Compilatorul C++ alocă un spațiu de memorie egal cu numărul maxim de elemente ale tabloului, rezervând octeți în funcție de tipul de bază al elementelor matricei.

Exemple:

Secvență de cod 4.1: Cod C++ pentru declararea unei matrici

```
1 // tipul elementelor este int iar nr maxim de linii
2 // si coloane este 20
3 int matrice[20][20];
4 // matrice cu elemente booleene
5 int matBool[6][5];
6 // matrice de siruri de caractere
7 string sir[10][10];
8
```

- Inițializarea unui tablou bidimensional

Ca și tablourile unidimensionale, matricele pot fi inițializate. Cele declarate ca variabile globale se inițializează prin definițiile lor la compilare, iar cele declarate ca variabile locale se inițializează prin declarațiile lor la execuție.

Inițializarea se poate face prin introducerea datelor de la tastatură/fișier, prin atribuire a unei valori date, sau prin utilizarea unei constante de tip tablou.

Secvență de cod 4.2: Cod C++ pentru initializarea unei matrice

```

1 //Versiune citire de la tastatura
2 int mat[20][20], linii, col;
3 cin >> linii >> col;
4 for (int i = 0; i < linii; i++)
5 {
6     for (int j = 0; j < col; j++)
7     {
8         cin >> mat[i][j];
9     }
10 }
11
12 //Versiune initializare cu o constanta data
13 int mat[20][20], linii, col;
14 cin >> linii >> col;
15 for (int i = 0; i < linii; i++)
16 {
17     for (int j = 0; j < col; j++)
18     {
19         mat[i][j] = 0;
20     }
21 }
22
23 //Versiune prin utilizarea unei constante de tip tablou
24 int mat[3][4] = {{ 1, 2, 3, 4 },
25                  { 5, 6, 7, 8 },
26                  { 9, 10, 11, 12 }};
27

```

- Memorarea unui tablou bidimensional

Ca și în cazul vectorilor, necesarul de memorie pentru păstrarea elementelor unui tablou bidimensional este direct proporțional cu tipul și cu dimensiunea lui, numărul total de octeți fiind: `sizeof(tip_elemente_matrice) * nr_maxim_linii * nr_maxim_coloane`. Elementele unui tablou sunt memorate într-o zonă de memorie contiguă, linie după linie.

Secvență de cod 4.3: Cod C++ pentru memorarea unui tablou bidimensional

```

1 int main()
2 {
3     int mat[3][2], k = 1;
4     //initializam elementele matricei valori cu de la 1
5     // la linii*coloane, prin intermediul variabilei k
6     // si afisam apoi locatiile lor in memorie
7
8     for (int i = 0; i < 3; i++)
9     {
10         cout << "linie noua \n";
11         for (int j = 0; j < 2; j++)
12         {
13             mat[i][j] = k++;
14             std::cout << &mat[i][j] << "\t" << mat[i][j] << std::endl;
15         }
16         std::cout << "\n";
17     }
18 }
19
20

```

Ieșirea pentru demo-ul de memorare este în Figura 4.1.

- Accesarea elementelor unui tablou bidimensional

006FFB58	1
006FFB5C	2
006FFB60	3
006FFB64	4
006FFB68	5
006FFB6C	6
006FFB70	7
006FFB74	8
006FFB78	9
006FFB7C	10

Figura 4.1: Output pentru memorarea unui tablou bidimensional

Amintim că o matrice poate fi privită ca un tablou de tablouri. Mai precis, o declarație de forma:

`int matrice[4][3]`

se poate interpreta astfel: variabila *matrice* este declarată ca fiind un tablou cu 4 elemente, *matrice*[0], *matrice*[1], *matrice*[2] și *matrice*[3], fiecare dintre acestea fiind tablouri cu elemente de tip întreg. Dacă este cazul, putem accesa liniile matricei folosind o astfel de referire. În ceea ce privește elementele matricei, accesul la acestea se face folosind numele variabilei, urmat între paranteze drepte de indicele său de linie, iar apoi de indicele său de coloană. De exemplu:

`matrice[0][2]`

se referă la elementul situat pe prima linie și pe a treia coloană a matricei. După cum am mai precizat în capitolele anterioare, în limbajul C++, indicii tablourilor încep numărătoarea de la valoarea 0 și se termină cu limita superioară - 1 (nrLinii-1/nrColoane-1).

- Citirea elementelor unui tablou bidimensional

Orice matrice este caracterizată prin dimensiunea sa, reprezentată de numărul de linii și respectiv de coloane, și în plus mulțimea elementelor sale. Citirea se poate face în diverse moduri: fie citire de la tastatura, fie dintr-un fișier de intrare.

Secvență de cod 4.4: Cod C++ pentru citirea unui tablou bidimensional

```

1 // citirea de la tastatura
2 void citireMatriceStatice(int& linii, int& col, int mat[20][20])
3 {
4     cout << " dati nr de linii ale matricei ";
5     cin >> linii;
6     cout << " \n dati nr de coloane ale matricei ";
7     cin >> col;
8     cout << " \n introduceți elementele matricei ";
9     for (int i = 0; i < linii; i++)
10    {

```

```

11         for (int j = 0; j < col; j++)
12         {
13             cin >> mat[i][j];
14         }
15     }
16 }
17 //citirea din fisier
18 void citireMatriceFisier(std::ifstream& f, int& linii, int& col, int mat[20][20])
19 {
20     f >> linii >> col;
21     for (int i = 0; i < linii; i++)
22     {
23         for (int j = 0; j < col; j++)
24         {
25             f >> mat[i][j];
26         }
27     }
28 }

```

- Afișarea elementelor matricei este similară cu citirea

Secvență de cod 4.5: Cod C++ pentru afișarea elementelor unui tablou bidimensional

```

1 void afisareMatriceStatice(int linii, int col, int mat[20][20])
2 {
3     for (int i = 0; i < linii; i++)
4     {
5         for (int j = 0; j < col; j++)
6         {
7             cout << mat[i][j] << " ";
8         }
9         cout << "\n";
10    }
11 }

```

Alocare dinamică

În această secțiune, vom vedea cum declarăm tablouri bidimensionale dinamic. Ca și în cazul vectorilor, alocarea dinamică a tablourilor bidimensionale va asigura faptul că elementele succesive ale liniilor tabloului bidimensional se vor afla în locații succesive de memorie, ceea ce înseamnă că accesarea matricelor alocate dinamic poate fi făcută ca și în cazul matricelor alocate static, folosind operatorul de indexare, `[.][.]`.

Alocarea dinamică de memorie pentru o matrice este importantă deoarece:

- permite o folosire eficientă a memoriei, evitând alocările estimative, acoperitoare;
- permite declararea unor matrice cu lungimi diferite ale liniilor;
- o matrice alocată dinamic poate fi modificată, ca și dimensiuni dacă este necesar, în cursul execuției programului.
- odată ce nu mai este nevoie de datele stocate în matrice, memoria folosită poate fi eliberată. Evident că acest lucru cade în sarcina programatorului.

- reprezintă o soluție mai bună la problema transmiterii matricelor ca parametri ai unor metode.

În această secțiune vom oferi câteva informații de bază legate de alocarea dinamică a matricelor. Pentru a o ilustra cât mai bine, vom da un exemplu și vom comenta succint sintaxa.

- Declararea, alocarea dinamică a unui tablou bidimensional, și eliberarea memoriei

După cum s-a văzut în capitolul **Structuri de date fundamentale. Prelucrarea tablourilor unidimensionale**, în C++ se folosesc pentru alocarea dinamică, operatorii `new` și `delete`. În cazul alocării dinamice a unei matrice cu ambele dimensiuni variabile, ideea este de a alocă dinamic un tablou de pointeri, în care fiecare pointer referă (arată către) o linie alocată la rândul ei dinamic.

<i>int**</i>	<i>int*</i>	<i>int</i>
<code>mat</code> —>	<code>mat[0]</code> —>	<code>mat[0][0]</code> <code>mat[0][1]</code> <code>mat[0][2]</code>
	<code>mat[1]</code> —>	<code>mat[1][0]</code> <code>mat[1][1]</code> <code>mat[1][2]</code>
	<code>mat[2]</code> —>	<code>mat[2][0]</code> <code>mat[2][1]</code> <code>mat[2][2]</code>

Astfel pentru o matrice de dimensiune $m \times n$ cu elemente de tip întreg de exemplu, `mat` va fi declarată ca fiind un pointer către un tablou (alocat dinamic de pointeri `int*`)

`int** mat`

Pentru ca matricea să aibă m linii, pentru tabloul de pointeri de tip `int*` trebuie să fie alocat un spațiu de dimensiune m :

`mat = new int*[m]`

Apoi, se parcurge fiecare element `mat[i]` al acestui tablou și îl inițializăm cu adresa unei linii de n elemente de tip `int`, alocată dinamic:

`for(int i = 0; i < m; i++)`
`mat[i] = new int[n];`

Metoda care face declararea și alocarea de memorie este prezentată mai jos. Dealocarea zonei de memorie corespunzătoare matricei *mat* se va face în ordine inversă alocării: se vor dealoca rând pe rând liniile matricei și numai la final se dealocă tabloul (coloana) pointerilor care reprezintă adresele de început ale liniilor matricei. Metoda de dealocare este și ea prezentată mai jos.

Secvență de cod 4.6: Cod C++ pentru alocarea și dealocarea unei matrice

```

1  #include <iostream>
2  using namespace std;
3  int** alocareDinamicaMatrice(int linii, int col)
4  {
5      int** mat = new int* [linii];
6      for (int i = 0; i < linii; i++)
7          mat[i] = new int[col];
8      return mat;
9  }
10
11 void dealocareMatrice(int** mat, int linii)
12 {
13     for (int i = 0; i < linii; i++)
14         delete [] mat[i];
15     delete [] mat;
16 }
```

Reiterăm observația conform căreia, în cazul folosirii alocării dinamice, programatorul are sarcina de a alocă și de a elibera zona de memorie folosită, atunci când nu mai este nevoie de variabilele declarate dinamic.

- Accesarea elementelor, citirea și afișarea componentelor unui tablou alocat dinamic.

Cele trei operații amintite mai sus se fac absolut asemănător cu varianta statică. Codul prezentat mai jos are metodele de citire și afișare, iar în metoda `main()` apeluri inclusiv pentru metodele de alocare și dealocare discutate anterior.

Secvență de cod 4.7: Cod C++ pentru citirea și afișarea unei matrice

```

1  #include <iostream>
2  using namespace std;
3  void citireMatrice(int** mat, int linii, int col) {
4      for (int i = 0; i < linii; i++) {
5          for (int j = 0; j < col; j++) {
6              cout << " mat[" << i << "][" << j << "]=";
7              cin >> mat[i][j];
8          }
9          cout << endl;
10     }
11 }
12
13 void afisareMatrice(int** mat, int linii, int col)
14 {
15     for (int i = 0; i < linii; i++) {
16         for (int j = 0; j < col; j++)
17             cout << mat[i][j] << " ";
18         cout << endl;
19     }
20 }
21 int main()
22 {
23
24     int** mat, linii, col;
25     cout << "Introduceți numărul de linii și coloane ale matricei";
```

```

26     cin >> linii >> col;
27     mat = alocareDinamicaMatrice(linii, col);
28     citireMatrice(mat, linii, col);
29     afisareMatrice(mat, linii, col);
30     dealocareMatrice(mat, linii);
31 }

```

În secțiunea imediat următoare, așa cum am procedat și în cazul vectorilor, vom prezenta o a treia variantă, mai modernă, de a lucra cu tablouri bidimensionale folosind containerele din STL.

4.1.3 Reprezentarea tablourilor bidimensionale folosind Standard Template Library (C++)

În STL nu există tablouri bidimensionale, dar prin combinarea containerelor secvențiale, cum ar fi **vector** sau **deque**, se pot obține matrici alocate dinamic. O matrice poate fi concepută ca un vector de vectori și există mai multe moduri în care poate fi definit un astfel de vector de vectori. Vom încerca în continuare să prezentăm o parte dintre ele.

1. Una dintre variante ar fi crearea matricei linie cu linie. Se citește pe rând câte o linie (reprezentată de un vector) și se adaugă în matrice folosind **push_back**.

Secvență de cod 4.8: Cod C++ pentru creare matrice (STL)

```

1  vector<vector<int>>> creareMatriceSTL(int linii, int col)
2  {
3      vector<vector<int>>> mat;
4      vector<int> linie;
5      int elem;
6      for (int i = 0; i < linii; i++)
7      {
8          for (int j = 0; j < col; j++)
9          {
10             cin >> elem;
11             linie.push_back(elem);
12         }
13         mat.push_back(linie);
14         // stergem toate elementele de pe linie
15         linie.clear();
16     }
17     return mat;
18 }

```

2. O cale mai simplă de a crea o matrice, este de a folosi constructorii $x = \text{vector}(m)$ și $\text{vector}(n, x)$, care crează un vector x de dimensiune m , respectiv un vector cu n elemente de de tipul x , fiecare având dimensiune m .

Secvență de cod 4.9: Cod C++ pentru creare matrice folosind constructori

```

1  vector<vector<int>>> creareMatriceSTL(int linii, int col)
2  {
3      vector<vector<int>>> mat(linii, vector<int>(col));
4      for (int i = 0; i < linii; i++)
5      {
6          for (int j = 0; j < col; j++)

```

```

7         {
8             cin >> mat[i][j];
9         }
10    }
11    return mat;
12 }

```

3. O altă soluție pentru a crea matrice cu ajutorul containerului **vector** este declararea unui vector de vectori vid, și apoi redimensionarea lui cu **resize()**.

Secvență de cod 4.10: Cod C++ pentru creare matrice folosind constructori

```

1 vector<vector<int>>> creareMatriceSTL(int linii, int col)
2 {
3     vector<vector<int>>> mat;
4     mat.resize(linii);
5     for (int i = 0; i < linii; i++)
6     {
7         mat[i].resize(col);
8     }
9     return mat;
10 }

```

4. O a patra soluție de a crea o matrice este cu ajutorul containerului **map**. Acesta este un container asociativ care păstrează elementele sub formă de perechi (*cheie, valoare*). Nu se poate ca două valori să aibă aceleași chei. Clasa șablon **map** supraîncarcă operatorul **[.]**, și mapa se folosește ca un șir asociativ astfel:

$$mapa[cheie] = valoare$$

Această caracteristică este folosită pentru a construi tablouri bidimensionale. Cum operatorul **[.][.]** nu există, se declară un map de map-uri și se aplică de două ori operatorul **[.]**. În mapa mare, cheia va fi indexul de linie, în mapa mică cheia va fi indexul de coloană.

Secvență de cod 4.11: Cod C++ pentru creare matrice folosind map

```

1 map<int, map<int, int>>> creareMatriceMap(int linii, int col)
2 {
3     map<int, map<int, int>>> mat;
4     for (int i = 0; i < linii; i++)
5     {
6         for (int j = 0; j < col; j++)
7         {
8             cin >> mat[i][j];
9         }
10    }
11    return mat;
12 }

```

Accesarea valorii asociate unei chei folosind operatorul de indexare **[.]** este o operație care se face în complexitate $O(\log n)$, unde n este numărul perechilor din mapă. Ceea ce înseamnă că în cazul mapei de mape, accesarea elementului $mat[i][j]$ se face în complexitate $O(\log n \times \log m)$

4.2 Tablouri bidimensionale - probleme rezolvate

În secțiunea care urmează vom utiliza câteva dintre metodele realizate în subcapitolul anterior, și anume, vom presupune că pentru prelucrarea tablourilor bidimensionale avem următoarea interfață predefinită:

Secvență de cod 4.12: Cod C++ Intefata de lucru cu tablouri bidimensionale

```

1 void citireMatriceStatice(int& linii, int& col, int mat[20][20]);
2 void citireMatriceFisier(std::ifstream& f, int& linii, int& col, int mat[20][20]);
3 void afisareMatriceStatice(int linii, int col, int mat[20][20]);
4 int** alocareDinamicaMatrice(int linii, int col);
5 void dealocareMatrice(int** mat, int linii);
6 void citireMatriceDinamica(int** mat, int linii, int col);
7 void citireMatriceDinamicaFisier(std::ifstream& f, int** mat, int& linii, int& col);
8 void afisareMatriceDinamica(int** mat, int linii, int col);
9 vector<vector<int>> creareMatriceSTL(int linii, int col);
10 void afisareMatriceSTL(vector<vector<int>> mat)
11

```

În contextul în care presupunem aceste metode implementate, vom prezenta numai metodele care rezolvă efectiv problemele propuse.

Problemele pe care le propunem în această secțiune sunt:

• Probleme cu parcurgeri de matrice

1. Se dă o matrice cu m linii și n coloane. Să se determine câte elemente ale matricei au toți vecinii numere prime.
2. Se dă o matrice pătratică de numere întregi, de dimensiune n . Să se calculeze.
 - (a) Suma elementelor de pe diagonala principală a matricei;
 - (b) Produsul elementelor de pe diagonala secundară a matricei;
 - (c) Media aritmetică a elementelor de sub diagonala principală a matricei;
 - (d) Minimul elementelor de deasupra diagonalei secundare a matricei;
3. Se dă o matrice pătratică de numere întregi, de dimensiune n . Diagonala principală și cea secundară, împart matricea în patru zone triunghiulare. Să se determine suma elementelor dintr-o zonă dată.
4. Se dă o matrice de numere întregi, de dimensiune $m \times n$. Să se afișeze șirul numerelor obținut prin parcurgerea în spirală a matricei, pornind din colțul stânga sus și mergând pe margine, către dreapta.

Exemplu: Pentru matricea

$$Mat[4][4] = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (4.1)$$

se va afișa șirul: 1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10.

5. Se dă o matrice pătratică de numere întregi, de dimensiune n . Să se afișeze șirul numerelor obținut prin parcurgerea în zig- zag a matricei date.

Exemplu: Pentru matricea

$$Mat[4][4] = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (4.2)$$

se va afișa șirul: 1, 5, 2, 3, 6, 9, 13, 10, 7, 4, 8, 11, 14, 15, 12, 16.

În continuare vom oferi posibile soluții la problemele enunțate anterior.

Pentru problema 1 avem:

- Date de intrare: o matrice și numărul ei de linii m și coloane n ;
- Date de ieșire: numărul de elemente ale matricei care au toți vecinii numere prime;
- Algoritm de rezolvare: Se reține într-o variabilă contor, numărul căutat. Pentru fiecare element al matricei vom reține într-o variabilă booleană *cond_verif* dacă toți vecinii sunt primi sau nu.

Pentru fiecare element al matricei se consideră vecinii (nord, sud, est, vest) care sunt în interiorul matricei și se verifică dacă aceștia sunt numere prime. Dacă răspunsul este fals, variabila booleană devine falsă. Dacă toți vecinii sunt primi, variabila booleană rămâne adevărată și contorul crește cu o unitate.

- Algoritmul scris în C++ prezintă metoda de verificare a faptului că un număr este prim, metoda care numără câte elemente satisfac condiția impusă de problemă și metoda main(). Alocarea dinamică a memoriei pentru matrice, citirea și de alocarea de memorie sunt facute conform metodelor din interfața prezentată anterior.

Secvență de cod 4.13: Cod C++ pentru problema 1

```

1
2  bool esteNumarPrim(int nr)
3  {
4      if (nr == 0 || nr == 1)
5          return false;
6      int nrDiv = 0;
7      for (int d = 2; d <= nr / 2; d++)
8      {
9          if (nr % d == 0)
10             nrDiv++;
11      }
12      if (nrDiv)
13          return false;
14      else
15          return true;
16  }
17

```

```

18 int numarareVeciniPrimi(int** mat, int linii, int col)
19 {
20     int contor = 0;
21     for (int i = 0; i < linii; i++)
22         for (int j = 0; j < col; j++) {
23             bool cond_verif = true;
24             if (i > 0 && !esteNumarPrim(mat[i - 1][j]))
25                 cond_verif = false;
26             if (i < linii - 1 && !esteNumarPrim(mat[i + 1][j]))
27                 cond_verif = false;
28             if (j > 0 && !esteNumarPrim(mat[i][j - 1]))
29                 cond_verif = false;
30             if (j < col - 1 && !esteNumarPrim(mat[i][j + 1]))
31                 cond_verif = false;
32             if (cond_verif)
33                 contor++;
34         }
35     return contor;
36 }
37
38 int main()
39 {
40     int** mat, linii, col, k;
41     cout << "Introduceti numarul de linii si coloane ale matricei";
42     cin >> linii >> col;
43     mat = alocareDinamicaMatrice(linii, col);
44     citireMatrice(mat, linii, col);
45     cout << "numarul de elemente cu toti vecinii primi este: ";
46     cout << numarareVeciniPrimi(mat, linii, col);
47     dealocareMatrice(mat, linii);
48 }

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu m linii și n coloane, adică complexitatea este $O(m \times n)$. Din punct de vedere al timpului de execuție, se parcurg toate elementele matricei și se verifică vecinii, care sunt cel mult 4 la număr. Așadar complexitatea algoritmului este $O(m \times n)$.

Pentru problema 2 avem:

- Date de intrare: o matrice pătratică și numărul ei de linii/coloane;
- Date de ieșire: un număr care, după caz, reprezintă:
 - * suma elementelor de pe diagonala principală;
 - * produsul elementelor de pe diagonala secundară;
 - * media aritmetică a elementelor de sub diagonala principală a matricei;
 - * minimul elementelor de deasupra diagonalei secundare a matricei;
- Algoritm de rezolvare: pentru rezolvarea acestei probleme, avem nevoie să cunoaștem câteva lucruri legate de parcurgerea matricelor pe diagonale, sub diagonale sau deasupra diagonalelor, toate aceste zone fiind caracterizate prin anumite relații între indicii de linie și de coloană. Astfel avem:
 - * **Diagonala principală** $i = j$;
 - * **Sub diagonala principală** $j < i$
 - * **Deasupra diagonalei principale** $j > i$

- * **Diagonala secundară** $i + j = n - 1$, unde n este dimensiunea matricei și indexarea este de la 0;
 - * **Sub diagonala secundară** $i + j > n - 1$;
 - * **Deasupra diagonalei secundare** $i + j < n - 1$,
- Algoritmul scris în C++ implică utilizarea mai multor funcții, printre care: una pentru citirea a elementelor matricei (oricare din interfața precizată la începutul acestei secțiuni), restul pentru rezolvarea fiecărei cerințe în parte. Pentru că metoda de citire este generică, pentru o matrice oarecare, vom verifica ulterior dacă numărul de linii este același cu numărul de coloane citit.

Secvență de cod 4.14: Cod C++ metodele care calculează suma și produsul elementelor de pe cele două diagonale, problema 2

```

1  int sumaDiagPrincipala(int dim, int mat[20][20])
2  {
3      int suma = 0;
4      // pe diagonala principala indicii elementelor
5      // matricei sunt egali i=j
6      for (int i = 0; i < dim; i++)
7      {
8          suma = suma + mat[i][i];
9      }
10     return suma;
11 }
12
13 int produsDiagSecundara(int dim, int mat[20][20])
14 {
15     int produs = 1;
16     // pe diagonala principala indicii elementelor
17     // matricei au proprietatea ca i+j=dim-1
18     for (int i = 0; i < dim; i++)
19     {
20         produs = produs * mat[i][dim-1-i];
21     }
22     return produs;
23 }

```

Secvență de cod 4.15: Cod C++ Continut pentru main(), problema 2

```

1  citireMatriceStatica(linii, coloane, matrice);
2  /* pentru ca metoda de citire este una generica,
3  pentru o matrice oarecare, verificam mai intai
4  daca matricea este patratica
5  */
6  if (linii == coloane)
7  {
8      int dim = linii;
9      cout << "suma elementelor de pe diagonala principala este ";
10     cout << sumaDiagPrincipala(dim, matrice) << "\n";
11     cout << "produsul elementelor de pe diagonala secundara este ";
12     cout << produsDiagSecundara(dim, matrice) << "\n";
13 }
14 else
15     cout << "matricea nu este patratica \n";

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu n linii și coloane, adică complexitatea este $O(n^2)$. Din punct de vedere al timpului de execuție, se parcurg toate elementele diagonalelor, folosind o singură buclă *for*

din cauza relației între indici. Așadar complexitatea algoritmului este $\Theta(n)$.

Secvență de cod 4.16: Cod C++ metodele care rezolva c si d, problema 2

```

1  int sumaDiagPrincipala(int dim, int mat[20][20])
2  double medieArSubDiagPr(int dim, int mat[20][20])
3  {
4      int suma = 0;
5      int contor = 0;
6      /* sub diagonala principala indicele de coloana j
7       este mai mic decat indicele de linie i; i<j
8       */
9      for (int i = 0; i < dim; i++)
10     {
11         for (int j = 0; j < i; j++)
12         {
13             suma += mat[i][j];
14             ++contor;
15         }
16     }
17     return (double)suma/contor;
18 }
19
20 int minDeasupraDiagSecundara(int dim, int mat[20][20])
21 {
22     int min = INT_MAX;
23     /* deasupra diagonale secundare i+j<dim-1
24     adica j<dim-1-i
25     */
26     for (int i = 0; i < dim; i++)
27     {
28         for (int j = 0; j < dim-1-i; j++)
29         {
30             if (min > mat[i][j])
31                 min = mat[i][j];
32         }
33     }
34     return min;
35 }

```

Secvență de cod 4.17: Cod C++ Continut pentru main(),c si d, problema 2

```

1  citireMatriceStatice(linii, coloane, matrice);
2  /* pentru ca metoda de citire este una generica,
3   pentru o matrice oarecare, verificam mai intai
4   daca matricea este patratica
5   */
6  if (linii == coloane)
7  {
8      int dim = linii;
9      cout << "media aritm a elementelor de sub diagonala principala este ";
10     cout << medieArSubDiagPr(dim, matrice) << "\n";
11     cout << "minimul elementelor de deasupra diagonalei secundare este ";
12     cout << minDeasupraDiagSecundara(dim, matrice) << "\n";
13 }
14 else
15     cout << "matricea nu este patratica \n";

```

- Analiza complexității: Din punct de vedere al spațiului de memorie, complexitatea este $O(n^2)$, ca și în cazurile anterioare. Din punct de vedere al timpului de execuție, se parcurg toate elementele triunghiurilor situate sub diagonala principală, respectiv deasupra diagonalei secundare. În cazul subpunctului c, pentru că indicele j ia valori condiționate de valoarea lui i , în for-ul pentru j se va intra: o dată dacă $i = 1$, de două ori pentru $i = 2$, ..., de $n - 1$ ori pentru $i = n - 1$.

Astfel numărul de operații va fi:

$$1 + 2 + \dots + n - 1 = \frac{n(n-1)}{2}$$

ceea ce înseamnă că algoritmul are complexitate $\Theta(n^2)$. Similar se calculează și pentru cazul d.

Pentru problema 3 avem:

- Date de intrare: o matrice pătratică și numărul ei de linii/coloane;
- Date de ieșire: un număr care, după caz, reprezintă suma unui triunghi specificat dintre cele 4 formate de diagonale;
- Algoritm de rezolvare: Vom considera cele 4 zone, identificate astfel:
 - Zona 1 (Nord): este cea de deasupra diagonalei principale și de deasupra diagonalei secundare;
 - Zona 2 (Est): este cea de deasupra diagonalei principale și de sub diagonală secundară;
 - Zona 3 (Sud): este cea de sub diagonală principală și de sub diagonală secundară;
 - Zona 4 (Vest): este cea de sub diagonală principală și de deasupra diagonalei secundare;

Se poate observa indicii de linie și coloană iau valori conditionate. Luăm în considerare, atât un caz cu număr par de linii și coloane, cât și unul cu număr impar de linii și coloane.

Pentru triunghiurile din Nord și Sud este ușor de văzut de unde până unde ia valori i , iar apoi j va lua valori în funcție de considerentele de la problema 1. Pentru triunghiurile din Est și Vest este mai ușor de văzut de unde până unde ia valori j , iar apoi i va lua valori în funcție de j . Vom avea astfel:

* Zona 1(Nord):

$$0 \leq i \leq \lfloor (n-1)/2 \rfloor - 1; i+1 \leq j \leq n-2-i$$

* Zona 2 (Est):

$$\lfloor n/2 \rfloor + 1 \leq j \leq n-1; n-j \leq i \leq j-1$$

* Zona 3(Sud):

$$\lfloor n/2 \rfloor + 1 \leq i \leq n-1; n-i \leq j \leq i-1$$

* Zona 4(Vest):

$$0 \leq j \leq [(n-1)/2] - 1; j+1 \leq i \leq n-2-j$$

- Algoritmul scris în C++ implemtează formulele anterioare. Metoda returneaza suma dintr-o anumită zonă, în funcție de parametrul primit.

Secvență de cod 4.18: Cod C++ Metoda folosita pentru rezolvarea problemei 3

```

1  int sumaZoneTriunghiulare(int dim, int mat[20][20], int zona)
2  {
3      // formulele pentru indici sunt cele explicate
4      // in partea teoretica
5
6      int suma = 0;
7
8      if (zona == 1)
9      {
10         for (int i = 0; i <= (dim-1) / 2 - 1; i++)
11             for (int j = i + 1; j <= dim - 2 - i; j++)
12                 suma += mat[i][j];
13     }
14     if (zona == 2)
15     {
16         for (int j = dim / 2 + 1; j <= dim - 1; j++)
17             for (int i = dim - j; i <= j - 1; i++)
18                 suma += mat[i][j];
19     }
20     if (zona == 3)
21     {
22         for (int i = dim / 2 + 1; i <= dim - 1; i++)
23             for (int j = dim - i; j <= i - 1; j++)
24                 suma += mat[i][j];
25     }
26     if (zona == 4)
27     {
28         for (int j = 0; j <= (dim - 1) / 2 - 1; j++)
29             for (int i = j + 1; i <= dim - 2 - j; i++)
30                 suma += mat[i][j];
31     }
32     return suma;
33 }
34 
```

Secvență de cod 4.19: Cod C++ Continut pentru main(), problema 3

```

1  citireMatriceStatica(linii, coloane, matrice);
2  /* pentru ca metoda de citire este una generica,
3  pentru o matrice oarecare, verificam mai intai
4  daca matricea este patratica
5  */
6  if (linii == coloane)
7  {
8      int dim = linii;
9      int zona;
10     do
11     {
12         cout << "introduceti codul zonei / intre 1 si 4: ";
13         cin >> zona;
14     } while (zona < 1 || zona > 4);
15
16     cout << "suma elementelor din zona : " << zona << "este ";
17     cout << sumaZoneTriunghiulare(dim, matrice, zona);
18 }
19 else
20     cout << "matricea nu este patratica \n";

```

- Verificarea corectitudinii (în acest caz) se face prin diverse testări.
- Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu n linii și coloane, adică complexitatea este $O(n^2)$. Din

punct de vedere al timpului de execuție, se parcurg toate elementele triunghiurilor. În cazul triunghiului din nord, pentru că indicele j ia valori condiționate de valoarea lui i , în for-ul pentru j se va intra: de $n - 2$ ori dacă $i = 0$, de $n - 4$ ori dacă $i = 1$, de $n - 6$ ori dacă $i = 2$ etc Se obține astfel o progresie aritmetică de rație 2: $n - 2, n - 4, n - 6, \dots$ care se termină fie în 1, fie în 2. În funcție de cum este n (impar, sau par). Presupunând n impar avem:

$$n - 2 + n - 4 + n - 6 + \dots + 1 = \frac{(n - 1)(n - 1)}{4}$$

Folosind formula cunoscută pentru calculul sumei unei progresii, obținem că algoritmul are complexitatea $\Theta(n^2)$. Similar se calculează complexitatea și pentru celelalte situații.

Pentru problema 4 avem:

- Date de intrare: o matrice cu numere întregi și numărul ei de linii și coloane;
- Date de ieșire: un șir care constă în elementele matricei afișate în spirală.
- Algoritm de rezolvare: Se poate observa că parcurgerea în spirală presupune parcurgerea marginilor matricei. După o parcurgere completă a marginilor de sus, de la dreapta, de jos și apoi de la stânga, rămânem cu o matrice de dimensiunea mai mică cu 2. Se poate observa de asemenea că:
 - Marginea de sus și cea de jos se parcurg între aceleași valori ale coloanelor (doar că în sensuri opuse). Aceasta ne poate duce cu gândul să fixăm o limită stânga și una dreapta, care după fiecare iterație vor scădea cu 1;
 - Marginea dreapta și cea de jos se parcurg între aceleași valori ale liniilor (doar că în sensuri opuse). Aceasta ne poate duce cu gândul să fixăm o limită sus și una dreapta, care după fiecare iterație vor scădea cu 1;

Vom face parcurgerea matricei pe chenar, în ordinea: linia de sus, coloana din dreapta, linia de jos, coloana din stânga. La fiecare iterație limitele stabilite vor scădea cu câte o unitate:

- Algoritmul scris în C++ implementează ideea expusă anterior. Metoda afișează elementele matricei, în ordinea dorită.

Secvență de cod 4.20: Cod C++ pentru problema 4

```
1 void parcurgereMatriceSpirala (int linii, int col, int mat[20][20])
2 {
```

```

3      int stanga = 0, dreapta = col-1;
4      int sus = 0, jos = linii - 1;
5      while (stanga <= dreapta && sus <= jos)
6      {
7          // afisam linia de sus
8          for (int i = stanga; i <= dreapta; i++)
9          {
10             cout << mat[sus][i] << " ";
11          }
12
13          // afisam coloana din dreapta
14          //avand grija sa nu afisam si
15          // coltul dreapta sus inca o data
16          for (int i = sus + 1; i <= jos-1; i++)
17          {
18             cout << mat[i][dreapta] << " ";
19          }
20
21          // afisam linia de jos daca ea nu coincide
22          // cumva cu cea de sus
23          if (sus < jos)
24          {
25             for (int i = dreapta; i >= stanga; i--)
26             {
27                 cout << mat[jos][i] << " ";
28             }
29          }
30
31          // afisam coloana stanga daca ea nu coincide
32          // cumva cu cea dreapta
33          //avand grija sa nu afisam si
34          // coltul de stanga jos inca o data
35          if (stanga < dreapta)
36          {
37             for (int i = jos-1; i >= sus+1; i--)
38             {
39                 cout << mat[i][stanga] << " ";
40             }
41          }
42          stanga++;
43          sus++;
44          dreapta--;
45          jos--;
46      }
47  }
48
49  int main()
50  {
51      int matrice[20][20];
52      int linii, coloane;
53      citireMatriceStatice(linii, coloane, matrice);
54      parcurgereMatriceSpirala(linii, coloane, matrice);
55  }
56

```

- Metoda propusă parcurge toate elementele matricei, deci va avea complexitate $O(mn)$.

Pentru problema 5 avem:

- Date de intrare: o matrice pătratică și dimensiunea ei;
- Date de ieșire: un șir care constă în elementele matricei afisate în zig - zag.
- Algoritm de rezolvare: Dacă pe exemplul dat, trasați linii între elementele afișate în șir, se poate vedea că parcurgerea matricei se face pe linii oblice paralele cu diagonala secundară a matricei, în sensuri alternative: o dată de jos în sus, apoi de sus în jos. Aceste paralele sunt caracterizate de o sumă constantă indicilor $i + j = k$, unde

$k = 0, 1, \dots, 2(n-1)$. Ceea ce înseamnă ca cel mai ușor mod de a descrie aceste paralele este prin intermediul lui k . El va fi cel după care vom itera. Se observă că pentru paralelele cu suma $k \leq n-1$, una dintre coordonate porneste mereu de la 0, fie indicele de linie, fie cel de coloană. Iar cealaltă, în contrapartidă, porneste mereu de la valoarea lui k și trebuie să ajungă la 0. Pe paralelele aflate sub diagonala secundară, un indice (fie el cel de linie, fie de coloană) va porni mereu de la valoarea $n-1$. Celălalt indice va porni în schimb de la valoarea lui k din care se scade valoarea indicelui fixat, și va merge până la $n-1$.

- Algoritmul scris în C++ implementează ideea expusă anterior. Metoda afișează elementele matricei, în ordinea dorită.

Secvență de cod 4.21: Cod C++ pentru problema 5

```

1 void parcurgereMatriceZigZag(int dim, int mat[20][20])
2 {
3     //variabila booleana sus da directia de parcurs a diagonalelor
4     bool sus = false;
5     int i, j;
6     for( int k = 0; k <= 2*(dim - 1) ; k++)
7     {
8         // daca suntem deasupra diagonalei secundare
9         if (k <= dim - 1)
10        {
11            if (sus)
12            {
13                for (i = k, j = 0; i >= 0; i--, j++)
14                    cout << mat[i][j] << " ";
15                sus = false;
16            }
17            else
18            {
19                for (j = k, i = 0; j >= 0; j--, i++)
20                    cout << mat[i][j] << " ";
21                sus = true;
22            }
23        }
24        // daca suntem sub diagonala secundara
25        else
26        {
27            if (sus)
28            {
29                for (i = dim-1, j = k-i; j <= dim-1; i--, j++)
30                    cout << mat[i][j] << " ";
31                sus = false;
32            }
33            else
34            {
35                for (j = dim-1, i = k-j; i <= dim-1; i++, j--)
36                    cout << mat[i][j] << " ";
37                sus = true;
38            }
39        }
40    }
41 }
42
43
44
45 int main()
46 {
47     int matrice[20][20];
48     int dim;
49     citireMatricePtratica(dim, matrice);
50     parcurgereMatriceZigZag(dim, matrice);
51 }
52

```

- Metoda propusă parcurge toate elementele matricei, deci va avea com-

plexitate $O(n^2)$.

• **Alte probleme cu matrice**

1. Se dă o matrice cu m linii și n coloane.
 - Să se permute circular coloanele matricei către dreapta cu o poziție. Exemplu: Permutare cu o poziție spre dreapta:

$$Mat = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} \rightarrow Mat' = \begin{bmatrix} 4 & 1 & 2 & 3 \\ 8 & 5 & 6 & 7 \\ 12 & 9 & 10 & 11 \end{bmatrix} \quad (4.3)$$

- Să se permute circular coloanele matricei către dreapta cu k poziții, $0 \leq k \leq n$. Exemplu: Permutare cu $k = 3$ poziții spre dreapta:

$$Mat = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} \rightarrow Mat' = \begin{bmatrix} 2 & 3 & 4 & 1 \\ 6 & 7 & 8 & 5 \\ 10 & 11 & 12 & 9 \end{bmatrix} \quad (4.4)$$

2. Se dă o matrice cu m linii și n coloane. Să se rotească matricea cu 90° în sensul acelor de ceas.

Exemplu:

$$Mat = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} \rightarrow MRot = \begin{bmatrix} 9 & 5 & 1 \\ 10 & 6 & 2 \\ 11 & 7 & 3 \\ 12 & 8 & 4 \end{bmatrix} \quad (4.5)$$

3. Se dă o matrice cu m linii și n coloane care are elemente distincte. Să se interschimbe coloanele pe care se află cel mai mic și cel mai mare element al matricei. Dacă acestea se află pe aceeași coloană, nu se va face nicio interschimbare.
4. Se dă o matrice cu m linii și n coloane având elemente în mulțimea $\{0, 1\}$. Se știe că matricea are exact k elemente egale cu 1. Se citește un număr q natural. Dacă el este divizor al lui k , atunci să se împartă matricea în k/q regiuni conexe, astfel încât, valorile de 1 să fie distribuite uniform în aceste zone (cu alte cuvinte, fiecare dintre regiuni trebuie să conțină exact q valori de 1).
O regiune se numește conexă dacă oricum am alege două celule ale ei, putem ajunge din una în cealaltă fără a ieși din regiunea respectivă.
5. Sudoku: Se dă o matrice pătratică de dimensiune 9×9 și cele 81 de numere care compun un joc de sudoku. Regula acestui joc este că orice

linie, coloană și orice pătrat de 3×3 celule al grilei trebuie să conțină o singură dată fiecare cifră cuprinsă între 1 și 9. Se dă un fișier text cu un joc considerat terminat. Să se valideze conform regulii amintite.

Exemplu: Sudoku.txt : 1, 8, 5, 4, 3, 6, 2, 9, 7, 4, 3, 7, 5, 9, 2, 8, 1, 6, 6, 9, 2, 8, 1, 7, 3, 4, 5, 3, 7, 6, 2, 8, 9, 4, 5, 1, 2, 1, 4, 3, 7, 5, 9, 6, 8, 9, 5, 8, 6, 4, 1, 7, 3, 2, 8, 6, 3, 1, 2, 4, 5, 7, 9, 7, 2, 1, 9, 5, 3, 6, 8, 4, 5, 4, 9, 7, 6, 8, 1, 2, 3;

Pentru problema 6 avem:

- Date de intrare: o matrice și numărul ei de linii m și coloane n ;
- Date de ieșire: matricea, în care coloanele sunt fie permutate cu o poziție la dreapta, fie cu k poziții la dreapta;
- Algoritm de rezolvare: Permutarea circulară către dreapta (cu o poziție sau mai multe) este echivalentă cu permutarea fiecărei linii către dreapta (cu o poziție sau mai multe).

În cazul permutării cu o singură poziție către dreapta, ultimul element al liniei devine primul. El va fi reținut într-o variabilă auxiliară. Restul elementelor se deplasează la dreapta cu o poziție.

Pentru a obține o permutare circulară la dreapta cu k poziții fără a folosi altă structură ajutătoare, putem proceda astfel pentru fiecare linie:

- Se inversează ordinea elementelor între pozițiile 0 și $n - 1$;
- Se inversează ordinea elementelor între pozițiile 0 și $k - 1$;
- Se inversează ordinea elementelor între pozițiile k și $n - 1$;
- Algoritmul scris în C++ are metode (prezentate separat) pentru fiecare dintre situațiile discutate anterior și funcția main() corespunzătoare. Alocarea dinamică a memoriei pentru matrice, citirea, afișarea matricei și de alocarea de memorie sunt facute conform metodelor din interfața prezentată anterior.

Secvență de cod 4.22: Cod C++ metoda care permuta circular cu o pozitie, problema 6

```

1 void permutareCircularaDreapta(int** mat, int linii, int col)
2 {
3     for (int i = 0; i < linii; i++) {
4         int aux = mat[i][col - 1];
5         for (int j = col - 1; j >= 0; j--)
6             mat[i][j] = mat[i][j - 1];
7         mat[i][0] = aux;
8     }
9 }
10 int main()
11 {
12     int** mat, linii, col;
13     cout << "Introduceti numarul de linii si coloane ale matricei";
14     cin >> linii >> col;
15     mat = alocareDinamicaMatrice(linii, col);

```

```

16     citireMatrice(mat, linii, col);
17     permutareCircularaDreapta(mat, linii, col);
18     afisareMatrice(mat, linii, col);
19     dealocareMatrice(mat, linii);
20
21 }

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu m linii și n coloane, adică complexitatea este $O(m \times n)$. Din punct de vedere al timpului de execuție, se parcurg toate elementele matricei și se mută la dreapta cu o poziție. Așadar complexitatea algoritmului este $O(m \times n)$.

Secvență de cod 4.23: Cod C++ metodele care permuta circular cu k pozitii, problema 6

```

1
2 // metoda inverseaza ordinea elementelor intr-un vector
3 // intre pozitiile specificate de ls si ld
4 void inversareOrdine(int* vector, int ls, int ld)
5 {
6     for (int i = ls; i < ls + (ld - ls + 1) / 2; i++)
7     {
8         swap(vector[i], vector[ls + ld - i]);
9     }
10 }
11
12 void kPermutareCircularaDreapta(int** mat, int linii, int col, int k)
13 {
14     // daca k > col, atunci:
15     k = k % col;
16     for (int i = 0; i < linii; i++) {
17         // se inverseaza ordinea elem pe linie
18         inversareOrdine(mat[i], 0, col - 1);
19         // se inverseaza ordinea elem de la 0 la k-1
20         inversareOrdine(mat[i], 0, k - 1);
21         // se inverseaza ordinea elem de la k la col-1
22         inversareOrdine(mat[i], k, col - 1);
23     }
24 }
25
26 int main()
27 {
28     int** mat, linii, col, k;
29     cout << "Introduceti numarul de linii si coloane ale matricei";
30     cin >> linii >> col;
31     mat = alocareDinamicaMatrice(linii, col);
32     citireMatrice(mat, linii, col);
33     cout << "Introduceti numarul de pozitii cu care vreti sa permutati (>0)";
34     cin >> k;
35     kPermutareCircularaDreapta(mat, linii, col, k);
36     afisareMatrice(mat, linii, col);
37     dealocareMatrice(mat, linii);
38
39 }

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu m linii și n coloane, adică complexitatea este $O(m \times n)$. Din punct de vedere al timpului de execuție: se parcurg toate liniile matricei. Pe fiecare linie se face o inversare totală a elementelor și apoi o inversare pe bucățile cuprinse între indicii $0 - k - 1$, respectiv $k - n - 1$. Aceasta, pe fiecare linie presupune un număr de operații proportional cu $2 * n/2$. Așadar complexitatea algoritmului rămâne $O(m \times n)$.

Pentru problema 7 avem:

- Date de intrare: o matrice și numărul ei de linii m și coloane n ;
- Date de ieșire: matricea rotită cu 90° la dreapta ;
- Algoritm de rezolvare: Vom construi o matrice auxiliară cu ajutorul căreia vom face rotirea.

Se poate observa ușor că prin rotirea cu 90° în sensul acelor de ceas, se produc următoarele schimbări:

- Liniile matricei inițiale sunt coloane în matricea rotită, iar coloanele devin linii;
- Ordinea se schimbă: prima linie devine ultima coloană, a doua linie devine penultima coloană, ..., ultima linie devine prima coloană;
- dimensiunile matricei se schimbă între ele;

Algoritmul scris, parcurge matricea inițială și copiază valorile ei în matricea auxiliară, ținând cont de observațiile anterioare;

- Algoritmul scris în C++ prezintă metoda de rotire și funcția main() corespunzătoare. Alocarea dinamică a memoriei pentru matrice, citirea, afișarea matricei și de alocarea de memorie sunt facute conform metodelor din interfața prezentată anterior.

Secvență de cod 4.24: Cod C++ metoda care rotește o matrice, problema 7

```

1
2  /* Atentie ca in aceasta metoda folosim o referinta
3  la pointerul mat pentru ca atunci cand se intra in metoda
4  rotire sa nu se faca o copie a acestuia. Daca s-ar face
5  copie, toate operatiile ar fi facute pe acea copie, iar la iesirea
6  din metoda, pointerul mat ar referi aceeași zona de memorie
7  ca si la intrarea in metoda
8  */
9  void rotire90(int** &mat, int& linii, int& col)
10 {
11     //creem matricea auxiliara care are dimensiunile
12     //invers fata de matricea initiala
13     int** matAux = alocareDinamicaMatrice(col, linii);
14
15     //parcurem matricea initiala pe linii, punand
16     elementele liniilor pe coloanele matricei auxiliare;
17     coloanele se parcurg de la coada la cap
18     */
19
20     for (int i = 0; i < linii; i++)
21     {
22         for (int j = 0; j < col; j++)
23         {
24             //indicele de coloana din matricea veche este
25             // indice de linie in matricea noua
26             matAux[j][linii-1-i] = mat[i][j];
27         }
28     }
29     // dealocam zona de mem corespunzatoare matricei vechi
30     // si acesteia ii dam valoarea celei noi (ca si pointer)
31     dealocareMatrice(mat, linii);
32     mat = matAux;
33     // modificam si dimensiunile matricei rotite
34     swap(linii, col);
35 }
36
37 int main()
38 {

```

```

39     int** mat, linii, col, k;
40     cout << "Introduceți numărul de linii și coloane ale matricei";
41     cin >> linii >> col;
42     mat = alocareDinamicaMatrice(linii, col);
43     citireMatrice(mat, linii, col);
44     rotire90(mat, linii, col);
45     afisareMatrice(mat, linii, col);
46     dealocareMatrice(mat, linii);
47
48 }

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu m linii și n coloane, adică complexitatea este $O(m \times n)$. Din punct de vedere al timpului de execuție, se parcurg toate elementele matricei și se mută în matricea auxiliară. Așadar complexitatea algoritmului este $O(m \times n)$.

Pentru problema 3 avem:

- Date de intrare: o matrice și numărul ei de linii m și coloane n ;
- Date de ieșire: matricea în care au fost interschimbate coloanele pe care se află elementul minim și respectiv elementul maxim al matricei;
- Algoritm de rezolvare: parcurgem matricea inițială pentru a determina elementul minim/maxim și respectiv coloanele pe care se află aceste elemente poziționate. Indicii celor două coloane sunt parametri transmiși prin referință metodei care face această determinare. Odată ce am determinat cei doi indici, folosim o metodă de interschimbare a vectorilor coloană. Metoda parcurge element cu element cele două coloane și face interschimbarea valorilor elementelor.
- Algoritmul scris în C++ prezintă metoda pentru determinarea coloanei minimului/maximului, metoda de interschimbare și funcția main() corespunzătoare. Alocarea dinamică a memoriei pentru matrice, citirea, afișarea matricei și de alocarea de memorie sunt facute conform metodelor din interfața prezentată anterior.

Secvență de cod 4.25: Cod C++ metoda care interschimba coloane, problema 3

```

1
2 void determinareMinMax(int** mat, int linii, int col, int& colMin, int& colMax)
3 {
4     int min = INT_MAX;
5     int max = INT_MIN;
6
7     for (int i = 0; i < linii; i++)
8         for (int j = 0; j < col; j++)
9             {
10                if (mat[i][j] < min)
11                {
12                    min = mat[i][j];
13                    colMin = j;
14                }
15                if (mat[i][j] > max)
16                {
17                    max = mat[i][j];
18                    colMax = j;
19                }
20            }

```

```

21     }
22 }
23 void interschimbareColoane(int** mat, int linii, int& col1, int& col2)
24 {
25     for (int i = 0; i < linii; i++)
26     {
27         swap(mat[i][col1], mat[i][col2]);
28     }
29 }
30 int main()
31 {
32     int** mat, linii, col, q;
33     cout << "Introduceti numarul de linii si coloane ale matricei";
34     cin >> linii >> col;
35     mat = alocareDinamicaMatrice(linii, col);
36     citireMatrice(mat, linii, col);
37     int colMin = 0, colMax = 0;
38     determinaMinMax(mat, linii, col, colMin, colMax);
39     interschimbareColoane(mat, linii, colMin, colMax);
40     afisareMatrice(mat, linii, col);
41     dealocareMatrice(mat, linii);
42 }
43

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu m linii și n coloane, adică avem complexitate de $O(m \times n)$. Din punct de vedere al timpului de execuție, se parcurg toate elementele matricei pentru a determina poziția minimului/maximului, ceea ce înseamnă complexitate $O(m \times n)$ (deoarece interschimbarea se face în timp liniar $O(m)$).

Pentru problema 4 avem:

- Date de intrare: o matrice și numărul ei de linii m și coloane n ;
 - Date de ieșire: o matrice cu aceleași dimensiuni ca cea inițială care conține împărțirea pe zone conexe a matricei inițiale;
 - Algoritm de rezolvare: Vom construi matricea care va conține împărțirea pe zone. Pentru ca zonele să fie conexe, matricea inițială se parcurge astfel încât celula curentă să fie vecină cu cea precedentă. Pentru aceasta se parcurg liniile, în mod obișnuit, dar coloanele se parcurg o dată de la stânga la dreapta, iar apoi de la dreapta la stânga. În timpul parcurgerii avem într-o variabilă *zona* memorat indexul zonei, iar într-o variabilă *contor*, numărul de valori de 1 care au fost găsite până la momentul curent. Când acest contor ajunge la valoarea q , am terminat de parcurs o zonă - contorul revine la 0, indexul zonei crește cu 1. Numai în cazul în care zona este ultima, aceste modificări nu se mai fac.
- Algoritmul scris, parcurge matricea inițială și copiază valorile ei în matricea auxiliară, ținând cont de observațiile anterioare;
- Algoritmul scris în C++ are o metodă care numără câte valori de 1 avem în matrice, pentru ca acest lucru să nu se mai citească de la tastatură. Metoda care face împărțirea matricei în zone returnează

matricea *zone*, care ulterior este afișată în `main()`. Alocarea dinamică a memoriei pentru matrice, citirea, afișarea matricei și de alocarea de memorie sunt facute conform metodelor din interfața prezentată anterior.

Secvență de cod 4.26: Cod C++ metoda care împarte o matrice în zone problema 4

```

1  int numarareValoril(int** mat, int linii, int col)
2  {
3      int contor = 0;
4      for (int i = 0; i < linii; i++) {
5          for (int j = 0; j < col; j++) {
6              if(mat[i][j]==1)
7                  contor++;
8          }
9      }
10     return contor;
11 }
12
13 int ** impartireZone(int ** mat, int linii, int col, int totalVall, int nrVall)
14 {
15     int nrZone = totalVall / nrVall;
16     int** zone = alocareDinamicaMatrice(linii, col);
17     int zona = 1, contor = 0;
18     for (int i = 0; i < linii; i++)
19         //pe liniile pare se face parcurgerea de la
20         //stanga la dreapta
21         if (i % 2 == 0)
22             for (int j = 0; j < col; j++) {
23
24                 zone[i][j] = zona;
25                 contor += mat[i][j];
26                 if (zona < nrZone && contor == nrVall) {
27                     contor = 0;
28                     zona++;
29                 }
30             }
31     else
32         for (int j = col - 1; j >= 0; j--) {
33             zone[i][j] = zona;
34             contor += mat[i][j];
35             if (zona < nrZone && contor == nrVall) {
36                 contor = 0;
37                 zona++;
38             }
39         }
40     return zone;
41 }
42
43 int main()
44 {
45     int** mat, linii, col, q;
46     cout << "Introduceti numarul de linii si coloane ale matricei";
47     cin >> linii >> col;
48     mat = alocareDinamicaMatrice(linii, col);
49     citireMatrice(mat, linii, col);
50     int totalVall = numarareValoril(mat, linii, col);
51     cout << "numarul de elemente egale cu 1 este: " << totalVall;
52     cout << "\n cate elemente egale cu 1 vreti intr-o zona: ";
53     // k trebuie sa fie divizor al lui totalVall
54     cin >> q;
55     if (totalVall % q == 0) {
56         int** zone = impartireZone(mat, linii, col, totalVall, q);
57         afisareMatrice(zone, linii, col);
58         dealocareMatrice(zone, linii);
59     }
60     dealocareMatrice(mat, linii);
61 }
62

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează două tablouri bidimensionale cu m linii și n coloane, adică

complexitatea este $O(m \times n)$. Din punct de vedere al timpului de execuție, se parcurg toate elementele matricei inițiale și se construiește matricea *zone*. Așadar complexitatea algoritmului este $O(m \times n)$.

Pentru problema 5 avem:

- Date de intrare: o matrice de dimensiune 9×9 , reprezentând o grila de sudoku;
- Date de ieșire: o variabilă booleană care confirmă dacă grila este una validă sau nu;
- Algoritm de rezolvare: Validarea grilei de sudoku se face prin validarea separată a fiecărei linii în parte, a fiecărei coloane în parte și a fiecărui pătrat de 3×3 în parte. Metodele de validare au toate la bază ideea folosirii unui vector de frecvență pentru a verifica numărul de apariții ale fiecărei cifre dintre 1 și 9 (pe linie, coloană sau în pătrat).
- Algoritmul scris în C++ prezintă cele trei metode de validare pentru linie/coloană/ pătrat și metoda care iterează prin toate liniile/-coloanele/pătratele grilei, validându-le.

Secvență de cod 4.27: Cod C++ metoda care validează un sudoku, problema 5

```

1      #include <stdio.h>
2      #include <fstream>
3
4      void citireSudoku(int sudoku[9][9]) {
5          std::ifstream fin("sudoku.in");
6          for (int i = 0; i < 9; i++)
7              for (int j = 0; j < 9; j++)
8                  fin >> sudoku[i][j];
9      }
10     // facem validările folosind un vector de frecvență
11
12     bool validePatrat(int sudoku[9][9], int iColt, int jColt)
13     {
14         int ap[10] = { 0 };
15         // se considera patrate de dim 3x3 avand coordonatele
16         // coltului stanga sus specificate pe param metodei
17         for (int i = iColt; i < iColt + 3; i++)
18             for (int j = jColt; j < jColt + 3; j++)
19                 ap[sudoku[i][j]]++;
20         for (int i = 1; i <= 9; i++)
21             if (ap[i] > 1)
22                 return false;
23         return true;
24     }
25     bool valideLinie(int sudoku[9][9], int linie)
26     {
27         int ap[10] = { 0 };
28         for (int i = 0; i < 9; i++)
29             ap[sudoku[linie][i]]++;
30         for (int i = 1; i <= 9; i++)
31             if (ap[i] > 1)
32                 return false;
33         return true;
34     }
35     bool valideColoana(int sudoku[9][9], int col) {
36         int ap[10] = { 0 };
37         for (int i = 0; i < 9; i++)
38             ap[sudoku[i][col]]++;
39         for (int i = 1; i <= 9; i++)
40             if (ap[i] > 1)
41                 return false;
42         return true;

```

```

43 }
44 void validareJoc() {
45     int sudoku[9][9];
46     citireSudoku(sudoku);
47     bool E = true;
48     for (int i = 0; i < 9; i++) {
49         if (!validareColoana(sudoku, i) || !validareLinie(sudoku, i))
50             E = false;
51     }
52
53     for (int i = 0; i < 9; i += 3)
54         for (int j = 0; j < 9; j += 3)
55             if (!validarePatrat(sudoku, i, j))
56                 E = false;
57     if (E == true)
58         printf("Tabela este valida.\n");
59     else
60         printf("Tabela nu este valida.\n");
61 }
62 int main() {
63     validareJoc();
64     return 0;
65 }

```

- Analiza complexității. Din punct de vedere al spațiului de memorie, se utilizează un tablou bidimensional cu 9 linii și 9 coloane, adică avem complexitate constantă $O(1)$. Din punct de vedere al timpului de execuție, se parcurg în cazul cel mai defavorabil toate elementele matricei, dar și complexitatea de timp este constantă $O(1)$.

4.3 Tablouri bidimensionale - probleme propuse

1. Se consideră o matrice oarecare. Să se determine numărul de coloane cu toate elementele pozitive.
2. Se consideră o matrice oarecare. Să se determine transpusa ei.
3. Se consideră o matrice pătratică. Să se determine care sunt elementele de pe diagonala principală a matricei care au cel mai mare număr de cifre.
4. Se dau două matrice de dimensiuni $m \times n$ și respectiv $n \times p$. Să se calculeze produsul celor două matrice date.
5. Se consideră o matrice pătratică. Să se afișeze maximul dintre sumele de pe fiecare pătrat concentric al matricei.
6. Se consideră o matrice oarecare. Să se șteargă liniile din matrice care au toate elementele numere cu suma cifrelor pară.
7. Se consideră o matrice oarecare. Să se insereze înaintea fiecărei coloane care are primul element palindrom, o nouă coloană care să conțină cel mai mare divizor comun al elementelor coloanei respective.
8. Se dă o matrice de dimensiune $m \times n$. Să se verifice câte elemente ale matricei au suma vecinilor orizontali, verticali și diagonali divizibilă la 5.

9. Se consideră o matrice oarecare. Să se ordoneze crescător elementele de pe fiecare linie a matricei date.
10. Se consideră o matrice oarecare. Să se ordoneze descrescător în funcție de prima cifră, elementele de pe fiecare coloană a matricei.
11. Se consideră o matrice pătratică. Să se completeze elementele ei astfel: pe diagonala principală elementele sunt egale cu 1, pe semidiagonalele alăturate diagonalei principale elementele au valoarea 2, pe următoarele semidiagonale valoarea 3 etc. Exemplu:

$$Mat = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 2 & 3 \\ 3 & 2 & 1 & 2 \\ 4 & 3 & 2 & 1 \end{bmatrix} \quad (4.6)$$

12. Se consideră o matrice oarecare. Să se rotească matricea cu 90 de grade, în sens trigonometric.
13. Se consideră o matrice oarecare. Să se rotească matricea cu 180 de grade, în sens trigonometric.
14. Să se completeze o matrice pătratică cu numerele lui Fibonacci, pe paralele la diagonala secundară și începând din colțul stânga sus. Fiecare paralelă va fi parcursă de sus în jos.
15. Se consideră o matrice pătratică. Să se verifice dacă există cel puțin o paralelă la diagonala principală care nu conține elemente prime.
16. Se consideră o matrice pătratică. Să se numere câți termeni ai șirului lui Fibonacci sunt în triunghiul din vest.
17. Se consideră o matrice pătratică. Să se afișeze numerele cu număr maxim de cifre pare aflate în triunghiul din nord.
18. Se consideră o matrice pătratică. Să se calculeze cmmdc al valorilor care au prima cifră pară, aflate în triunghiul din est.
19. Se consideră o matrice oarecare. Să se permute circular, spre stanga, fiecare linie a matricei cu un număr de poziții egal cu prima cifră a ultimului element de pe linie.
20. Un etaj al unei clădiri este reprezentat schematic sub forma unei matrice ce conține valorile -1 și 0, unde -1 reprezintă zid iar 0 reprezintă spațiu liber. Pereții sunt de grosime 1 și ușile nu sunt marcate (se consideră ca și pereții). În aceste condiții:

- Să se determine numărul de încăperi ale etajului respectiv;
 - Să se determine încăperea cu suprafață maximă;
 - Să se determine ce perete poate fi dărâmat (o poziție marcată cu -1 va fi marcată cu 0) astfel încât să se obțină o încăpere cu suprafață maximă?
21. Într-un magazin de animale sunt așezate $m \times n$ acvarii (identice ca dimensiuni), dispuse pe m linii, și n coloane adiacente. Fiecare acvariu conține câte un cameleon. Cameleonii pot fi la momentul inițial de culoare verde (codificată cu 0) sau galbenă (codificată cu 1). La fiecare moment de timp $t = 1, 2, 3, \dots$ cameleonii își schimbă simultan culoarea după următoarea regulă: fiecare cameleon se colorează în culoarea pe care au avut-o la momentul $t-1$ majoritatea cameleonilor din acvariile învecinate (sunt cel mult 8 acvarii vecine). În cazul în care numărul cameleonilor vecini verzi este egal cu numărul cameleonilor vecini galbeni, cameleonul studiat își va păstra culoarea. Se cere:
- Să se determine și să se afișeze (în fișier sau în consolă) culorile cameleonilor la un anumit moment $t > 0$.
 - Să se determine (într-o manieră eficientă) zona - de forma dreptunghiulară cea mai mare, care conține cameleoni de aceeași culoare.
22. Matrice rare. Se consider o matrice A de dimensiuni $n \times n$ cu p elemente nenule (p mult mai mic decât n^2). O astfel de matrice se mai numește și matrice rară. Matricea este memorată economic sub forma unui vector de structuri. Structura conține trei componente: $\{Va, La, Ca\}$, care rețin valoarea unui element nenul, și respectiv indicele de linie și de coloană, adică poziția elementului în matrice. Să se scrie un program care din vectorul cu componente de tip $\{Va, La, Ca\}$ reface matricea A . În plus, pentru două matrice rare A și B memorate prin $\{Va, La, Ca\}$ și respectiv $\{Vb, Lb, Cb\}$ să se calculeze suma, fără a reconstitui matricile, și folosind doar memorarea economică.

Capitolul 5

Algoritmi de căutare și sortare

Operația de ordonare a unor componente în funcție de diverse criterii este foarte des întâlnită în practică. Se cunosc o mulțime de algoritmi de sortare, majoritatea având o logică destul de simplă. În acest capitol vom prezenta câteva metode de sortare mai speciale (criteriul de selecție fiind complexitatea scăzută), și anume Quick Sort (sortarea rapidă) și Merge Sort (sortare prin interclasare). În secțiunea de sortări vom documenta și maniera în care lucrăm cu sortarea în C++, punând accent pe biblioteca `<algorithm>`, dar și pe maniera în care folosim algoritmul `sort` din această bibliotecă (definirea unui criteriu de comparare folosind comparator, definirea unui criteriu de comparare folosind supraîncărcarea unui operator, definirea unui criteriu de comparare folosind expresie lambda).

O subproblemă des întâlnită în practică este și căutarea. În acest capitol vom studia și tipurile de căutare clasice: secvențială și binară, însă vom trata și câteva variante diverse de căutare după criterii multiple.

5.1 Tipuri de algoritmi de căutare. Aspecte teoretice

Presupunem că avem o colecție de obiecte $R_1, R_2, \dots, R_n$, iar fiecare obiect este identificat printr-o caracteristică numită cheie. Pentru o valoare dată k , problema pe care o rezolvă algoritmi de căutare este găsirea obiectului din colecție care are cheia k . Căutarea poate fi cu succes, dacă obiectul cu cheia k a fost localizat, sau fără succes, dacă nu a fost găsit nici un obiect cu cheia k . Structurile de date ce pot fi utilizate pentru a stoca colecția de obiecte sunt tablourile, listele înlănțuite sau arborii de căutare. Algoritmii sunt clasificați în funcție de mecanismul de căutare.

5.1.1 Căutare secvențială

Căutarea secvențială presupune parcurgerea secvențială a unui tablou unidimensional și compararea fiecărui element din tablou cu elementul căutat până

când se găsește. Complexitatea unui astfel de procedeu, luând în calcul un tablou de lungime n este:

- Caz favorabil - elementul căutat este pe prima poziție, atunci $\Theta(1)$;
- Caz defavorabil - elementul căutat nu se găsește în tablou (sau există pe ultima poziție) - trebuie să parcurgem toate elementele, atunci avem $\Theta(n)$;
- Caz mediu: există $n+1$ situații posibile, probabilitate $\frac{1}{n+1}$, atunci complexitatea este: $\frac{1}{n+1} * 1 + \frac{1}{n+1} * 2 + \dots + \frac{1}{n+1} * n + \frac{1}{n+1} * n = \dots \approx \frac{1}{n+1} * \frac{(n+1)(n+2)}{2} = \frac{n+2}{2} = \Theta(n)$

Amintim antetul posibil pentru metoda de căutare secvențială (oferim mai multe variante, apelarea lor depinzând de contextul problemei):

Secvență de cod 5.1: Cod C++ pentru căutare secvențială (variante)

```

1 //se poate returna pozitia pe care se gaseste elementul dorit, elem
2 int cautareSecventiala1(int vector[50], int dimension, int elem)
3 //true/false, in functie de situatia elementului cautat
4 bool cautareSecventiala2(int vector[50], int dimension, int elem)
5 //se poate afisare in interiorul metodei daca elementul se gaseste sau nu in vectorul
  ↳ de numere
6 void cautareSecventiala3(int vector[50], int dimension, int elem)

```

5.1.2 Căutare binară

Căutarea binară se bazează pe tehnica de programare "Divide et Impera". Se consideră că tabloul în care se face căutarea este ordonat. Elementul căutat este "verificat" cu mijlocul vectorului: dacă elementul este egal cu mijlocul, căutarea se termină, dacă nu sunt egale, atunci se compară valoarea mijlocului cu cea a elementului de căutat. Dacă elementul este mai mare se continuă căutarea de la mijlocul listei până la sfârșit, iar dacă este mai mic se continuă căutarea de la început până la mijloc. Se continuă procesul până când găsim elementul sau ajungem să avem cazuri particulare (un singur element în subsecvență). Complexitatea unui astfel de procedeu, luând în calcul un tablou ordonat de lungime n este:

- Caz favorabil - elementul căutat este la mijlocul tabloului (este găsit pe prima poziție verificată), atunci complexitatea este $\Theta(1)$;
- Caz defavorabil - elementul se găsește pe ultima poziție verificată (sau nu se găsește deloc), atunci complexitatea este $\Theta(\log_2 n)$;
- Caz mediu, presupunem $n = 2^k$ ($\log_2 n = k$)
 - Găsim elementul la prima comparație, probabilitate $\frac{1}{n}$, număr de operații 1

- Găsim elementul la a doua comparație, probabilitate $\frac{2}{n}$, număr de operații 2
- ...
- Găsim elementul la a k-a comparație, probabilitate $\frac{2^{k-1}}{n}$, număr de operații k

Dacă luăm suma: $\sum_{i=1}^k \frac{2^{i-1}}{n} * i = \frac{1}{n} * \sum_{i=1}^k i * 2^{i-1} = \dots < \log_2 n$. Complexitate medie este $O(\log_2 n)$.

- Complexitate totală este: $O(\log_2 n)$

Atenție că pentru căutare binară există posibilitatea de a scrie o funcție iterativă, dar și o funcție recursivă. Pentru varianta recursivă, pentru calcul de complexitate se folosește Teorema Master (vezi capitol "Analiza eficienței algoritmilor. Complexități").

Amintim antetul posibil pentru metoda de căutare secvențială (oferim mai multe variante, apelarea lor depinzând de contextul problemei):

Secvență de cod 5.2: Cod C++ pentru căutare binară (variante)

```

1 //se poate returna pozitia pe care se gaseste elementul dorit, elem
2 int cautareBinara1(int vector[50], int dimension, int elem, int start, int end)
3 //true/false, in functie de situatia elementului cautat
4 bool cautareBinara2(int vector[50], int dimension, int elem, int start, int end)
5 //se poate afisare in interiorul metodei daca elementul se gaseste sau nu in vectorul
  ↪ de numere
6 void cautareBinara3(int vector[50], int dimension, int elem, int start, int end)
```

Cum căutarea binară are o complexitate logaritmică (vs căutare secvențială, unde avem complexitate liniară), putem să folosim acest tip de căutare direct din STL (Standard Template Library)¹.

Prototipul pentru căutare binară este:

binary_search(begin, end, valuetofind), unde *begin*: adresa primului element din vector, *end*: adresa ultimului element din vector, *valuetofind*: elementul căutat.

SAU

binary_search(begin, end, valuetofind, order_comparison), unde *begin*: adresa primului element din vector, *end*: adresa ultimului element din vector, *valuetofind*: elementul căutat, *order_comparison* predicat binar care poate să fie true sau false.

Secvență de cod 5.3: Cod C++ pentru căutare binară (STL, variante)

```

1 #include <algorithm>
2 int main()
3 {
4     int coef[] = { 1, 5, 8, 9, 10, 15, 30 };
5     int asize = sizeof(coef) / sizeof(coef[0]);
6
7     if (binary_search(coef, coef + 7, 15))
8         cout << "\n Element gasit";
9     else
```

¹http://www.cplusplus.com/reference/algorithm/binary_search/

```

10         cout << "\n Element NEgasit";
11     }
12     //-----
13     #include <iostream>
14     #include <algorithm>
15     #include <vector>
16     using namespace std;
17
18     bool compare_string_by_length(string i, string j)
19     {
20         return (i.size() == j.size());
21     }
22
23     int main()
24     {
25         int inputs[] = { 1, 4, 6, 7, 8, 9, 10 };
26         vector<int> v(inputs, inputs + 7);
27
28         cout << binary_search(v.begin(), v.end(), 7);
29
30         cout << binary_search(v.begin(), v.end(), 200);
31
32         vector<string> sol = { "anabela", "babau", "popa", "sebas" };
33
34         cout << binary_search(sol.begin(), sol.end(), "algo", compare_string_by_length);
35
36         /* cautam stringul care are aceeasi lungime ca "algo" */
37         return 0;
38     }
39
40 
```

5.1.3 Alte tipuri de căutări

Căutarea ternară

Căutarea ternară este o metodă de căutare, extindere a căutării binare, aplicată căutării unui element într-un tablou ordonat. Algoritmul căutării binare divide tabloul în două părți, în timp ce căutarea ternară îl împarte în trei părți. Căutarea ternară, ca și cea binară, este un algoritm ce se bazează pe tehnica de programare Divide et Impera, la fiecare iterație se neglijează două treimi din tablou și se continuă căutarea pe treimea rămasă.

Exemplu:

Pentru tabloul din Figura 5.1 cu 12 elemente și numărul căutat 13, algoritmul execută următorii pași:

- PASUL 1: se împarte tabloul în trei cu ajutorul variabilelor mijloc1 și mijloc2; elementele aflate la indecșii reprezentați de acestea nu sunt egale cu 13, secvența de căutare devine secvența cuprinsă între cele două poziții;
- PASUL 2: variabilele mijloc1 și mijloc2 împart secvența de căutare în trei; elementele aflate la indecșii reprezentați de acestea nu sunt egale cu 13, secvența de căutare devine secvența de după indexul mijloc1;
- PASUL 3: variabilele mijloc1 și mijloc2 sunt egale; 13 se află pe poziția indicată de acestea; algoritmul se termină cu succes.

Implementarea în C++ a acestui algoritm este:

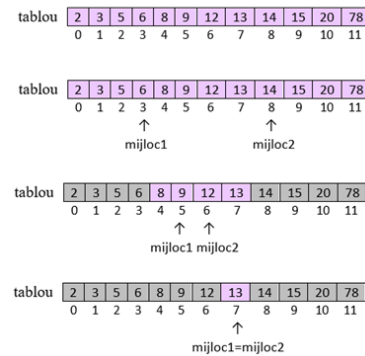


Figura 5.1: Exemplu căutare ternară

Secvență de cod 5.4: Cod C++ pentru căutare ternară

```

1
2 #include <iostream>
3
4 void citire(int tablou[], int &lungime)
5 {
6     std::cout << "Introduceți lungimea sirului: ";
7     std::cin >> lungime;
8     std::cout << "Introduceți un sir cu " << lungime << " elemente sortate crescator:\n";
9     for (int index = 0; index < lungime; index++)
10         std::cin >> tablou[index];
11 }
12
13 int CautareTernara(int tablou[], int lungime, int element)
14 {
15     int start = 0; //inceputul secventei de cautare
16     int end = lungime - 1; //sfarsitul secventei de cautare
17     int mijloc1, mijloc2;
18     while (start <= end)
19     {
20         //impartirea secventei de cautare in 3
21         mijloc1 = start + (end - start) / 3;
22         mijloc2 = end - (end - start) / 3;
23         if (tablou[mijloc1] == element) return mijloc1;
24         if (tablou[mijloc2] == element) return mijloc2;
25         if (element < tablou[mijloc1])
26             //elementul se afla in prima treime a secventei
27             end = mijloc1 - 1;
28         else
29             if (element > tablou[mijloc2])
30                 //elementul se afla in ultima treime a secventei
31                 start = mijloc2 + 1;
32             else
33                 //elementul se afla in partea din mijloc a secventei
34                 {
35                     start = mijloc1 + 1;
36                     end = mijloc2 - 1;
37                 }
38     }
39     //elementul nu apartine tabloului
40     return -1;
41 }
42 void main()
43 {
44     int tablou[100], lungime;
45     citire(tablou, lungime);
46     int element;
47     std::cout << "Introduceți elementul cautat: ";
48     std::cin >> element;

```

```

49     int pozitie = CautareTernara(tablou, lungime, element);
50     if (pozitie >= 0) std::cout << "Elementul se afla pe pozitia " << pozitie+1 << '\n'
51         ← ;
52     else std::cout << "Elementul nu apartine sirului\n";
    }

```

Complexitatea de timp a algoritmului, în condițiile în care tabloul, respectiv secvența în care se efectuează căutarea, conține n elemente, este $O(\log_3 n)$. Prin comparație cu căutarea binară care are complexitatea $O(\log_2 n)$, căutarea ternară pare la prima vedere să aibă o complexitate mai bună. Dacă analizăm pe relația de recurență a algoritmilor numărul de operații în cazul defavorabil pentru ambele căutări vom obține:

- $T(n) = T(\frac{n}{2}) + 2$, $T(1) = 1$ rezultă $2 * \log_2 n$ operații în cazul căutării binare;
- $T(n) = T(\frac{n}{3}) + 4$, $T(1) = 1$ rezultă $4 * \log_3 n$ operații în cazul căutării ternare;

Cum $2 * \log_3 2 > 1$ înseamnă că se execută, în total, mai multe operații în cazul căutării ternare, de aceea căutarea binară este preferată de cele mai multe ori.

O aplicație a căutării ternare este găsirea minimului unei funcții reale $f : [a, b] \rightarrow \mathbb{R}$ continuă și unimodală (care admite un singur minim relativ pe intervalul de definiție). Se notează $x^* = \inf(f(x))$ cu $x \in [a, b]$. Fie două puncte $m_1, m_2 \in [a, b]$ astfel încât $a \leq m_1 \leq m_2 \leq b$, atunci sunt posibile următoarele situații:

$$\begin{cases} f(m_1) < f(m_2), x^* \in [m_1, b] \\ f(m_1) = f(m_2), x^* \in [m_1, m_2] \\ f(m_1) > f(m_2), x^* \in [a, m_2] \end{cases}$$

Așadar, funcția odată evaluată în două puncte, intervalul de căutare a minimului se poate restrânge. Scopul este ca după un număr fixat N de evaluări ale funcției să obținem un interval de incertitudine cât mai mic posibil pentru x^* . Algoritmul pentru rezolvare utilizează principiul de împărțire al căutării ternare. Similar se procedează pentru găsirea maximului funcției.

Căutarea prin salturi

Căutarea prin salturi sau căutarea în blocuri se referă la un algoritm de căutare într-un șir sortat. Ideea de bază constă în verificarea a cât mai puține elemente prin salturi înainte cu pași ficși sau sărind peste anumite elemente în locul verificării tuturor elementelor. Să presupunem că avem un șir $arr[]$ de mărime n și un bloc (care trebuie sărit) de mărime m . Apoi căutăm la indicii $arr[0], arr[m], arr[2 * m], \dots, arr[k * m]$ și așa mai departe. Odată ce găsim intervalul $(arr[k * m] < x < arr[(k + 1) * m])$, efectuăm o operație de căutare liniară din indexul $k * m$ pentru a găsi elementul x .

Exemplu:

Să luăm în considerare următorul șir: (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610). Lungimea șirului este 16. Rezultatul căutării va găsi valoarea de 55 cu următorii pași, presupunând că dimensiunea blocului care urmează să fie sărită este de 4.

- PASUL 1: Salt de la indexul 0 la indexul 4;
- PASUL 2: Salt de la indexul 4 la indexul 8;
- PASUL 3: Salt de la indexul 8 la indexul 16;
- PASUL 4: Deoarece elementul de la indexul 16 este mai mare de 55, vom face un pas înapoi pentru a ajunge la indexul 9;
- PASUL 5: efectuăm o căutare liniară din indexul 9 pentru a obține elementul 55.

Complexitatea de timp a algoritmului se determină luând în calcul numărul operațiilor efectuate, care este: $\frac{n}{m} + m - 1$, unde m este mărimea saltului, iar n este lungimea șirului în care se face căutarea. Ca să aflăm pentru ce valoare a lui m această expresie are valoare minimă, diferențiem în funcție de m : $\frac{n}{-m^2} + 1 = 0 \Rightarrow n = m^2 \Rightarrow m = \sqrt{n}$. Astfel, valoarea optimă pentru m este $\sqrt{n}$. Complexitatea de timp a acestui algoritm este, precum am prezentat mai sus de $O(\sqrt{n})$. Astfel, algoritmul căutării prin salturi este mai rapid decât cel al căutării secvențiale, dar mai lent decât cel al căutării binare. Algoritmul poate fi modificat prin efectuarea căutărilor prin salturi pe mai multe nivele, înainte de a efectua, în cele din urmă, căutarea liniară. Pentru o căutare la nivel k , dimensiunea optimă a blocului pentru nivelul 1 (numărând de la 1) este $n^{\frac{k-1}{k}}$. Algoritmul modificat va efectua k salturi înapoi și se execută în timpul $O(k * n^{\frac{1}{k-1}})$. Un avantaj al căutării prin salturi este complexitatea mai mică decât cea a căutării secvențiale, dar un neajuns ar fi acela că funcționează doar pentru șirurile sortate.

Implementarea în limbajul C++ a acestui algoritm este:

Secvență de cod 5.5: Cod C++ pentru căutare prin salturi

```

1  #include <iostream>
2  #include <math.h>
3  int min(int a, int b)
4  {
5      return (a < b) ? a : b;
6  }
7  int jumpSearch(int arr[], int x, int dim)
8  {
9      // calcularea pasului de sarit in cautare
10     int step = (int)sqrt(dim);
11
12     // cautarea blocului in care se afla elementul (daca este prezent)
13     int prev = 0;
14     while (arr[min(step, dim) - 1] < x)
```

```

15     {
16         prev = step;
17         step +=(int)sqrt(dim);
18         if (prev >= dim)
19             return -1;
20     }
21
22     //cautarea liniara a elemetului in bloc, pornind de la prev
23     while (arr[prev] < x)
24     {
25         prev++;
26         // daca ajungem la sfarsitul blocului inseamna ca elemntul nu exista
27         if (prev == min(step,dim))
28             return -1;
29     }
30     // daca elementul exista
31     if (arr[prev] == x)
32         return prev;
33
34     return -1;
35 }
36 void read(int arr[], int &dim)
37 {
38     std::cout << "Dati dimensiunea sirului\n";
39     std::cin >> dim;
40     std::cout << "Dati sirul sortat crescator\n";
41     for (int index = 0; index < dim; index++)
42         std::cin >> arr[index];
43 }
44 void main()
45 {
46     int arr[100], dim, x;
47     read(arr, dim);
48     std::cout << "Dati elementul cautat\n";
49     std::cin >> x;
50     int index=jumpSearch(arr, x, dim); //index va lua pozitia lui x
51     if (index >= 0)
52         std::cout << "Elementul " << x << " se afla pe pozitia " << index<<"\n";
53     else
54         std::cout << "Elementul nu se afla in sir\n";
55 }

```

Căutarea prin interpolare

Căutarea prin interpolare este un algoritm pentru căutarea unei chei date într-un șir indexat care a fost ordonat. Ea este similară cu modul în care oamenii caută printr-un dicționar un anumit cuvânt. O persoana care caută, de exemplu, un cuvânt care începe cu litera "S" va presupune, în general, că acesta se afla în ultimul sfert din dicționar. Astfel, ea va deschide mai întâi acea parte și apoi va lua o decizie bazată pe ceea ce descoperă cu privire la unde să caute în continuare. Cu alte cuvinte, oamenii utilizează de obicei câteva cunoștințe despre distribuția așteptată a valorilor cheie pentru a "calcula" unde să se uite în continuare. În fiecare etapă de căutare se calculează unde, în spațiul de căutare rămas, elementul căutat ar putea fi, pe baza valorilor de la limitele spațiului de căutare și a valorii cheii căutate, de obicei printr-o interpolare liniară. Valoarea cheie găsită la această poziție estimată este apoi comparată cu valoarea cheie solicitată. Dacă nu sunt egale, atunci, în funcție de comparație, spațiul de căutare rămas este redus la partea dinainte sau de după poziția estimată.

Pentru a găsi poziția în care să se realizeze căutare se utilizează următoarea

formulă:

$$pos = low + \left[\frac{(x - arr[low]) * (high - low)}{(arr[high] - arr[low])} \right]$$

unde x este cheia căutată, pos este poziția posibilă, iar $high$ și low sunt limitele superioară și inferioară ale spațiului de căutare. Formula caută să returneze o valoare mai mare pentru pos dacă x este mai apropiat de $arr[high]$ și mai mică dacă acesta este mai apropiat de $arr[low]$. Algoritmul de căutare prin interpolare este similar cu cel de căutare binară cu excepția partițiilor:

- PASUL 1: într-un ciclu, se calculează valoarea lui pos folosind formula de mai sus;
- PASUL 2: Dacă este o potrivire, se returnează indexul elementului și se iese din buclă;
- PASUL 3: Dacă elementul căutat este mai mic, se va calcula poziția posibilă în subșirul stâng, altfel în cel drept;
- PASUL 4: Se repetă până când cheia este găsită sau până când lungimea subșirului rămas se reduce la zero.

Complexitatea de timp a algoritmului de căutare prin interpolare este, în cazul favorabil (atunci când elementele sunt uniform distribuite), $O(\log(\log n))$. În cazul defavorabil, de exemplu când elementele cresc exponențial, complexitatea poate ajunge la $O(n)$.

Astfel, în comparație cu algoritmul de căutare binară care merge mereu în mijlocul subșirului și care are o complexitate de $O(\log n)$, căutarea prin interpolare poate da un randament mai bun *în cazul șirurilor uniform distribuite*. Acest lucru se poate observa și din graficul din Figura 5.2 care reprezintă funcțiile $\log n$ și $\log(\log n)$. Avantajul major al acestui tip de căutare este dat de complexitatea bună din cazul șirurilor uniform distribuite, dar limitarea acesteia constituie un neajuns.

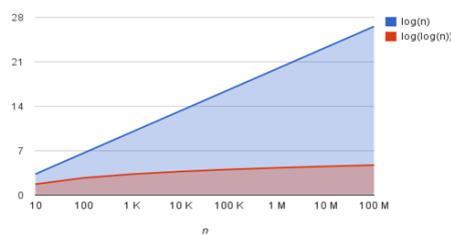


Figura 5.2: Reprezentare grafică a funcțiilor $\log n$ și $\log(\log n)$

Implementarea în limbajul C++ a acestui algoritm este:

Secvență de cod 5.6: Cod C++ pentru căutare prin interpolare

```

1  #include <iostream>
2  int interpolationSearch(int arr[], int x, int dim)
3  {
4      //initializam capetele
5      int low = 0;
6      int high = dim - 1;
7      //elementul cautat trebuie sa se afle intre capete, daca nu returnam -1
8      while (low <= high && x >= arr[low] && x <= arr[high])
9      {
10         //calculam pozitia posibila in functie de apropierea elementului cautat de unul
11         //↪ dintre capete
12         int pos = low + (((high - low) / (arr[high] - arr[low])) * (x - arr[low]));
13         //daca elementul de pe pozitia pos corespunde
14         if (arr[pos] == x)
15             return pos;
16         //daca x este mai mare continuam cautarea in partea superioara
17         if (arr[pos] < x)
18             low = pos + 1;
19         //iar daca este mai mic in partea inferioara
20         else
21             high = pos - 1;
22     }
23     return -1;
24 }
25 void read(int arr[], int &dim)
26 {
27     std::cout << "Dati dimensiunea sirului\n";
28     std::cin >> dim;
29     std::cout << "Dati sirul sortat crescator\n";
30     for (int index = 0; index < dim; index++)
31         std::cin >> arr[index];
32 }
33 void main()
34 {
35     int arr[100], dim, x;
36     read(arr, dim);
37     std::cout << "Dati elementul cautat\n";
38     std::cin >> x;
39     int index = interpolationSearch(arr, x, dim); //index va lua pozitia lui x
40     if (index >= 0)
41         std::cout << "Elementul " << x << " se afla pe pozitia " << index << "\n";
42     else
43         std::cout << "Elementul nu se afla in sir\n";
44 }

```

Căutarea exponențială

Căutarea exponențială este un algoritm utilizat pentru a efectua o căutare într-un șir sortat infinit, dar poate fi utilizat și pe șiruri mărginite. Ideea de bază a acestui algoritm este determinarea intervalului în care se află numărul căutat și aplicarea căutării binare pe intervalul identificat. Intervalul corespunde subșirului de la indexul 2^{k-1} la indexul 2^k , k fiind primul exponent pentru care elementul de la indexul 2^k este mai mare decât numărul căutat.

Exemplu:

Aplicăm pas cu pas algoritmul de căutare exponențială pentru tabloul din Figura 5.3 cu 15 elemente și numărul căutat 7. Se execută următorii pași:

- PASUL 1: (prima secvență din Figura 5.3) exponentul k este 0, elementul de la indexul $2^0 - 1$ nu este mai mare decât 7, deci indexul crește;
- PASUL 2: exponentul k este 1, elementul de la indexul $2^1 - 1$ nu este mai mare decât 7, deci indexul crește;

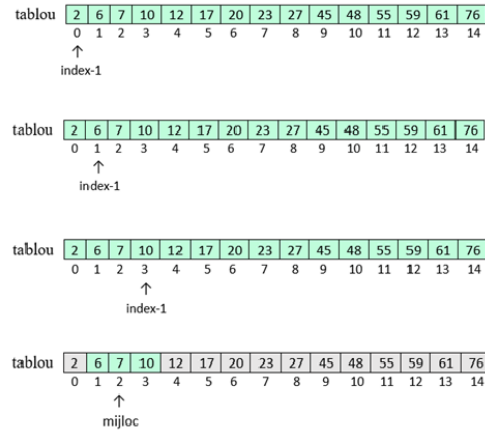


Figura 5.3: Exemplu căutare exponențială

- PASUL 3: exponentul k este 2, elementul de la indexul $2^2 - 1$ este mai mare decât 7, deci se execută căutare binară pe intervalul de la indexul $2^1 - 1$ la indexul $2^2 - 1$;
- PASUL 4: secvența de căutare este de la 1 la 3; variabila mijloc Împarte În două secvența; 7 se află pe poziția indicată de acesta; algoritmul se termină cu succes.

Complexitatea de timp a algoritmului de căutare exponențială, în condițiile în care i reprezintă poziția pe care se află elementul căutat (sau pe care ar trebui să se afle, dacă acesta nu aparține șirului), este $O(\log_2 i)$. Prima etapă a algoritmului, care constă în identificarea intervalului de apartenență a elementului căutat, are complexitatea $O(\log_2 i)$ deoarece se caută primul index 2^k cu elementul corespunzător mai mare decât cel căutat, deci $\log_2 i < k \leq \log_2 i + 1$. A doua etapă a algoritmului constă în efectuarea unei căutări binare în secvența de la indexul 2^{k-1} la indexul 2^k . Cum complexitatea algoritmului de căutare binară este $O(\log_2 n)$, unde n este lungimea secvenței de căutare, (în acest caz $n = 2^{(k-1)}$, de la 2^{k-1} la 2^k), complexitatea celei de a doua părți a algoritmului este $O(\log_2 i)$.

În concluzie, complexitatea totală a algoritmului de căutare exponențială este egală cu $O(\log_2 i) + O(\log_2 i) = 2 * O(\log_2 i) = O(\log_2 i)$. Astfel, acest algoritm poate avea performanțe mai bune decât algoritmul de căutare binară, *dacă poziția elementului căutat este mai aproape de începutul șirului*. O caracteristică importantă a acestui algoritm este aplicabilitatea pe șiruri ordonate infinite.

Implementarea În limbajul C++ a acestui algoritm este:

Secvență de cod 5.7: Cod C++ pentru căutare exponențială

```

1  #include <iostream>
2
3  void citire(int tablou[], int &lungime)
4  {
5      std::cout << "Introduceti lungimea sirului: ";
6      std::cin >> lungime;
7      std::cout << "Introduceti un sir cu " << lungime << " elemente sortate crescator:\n";
8      for (int index = 0; index < lungime; index++)
9          std::cin >> tablou[index];
10 }
11
12 int CautareBinara(int tablou[], int start, int end, int element)
13 {
14     //start este inceputul secventei de cautare
15     //end este sfarsitul secventei de cautare
16     int mijloc;
17     while (start <= end)
18     {
19         //impartirea secventei de cautare in 2
20         mijloc = (start + end) / 2;
21         if (tablou[mijloc] == element) return mijloc;
22         if (element < tablou[mijloc])
23             //elementul se afla in prima jumatate a secventei
24             end = mijloc - 1;
25         else
26             //elementul se afla in a doua jumatate a secventei
27             start = mijloc + 1;
28     }
29     //elementul nu apartine tabloului
30     return -1;
31 }
32
33 int CautareExponentiala(int tablou[], int lungime, int element)
34 {
35     int index = 1;
36     while (tablou[index - 1] <= element && index < lungime)
37         index *= 2;
38     return CautareBinara(tablou, index/2-1, index-1, element);
39 }
40
41 void main()
42 {
43     int tablou[100], lungime;
44     citire(tablou, lungime);
45     int element;
46     std::cout << "Introduceti elementul cautat: ";
47     std::cin >> element;
48     int pozitie = CautareExponentiala(tablou, lungime, element);
49     if (pozitie >= 0) std::cout << "Elementul se afla pe pozitia " << pozitie + 1 << '\n';
50     else std::cout << "Elementul nu apartine tabloului\n";
51 }

```

5.2 Tipuri de algoritmi de sortare. Aspecte teoretice

O altă subproblemă des întâlnită în programare este sortarea. Un algoritm de sortare este un algoritm alcătuit dintr-o serie de instrucțiuni care au ca date de intrare o structură de date (unidimensională, bidimensională, etc), efectuează diverse operații asupra structurii respectiv și au ca date de ieșire o structură sortată după un anumit criteriu. Există mulți factori pe care trebuie să-i luăm în considerare atunci când alegem să utilizăm un algoritm de sortare, printre care eficiența (timpul de execuție și memoria folosită) și stabilitatea. Fiind o problemă des întâlnită, se pot observa diverse implementări ale algoritmilor de sortare în limbajele de programare pe care le utilizăm (de exemplu: în C++ avem sort-ul

din STL).

Avantajele cunoașterii algoritmilor de sortare diverși:

- Tehnici similare sunt eficiente în abordarea altor probleme;
- Adesea folosim algoritmi de sortare ca punct de plecare în rezolvarea altor probleme.

Bineînțeles că, la prima vedere, problema sortării pare una banală, însă abordarea poate să devină complexă, în cazul în care avem anumite pretenții de eficiență, modularitate, timp, proprietăți input, etc.

Pentru simplificarea capitolului acestuia, dar și pentru că metodele de sortare în $O(n^2)$ sunt cunoscute încă din liceu, în această secțiune vom trata algoritmi eficienți de sortare, cum sunt Quick Sort și Merge Sort, iar, în plus, vom oferi detalii despre lucrul elegant și modern cu sort-ul din C++ (cu diverse scenarii).

5.2.1 Sortarea rapidă - Quick Sort

Datorită performanțelor sale deosebite, algoritmul Quick Sort este unul dintre cei mai utilizați algoritmi de sortare până în acest moment, bazându-se pe tehnica "Divide et Impera". Quick Sort este un algoritm ce poate fi descris foarte simplu la modul teoretic, însă scrierea acestuia sub formă de cod sursă a pus întotdeauna probleme. Cercetătorii din domeniul informaticii au avut nevoie de ani de zile pentru a obține o implementare practică a acestui algoritm.

Ideea algoritmului de sortare rapidă - Quick Sort:

- Se rearanjează și apoi se împarte tabloul unidimensional, de dimensiune n , $\text{arr}[1..n]$ în două subtablouri $\text{arr}[1..m]$ și $\text{arr}[m+1..n]$ astfel încât componentele lui $\text{arr}[1..m]$ sunt mai mici decât $\text{arr}[m+1..n]$; este importantă o rearanjare anterioară, pentru a evita diverse probleme legate de complexitate
- Se sortează fiecare dintre subtablourile obținute aplicând exact aceeași strategie de împărțire
- Se concatenează subtablourile sortate

Algoritmul Quick Sort este un algoritm de sortare prin pivotaj. Astfel, subliniem câteva idei pentru construirea/alegerea unui pivot:

- Se alege o valoare aleatoare din tablou (prima, ultima sau una arbitrară), aceasta reprezentând valoarea pivotului

- Se rearanjează elementele tabloului astfel încât toate elementele care sunt mai mici decât valoarea aleasă să se afle în prima parte (stânga) a tabloului, iar valorile mai mari decât pivotul să se afle în partea a doua (dreapta) a tabloului
- Se pune valoarea pivotului pe poziția sa finală (astfel încât în stânga să avem elementele mai mici decât pivotul, iar în dreapta, elementele mai mari ca valoarea pivotului)

Un bun pivot împarte tabloul curent în două subtablouri de dimensiuni apropiate (partiționare echilibrată). Alegerea unui bun pivot este direct proporțională cu obținerea unei complexități mai bune.

O idee de rearanjare a elementelor:

- Se folosesc doi indici: unul care pornește de la poziția primul element (limităInferioară) iar celălalt care pornește de la poziția ultimului element (limităSuperioară)
- Se incrementează/decrementează indicii până când se identifică o inversiune (o pereche de indici $i < j$ cu proprietatea că $\text{arr}[i] > \text{pivot}$ și $\text{arr}[j] < \text{pivot}$)
- Se repară situația inversiunii prin interschimbarea elementelor
- Se continuă procesul până când indicii se "suprapun"

Implementarea în limbajul C++ a acestui algoritm este:

Secvență de cod 5.8: Cod C++ pentru metoda de sortare rapidă (Quick Sort)

```

1 void QuickSort(int arr[], int left, int right)
2 {
3     int i = left, j = right;
4     int tmp;
5     int pivot = arr[(left + right) / 2];
6     /*Partiționare */
7     while (i <= j) {
8         while (arr[i] < pivot)
9             i++;
10        while (arr[j] > pivot)
11            j--;
12        if (i <= j) {
13            tmp = arr[i];
14            arr[i] = arr[j];
15            arr[j] = tmp;
16            i++;
17            j--;
18        }
19    }
20
21    /* Recursivitate */
22
23    if (left < j)
24        Quick Sort(arr, left, j);
25    if (i < right)
26        Quick Sort(arr, i, right);
27 }
```

Complexitatea algoritmului de pivotaj se obține luând în calcul: dimensiunea problemei ($n = \text{limităSuperioară} - \text{limităInferioară} + 1$) și operația dominantă (comparația în care intervin elemente ale lui x); deci $T(n) = n + c$, $c = 0$ dacă $i = j$ și $c = 1$ dacă $i = j + 1$. Deci $T(n)$ aparține lui $\Theta(n)$.

Pentru *analiza complexității* totale a algoritmului se pleacă de la două situații generate de poziția pe care o ocupă pivotul în urma operației de partiționare. Astfel, în cel mai favorabil caz, pivotul este plasat în cadrul fiecărei subprobleme pe poziția din mijloc a subtabloului, în această situație problema fiind descompusă în două subprobleme de aceeași dimensiune. Cum fiecare subtablou de m elemente necesită maxim m comparații, este clar că pentru un număr de m autoapeluri pentru subtablouri de dimensiune n vor fi necesare $O(n)$ comparații. Cum există $\log n$ nivele, rezultă că algoritmul Quick Sort are o complexitate $O(n \log n)$ în cazul cel mai favorabil.

Există însă situații când această complexitate este distrusă: de exemplu un tablou care are deja elementele sortate - cazul cel mai defavorabil. La descompunerea problemei inițiale se va obține o singură subproblemă care urmărește sortarea unui tablou cu $n - 1$ elemente. Deoarece numărul de nivele din recursivitate este n , complexitatea algoritmului degenerază către $O(n^2)$. Pentru a evita această deteriorare a complexității, de obicei se efectuează o permutare aleatoare a tabloului inițial înaintea aplicării algoritmului Quick Sort. De asemenea, trebuie reținut că acest algoritm de sortare presupune alocarea implicită a unei stive (datorată recursivității) de lungime ce variază între $\log n$ și n .

Câteva observații importante de reținut pentru algoritmul de sortare rapidă:

- Cu toate că este un algoritm de sortare foarte rapid, utilizarea Quick Sort poate conduce la umplerea stivei atunci când se utilizează seturi largi de date.
- Datorită complexității, algoritmul nu este o alegere practică atunci când se dorește ordonarea unor seturi de date de dimensiune redusă;
- Modul în care este ales elementul care joacă rol de pivot are un impact major asupra performanțelor algoritmului. În cazul cel mai defavorabil, ordinul de complexitate al algoritmului este egal cu $O(n^2)$ și se obține în situația în care elementele vectorului sunt deja ordonate, iar pe post de pivot se alege elementul cel mai din stânga. Dacă alegerea pivotului se face însă aleatoriu, atunci complexitatea algoritmului devine din nou egală cu $O(n * \log_2 n)$;
- Cea mai complexă problemă în cadrul algoritmului de sortare rapidă o reprezintă alegerea corectă a pivotului, căci corespunzător alegerii corecte a pivotului algoritmul se comportă eficient, iar ineficiența algoritmului este dată uneori de alegerea necorespunzătoare a pivotului, rezultând o performanță scăzută. Însă, dacă la fiecare pas de sortare se va alege un pivot corect, atunci algoritmul va funcționa după în $O(n * \log_2 n)$;

- Quick Sort este sensibil la ordinea datelor de intrare. Nu este o metodă stabilă;
- Quick Sort reprezintă unul dintre cei mai rapizi algoritmi de sortare. Datorită acestui lucru, algoritmul reprezintă o bună alegere în situațiile în care viteza este foarte importantă, indiferent de dimensiunea seturilor de date ce se doresc a fi sortate. Cu toate acestea nu trebuie scăpat din vedere faptul că algoritmul nu este stabil, iar performanțele acestuia sunt modeste atunci când operează asupra unor liste aproape ordonate;
- Quick Sort are performanțe slabe în cazul sortărilor banale, atunci când operează asupra unor liste aproape ordonate;
- Quick Sort are performanțe deosebite în cazul tablourilor dezordonate. De asemenea, dacă pivotul se alege întotdeauna la mijloc (mediana), atunci tabloul sortat invers devine cazul optim al sortării Quick Sort.

5.2.2 Sortare prin interclasare - Merge Sort

Ideea algoritmului de sortare prin interclasare - MergeSort:

- Se împarte tabloul unidimensional, $arr[1..n]$, de dimensiune n în două subtablouri $arr[1..[n/2]]$ și $arr[[n/2]+1..n]$
- Se sortează fiecare subtablou obținut mai sus
- Se interclasează elementele subtablourilor $arr[1..[n/2]]$ și $arr[[n/2]+1..n]$ și se construiește tabloul sortat $aux[1..n]$. Se mută tot conținutul tabloului auxiliar în $arr[1..n]$
- Algoritmul clasic de interclasarea presupune construirea unui tablou nou sortat pornind de la alte două tablouri sortate, de dimensiuni diferite

Principalul dezavantaj al sortării prin interclasare este faptul că utilizează un tablou adițional de dimensiunea tabloului de sortat.

Pentru a determina complexitatea algoritmului de sortare prin interclasare (algoritm recursiv), stabilim relația de recurență:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2 * T(\frac{n}{2}) + n & altfel \end{cases}$$

Presupunem $n = 2^k$ și înlocuim rând pe rând, astfel obținând:

$$T(2^k) = 2 * T(2^{k-1}) + 2^k$$

$$2 * T(2^{k-1}) = 2^2 * T(2^{k-2}) + 2^k$$

$$2^2 * T(2^{k-2}) = 2^3 * T(2^{k-3}) + 2^k$$

...

$$2^{k-1} * T(2^1) = 2^k * T(2^0) + 2^k$$

$$T(2^k) = k * 2^k$$

Dacă ne întoarcem la notația făcută, atunci $k = \log n$, deci $T(n) \in \Theta(n * \log_2 n)$.

Pseudocodul/Codul pentru algoritmul de sortare prin interclasare nu a fost prezentat în acest material, însă acesta, fiind un algoritm foarte cunoscut, se găsește în toate elementele bibliografice amintite. Sugerăm parcurgerea acestuia dintr-unul dintre materiale.

5.2.3 Sortarea în Standard Template Library

Standard Template Library (STL) are trei componente cheie și anume: containerii (structuri de date sub formă de template-uri), iteratorii și algoritmii.

Introsort sau introspective sort este un algoritm de sortare hibrid care oferă performanțe medii rapide și performanțe asimptotic optime. Algoritmul începe să lucreze cu Quick Sort, dar când recursivitatea atinge o anumită adâncime (baza-tăpe numărul elementelor ce urmează să fie sortate) algoritmul continuă sortarea cu ajutorul metodei heapSort². Algoritmul folosește, de asemenea, insertion sort când numărul de elemente de sortat este destul de mic. Un posibil dezavantaj al introsort-ului este spațiul de cod suplimentar necesar, aproximativ dublu față de Quick Sort sau Heap Sort singur.

Deși ambii algoritmi din care Introsort este format dau complexitatea $O(n * \log(n))$ pe cazul mediu, aceasta aduce îmbunătățiri folosindu-se de viteza algoritmului Quick Sort și evitând complexitatea pătratică a acestuia în cazurile defavorabile folosindu-se de heapsort atunci când un astfel de caz este întâlnit.

Introsort-ul, ca și Quick Sort-ul, nu este stabil (nu păstrează ordinea elementelor echivalente) de aceea este încă nevoie de o cerință separată pentru o sortare stabilă. În STL, cerințele pentru sortarea stabilă sunt scrise printr-un algoritm Merge Sort pentru a fi satisfăcătoare ([std::stable_sort](#)).

Partiționarea la Introsort

Aceste valori sunt alese empiric ca fiind aproximative din cauza diferitelor teste și cercetări efectuate.

1. Dacă dimensiunea partiției poate depăși limita maximă în adâncime atunci introsort comută cu heapsort. Definim limita în adâncime maximă ca fiind $2 * \log N$.

²<https://en.wikipedia.org/wiki/Heapsort>

2. Dacă dimensiunea partiției este prea mică, atunci Quick Sort se descompune în insertion sort. Definim această dimensiune ca fiind 16. Deci, dacă dimensiunea partiției este mai mică de 16 atunci vom face insertion sort.

Câteva detalii despre Insertion Sort:

Spre deosebire de alți algoritmi de sortare, sortarea prin inserție este folosită destul de des pentru sortarea tablourilor cu număr mic de elemente. De exemplu, poate fi folosit pentru a îmbunătăți rutina de sortare rapidă. Sortarea prin inserție seamănă oarecum cu sortarea prin selecție. Tabloul este împărțit imaginar în două părți, o parte sortată și o parte nesortată. La început, partea sortată conține primul element al tabloului și partea nesortată conține restul tabloului. La fiecare pas, algoritmul ia primul element din partea nesortată și îl inserează în locul potrivit al părții sortate. Când partea nesortată nu mai are nici un element, algoritmul se oprește. Astfel, cazul mediu al aceluiași algoritm este $O(n^2)$, iar cazul defavorabil este $O(n^2)$.

Secvență de cod 5.9: Cod C++ pentru sortarea prin inserție (Insertion Sort)

```

1  #include <iostream>
2  #include <fstream>
3
4  void insertionSort(int arr[], int dim)
5  {
6      int indexAux;
7      for (int index = 0; index < dim; index++)
8      {
9          indexAux = index;
10         while (indexAux > 0 && arr[indexAux] < arr[indexAux - 1])
11         {
12             int temp = arr[indexAux];
13             arr[indexAux] = arr[indexAux - 1];
14             arr[indexAux - 1] = temp;
15             indexAux--;
16         }
17     }
18 }
19 int main()
20 {
21     int dim;
22     int *sel;
23     ifstream f("intrare.txt");
24     ofstream g("iesire.txt");
25     f >> dim;
26     sel = new int[dim];
27     for (int index = 0; index < dim; ++index)
28         f >> sel[index];
29     insertionSort(sel, dim);
30     for (int index = 0; index < dim; ++index)
31         g << sel[index] << " ";
32
33     return 0;
34 }
```

3. Dacă dimensiunea partiției este sub limită și nu prea mică (adică între 16 și $2 * \log N$), atunci se efectuează un simplu Quick Sort.

Secvență de cod 5.10: Cod C++ pentru Intro Sort

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <math.h>
```

```

5
6 //functie care interchimba valorile a 2 variabile
7 void swapValue(int &a, int &b)
8 {
9     int temp = a;
10    a = b;
11    b = temp;
12    return;
13 }
14
15 // functie care partitioneaza vectorul si returneaza punctul de partitie
16 int partition(std::vector<int> &arr, int left, int right)
17 {
18     int pivot = arr[right]; //alegem pivotul
19     int index = left;
20
21
22     for (int index2 = left; index2 < right; ++index2)
23     {
24         if (arr[index2] <= pivot)
25         {
26             swapValue(arr[index2], arr[index]);
27             index++;
28         }
29     }
30
31     arr[right] = arr[index];
32     arr[index] = pivot;
33
34     return index;
35 }
36
37 // functia recursiva pentru quick sort
38 void quickSortRecursive(std::vector<int> &arr, int left, int right)
39 {
40     if (left < right)
41     {
42         int paritionPoint = partition(arr, left, right);
43         quickSortRecursive(arr, left, paritionPoint - 1);
44         quickSortRecursive(arr, paritionPoint + 1, right);
45     }
46 }
47
48 // functia maxHeapify muta in nodul index cea mai mare valoare
49 void maxHeapify(std::vector<int> &arr, int heapSize, int index)
50 {
51     int left = (index + 1) * 2 - 1;
52     int right = (index + 1) * 2;
53     int largest = 0;
54
55     //gaseste maximul dintre arr[index], arr[left] si arr[right]
56     if (left < heapSize && arr[left] > arr[index])
57         largest = left;
58     else
59         largest = index;
60
61     if (right < heapSize && arr[right] > arr[largest])
62         largest = right;
63
64     //daca maximul nu este deja parinte, atunci interchimba si continua reconstituirea
65     ↪ heap-ului
66     if (largest != index)
67     {
68         swapValue(arr[index], arr[largest]);
69         maxHeapify(arr, heapSize, largest);
70     }
71 }
72
73 //functia pentru heapsort
74 void heapSort(std::vector<int> &arr, int count)
75 {
76     int heapSize = count;
77
78     // se construiesc heap-ul
79     for (int index = (heapSize - 1) / 2; index >= 0; --index)
80         maxHeapify(arr, heapSize, index);
81
82     //se extrage cate un element si se pune la inceputul vectorului, apoi se reapeleaza
83     ↪ maxHeapify
84     for (int index = count - 1; index > 0; --index)

```

```

83     {
84         swapValue(arr[index], arr[0]);
85         --heapSize;
86         maxHeapify(arr, heapSize, 0);
87     }
88 }
89
90 //functie care sorteaza un vector folosind sortarea prin insertie
91 void insertionSort(std::vector<int> &arr, int count)
92 {
93     for (int index = 1; index < count; ++index)
94     {
95         int index2 = index;
96         while ((index2 > 0))
97         {
98             if (arr[index2 - 1] > arr[index2])
99             {
100                 swapValue(arr[index2 - 1], arr[index2]);
101                 --index2;
102             }
103             else break;
104         }
105     }
106 }
107
108 }
109
110 // functia pentru introsort care selecteaza una din cele 3 tipuri de sortari in functie
111 // de elemente care urmeaza sa fie sortate
112 void introsort(std::vector<int> &arr, int count)
113 {
114     int partitionSize = partition(arr, 0, count - 1);
115     if (partitionSize < 16)
116         insertionSort(arr, count);
117     else if (partitionSize > (2 * log(count)))
118         heapSort(arr, count);
119     else
120         quickSortRecursive(arr, 0, count - 1);
121 }
122
123 // afisare vector
124 void printArray(std::vector<int> arr, int n)
125 {
126     for (int i = 0; i < n; i++)
127         std::cout << arr[i] << " ";
128     std::cout << "\n";
129 }
130
131 int main()
132 {
133     std::random_device rd;
134     std::mt19937 mt(rd());
135     std::uniform_int_distribution<int> dist(-10000, 10000);
136
137     std::vector<int> arr;
138     int n;
139     std::cin >> n;
140
141     for (int index = 0; index < n; ++index)
142         arr.push_back(dist(mt));
143
144     introsort(arr, n);
145
146     printArray(arr, n);
147
148     system("pause");
149
150     return 0;
151 }

```

C++ STL oferă posibilități de sortarea a unui vector sau un array (elemente cu acces aleatoriu). Algoritmul de sortare este definit in biblioteca `<algorithm>`.

Variantele sunt următoarele ³:

- (1): elementele sunt comparate folosind operatorul "<"
- (2): elementele sunt comparate folosind funcția binară de comparare "comp".

Secvență de cod 5.11: Cod C++ pentru sort() din STL

```

1 // default (1)
2 template <class RandomAccessIterator>
3 void sort (RandomAccessIterator first, RandomAccessIterator last);
4 // custom (2)
5 template <class RandomAccessIterator, class Compare>
6 void sort (RandomAccessIterator first, RandomAccessIterator last, Compare comp);

```

Funcția acceptă două elemente din interval ca și argumente și returnează o valoare de tip boolean. Valoarea returnată indică dacă elementul trecut ca primul argument este considerat a merge înaintea celui de-al doilea argument, în ordinea specifică pe care o definește. Funcția nu trebuie să modifice niciunul dintre argumentele sale. Aceasta poate fi: fie o funcție pointer, fie o funcție obiect.

Antetul acestei funcții ar fi:

Secvență de cod 5.12: Antet comparator sort

```

1 bool comp(const Type1 &a, const Type2 &b);

```

Pentru acest tip de sortare, ordinea elementelor identice NU este garantată, vezi informații suplimentare despre `std::stable_sort`⁴.

Așadar, în C++ avem varianta de a utiliza funcții de sortare (eficiente) deja implementate pentru noi. Ele există, singura provocare fiind identificarea corectă a situației pe care o căutăm. Mai jos vom enumera variantele, iar în funcție de problema, fiecare poate alege alternativa dorită.

- Avem un șir de numere întregi (sau orice tip primitiv de date). Cum fac sortarea?
 - Sortarea se poate face scriind oricare dintre funcțiile de sortare studiate (Quick Sort, Merge Sort dacă dorim complexitate $O(n * \log n)$).
 - Apelăm ceva existent din C++, adică funcția `sort()`, amintită mai sus, vezi **5.13**.

Secvență de cod 5.13: Cod C++ pentru sortarea unui array de int cu sort() STL

```

1 #include <algorithm>
2 #include <iostream>
3 int main()
4 {
5     int arr[] = { 1, 5, 8, 9, 6, 7, 3, 4, 2, 0 };
6     int dim = sizeof(arr) / sizeof(arr[0]);
7
8     std::sort(arr, arr + dim);
9
10    std::cout << "\nArray after sorting using "

```

³<http://www.cplusplus.com/reference/algorithm/sort/>

⁴https://helloacm.com/c-coding-reference-sort-and-stable_sort/

```

11     "default sort is : \n";
12     for (int index = 0; index < dim; ++index)
13         std::cout << arr[index] << " ";
14
15     return 0;
16 }

```

Avantajul e clar, nu scriem noi funcția de sortare de la zero, plus că prin `sort()` din C++ ne asigurăm că folosim o variantă eficientă de sortare.

- Avem un șir de obiecte (oricare tip introdus de noi). Cum fac sortarea?
 - Adaptez una dintre variante de sortări pe care le știu.

În acest caz, adaptat înseamnă că definesc tipul de dată dorit (printr-o structură sau o clasă), iar apoi modific parametrii funcției de sortare, plus toate comparațiile pe care le implică această metodă de sortare, vezi 5.14. În acest exemplu am introdus structura dorită, "Eveniment", caracterizat prin două atribute: durata și probabilitate. Apoi, am definit ce înseamnă să compar două evenimente (operație de care avem nevoie în operația de sortare), apoi ce înseamnă atribuirea între două variabile de tip "Eveniment", plus am adaptat funcția de Quick Sort, corespunzător cu tipul de date dorit, "Eveniment".

Secvență de cod 5.14: Cod C++ pentru sortarea unui array de obiecte cu o funcție de Quick Sort

```

1  struct Eveniment
2  {
3      int durata;
4      float probabilitate;
5  };
6
7  bool comparaEvenimente(Eveniment &e1, Eveniment &e2)
8  {
9      return (e1.probabilitate / e1.durata < e2.probabilitate / e2.durata);
10 }
11
12 void egal(Eveniment &e1, Eveniment &e2)
13 {
14     e1.durata = e2.durata;
15     e1.probabilitate = e2.probabilitate;
16 }
17
18 void quickSort(Eveniment array[100], int st, int dr)
19 {
20     int indexStg = st, indexDr = dr;
21     Eveniment aux;
22     Eveniment mij = array[(st + dr) / 2];
23     while (indexStg <= indexDr)
24     {
25         while (comparaEvenimente(array[indexStg], mij))
26             indexStg++;
27         while (comparaEvenimente(mij, array[indexDr]))
28             indexDr--;
29         if (indexStg <= indexDr)
30         {
31             egal(aux, array[indexStg]);
32             egal(array[indexStg], array[indexDr]);
33             egal(array[indexDr], aux);
34             indexStg++;
35             indexDr--;
36         }
37     }
38     if (st < indexDr) quickSort(array, st, indexDr);

```

```

39     if (indexStg < dr) quickSort(array, indexStg, dr);
40 }

```

- Apelăm ceva existent din C++, adică funcția `sort()`, amintită mai sus, DAR precizând criteriul de sortare.

* Criteriul de sortare precizat prin funcție comparator, vezi 5.15.

Secvență de cod 5.15: Cod C++ pentru sortarea unui array de obiecte cu `sort()` STL

```

1      #include <algorithm>
2  #include <iostream>
3
4  struct Eveniment
5  {
6      int durata;
7      float probabilitate;
8  };
9
10 bool comparaEvenimente(Eveniment &e1, Eveniment &e2)
11 {
12     return (e1.probabilitate / e1.durata < e2.probabilitate / e2.
13             ↪ durata);
14 }
15
16 int main()
17 {
18     Eveniment arr[] = { {6,0.3}, {1,0.2}, {2,0.4}, {4,0.7} };
19     int dim = sizeof(arr) / sizeof(arr[0]);
20
21     std::sort(arr, arr + dim, comparaEvenimente);
22
23     std::cout << "Evenimente sort : \n";
24     for (int index = 0; index < dim; index++)
25         std::cout << "[" << arr[index].durata << ", " << arr[index].
26             ↪ probabilitate << "]" << " ";
27
28     return 0;
29 }

```

* Criteriul de sortare precizat prin supraîncărcarea⁵ operatorului `<` pentru obiectele de tip "Eveniment", vezi 5.17.

Secvență de cod 5.16: Cod C++ pentru sortarea unui array de obiecte cu `sort()` STL

```

1      #include <algorithm>
2  #include <iostream>
3
4  struct Eveniment
5  {
6      int durata;
7      float probabilitate;
8
9      bool operator<(const Eveniment &object) const
10     {
11         return (probabilitate / durata < object.probabilitate / object
12             ↪ .durata);
13     }
14 };
15
16 int main()
17 {
18     Eveniment arr[] = { {6,0.3}, {1,0.2}, {2,0.4}, {4,0.7} };
19     int dim = sizeof(arr) / sizeof(arr[0]);
20
21     std::sort(arr, arr + dim);

```

⁵<https://en.cppreference.com/w/cpp/language/operators>

```

21         std::cout << "Evenimente sort : \n";
22     for (int index = 0; index < dim; index++)
23     {
24         std::cout << "[" << arr[index].durata << ", " << arr[index].
                ↪ probabilitate << "]" ";
25     }
26     return 0;
27 }

```

* Criteriul de sortare precizat prin funcție obiect (functor).

Într-o expresie, apelul unei funcții este înlocuit prin rezultatul în-tors de funcție. Misiunea funcției poate fi preluată de către un obiect. În acest scop se supraîncarcă operatorul apel de funcție sub forma function `operator()`. Obiectele aparținând claselor care au supraîncărcat operatorul apel de funcție sunt obiecte funcții sau functori. Folosim mai departe exemplul de mai sus, dar în care supraîncărcăm operatorul `()`, vezi 5.17. În acest exemplu, am utilizat și un container pentru a reține grupul de evenimente.

Secvență de cod 5.17: Cod C++ pentru sortarea unui array de obiecte cu `sort()` STL

```

1  #include <algorithm>
2  #include <iostream>
3  #include <vector>
4
5  struct Eveniment
6  {
7      int durata;
8      float probabilitate;
9
10     bool operator()(const Eveniment & eveniment1, const Eveniment &
                ↪ eveniment2) const
11     {
12         return (eveniment1.proabilitate / eveniment1.durata <
                ↪ eveniment2.proabilitate / eveniment2.durata);
13     }
14 };
15
16 int main()
17 {
18     Eveniment arr[] = { {6,0.3}, {1,0.2}, {2,0.4}, {4,0.7} };
19     std::vector<Eveniment> myVectorEveniment(arr, arr + 4);
20     int dim = sizeof(arr) / sizeof(arr[0]);
21
22     // SAU std::sort(arr, arr+dim, Eveniment());
23     std::sort(myVectorEveniment.begin(), myVectorEveniment.end(),
                ↪ Eveniment());
24
25     std::cout << "Evenimente sort : \n";
26
27     // SAU
28     /*
29     for (int index = 0; index < dim; index++)
30     {
31         std::cout << "[" << arr[index].durata << ", " << arr[index].
                ↪ probabilitate << "]" ";
32     }
33     */
34     for (std::vector<Eveniment>::iterator it = myVectorEveniment.begin()
                ↪ (); it != myVectorEveniment.end(); ++it)
35     {
36         std::cout << " " << (*it).durata << " " << (*it).probabilitate
                ↪ << std::endl;
37     }
38     return 0;
39 }

```

* Criteriul de sortare precizat prin funcții lambda (C++11), vezi 5.19.

Funțiile lambda sunt funcții fără nume și definesc funcționalitatea în locul unde sunt definite. Funțiile lambda sunt utilizate frecvent în locul functorilor sau a predicatelor în algoritmi STL.

Despre definirea unei funcții lambda:

- Avem sintaxă: `[]() -> {...}`
- `[]` - utilizat pentru a captura variabile referință sau copii de variabile;
- `()` - se specifică parametrii;
- `->` - specifică tipul de return pentru expresii lambda mai sofisticate;
- `{}` - include expresii și bucăți de cod;
- `[]` - capturarea variabilelor are o semnificație diferită. Semnalăm câteva situații: `[a, &b]` - variabila `a` este capturată prin copiere și variabila `b` prin referință; `[this]` - capturează obiectul curent (`*this`) prin referință; `[&]` - capturează toate variabile automate folosite în corpul expresiei lambda prin referință, de asemenea captează și obiectul `this` prin referință dacă este cazul; `[=]` - capturează toate variabile automate folosite în corpul expresiei lambda prin copiere deasemenea captează și obiectul `this` prin referință dacă este cazul; `[]` - nu capturează nimic.

Secvență de cod 5.18: Cod C++ pentru sortarea unui array de obiecte cu `sort()` STL

```

1  #include <algorithm>
2  #include <iostream>
3  #include <vector>
4
5  struct Eveniment
6  {
7      int durata;
8      float probabilitate;
9  };
10
11 int main()
12 {
13     Eveniment arr[] = { {6,0.3}, {1,0.2}, {2,0.4}, {4,0.7} };
14     //std::vector<Eveniment> myVectorEveniment(arr, arr + 4);
15     int dim = sizeof(arr) / sizeof(arr[0]);
16
17     std::sort(arr, arr + dim, [](const Eveniment & eveniment1, const
18         ↪ Eveniment & eveniment2)
19     {
20         return (eveniment1.probabilitate / eveniment1.durata <
21             ↪ eveniment2.probabilitate / eveniment2.durata);
22     });
23     std::cout << "Evenimente sort : \n";
24
25     for (int index = 0; index < dim; index++)
26         std::cout << "[" << arr[index].durata << ", " << arr[index].
27             ↪ probabilitate << "]" << " ";
28
29     return 0;
30 }
```

5.2.4 Ghid pentru alegerea unui algoritm potrivit de sortare

Prezentăm câteva criterii pentru alegerea unui algoritm de sortare:

- câteva elemente de sortat $\Rightarrow$ Insertion Sort
- elementele sunt în mare parte sortate deja $\Rightarrow$ Insertion Sort
- pentru un caz mediu favorabil $\Rightarrow$ Quick Sort
- puțin cod de scris $\Rightarrow$ Insertion Sort
- sortare complexă folosită în orice caz $\Rightarrow$ Introsort
- sortare rapidă și eficientă (aproximativ peste 100 de valori) $\Rightarrow$ `std::sort` (STL)

Din punct de vedere tehnic, introsort-ul îndeplinește mai bine cerința de complexitate decât quick sort-ul, deoarece heapsort-ul garantează complexitatea $O(N \log(N))$ în cel mai rău caz, în timp ce quick sort-ul are timp pătratic în cel mai rău caz. Cu toate acestea, heapsort-ul este "mai lent" decât quick sort-ul în cazul mediu, în sensul că heapsort-ul efectuează $C * N \log N$, în timp ce quick sort-ul are performanță $D * N \log N$, D fiind semnificativ mai mică decât C (D și C sunt constante). Cu alte cuvinte, costul per comparație al heapsort-ului este mai mare decât cel al quick sort-ului.

Pentru a obține cea mai bună complexitate, introsort-ul începe cu quick sort - un algoritm recursiv -, dar atunci când adâncimea recursiunii devine prea mare (ceea ce înseamnă că intră într-un comportament degenerat reprezentând cel mai rău caz), trece la heapsort.

Pentru a obține rezultatele afișate în tabelele următoare 5.1, 5.2, 5.3, am executat fiecare încercare de 27 de ori pe același calculator și am dat deoparte cel mai bun și cel mai defavorabil rezultat de performanță. În aceste tabele este prezentată media celor 25 de studii rămase. Am creat mai multe seturi de date de referință pentru a arăta cum se compară algoritmii prezentați în acest proiect. De reținut este faptul că valorile reale ale tabelelor generate sunt mai puțin importante, deoarece reflectă hardware-ul specific pe care au fost executate valorile de referință. Însă putem observa că pe valori mici, în rezultatele de față, sort-ul din STL este mai încet decât restul, iar pe valori mari acesta este cel mai rapid. În schimb, introsort-ul, pe majoritatea valorilor mici este cel mai rapid, deoarece folosește insertionsort-ul, iar pe valori mari, acesta se mișcă foarte greu din cauza heapsort-ului. Un alt aspect important este faptul că heapsort-ul este în medie de 2 până la 5 ori mai lent decât quick sort-ul.

Tabelul 5.1: Rezultate de performanță (în secunde) pe valori random (între -10000 și 10000)

n	Heap Sort	Quick Sort	Introsort	std::sort
10	$3.37E - 05$	$2.37E - 05$	$2.66E - 05$	$8.98E - 06$
114	0.001027	0.000522	0.001117	0.000117
535	0.006747	0.003255	0.007181	0.000699
1024	0.014384	0.006527	0.015064	0.001431
36021	0.804742	0.341126	0.824925	0.073775
212980	5.80652	2.44253	5.903153	0.458173
780450	23.3729	9.759863	23.80737	1.50684
4120531	140.1421	59.0482	142.6844	7.9022

Tabelul 5.2: Rezultate de performanță (în secunde) pe valori ordonate crescător (între -10000 și 10000)

n	Heap Sort	Quick Sort	Introsort	std::sort
10	$3.88E - 05$	$1.64E - 05$	$1.38E - 05$	$4.81E - 05$
114	$1.10E - 03$	0.000262	0.00117	$3.78E - 05$
534	0.007169	0.001493	0.007423	0.000261
1024	0.015118	0.003118	0.015881	0.000538
36021	0.83283	0.183954	0.858817	0.033722
212980	5.88332	1.3982	6.07857	0.219399
780450	22.2326	6.0824	23.0056	0.795593
4120531	131.07	39.3006	129.966	6.20361

Tabelul 5.3: Rezultate de performanță (în secunde) pe valori ordonate descrescător (între -10000 și 10000)

n	Heap Sort	Quick Sort	Introsort	std::sort
10	$3.95E - 05$	$1.67E - 05$	$1.41E - 01$	$5.45E - 06$
114	$1.08E - 03$	0.000261	0.001162	$3.85E - 05$
534	0.006999	0.001566	0.008728	0.000262
1024	0.015147	0.003094	0.016763	0.000557
36021	0.83509	0.183461	0.860815	0.033119
212980	5.84654	1.40885	6.0718	0.224486
780450	22.3041	6.1178	22.9749	0.779508
4120531	129.154	39.4709	144.177	4.04703

5.3 Algoritmi de căutare și sortare - probleme rezolvate

Problemele pe care le propunem în această secțiune sunt:

1. Implementați eficient metodele de căutare/inserare/ștergere într-un șir sortat. Precizați complexitățile găsite.
2. Se citește o matrice cu n linii și m coloane. Să se genereze un șir de 100 numere random și să se afișeze câte dintre aceste elemente sunt pe prima linie a matricei. Obs: Se va aplica o operație de sortare asupra liniilor matricei astfel încât prima coloană să fie sortată crescător, ulterior se va apela căutarea binară.
3. Se citește o matrice pătratică din fișier txt. Să se sorteze crescător elementele de pe liniile pare, și descrescător valorile multiplu de 3 de pe liniile impare. Observație: Pentru reținerea matricei se va folosi o variabilă de tip `std::vector<std::vector>`, iar pentru sortare se va utiliza funcția `std::sort`.
4. Să se sorteze o stivă de numere întregi. Posibilități: recursiv sau cu o stivă temporară.
5. Fiind dați doi vectori de dimensiune n și o valoare k , să se reorganizeze elementele din vectori, astfel $a[i] + b[i] \geq k$. Dacă cei doi vectori nu se pot reorganiza astfel încât să se poată îndeplini relația precizată, atunci se afișează "Nu".

Pentru problema 1 propunem funcții pentru cele trei operații cerute în problemă, vezi cod C++ asociat 5.19.

Secvență de cod 5.19: Cod C++ pentru problema 1

```

1  #include <iostream>
2
3  int cautareBinara(int vector[100], int lenght, int searchedElement)
4  {
5      bool isFound = false;
6      int lowerBound = 0, upperBound, middle;
7      upperBound = lenght - 1;
8      while (lowerBound <= upperBound && !isFound)
9      {
10         middle = (lowerBound + upperBound) / 2;
11         if (vector[middle] == searchedElement)
12             isFound = true;
13         else if (searchedElement < vector[middle])
14             upperBound = middle - 1;
15         else
16             lowerBound = middle + 1;
17     }
18     if (isFound)
19         return middle;
20     else
21         return -1;
22 }
23
24 void insertSorted(int vector[100], int &lenght, int insertedNumber)
25 {

```

```

26     int index;
27     for (index = lenght - 1; index >= 0; index--)
28         if (vector[index] > insertedNumber)
29             vector[index + 1] = vector[index];
30         else break;
31     vector[index + 1] = insertedNumber;
32     lenght++;
33 }
34
35 void deleteSorted(int vector[100], int &lenght, int deletedElement)
36 {
37     if (cautareBinara(vector, lenght, deletedElement) != -1)
38     {
39         for (int index = cautareBinara(vector, lenght, deletedElement); index < lenght
40             ↪ - 1; index++)
41             vector[index] = vector[index + 1];
42         lenght--;
43     }
44     else
45         std::cout << "Elementul pe care doriti sa il stergeti nu se afla in vector.";
46 }

```

Comentăm și complexitățile obținute:

- Pentru funcția "cautareBinara" avem $O(\log lenght)$, unde $lenght$ este lungimea tabloului unidimensional considerat;
- Pentru funcția "insertSorted" avem mai multe situații: caz favorabil - elementul de inserat e mai mare ca toate elementele din vector $\Rightarrow O(1)$, caz nefavorabil - elementul de inserat e mai mic ca toate elementele din vector $\Rightarrow O(lenght)$, caz mediu - cazuri posibile: $lenght + 1$, deci dacă D este domeniul problemei, atunci:

$$\sum_{I \in D} P(I) * E(I) = \frac{1}{lenght + 1} (1 + 2 + \dots + lenght) = \frac{lenght}{2} \Rightarrow O(lenght)$$

- Pentru funcția "deleteSorted" avem mai multe situații: caz favorabil - elementul de eliminat e pe ultima poziție din vector $\Rightarrow O(1)$, caz nefavorabil - elementul de inserat e pe prima poziție din vector/nu există $\Rightarrow O(lenght)$, caz mediu - cazuri posibile: $lenght + 1$, deci

$$\sum_{I \in D} P(I) * E(I) = \frac{1}{lenght + 1} (1 + 2 + \dots + lenght) = \frac{lenght}{2} \Rightarrow O(lenght)$$

Pentru problema 2 propunem funcții pentru diversele operații cerute în problemă, vezi cod C++ asociat 5.20.

Secvență de cod 5.20: Cod C++ pentru problema 3

```

1 #include <iostream>
2 #include <fstream>
3 #include <stdlib.h>
4 #include <time.h>
5 #include <string>
6 using namespace std;
7 void readMatrixFromFile(int matrice[10][10], int& lineNumber, int& columnNumber, string
8     ↪ filePath)

```

```

9      ifstream myfile(filePath);
10     if (myfile.is_open())
11     {
12         myfile >> lineNumber >> columnNumber;
13         for (int currentLine = 0; currentLine < lineNumber; currentLine++)
14             for (int currentColumn = 0; currentColumn < columnNumber; currentColumn++)
15                 myfile >> matrice[currentLine][currentColumn];
16         myfile.close();
17     }
18     else
19         cout << "Unable to open file";
20 }
21 void generateRandomVector(int vector[100])
22 {
23     for (int i = 0; i < 100; i++)
24         vector[i] = rand() % 1000;
25 }
26 void swap2Columns(int matrice[10][10], int lineNumber, int firstColumnToSwap, int
27     secondColumnToSwap)
28 {
29     for (int currentLine = 0; currentLine < lineNumber; currentLine++)
30     {
31         int auxiliar = matrice[currentLine][firstColumnToSwap];
32         matrice[currentLine][firstColumnToSwap] = matrice[currentLine][
33             ↪ secondColumnToSwap];
34         matrice[currentLine][secondColumnToSwap] = auxiliar;
35     }
36 }
37 void swap2Lines(int matrice[10][10], int columnNumber, int firstLineToSwap, int
38     ↪ secondLineToSwap)
39 {
40     for (int currentColumn = 0; currentColumn < columnNumber; currentColumn++)
41     {
42         int auxiliar = matrice[firstLineToSwap][currentColumn];
43         matrice[firstLineToSwap][currentColumn] = matrice[secondLineToSwap][
44             ↪ currentColumn];
45         matrice[secondLineToSwap][currentColumn] = auxiliar;
46     }
47 }
48 void sortFirstLine(int matrice[10][10], int lineNumber, int columnNumber)
49 {
50     for (int currentColumn = 0; currentColumn < columnNumber - 1; currentColumn++)
51         for (int nextColumn = currentColumn + 1; nextColumn < columnNumber; nextColumn
52             ↪ +++)
53             if (matrice[0][currentColumn] > matrice[0][nextColumn])
54                 swap2Columns(matrice, columnNumber, currentColumn, nextColumn);
55 }
56 void sortFirstColumn(int matrice[10][10], int lineNumber, int columnNumber)
57 {
58     for (int currentLine = 0; currentLine < lineNumber - 1; currentLine++)
59         for (int nextLine = currentLine + 1; nextLine < lineNumber; nextLine++)
60             if (matrice[currentLine][0] > matrice[nextLine][0])
61                 swap2Lines(matrice, columnNumber, currentLine, nextLine);
62 }
63 int cautareBinaraLinie(int matrice[10][10], int columnNumber, int searchedElement)
64 {
65     bool isFound = false;
66     int lowerBound = 0, upperBound, middle;
67     upperBound = columnNumber - 1;
68     while (lowerBound <= upperBound && !isFound)
69     {
70         middle = (lowerBound + upperBound) / 2;
71         if (matrice[0][middle] == searchedElement)
72             isFound = true;
73         else if (searchedElement < matrice[0][middle])
74             upperBound = middle - 1;
75         else
76             lowerBound = middle + 1;
77     }
78     if (isFound)
79         return middle;
80     else
81         return -1;
82 }
83 int cautareBinaraColoana(int matrice[10][10], int lineNumber, int searchedElement)
84 {
85     bool isFound = false;
86     int lowerBound = 0, upperBound, middle;
87     upperBound = lineNumber - 1;

```

```

85     while (lowerBound <= upperBound && !isFound)
86     {
87         middle = (lowerBound + upperBound) / 2;
88         if (matrice[middle][0] == searchedElement)
89             isFound = true;
90         else if (searchedElement < matrice[middle][0])
91             upperBound = middle - 1;
92         else
93             lowerBound = middle + 1;
94     }
95     if (isFound)
96         return middle;
97     else
98         return -1;
99 }
100 void countCommonElementsLinie(int matrice[][10], int columnNumber, int vector[100])
101 {
102     int numarElementePrezente = 0;
103     for (int contor = 0; contor < 100; contor++)
104         if (cautareBinaraLinie(matrice, columnNumber, vector[contor]) != -1)
105             numarElementePrezente++;
106     std::cout << numarElementePrezente;
107 }
108 void countCommonElementsColoana(int matrice[][10], int lineNumber, int vector[100])
109 {
110     int numarElementePrezente = 0;
111     for (int contor = 0; contor < 100; contor++)
112         if (cautareBinaraColoana(matrice, lineNumber, vector[contor]) != -1)
113             numarElementePrezente++;
114     std::cout << numarElementePrezente;
115 }
116
117 int main()
118 {
119     srand(time(NULL));
120     int vector[100], matrice[10][10], lineNumber, columnNumber;
121     string filePathRead = "fileIn.txt";
122     string filePathWrite = "fileOut.txt";
123     generateRandomVector(vector);
124     readMatrixFromFile(matrice, lineNumber, columnNumber, filePathRead);
125     //sortFirstLine(matrice, lineNumber, columnNumber);
126     //cout << "Numar elemente comune intre vector si prima linie a matricei sortate: ";
127     //countCommonElementsLinie(matrice, columnNumber, vector);
128     sortFirstColumn(matrice, lineNumber, columnNumber);
129     cout << "Numar elemente comune intre vector si prima coloana a matricei sortate: ";
130     countCommonElementsColoana(matrice, lineNumber, vector);
131 }

```

Comentăm ceea ce considerăm important în rezolvarea problemei de mai sus, din 5.20:

- Metoda "readMatrixFromFile" se ocupă de citirea numerelor din matricea considerată, iar ca și parametrii are matricea, numărul de linii, numărul de coloane, plus numele fișierului (string) din care se face citirea. Atenție la transmiterea parametrilor.
- Metoda "generateRandomVector" se ocupă de generarea random a 100 de numere (între 0 și 999), prin utilizarea funcției rand()⁶.
- Metoda "swap2Columns" se ocupă de interschimbarea a două coloane din matrice, a căror indice se transmite prin parametrii. Idem metoda "swap2Lines", dar pentru două linii precizate ca parametrii.
- Metodele de "cautareBinaraLinie" și "cautareBinaraColoana" se ocupă de

⁶<http://www.cplusplus.com/reference/cstdlib/rand/>

căutare binară pe prima linie/coloană, adică pe matrice[0][something], respectiv matrice[something][0].

Pentru problema 3 propunem următoarea rezolvare în C++, vezi 5.21.

Secvență de cod 5.21: Cod C++ pentru problema 3

```

1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  #include <vector>
5  #include <algorithm>
6  using namespace std;
7
8  void readMatrixFromFile(vector<vector<int>> &matrice, string filePath)
9  {
10     std::ifstream myfile(filePath);
11     if (myfile.is_open())
12     {
13         int lineNumber, columnNumber, element;
14         myfile >> lineNumber >> columnNumber;
15         for (int i = 0; i < lineNumber; i++)
16         {
17             vector<int> v; //the lines covered one by one
18             for (int j = 0; j < columnNumber; j++)
19             {
20                 myfile >> element;
21                 v.push_back(element);
22             }
23             matrice.push_back(v);
24         }
25         myfile.close();
26     }
27     else
28         cout << "Unable to open file";
29 }
30
31 void sortEvenLines(vector<vector<int>>& mat)
32 {
33     for (int i = 0; i < mat.size(); i += 2)
34         sort(mat.at(i).begin(), mat.at(i).end());
35 }
36
37 bool myFunction(int a, int b)
38 {
39     if (a > b)
40         return true;
41     return false;
42 }
43
44 void sortOddLines(vector<vector<int>>& mat)
45 {
46     for (int i = 1; i < mat.size(); i += 2)
47     {
48         vector<int> vecValue, vecPosition;
49         for (int j = 0; j < mat.at(i).size(); j++)
50         {
51             int x = mat.at(i).at(j);
52             if (x % 3 == 0)
53             {
54                 vecPosition.push_back(j);
55                 vecValue.push_back(mat.at(i).at(j));
56             }
57         }
58         sort(vecValue.begin(), vecValue.end(), myFunction);
59         for (int k = 0; k < vecPosition.size(); k++)
60             mat.at(i).at(vecPosition[k]) = vecValue.at(k);
61     }
62 }
63
64 void writeMatrix(vector<vector<int>> const& mat)
65 {
66     for (vector<int> row : mat)
67     {
68         for (int val : row)
69             cout << val << " ";
70         cout << '\n';
71     }

```



```

72 }
73
74 int main ()
75 {
76     vector<vector<int>>> matrice;
77     string filePathRead = "fileIn.txt";
78     readMatrixFromFile(matrice, filePathRead);
79     sortEvenLines(matrice);
80     sortOddLines(matrice);
81     writeMatrix(matrice);
82 }

```

Pentru problema 4 propunem următoarea rezolvare în C++, vezi 5.22. În această rezolvare ne folosim de containerul de tip stivă din STL, dar și de proprietățile specifice unei structuri de tip stivă (LIFO). Folosim o stivă suplimentară care reține elementele în ordine crescătoare (la fiecare pas verificăm cum este valoarea elementului din vârful (top) din stiva dată față de elementul din vârful stivei ajutătoare). În rezolvare se folosesc diverse funcții membre stivei (din interfața specifică⁷).

Secvență de cod 5.22: Cod C++ pentru problema 4

```

1  #include <iostream>
2  #include <stack>
3  using namespace std;
4
5  stack<int> sortStack(stack<int> myStack)
6  {
7      stack<int> helpStack;
8      while (!myStack.empty())
9      {
10         int aux = myStack.top();
11         myStack.pop();
12         while (!helpStack.empty() && helpStack.top() > aux)
13         {
14             myStack.push(helpStack.top());
15             helpStack.pop();
16         }
17         helpStack.push(aux);
18     }
19     return helpStack;
20 }
21
22 int main()
23 {
24     int number;
25     stack<int> myStack;
26     for (int i = 0; i < 6; i++)
27     {
28         cin >> number;
29         myStack.push(number);
30     }
31     stack<int> helpStack = sortStack(myStack);
32     for (int i = 0; i < myStack.size(); i++)
33     {
34         cout << helpStack.top() << ' ';
35         helpStack.pop();
36     }
37 }

```

Pentru problema 5 avem:

- Date de intrare: cei doi vectori de numere întregi, de dimensiune n ;
- Date de ieșire: un mesaj corespunzător rezultatului, "Da" pentru situația că vectorii respectă condiția impusă sau "Nu" pentru contrar;

⁷<http://www.cplusplus.com/reference/stack/stack/>

- Algoritm de rezolvare: sortăm crescător pe unul dintre vectori (primul) și descrescător pe celălalt și testăm dacă avem îndeplinită condiția impusă, vis a vis de k -ul dat. Pentru sortare folosim `std::sort`, o varianta pentru a sorta crescător (cu doi parametri), iau o altă variantă în care funcția primește al treilea parametru, o funcție obiect (functor) binară care face comparația identică cu operatorul $>$ (adică verifică dacă primul parametru este mai mare decât al doilea). `std::greater` este definit cu același comportament ca și 5.23;

Secvență de cod 5.23: Cod C++ echivalent pentru `std::greater`

```

1 template <class T> struct greater : binary_function<T,T,bool> {
2     bool operator() (const T& x, const T& y) const {return x>y;}
3 };

```

- Algoritmul scris în C++ asociat problemei este 5.24

Secvență de cod 5.24: Cod C++ pentru problema 5

```

1 #include <iostream>
2 #include <algorithm>
3 #include <functional>
4
5 void citeste(int array[], int len) {
6     for (int index = 0; index < len; index++)
7         std::cin >> array[index];
8 }
9
10 int main() {
11     int a[1000], b[100], n, k;
12     std::cin >> n >> k;
13     citeste(a, n);
14     citeste(b, n);
15     std::sort(a, a + n);
16     std::sort(b, b + n, std::greater<int>());
17     for (int index = 0; index < n; index++) {
18         if (a[index] + b[index] < k) {
19             std::cout << "Nu";
20             return 0;
21         }
22     }
23     std::cout << "Da, se indeplineste conditia data!";
24
25     return 0;
26 }

```

- Analiza complexității: complexitatea acestei probleme se rezumă la complexitatea (dublă) a șirurilor de dimensiunea n . Cum am folosit `std::sort`, atunci complexitatea este: $O(2 * n * \log n) = O(n * \log n)$.

Pentru toate problemele rezolvate mai sus, am respectat câteva reguli de scriere de cod (amintite și în capitolele anterioare), de aceea, oferim mai jos câteva sfaturi practice de scriere de cod curat (**Clean Code tips**):

- Keep it simple, stupid (KISS)
- Don't repeat yourself (DRY)
- Be precise

- Only one responsibility
- Describe the intention
- Use explanatory names
- Avoid abbreviation
- Use names from the domain language
- Delete commented & unused code
- Keep functions/classes short
- Avoid magic numbers
- Simplify complex conditional expressions

De observat faptul că, comentariile și formatul codului pot face codul mai ușor de citit și de înțeles. De asemenea, o denumire corectă (vis a vis de contextul în care se folosește variabila respectivă), poate să ofere o claritate maximă a codului pe care îl scriem. Nu în ultimul rând, scrierea de cod reutilizabil/generic este o obligație pentru orice programator, nu o opțiune.

5.4 Algoritmi de căutare și sortare - probleme propuse

Fie următoarele probleme:

1. Se dau niște date calendaristice (citite dintr-un fișier text). Să se verifice dacă sunt valide (zi, lună, an) și apoi cele valide să se sorteze crescător.

Observații de implementare:

- Pe fiecare linie a fișierului de intrare este câte o dată calendaristică. De asemenea, pe prima linie a fișierului există numărul de date considerate;
 - O dată calendaristică se poate reține ca și un obiect "DataCalendaristica", cu attribute: zi, lună, an;
 - încurajăm folosirea unei metode de validare, care verifică validitatea unei date calendaristice;
 - sortarea crescătoare trebuie să țină cont de toate cele 3 attribute (și tratate și cazurile în care avem aceeași zi/lună).
2. Să se transforme algoritmul clasic de sortare prin numărare astfel încât să poată fi utilizat și pentru sortarea vectorilor care conțin valori întregi negative.

3. Se citește un vector de n numere întregi ordonat descrescător dintr-un fișier. Se citește un al doilea vector de m numere întregi. Folosind căutarea binară să se determine care elemente din al doilea vector apar în primul și pe ce poziții. Rezultatul se va scrie într-un alt fișier.
Care este complexitatea obținută?
4. Fie un tablou ordonat crescător de n numere întregi distincte. Să se determine un indice m , $0 \leq m < n$, cu $arr[m] = m$, dacă este posibil. Indicație: Folosim ideea de la căutarea binară.
5. Se citește un vector de n elemente din fișier. Să se sorteze crescător vectorul și să se returneze pozițiile fiecărui element sortat din vectorul inițial. Exemplu: Se citește: 35, 21, 8, 71, 93, 4, 20. Se afișează: 6, 3, 7, 2, 1, 4, 5.

Pentru toate problemele de mai sus să se țină cont de principiile de cod curat enunțate în această secțiune. De asemenea, încurajăm folosirea containerelor, plus a structurilor (după caz), `struct` (care permit introducerea obiectelor).

Capitolul 6

Algoritmi Divide et Impera

6.1 Aspecte teoretice

Divide et Impera este o metodă de programare care se aplică problemelor care pot fi descompuse în subprobleme independente, similare problemei inițiale, de dimensiuni mai mici și care pot fi rezolvate foarte ușor. Procesul se reia până când (în urma descompunerilor repetate) se ajunge la probleme care admit rezolvare imediată. Această tehnică se poate implementa atât iterativ, cât și recursiv.

Metoda se poate aplica în rezolvarea unei probleme care îndeplinește următoarele condiții:

- se poate descompune în (două sau mai multe) subprobleme;
- aceste subprobleme sunt independente una față de alta (subproblema nu se rezolvă pe baza alteia și nu se folosește rezultatul celeilalte);
- aceste subprobleme sunt similare cu problema inițială;
- la rândul lor subproblemele se pot descompune (dacă este necesar) în alte subprobleme mai simple;
- aceste subprobleme simple au soluții rapide.

Etapele rezolvării unei probleme inițiale cu Divide et Impera sunt:

- (Împarte) descompunerea problemei inițiale în subprobleme independente, similare problemei de bază, dar de dimensiuni mai mici;
- descompunerea treptată a subproblemelor în alte subprobleme din ce în ce mai simple, până când se pot rezolva imediat, prin algoritmul simplificat;
- (Stăpânește) rezolvarea subproblemelor mici/simple;

- (Combină) combinarea soluțiilor găsite pentru construirea soluțiilor subproblemelor;
- combinarea ultimelor soluții determină obținerea soluției problemei inițiale.

Complexitatea algoritmilor de tipul Divide et Impera se calculează după formula:

$$T(n) = D(n) + S(n) + C(n)$$

unde $D(n)$, $S(n)$ și $C(n)$ reprezintă complexitățile celor 3 pași descriși mai sus: împarte (divide), stăpânește (impera) și combină. Algoritmii divide et impera au o bună comportare în timp, dacă dimensiunile subproblemelor în care se împarte subproblema sunt aproximativ egale. Dacă lipsesc fazele de combinare a soluțiilor, viteza acestor algoritmi este și mai mare (ex. căutarea binară). În majoritatea cazurilor, descompunerile înjumătățesc dimensiunea problemei, un exemplu tipic reprezentându-l sortarea prin interclasare (mergeSort).

6.1.1 Studii de caz pe diverse probleme

- Calculul puterii unui număr, x^n : algoritm clasic ($O(n)$) vs algoritm Divide et Impera.

Discuție după n :

- n par: $x^{n/2} * x^{n/2}$;
- n impar: $x * x^{n/2} * x^{n/2}$;
- (caz particular) n este 0: 1.

Complexitate: $T(n) = T(n/2) + \Theta(1)$. Total $O(\log n)$ (refolosind calculul pe ramuri de recursivitate)

- Calculul maximului unui tablou $x[1..n]$, vezi algoritm Divide et Impera în Figura 6.1. Problema se rezolvă pe baza relației:

$$\min(a_1, a_2, \dots, a_n) = \min(\min(a_1, a_2, \dots, a_m), \min(a_{m+1}, \dots, a_n))$$

Analiza complexității se face stabilind: dimensiunea problemei (n), operația dominantă (comparația) și relația de recurență:

$$T(n) = \begin{cases} 0 & n = 1 \\ T(n/2) + T(n - n/2) + 1 & n > 1 \end{cases}$$

$$T(n) = \begin{cases} 0 & n = 1 \\ 2T(n/2) + 1 & n > 1 \end{cases}$$

```

maximDivide(x[s..d])
IF s=d then
    RETURN x[s]
ELSE
    m ← (s+d) DIV 2
    max1 ← maxim(x[s..m])
    max2 ← maxim(x[m+1..d])
    if max1 > max2 then
        RETURN max1
    ELSE
        RETURN max2
ENDIF
ENDIF

```

Figura 6.1: Pseudocod pentru determinare maxim dintr-un șir cu Divide et Impera

Caz particular: $n = 2^m$. Astfel avem (substituție inversă):

$$T(2^m) = 2T(2^{m-1}) + 1$$

$$T(2^{m-1}) = 2T(2^{m-2}) + 1 \mid * 2$$

...

$$T(2) = 2T(1) + 1 \mid * 2^{m-1}$$

$$T(1) = 0$$

$$T(n) = 1 + \dots + 2^{m-1} = 2^m - 1 = n - 1$$

Deci, $T(n)$ aparține lui $\Theta(n)$.

Atenționăm că deducerea complexității se poate face și cu Teorema master.

- Algoritmul Karatsuba¹ de înmulțire a două numere mari (algoritm clasic în $O(n^2)$)

Să presupunem că x și y sunt două numere întregi scrise în sistemul binar, fiecare având n biți. Pentru ușurința calculelor, putem presupune că n este o putere a lui 2. Dacă rezolvăm clasic (iterativ), atunci complexitatea obținută este $O(n^2)$.

Vom încerca o abordare Divide et Impera, împărțind cele două numere în jumătăți, astfel: $x = x_l, x_r = 2^{n/2}x_l + x_r$ și $y = y_l, y_r = 2^{n/2}y_l + y_r$. Atunci

¹https://en.wikipedia.org/wiki/Karatsuba_algorithm

avem produsul $xy = 2^n x_l y_l + 2^{n/2}(x_l y_r + x_r y_l) + x_r y_r$. Adunările și înmulțirile cu puteri ale lui 2 au complexitate liniară, deci operațiile semnificative sunt cele 4 înmulțiri $(x_l y_l, x_l y_r, x_r y_l, x_r y_r)$. Relația de recurență este:

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

Deci $T(n) = O(n^2)$, niciun progres vizibil. Însă, putem utiliza relația următoare:

$$(a + bi)(c + di) = ac - bd + (bc + ad)i = ac - bd - i(ac - bd) + i(a + b)(c + d)$$

În prima exprimare avem 4 înmulțiri, în a doua avem doar 3 (Gauss). Așadar $T(n) = 3T\left(\frac{n}{2}\right) + n$. Pseudocodul pentru algoritm este în Figura 6.2.

```
multiplyKaratsuba(x, y)
n=maxim(size x, size y)
IF n=1
    RETURN x * y
ELSE
    //[n/2] biti left, right x, obținem xl, xr
    //[n/2] biti left, right y, obținem yl, yr
    P1=multiplyKaratsuba(xl, yl)
    P2=multiplyKaratsuba(xl+xr, yl+yr)
    P3=multiplyKaratsuba(xr, yr)
    RETURN P1*2^n + (P2-P1-P3)*2^(n/2)+P3
ENDIF
```

Figura 6.2: Pseudocod pentru multiplyKaratsuba

Deși scăderea de la 4 la 3 pare nesemnificativă, algoritmul nostru este unul recursiv, deci ea se face la fiecare nivel. Folosind teorema master, complexitatea algoritmului nostru este: $O(n^{\log_2 3}) = O(n^{1.585})$.

- Înmulțirea matricelor: algoritm clasic ($O(n^3)$) sau cu algoritmul lui Strassen. Pentru matricele A și B de $n \times n$ elemente, dorim să obținem matricea produs $C = A \cdot B$. Algoritmul clasic provine direct din definiția înmulțirii a două matrice și necesită n^3 înmulțiri și $(n-1)n^2$ adunări scalare, astfel că timpul necesar pentru calcularea lui C este $O(n^3)$. Intenția este să găsim un algoritm de înmulțire matriceală al cărui timp să fie într-un ordin mai mic decât n^3 . Este evident că $O(n^2)$ este o limită inferioară pentru orice algoritm de înmulțire matriceală, deoarece trebuie să parcurgem cele n^2 elemente ale lui C .

Tehnica Divide et Impera sugerează un alt mod de calcul al matricei C : vom presupune că n este o putere a lui 2. Partiționăm pe A și pe B în câte

patru submatrice de $n/2 \times n/2$ elemente fiecare. Matricea produs C se poate calcula conform formulei pentru produsul matricelor de 2×2 elemente:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

unde $C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$ etc. Până aici nu am câștigat nimic față de complexitatea metodei clasice. Este de dorit ca în calcularea submatricelor C să folosim cât mai puține înmulțiri, chiar dacă prin aceasta mărim numărul de adunări (adunarea matricelor necesită un timp pătratic). Astfel, Strassen a propus o nouă metodă de calculare a submatricelor C_{ij} , care pentru $n = 2$ utilizează 7 înmulțiri și 18 adunări și scăderi. Astfel, se calculează șapte matrice de $n/2 \times n/2$ elemente:

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{22})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

Este ușor de verificat că matricea produs C se obține astfel: $C_{11} = P + S - T + V$, $C_{12} = R + T$, $C_{21} = Q + S$, $C_{22} = P + R - Q + U$. Astfel, timpul total pentru noul algoritm divide et impera este $T(n) \in 7 * T(n/2) + \Theta(n \log 7)$. Cum $\log 7 < 2.81$, rezultă că avem complexitatea $O(n^{2.81})$, algoritmul lui Strassen fiind mai eficient decât algoritmul clasic de înmulțire matriceală.

- Calculul pentru al n -lea număr din șirul lui Fibonacci

Problema determinării numerelor din șirul lui Fibonacci este o problemă clasică. Sunt mai multe metode utilizate, însă aici ne propunem să calculăm $\text{fib}(n)$ în timp $O(\log n)$ cu tehnica Divide et Impera. Vom porni de la următoarea ecuație matriceală:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} \text{fib}(n) \\ \text{fib}(n-1) \end{bmatrix} = \begin{bmatrix} \text{fib}(n+1) \\ \text{fib}(n) \end{bmatrix}$$

Efectuând calculul în partea stângă obținem:

$$\begin{bmatrix} a * \text{fib}(n) + b * \text{fib}(n-1) \\ c * \text{fib}(n) + d * \text{fib}(n-1) \end{bmatrix} = \begin{bmatrix} \text{fib}(n+1) \\ \text{fib}(n) \end{bmatrix}$$

De unde obținem:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

Rezultă că:

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^* \begin{bmatrix} \text{fibonacci}(n) & \text{fibonacci}(n-1) \\ \text{fibonacci}(n-1) & \text{fibonacci}(n-2) \end{bmatrix} = \begin{bmatrix} \text{fibonacci}(n+1) & \text{fibonacci}(n) \\ \text{fibonacci}(n) & \text{fibonacci}(n-1) \end{bmatrix}$$

Deci, putem scrie că:

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} \text{fibonacci}(2) & \text{fibonacci}(1) \\ \text{fibonacci}(1) & \text{fibonacci}(0) \end{bmatrix}$$

Astfel, putem utiliza algoritmul de exponențiere logaritmică pentru o matrice de ordin 2 pentru a determina orice număr Fibonacci:

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \begin{bmatrix} \text{fibonacci}(n+1) & \text{fibonacci}(n) \\ \text{fibonacci}(n) & \text{fibonacci}(n-1) \end{bmatrix}$$

- Căutarea binară²

O problemă des întâlnită în rezolvarea problemelor este căutarea. Dacă tabloul în care se face căutarea este sortat, atunci se poate apela la căutarea binară. Prin rezolvarea clasică (comparând pe rând valoarea căutată cu fiecare componentă a vectorului) în cazul unei căutări cu succes, numărul mediu de comparații este $n/2$, iar în cazul unei căutări fără succes numărul de comparații este egal cu n .

Este de precizat faptul că algoritmul de căutare binară se bazează pe tehnica reducerii (doar una dintre subprobleme este rezolvată). Algoritmul iterativ de căutare binară este destul de cunoscut (prezent în oricare dintre elementele bibliografice), de aceea o să facem doar câteva observații în legătură cu complexitatea lui.

- Caz favorabil $\Theta(1)$
- Caz defavorabil $\Theta(\log_2 n)$
- Caz mediu: presupunem $n = 2^k$ ($\log_2 n = k$)
 - * Găsim elementul la prima comparație, probabilitate $\frac{1}{n}$, număr de operații 1
 - * Găsim elementul la a doua comparație, probabilitate $\frac{2}{n}$, număr de operații 2

²Căutarea binară este definită și utilizată și în capitolul "Algoritmi de căutare și sortare"

- * ...
- * Găsim elementul la a k-a comparație, probabilitate $\frac{2^{k-1}}{n}$, număr de operații k
- * Dacă luăm suma: $\sum_{i=1}^k \frac{2^{i-1}}{n} * i = \frac{1}{n} * \sum_{i=1}^k i * 2^{i-1} = \dots < \log_2 n$
- * Complexitate medie este $O(\log_2 n)$
- * Complexitate totală este $O(\log_2 n)$

Algoritmul recursiv este și el cunoscut (iar pseudocodul este pe platforma elearning UnitBv, Modulul "Căutări și sortări"), de aceea vom comenta doar calcul de complexitate.

- Primul pas: stabilirea formulei de recurență. Presupunem că: $end - start = n$, atunci

$$T(n) = \begin{cases} 1 & n \leq 1 \\ T(\frac{n}{2}) + 1 & altfel \end{cases}$$

- La algoritmi recursivi e important să avem și termenul constant
- Complexitatea este $O(\log_2 n)$
- Cazul mediu: notăm $n = 2^k$ și calculăm toate valorile $T(2^k) = T(2^{k-1}) + 1, \dots, T(2^1) = T(2^0) + 1$
Adunăm toate aceste valori și obținem $T(2^k) = k + 1$, unde $k = \log_2 n$ deci complexitatea se obține $O(\log_2 n)$.

• Metode eficiente de sortare - Quick Sort³

Ideea algoritmului de sortare rapidă - Quick Sort:

- Se rearanjează și apoi se împarte tabloul unidimensional, de dimensiune n , $arr[1..n]$ în două subtablouri $arr[1..m]$ și $arr[m+1..n]$ astfel încât componentele lui $arr[1..m]$ sunt mai mici decât $arr[m+1..n]$; este importantă o rearanjare anterioară, pentru a evita diverse probleme legate de complexitate
- Se sortează fiecare dintre subtablourile obținute aplicând exact aceeași strategie de împărțire
- Se concatenează subtablourile sortate

Algoritmul Quick Sort este un algoritm de sortare prin pivotaj. Astfel, subliniem câteva idei pentru construirea/alegerea unui pivot:

- Se alege o valoare aleatoare din tablou (prima, ultima sau una arbitrară), aceasta reprezentând valoarea pivotului

³Sortarea rapidă este definită și utilizată și în capitolul "Algoritmi de căutare și sortare"

- Se rearanjează elementele tabloului astfel încât toate elementele care sunt mai mici decât valoarea aleasă să se afle în prima parte (stânga) a tabloului, iar valorile mai mari decât pivotul să se afle în partea a doua (dreapta) a tabloului
- Se pune valoarea pivotului pe poziția sa finală (astfel încât în stânga să avem elementele mai mici decât pivotul, iar în dreapta, elementele mai mari ca valoarea pivotului)

Un bun pivot împarte tabloul curent în două subtablouri de dimensiuni apropiate (partiționare echilibrată). Alegerea unui bun pivot este direct proporțională cu obținerea unei complexități mai bune.

O idee de rearanjare a elementelor:

- Se folosesc doi indici: unul care pornește de la poziția primul element (limităInferioară) iar celălalt care pornește de la poziția ultimului element (limităSuperioară)
- Se incrementează/decrementează indicii până când se identifică o inversiune (o pereche de indici $i < j$ cu proprietatea că $\text{arr}[i] > \text{pivot}$ și $\text{arr}[j] < \text{pivot}$)
- Se repară situația inversiunii prin interschimbarea elementelor
- Se continuă procesul până când indicii se "suprapun"

Complexitatea algoritmului de pivotaj se obține luând în calcul: dimensiunea problemei ($n = \text{limităSuperioară} - \text{limităInferioară} + 1$) și operația dominantă (comparația în care intervin elemente ale lui x); deci $T(n) = n + c$, $c = 0$ dacă $i = j$ și $c = 1$ dacă $i = j + 1$. Deci $T(n)$ aparține lui $\Theta(n)$.

Pentru analiza complexității totale a algoritmului se pleacă de la două situații generate de poziția pe care o ocupă pivotul în urma operației de partiționare. Astfel, în cel mai favorabil caz, pivotul este plasat în cadrul fiecărei subprobleme pe poziția din mijloc a subtabloului, în această situație problema fiind descompusă în două subprobleme de aceeași dimensiune. Cum fiecare subtablou de m elemente necesită maxim m comparații, este clar că pentru un număr de m autoapeluri pentru subtablouri de dimensiune $n \text{ div } m$ vor fi necesare $O(n)$ comparații. Cum există $\log n$ nivele, rezultă că algoritmul Quick Sort are o complexitate $O(n \log n)$ în cazul cel mai favorabil.

Există însă situații când această complexitate este distrusă: de exemplu un tablou care are deja elementele sortate - cazul cel mai defavorabil. La descompunerea problemei inițiale se va obține o singură subproblemă care urmărește sortarea unui tablou cu $n - 1$ elemente. Deoarece numărul de

nivele din recursivitate este n , complexitatea algoritmului degenerază către $O(n^2)$. Pentru a evita această deteriorare a complexității, de obicei se efectuează o permutare aleatoare a tabloului inițial înaintea aplicării algoritmului Quick Sort. De asemenea, trebuie reținut că acest algoritm de sortare presupune alocarea implicită a unei stive (datorată recursivității) de lungime ce variază între $\log n$ și n .

În cele ce urmează vom comenta analiza complexității, cazul mediu. Plecăm de la câteva ipoteze:

- Fiecare pas de partiționare necesită cel mult $n + 1$ comparații
- Există poziții posibile pentru pivot (o poziție notată cu q). Presupunem că fiecare dintre aceste poziții are aceeași șansă de a fi selectată (astfel $Probabilitate(q) = \frac{1}{n}$)
- Dacă pivotul se află pe poziția q atunci numărul de comparații este dat prin $T_q(n) = T(q - 1) + T(n - q) + (n + 1)$

Numărul mediu de comparații este:

$$\begin{aligned} T(n) &= (T_1(n) + \dots + T_n(n))/n = \\ &= ((T(0) + T(n-1)) + (T(1) + T(n-2)) + \dots + (T(n-1) + T(0)))/n + (n+1) = \\ &= 2(T(0) + T(1) + \dots + T(n-1))/n + (n+1) \end{aligned}$$

Deci

$$\begin{aligned} n * T(n) &= 2(T(0) + T(1) + \dots + T(n-1)) + n(n+1) \\ (n-1) * T(n-1) &= 2(T(0) + T(1) + \dots + T(n-2)) + (n-1)n \end{aligned}$$

Calculând diferența dintre ultimele două egalități obținem:

$$\begin{aligned} n * T(n) &= (n+1) * T(n-1) + 2n \\ T(n) &= (n+1)/n * T(n-1) + 2 \end{aligned}$$

Prin substituție inversă:

$$\begin{aligned} T(n) &= (n+1)/n * T(n-1) + 2 \\ T(n-1) &= n/(n-1) * T(n-2) + 2 | * (n+1)/n \\ T(n-2) &= (n-1)/(n-2) * T(n-3) + 2 | * (n+1)/(n-1) \\ &\dots \\ T(2) &= 3/2 * T(1) + 2 | * (n+1)/3 \end{aligned}$$

$$T(1) = 0 \mid * (n + 1)/2$$

$$T(n) = 2 + 2(n + 1)(1/n + 1/(n - 1) + \dots + 1/3) \approx 2(n + 1)(\ln n - \ln 3) + 2$$

Pentru ultima relația s-a folosit seria armonică: $1 + 1/2 + 1/3 + \dots + 1/n = \ln n + \text{const.}$ Astfel, în cazul mediu ordinul de complexitate este $n * \log(n)$.

Pseudocodul/Codul pentru algoritmul de sortare rapidă nu a fost prezentat în acest material, însă acesta, fiind un algoritm foarte cunoscut, se găsește în toate elementele bibliografice amintite. Sugerăm parcurgerea acestuia dintr-unul dintre materiale.

- Metode eficiente de sortare - MergeSort⁴

Ideea algoritmului de sortare prin interclasare - MergeSort:

- Se împarte tabloul unidimensional, $\text{arr}[1..n]$, de dimensiune n în două subtablouri $\text{arr}[1..[n/2]]$ și $\text{arr}[[n/2]+1..n]$
- Se sortează fiecare subtablou obținut mai sus
- Se interclasează elementele subtablourilor $\text{arr}[1..[n/2]]$ și $\text{arr}[[n/2]+1..n]$ și se construiește tabloul sortat $\text{aux}[1..n]$. Se mută tot conținutul tabloului auxiliar în $\text{arr}[1..n]$
- Algoritmul clasic de interclasarea presupune construirea unui tablou nou sortat pornind de la alte două tablouri sortate, de dimensiuni diferite

Principalul dezavantaj al sortării prin interclasare este faptul că utilizează un tablou adițional de dimensiunea tabloului de sortat.

Pentru a determina complexitatea algoritmului de sortare prin interclasare (algoritm recursiv), stabilim relația de recurență:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2 * T(\frac{n}{2}) + n & \text{altfel} \end{cases}$$

Presupunem $n = 2^k$ și înlocuim rând pe rând, astfel obținând:

$$T(2^k) = 2 * T(2^{k-1}) + 2^k$$

$$2 * T(2^{k-1}) = 2^2 * T(2^{k-2}) + 2^k$$

$$2^2 * T(2^{k-2}) = 2^3 * T(2^{k-3}) + 2^k$$

⁴Sortarea prin interclasare este definită și utilizată și în capitolul "Algoritmi de căutare și sortare"

$$\begin{array}{c}
 \dots \\
 2^{k-1} * T(2^1) = 2^k * T(2^0) + 2^k \\
 \hline
 T(2^k) = k * 2^k
 \end{array}$$

Dacă ne întoarcem la notația făcută, atunci $k = \log n$, deci $T(n) \in \Theta(n * \log_2 n)$.

Pseudocodul/Codul pentru algoritmul de sortare prin interclasare nu a fost prezentat în acest material, însă acesta, fiind un algoritm foarte cunoscut, se găsește în toate elementele bibliografice amintite. Sugerăm parcurgerea acestuia dintr-unul dintre materiale.

- Convoluție și transformata Fourier. Înmulțirea polinoamelor cu Divide et Impera.

Fiind date două polinoame $A(X)$ și $B(X)$, dorim să obținem produsul: $C(X) = A(X) * B(X)$, $\forall X$. Pentru simplitate, se considera gradul celor două polinoame ca fiind egal $n - 1$, lucru ce se poate obține ușor prin adăugarea unor zerouri ne semnificative corespunzător coeficienților lipsă. Prin urmare, coeficienții produsului vor fi:

$$c_k = \sum_{j=0}^k (a_j * b_{k-j}), k = 0, \dots, 2(n-1)$$

Se observă că acest produs este echivalent cu *operația de convoluție* dintre un vector A și un vector coloana B , însemnând produsul tuturor posibilelor shiftari relative, rezultatul arătând modul în care cele două funcții (de obicei) se influențează reciproc. Această tehnică e folosită în prelucrarea audio/de imagini, în operații de uniformizare (diminuarea sunetului de fundal, blur etc.).

Întrepretând astfel cele două polinoame, vectorul coeficienților lui C se numește convoluția celor doi vectori ai coeficienților A și B și e notată, de obicei $C = A \otimes B$.

În continuare, se dorește implementarea unui algoritm care să calculeze deci convoluția a doi vectori. Acest lucru poate fi obținut prin metode diferite:

- implementarea naivă, cu o complexitate $O(n^2)$;
- implementare Divide et Impera bazată pe tehnicile folosite în algoritmul lui Strassen, obținând o complexitate de $O(n * \ln 3)$;
- implementarea folosind Transformata Fourier: $O(n * \log n)$.

Până acum, am definit polinoamele printr-un vector de coeficienti, Însă, pentru a putea ajunge la complexitatea anunțată în introducere, este necesar să considerăm o altă metodă de reprezentare: *reprezentarea prin puncte*.

Conform Teoremei fundamentale a algebrei, pentru a defini un polinom de grad $n-1$, este suficient să determinăm un set de cel puțin n puncte distincte (x_k, y_k) din planul complex, unde $A(x_k) = y_k$. Acest lucru se face destul de ușor, prin selectarea unor valori x_k și determinarea valorii polinomului în acel punct, operație numită *evaluare*.

Dacă considerăm forma matriceală a produsului dintre matricea Vandermonde în care plasăm pe coloane cele n valori alese pentru a calcula valoarea polinomului și coloana coeficienților, rezultatul va reprezenta coloana valorilor polinomului în fiecare punct considerat:

$$V * A = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix} \quad (6.1)$$

Astfel, trecerea de la vectorul de coeficienți la reprezentarea pe puncte prin evaluare se face în $O(n^2)$. Operația inversă se va numi *interpolare* și va face trecerea de la un set de puncte la coeficienții asociați unui polinom, având aceeași complexitate. în mod similar, putem calcula coloana coeficienților unui vector $A = V^{-1} * Y$, folosind metoda eliminării a lui Gauss $O(n^3)$ sau, de asemenea $O(n^2)$, dacă V^{-1} este calculat în prealabil.

Avantajul acestei reprezentări constă în faptul că, date fiind două polinoame de gradul n printr-un set de astfel de puncte calculate pentru aceleași valori ale lui x_k , înmulțirea se poate face liniar: pentru orice x_k , $C(x_k) = A(x_k) * B(x_k)$, cu mențiunea că, datorită faptului că rezultatul înmulțirii a două polinoame de gradul n este un polinom de gradul $2n$, va fi necesară extinderea acestei reprezentări la un set de $2n$ puncte distincte.

Așadar, fiind date reprezentările extinse pentru două polinoame:

$$A(X) : \{(x_0, y_0), (x_1, y_1), \dots, (x_{2n-1}, y_{2n-1})\}$$

$$B(X) : \{(x_0, y'_0), (x_1, y'_1), \dots, (x_{2n-1}, y'_{2n-1})\}$$

Rezultatul înmulțirii va fi:

$$C(X) : \{(x_0, y_0 * y'_0), (x_1, y_1 * y'_1), \dots, (x_{2n-1}, y_{2n-1} * y'_{2n-1})\}$$

Așadar, performanța algoritmului de înmulțire pe puncte depinde de modul în care se face evaluarea și interpolarea, motiv pentru care vom alege valori

speciale pentru x_k considerați, așa numitele n rădăcini complexe ale unității, pe care le vom discuta în cele ce urmează.

Alegând astfel punctele, putem evalua un vector de coeficienți cu o complexitate $O(n * \log n)$ folosindu-ne de Transformata Fourier Discretă (DFT), al cărei algoritm de obținere se numește Fast Fourier Transform (FFT). Totodată, operația de interpolare se va folosi de Transformata Fourier Discretă Inversă (IDFT), cu algoritmul de obținere Inverse Fast Fourier Transform (IFFT), având aceeași complexitate $O(n * \log n)$.

Cei doi algoritmi folosiți în procesul de conversie (FFT și IFFT) urmăresc modelul unei implementări Divide Et Impera, deci, vom conveni ca polinoamele introduse ca input vor avea gradul o putere a lui 2 (lucru de care ne putem asigura prin adăugarea unor zerouri nesemnificative corespunzător coeficienților lipsă). De asemenea, datorită modificării gradului la $2n$ în urma înmulțirii, va fi nevoie să considerăm $2n$ rădăcini complexe ale unității. În cele ce urmează, definim câteva elemente legate de rădăcinile complexe ale unității și transformata Fourier.

– Rădăcinile complexe ale unității

O rădăcină complexă a unității este un număr ω astfel încât $\omega * n = 1$. Pentru orice număr n , există exact n rădăcini complexe ale unității ce se obțin din formula: $e^{\frac{2\pi i k}{n}}$, pentru $k = 0, \dots, n-1$, unde $e^{iu} = \cos(u) + i * \sin(u)$, iar u este un unghi în radiani. Aceste rădăcini ale unității au proprietatea că sunt egal distribuite de-a lungul cercului trigonometric.

De exemplu, pentru $n = 4$, cele 4 rădăcini ale unității sunt $1, -1, i, -i$, iar pentru $n = 8$, $\pm i, \pm 1, \pm(\frac{1}{\sqrt{2}} + i\frac{1}{\sqrt{2}}), \pm(-\frac{1}{\sqrt{2}} + i\frac{1}{\sqrt{2}})$.

Valorile $\omega_8^0, \omega_8^1, \dots, \omega_8^8$ din Figura 6.6 reprezintă cele 8 rădăcini ale unității, unde $\omega_8 = e^{\frac{2\pi i}{8}}$ este rădăcina principală a unității întrucât restul rădăcinilor reprezintă o putere a acesteia. Cele n rădăcini complexe ale unității formează împreună cu operația de înmulțire un grup izomorf cu $(Z_n, +)$, întrucât $\frac{n}{n} = \frac{0}{n} = 1$. Astfel, dintre proprietățile rădăcinilor unității, algoritmul FFT se folosește de faptul că:

$$\omega_n^n = 1, \omega_n^{n+k} = \omega_n^k, \omega_n^{\frac{n}{2}} = -1, (\omega_n^{k+\frac{n}{2}})^2 = (\omega_n^k)^2$$

Ultima proprietate se numește *Lema de înjumătățire* și este esențială pentru reducerea complexității în abordarea Divide Et Impera a algoritmului, întrucât ne demonstrează că cele n rădăcini pare ale unității sunt echivalente cu cele $\frac{n}{2}$ rădăcini ale unității (ultimele $\frac{n}{2}$ rădăcini sunt negativele primelor $\frac{n}{2}$). Deci, considerând gradul polinoamelor o putere a lui 2, vom putea face o împărțire corectă în subprobleme.

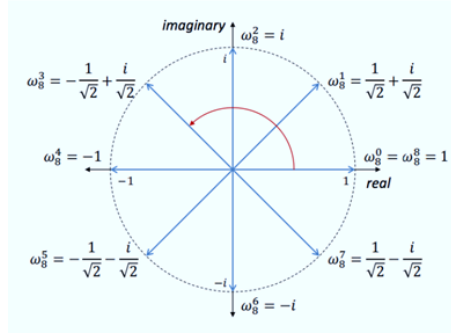


Figura 6.3: Rădăcini ale unității

Așadar, cunoscând aceste puncte, vom determina Transformata fourier discretă a unui vector de coeficienți. Calculăm pentru fiecare rădăcina a unității valoarea polinomului $A(X)$ am obținut:

$$y_k = \sum_{k=0}^{n-1} (a_k * \omega_n^{jk}), \forall j = 0, \dots, n-1$$

adică produsul matriceal:

$$\begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_1 & \omega_1^2 & \dots & \omega_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \dots & \omega_n^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix} \quad (6.2)$$

Vectorul $y = (y_0, y_1, \dots, y_{n-1})$ format din așezarea în linie a y_k se va numi Transformata Fourier Discretă (DFT) a vectorului de coeficienți ai polinomului $A(X)$, $a = (a_0, a_1, \dots, a_{n-1})$ și se notează $y = DFT_n(a)$. în cele ce urmează vom descrie algoritmul folosit pentru a obține această formă.

- FFT (Fast Fourier Transform)⁵ Fast Fourier Transform (FFT) este un algoritm Divide et Impera de complexitate $O(n * \log n)$. în contextul înmulțirii polinoamelor, acesta urmărește următoarea abordare recursivă: împarte vectorul de coeficienți în alți 2 vectori de coeficienți: unul corespunzător indicilor pari, iar celălalt, pentru indicii impari:

$$A^{[0]}(X) = a_0 + a_2 * X + a_4 * X^2 + \dots + a_{n-2} * X^{\frac{n}{2}-1}$$

⁵De văzut notițele de aici: https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-046j-design-and-analysis-of-algorithms-spring-2015/lecture-notes/MIT6_046JS15_writtenlec3.pdf

$$A^{[1]}(X) = a_1 + a_3 * X + a_5 * X^2 + \dots + a_{n-1} * X^{\frac{n}{2}-1}$$

Urmează evaluarea celor două polinoame de grad $\frac{n}{2}$ în punctele

$$(\omega_n^0)^2, (\omega_n^1)^2, \dots, (\omega_n^{n-1})^2$$

și combinarea rezultatului după formula:

$$A(X) = A^{[0]}(X^2) + X * A^{[1]}(X^2)$$

Conform *Lemei de înjumătățire*, punctele în care evaluăm cele două polinoame de grad $\frac{n}{2}$ nu sunt n valori distincte, ci $\frac{n}{2}$ valori distincte, suficiente pentru definirea fiecărui polinom. Așadar, repetând acești pași vom evalua recursiv polinoame de grad $\frac{n}{2}$ la $\frac{n}{2}$ rădăcini ale unității. Formula de recurență va fi:

$$T(n) = 2 * T\left(\frac{n}{2}\right) + O(n) = O(n * \log n)$$

întrucât funcția se reapelează de două ori, iar combinarea rezultatelor se face în timp liniar. O implementare în pseudocod a algoritmului ar putea fi în Figura 6.4:

```

RECURSIVE-FFT(a)
1  n = a.length           // n is a power of 2
2  if n == 1
3      return a
4   $\omega_n = e^{2\pi i/n}$ 
5   $\omega = 1$ 
6   $a^{[0]} = (a_0, a_2, \dots, a_{n-2})$ 
7   $a^{[1]} = (a_1, a_3, \dots, a_{n-1})$ 
8   $y^{[0]} = \text{RECURSIVE-FFT}(a^{[0]})$ 
9   $y^{[1]} = \text{RECURSIVE-FFT}(a^{[1]})$ 
10 for k = 0 to n/2 - 1
11      $y_k = y_k^{[0]} + \omega y_k^{[1]}$ 
12      $y_{k+(n/2)} = y_k^{[0]} - \omega y_k^{[1]}$ 
13      $\omega = \omega \omega_n$ 
14 return y                // y is assumed to be a column vector

```

Figura 6.4: Pseudocod pentru FFT

– IFFT (Inverse Fast Fourier Transform)

Inverse Discrete Fourier Transform (IFFT) este un algoritm folosit pentru a face conversia dintre reprezentarea prin puncte și cea prin coeficienți a unui polinom $A = V^{-1} * Y$. Pentru a face acest lucru fără a modifica complexitatea totală a algoritmului trebuie să găsim o modalitate convenabilă de a inversa matricea V , fără a face calculul efectiv.

Deoarece am folosit rădăcinile complexe ale unității ca puncte de referință, vom putea folosi proprietatea conjugatului unui număr complex: $\omega = a + ib \Rightarrow \bar{\omega} = a - ib, \omega = e^{\frac{2\pi i}{n}} \Rightarrow \bar{\omega} = e^{-\frac{2\pi i}{n}}$.

Astfel, folosind proprietățile rădăcinilor complexe ale unității, se poate demonstra că:

$$V * \bar{V} = n * I_n \Rightarrow V^{-1} = \frac{1}{n} \bar{V} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_1^{-1} & \omega_1^{-2} & \dots & \omega_1^{-(n-1)} \\ & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{-(n-1)} & \omega_n^{-2(n-1)} & \dots & \omega_n^{-(n-1)(n-1)} \end{bmatrix} \quad (6.3)$$

Se observă că, pentru a obține IDFT putem aplica același algoritm folosit la obținerea DFT, modificând $\omega = e^{\frac{2\pi i}{n}}$ cu $\bar{\omega} = e^{-\frac{2\pi i}{n}}$ și împărțind partea reală a rezultatului la n , căruia îi aplicăm ulterior o operație de rotunjire.

Codul pentru algoritmul de înmulțire a polinoamelor (alese la întâmplare dintr-un set) cu FFT este prezentat în secvența 6.1.

Secvență de cod 6.1: Algoritm pentru înmulțire a polinoamelor cu Divide et Impera

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <fstream>
5  #include <stdlib.h>
6  #include <complex>
7  #include <math.h>
8  #include <time.h>
9
10 using namespace std;
11
12 const double PI = acos(-1);
13 string filePath = "fileIn.txt";
14
15 enum {
16     direct,
17     invers
18 };
19
20 void readPolynomsFromFile(vector<vector<int>>& polinoame, string filePath)
21 {
22     ifstream myfile(filePath);
23     if (myfile.is_open())
24     {
25         while (!myfile.eof())
26         {
27             int gradCurent;
28             vector<int> polinomCurent;
29             myfile >> gradCurent;
30             polinomCurent.resize(gradCurent + 1);
31             for (int coefficient = 0; coefficient < polinomCurent.size();
32                 coefficient++)
33                 myfile >> polinomCurent.at(coefficient);
34             polinoame.push_back(polinomCurent);
35             myfile.close();
36         }
37     }

```

```

38         cout << "Unable to open file.";
39     }
40
41     void writePolynom(vector<int> toPrint)
42     {
43         bool isNOTFirstEntry = 0;
44         for (int coefficient = 0; coefficient < toPrint.size(); coefficient++)
45         {
46             if ((coefficient == 0 && toPrint.at(coefficient) != 0) || toPrint.size() ==
47                 ↪ 1)
48             {
49                 cout << toPrint.at(0) << ' ';
50                 isNOTFirstEntry = 1;
51             }
52             if (toPrint.at(coefficient) != 0 && coefficient != 0)
53             {
54                 if (toPrint.at(coefficient) > 0 && isNOTFirstEntry)
55                     cout << "+";
56                 if (toPrint.at(coefficient) != 1 && toPrint.at(coefficient) != -1)
57                     cout << toPrint.at(coefficient);
58                 if (toPrint.at(coefficient) == -1)
59                     cout << "-";
60                 if (coefficient == 1)
61                     cout << "X ";
62                 else
63                     cout << "X^" << coefficient << ' ';
64                 isNOTFirstEntry = 1;
65             }
66             cout << "\n";
67         }
68     }
69     vector<int> pickRandomPolynom(vector<vector<int>> polinoame)
70     {
71         return polinoame.at(rand() % polinoame.size());
72     }
73
74     vector<complex<double>> FFT(vector<complex<double>> myPolinom, int type)
75     {
76         if (myPolinom.size() == 1)
77             return vector<complex<double>>(1, myPolinom.at(0));
78         complex<double> nThrootOfUnity;
79         double angle = (2 * PI) / (myPolinom.size());
80         if (type == direct)
81             nThrootOfUnity = complex<double>(cos(angle), sin(angle));
82         else if (type == invers)
83             nThrootOfUnity = complex<double>(cos(angle), -sin(angle));
84         complex<double> currentrootOfUnity = 1;
85         vector<complex<double>> myPolinomEvenHalf, myPolinomOddHalf;
86         myPolinomEvenHalf.resize(myPolinom.size() / 2);
87         myPolinomOddHalf.resize(myPolinom.size() / 2);
88         for (int index = 0, indexEven = 0, indexOdd = 0; index < myPolinom.size();
89             ↪ index++)
90         {
91             if (index % 2 == 0)
92             {
93                 myPolinomEvenHalf.at(indexEven) = myPolinom.at(index);
94                 indexEven++;
95             }
96             else
97             {
98                 myPolinomOddHalf.at(indexOdd) = myPolinom.at(index);
99                 indexOdd++;
100             }
101         }
102         vector<complex<double>> y0 = FFT(myPolinomEvenHalf, type);
103         vector<complex<double>> y1 = FFT(myPolinomOddHalf, type);
104         vector<complex<double>> y;
105         y.resize(myPolinom.size());
106         for (int index = 0; index < (myPolinom.size() / 2); index++)
107         {
108             y.at(index) = y0.at(index) + currentrootOfUnity * y1.at(index);
109             y.at(index + (y.size() / 2)) = y0.at(index) - currentrootOfUnity * y1.at(
110                 ↪ index);
111             currentrootOfUnity = currentrootOfUnity * nThrootOfUnity;
112         }
113         return y;
114     }
115     vector<complex<double>> getProduct(vector<complex<double>> DFTfirst, vector<

```

```

115     ↪ complex<double>> DFTSecond)
116 {
117     vector<complex<double>> prod;
118     prod.resize(DFTfirst.size());
119     for (int index = 0; index < DFTfirst.size(); index++)
120         prod.at(index) = DFTfirst.at(index) * DFTSecond.at(index);
121     return prod;
122 }
123 void getNewSize(vector<complex<double>>& first, vector<complex<double>>& second)
124 {
125     int newsize = 1;
126     while (newsize < first.size() + second.size())
127         newsize *= 2;
128     first.resize(newsize);
129     second.resize(newsize);
130 }
131
132 vector<int> calculateProduct(vector<int> firstPolinom, vector<int> secondPolinom)
133 {
134     if ((firstPolinom.size() == 1 && firstPolinom.at(0) == 0) || (secondPolinom.
135         ↪ size() == 1 && secondPolinom.at(0) == 0))
136         return vector<int>{0};
137     vector<complex<double>> first(firstPolinom.begin(), firstPolinom.end());
138     vector<complex<double>> second(secondPolinom.begin(), secondPolinom.end());
139     getNewSize(first, second);
140     vector<complex<double>> firstPolinomDETFom = FFT(first, direct);
141     vector<complex<double>> secondPolinomDETFom = FFT(second, direct);
142     vector<complex<double>> product = getProduct(firstPolinomDETFom,
143         ↪ secondPolinomDETFom);
144     vector<complex<double>> productWithCoefficients = FFT(product, invers);
145     vector<int> result(productWithCoefficients.size());
146     for (int i = 0; i < productWithCoefficients.size(); i++)
147         result[i] = (round(productWithCoefficients[i].real())) /
148         ↪ productWithCoefficients.size();
149     return result;
150 }
151 void getProductOf2RandomPolynoms(vector<vector<int>> polinoame)
152 {
153     vector<int> firstPolinom = pickRandomPolynom(polinoame);
154     vector<int> secondPolinom = pickRandomPolynom(polinoame);
155     cout << "Rezultatul inmultirii dintre \n";
156     writePolynom(firstPolinom);
157     cout << "si \n";
158     writePolynom(secondPolinom);
159     vector<int> rezultat = calculateProduct(firstPolinom, secondPolinom);
160     cout << "este: \n";
161     writePolynom(rezultat);
162 }
163
164 int main()
165 {
166     srand(time(NULL));
167     vector<vector<int>> polinoame;
168     readPolynomsFromFile(polinoame, filePath);
169     getProductOf2RandomPolynoms(polinoame);
170 }

```

6.2 Algoritmi Divide et Impera - Probleme rezolvate

Fie următoarele probleme a căror rezolvare implică detalii despre metoda Divide et Impera sau a căror rezolvare implică cunoștințe despre probleme clasice specifice Divide et Impera (sortări, căutare binară, etc):

1. Se consideră polinomul $p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$, unde $a_i \neq 0, \forall i$. Numărul minim de înmulțiri care se pot efectua pentru a evalua polinomul în punctul x este:

- A. 3
- B. 4
- C. 6
- D. 9

Explicație: $p(x) = a_0 + x(a_1 + x(a_2 + a_3x))$

2. Complexitatea pentru funcția de mai jos este: (presupunem $n > 0$)

Secvență de cod 6.2: Cod C++ pentru funcția recursive() de la exercițiul 8.1

```

1  int recursive (int n)
2  {
3      if (n == 1)
4          return 1;
5      else
6          return recursive(n-1) + recursive(n-1);
7  }
```

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(2^n)$

Explicație: Relația de recursivitate pentru algoritmul de mai sus este:

$$T(n) = \begin{cases} c_1 & n = 1 \\ 2T(n-1) + c & \text{altfel} \end{cases}$$

Astfel, vom avea: $T(n) = 2(2T(n-2)+c)+c = 4T(n-2)+3c$, $T(n) = 8T(n-3)+6c+c = 8T(n-3)+7c$, $T(n) = 16T(n-4)+14c+c = 16T(n-4)+15c$, ..., $T(n) = 2^{n-1}T(1) + (2^{n-1} - 1)c$, deci $T(n) = O(2^n)$.

3. Se consideră un tablou unidimensional de dimensiune dată. Să se afle cmmdc-ul elementelor din tablou cu Divide et Impera. Care este complexitatea obținută?
4. Se citește un vector de n numere întregi ordonat descrescător din fișier txt dat. Se citește un al doilea vector de m numere întregi din consolă. Să se determine care elemente din al doilea vector apar în primul și pe ce poziții.
5. Să se determine al k -lea element în ordine crescătoare dintr-un vector cu n elemente, fără a sorta complet vectorul.
6. Se consideră o matrice. Se numește "vârf", un element care este mai mare sau egal ca cei patru vecini, sus, stânga, dreapta, jos. De exemplu, pentru $A[i][j]$ avem $A[i-1][j]$, $A[i+1][j]$, $A[i][j-1]$, $A[i][j+1]$. Pentru cazurile periferice, considerăm că vecinii care lipsesc sunt $-\infty$. Să se caute astfel de elemente "vârf" într-o matrice dată.

Exemplu:

$$\begin{bmatrix} 10 & 20 & 15 \\ 21 & 30 & 14 \\ 7 & 16 & 32 \end{bmatrix} \quad (6.4)$$

Răspunsul pe care îl căutăm este: 30 (sau 32).

Pentru problema 3 avem:

- Date de intrare: vectorul de numere întregi + dimensiunea acestuia;
- Date de ieșire: valoarea cmmdc calculată pentru între toate numerele din tabloul dat;
- Algoritm de rezolvare: pentru rezolvarea acestei probleme o să folosim principiul metodei de Divide et Impera, și anume: împățim problema în care multe probleme de același tip și repetăm împărțirea până în momentul în care ajungem la cazuri particulare, rezolvabile banal (de obicei, ajungem la dimensiune 1 sau 2). În cazul nostru, cazul particular pe care îl urmărim este: cazul tablourilor de 1 sau 2 elemente, deoarece în aceste cazuri putem spune că problema este absolut rezolvabilă (prin calcul de cmmdc între două numere sau returnarea directă a numărului).

Astfel, rezolvarea problemei mari, implică rezolvarea mai multor probleme (de dimensiune redusă), dar de același tip, adică operația esențială este de cmmdc. Totul o să se reducă la calculul cmmdc-ului între două secvențe (left, right), unde concluzionăm că cmmdc-ul tuturor elementelor este cmmdc-ul calculat între partea stângă din tablou și partea dreaptă.

Pentru calculul cmmdc-ului se poate utiliza orice algoritm cunoscut, în cazul nostru s-a folosit algoritmul prin scăderi repetate.

- Algoritmul scris în C++ este 6.3. Se poate observa clar împărțirea problemei în două secvențe, caz particular x, y , fiecare reținând cmmdc-ul format pe partea stângă a tabloului, respectiv pe partea dreaptă.

Secvență de cod 6.3: Cod C++ pentru problema 3

```

1  #include <iostream>
2  using namespace std;
3
4  int cmmdcVector(int vector[100], int left, int right)
5  {
6      if (left == right) return vector[left];
7      else
8      {
9          int x, y;
10         x = cmmdcVector(vector, left, (left + right) / 2);
11         y = cmmdcVector(vector, (left + right) / 2 + 1, right);
12         while (x != y)
13             if (x > y) x = x - y;
14             else y = y - x;
15         return x;
16     }
17 }
18
19 int main()
20 {
21     int elem[100], dim;
22     cin >> dim;
23     for (int index = 0; index < dim; index++)
24         cin >> elem[index];
25     cout << cmmdcVector(elem, 0, dim - 1);
26
27     return 0;
28 }

```

- Analiza complexității. Din punct de vedere al complexității de timp, o abordare Divide et Impera pentru această problemă implică trecerea prin toate componentele vectorului. De aici, complexitatea obținută este $O(n)$ - idem varianta clasică de obținere cmmdc (calcul cmmdc două câte două elemente, până se termină șirul de numere).

De menționat este faptul că, în general, rezolvarea unei probleme cu Divide et Impera poate duce la o complexitate mai bună decât rezolvare clasică (vezi căutare secvențială vs căutarea binară, vezi metode de sortare eficiente), însă în acest caz, problema considerată nu este o problemă specifică Divide et Impera (este o problemă pentru care alegem voit rezolvarea cu Divide et Impera nu pentru complexitate mai bună, înă pentru dezvoltarea capacităților de a înțelege principiile car stau la bază metodei de Divide et Impera).

Pentru problema 4 avem:

- Date de intrare: cei doi vectori de numere întregi + dimensiunile acestora;
- Date de ieșire: afișarea acelor elemente din al doilea șir care apar în primul, plus pozițiile pe care apar;

- Algoritm de rezolvare: pentru rezolvarea acestei probleme o să utilizăm căutarea binară. De ce? Pentru că în enunțul problemei avem o caracteristică a vectorului sursă (cel în care căutăm), și anume "ordonat descrescător". Această proprietate trebuie să o folosim în rezolvare, de aceea, o să aplicăm căutare binară pentru fiecare dintre elementele celui de al doilea șir luat în considerare ("verificare"). pentru fiecare element din vectorul "verificare" se aplică căutarea binară, cu afișare inclusiv a poziției unde se află (în cazul în care testă un element care nu se află în sursă, atunci avem valoarea -1).
- Algoritmul scris în C++ este 6.5. Rezolvarea conține două metode: una care se ocupă cu citirea input-ului problemei, iar o alta este funcția de căutare binară clasică.

Secvență de cod 6.4: Cod C++ pentru problema 4

```

1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  void citire(int &dimSursa, int sursa[101], int &dimVerificare, int verificare
    ↪ [101])
6  {
7      ifstream f("date.in");
8      f >> dimSursa;
9      for (int index = 0; index < dimSursa; index++)
10         f >> sursa[index];
11     cin >> dimVerificare;
12     for (int index = 0; index < dimVerificare; index++)
13         cin >> verificare[index];
14     f.close();
15 }
16
17 int cautare_binara(int dim, int vector[101], int elem)
18 {
19     int mij, i = 0, j = dim;
20     while (i <= j)
21     {
22         mij = (i + j) / 2;
23         if (vector[mij] == elem)
24             return mij;
25         if (vector[mij] > elem)
26             i = mij + 1;
27         else if (vector[mij] < elem)
28             j = mij - 1;
29     }
30     return -1;
31 }
32
33 int main()
34 {
35     int n, m, a[101], v[101];
36     citire(n, v, m, a);
37     for (int index = 0; index < m; index++)
38     {
39         int c = cautare_binara(n, v, a[index]);
40         if (c != -1)
41             cout << a[index] << " " << c + 1 << "\n";
42     }
43     return 0;
44 }

```

- Analiza complexității.

Din punct de vedere al complexității de timp, calculul se face astfel: avem un vector "sursă" de dimensiune "dimSursa", plus un vector "verificare", de

dimensiune "dimVerificare"; pentru aceștia s-a procedat astfel: s-a aplicat căutare binară pentru fiecare lement din vectorul "verificare", element căutat în vectorul sursă. Astfel, complexitatea obținută este: $dimVerificare * O(\log dimSursa)$.

Pentru problema 5 avem:

- Date de intrare: vectorul de numere întregi + dimensiune acesta, n ;
- Date de ieșire: al k -lea element în ordine crescătoare din vector;
- Algoritm de rezolvare: pentru a obține al k -lea element, luăm în calcul faptul că al k -lea semnifică faptul că sunt $k - 1$ elemente mai mici ca el în șirul considerat.
- Algoritmul scris în C++ este 6.6. În rezolvare(pentru diversivitatea exemplelor de probleme rezolvate) s-a folosit `std::vector`.

Secvență de cod 6.5: Cod C++ pentru problema 4

```

1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  void readVector(std::vector<int>& vec)
6  {
7      int lenght;
8      std::cout << "Introduceti lungimea vectorului: ";
9      std::cin >> lenght;
10     vec.resize(lenght);
11     std::cout << "Introduceti elementele vectorului: ";
12     for (int index = 0; index < lenght; index++)
13         std::cin >> vec[index];
14 }
15
16 int getPivot(std::vector<int>& vec, int lowerBound, int upperBound)
17 {
18     int pivot = vec[upperBound];
19     int smallerThanPivotIndex = lowerBound - 1;
20     for (int currentIndex = lowerBound; currentIndex < upperBound; currentIndex
21         ↪ ++ )
22     {
23         if (vec[currentIndex] <= pivot)
24         {
25             smallerThanPivotIndex++;
26             std::swap(vec[smallerThanPivotIndex], vec[currentIndex]);
27         }
28     }
29     std::swap(vec[smallerThanPivotIndex + 1], vec[upperBound]);
30     return smallerThanPivotIndex + 1;
31 }
32
33 int findKthElement(int k, std::vector<int>& vec, int lowerBound, int upperBound)
34 {
35     if (k >= 1 && k <= upperBound + 1)
36     {
37         int pivot = getPivot(vec, lowerBound, upperBound);
38         if (pivot + 1 == k)
39             return vec.at(pivot);
40         if (pivot + 1 > k)
41             return findKthElement(k, vec, lowerBound, pivot - 1);
42         return findKthElement(k, vec, pivot + 1, upperBound);
43     }
44     return -1;
45 }
46
47 void validateResult(int k, int kthElement)
48 {

```

```

48     if (kthElement != -1)
49     {
50         std::cout << "In ordine crescatoare, ";
51         if (k == 1)
52             std::cout << "primul element din vector este: " << kthElement << "\n"
53             ↪ ;
54         else
55             std::cout << "al " << k << "-lea element din vector este: " <<
56             ↪ kthElement << "\n";
57     }
58     else if (k < 0)
59         std::cout << "Va rugam introduceti un indice valid\n";
60     else
61         std::cout << "Vectorul dat are mai putin de " << k << " elemente\n";
62 }
63
64 void getKthElement(std::vector<int>& vec)
65 {
66     int k;
67     std::cout << "Introduceti indicele elementului cautat: ";
68     std::cin >> k;
69     while (k != 0)
70     {
71         int kthElement = findKthElement(k, vec, 0, vec.size() - 1);
72         validateResult(k, kthElement);
73         std::cout << "Introduceti indicele elementului cautat: ";
74         std::cin >> k;
75     }
76 }
77
78 int main()
79 {
80     std::vector<int> vec;
81     readVector(vec);
82     getKthElement(vec);
83 }

```

- Analiza complexității.

Cât credeți că este complexitatea pentru algoritmul propus mai sus? Ce se întâmplă (din punct de vedere complexitate) dacă se sorta vectorul și apoi se accesa elementul al k -lea?

Pentru problema 6 algoritmul de rezolvare constă în utilizarea corectă a algoritmului de căutare binară. Astfel, se consideră următorii pași:

- Se consideră coloana de la mijloc și se caută elementul maxim de pe aceasta;
- Presupunem că indexul coloanei de la mijloc este "mid", iar valoarea elementului maxim calculat este "max", iar elementul maxim este "mat[max_index][mid]";
- Dacă $max \geq A[index][mid - 1] \& max \geq A[index][pick + 1]$, atunci max este "vârf" și se returnează max;
- Dacă $max < mat[max_index][mid - 1]$, atunci se reapelează pe ramura stângă;
- Dacă $max < mat[max_index][mid + 1]$, atunci se reapelează pe ramura dreaptă.

Astfel, codul C++ asociat pentru 6 este 6.6.

Secvență de cod 6.6: Cod C++ pentru problema 6

```

1  #include<iostream>
2  using namespace std;
3
4  const int MAX = 1000;
5  int findMax(int arr[][MAX], int rows, int mid, int &max)
6  {
7      int max_index = 0;
8      for (int index = 0; index < rows; index++)
9      {
10         if (max < arr[index][mid])
11         {
12             max = arr[index][mid];
13             max_index = index;
14         }
15     }
16     return max_index;
17 }
18
19 int findPeakRec(int arr[][MAX], int rows, int columns, int mid)
20 {
21     int max = 0;
22     int max_index = findMax(arr, rows, mid, max);
23
24     if (mid == 0 || mid == columns - 1)
25         return max;
26
27     if (max >= arr[max_index][mid - 1] &&
28         max >= arr[max_index][mid + 1])
29         return max;
30
31     if (max < arr[max_index][mid - 1])
32         return findPeakRec(arr, rows, columns, mid - mid / 2);
33
34     return findPeakRec(arr, rows, columns, mid + mid / 2);
35 }
36
37
38 int findPeak(int arr[][MAX], int rows, int columns)
39 {
40     return findPeakRec(arr, rows, columns, columns / 2);
41 }
42
43 int main()
44 {
45     int arr[][MAX] = { { 10, 8, 10, 10 },
46                        { 14, 13, 12, 11 },
47                        { 15, 9, 11, 21 },
48                        { 16, 17, 19, 20 } };
49
50     int rows = 4, columns = 4;
51     cout << findPeak(arr, rows, columns);
52     return 0;
53 }

```

Complexitatea pentru o astfel de direcție de rezolvare este: $O(rows * \log(columns))$ (luat în calcul complexitatea pentru căutare binară).

Spațiu auxiliar considerat: $O(\frac{columns}{2})$ pentru apelurile recursive.

6.3 Algoritmi Divide et Impera - Probleme propuse

Problemele pe care le propunem în această secțiune sunt:

1. Presupunând că putem să facem înmulțirea a două matrice de ordin 3 cu 21 de înmulțiri de scalari, cât de rapid se poate face înmulțirea dintre două matrice de ordin n ?
 - A. $\Theta(n^{\log_3 21})$
 - B. $\Theta(n^{\log_2 21})$
 - C. $\Theta(n^{\log_9 21})$
 - D. $\Theta(n^2)$
2. Care dintre răspunsuri este adevărat în cazul în care algoritmul din Figura 6.5 se apelează pentru un tablou arr , de dimensiune dată, $n > 1$, iar la analiza complexității se consideră că operația dominantă este adunarea:

```

    algoritm(x[s..d])
    IF s>d THEN
        return 0
    ELSE
        IF s=d THEN
            return x[s]
        ELSE
            m ← (s+d)/2
            rez1=algoritm(x[s..m])
            rez2=algoritm(x[m+1..d])
            return rez1+rez2
        ENDIF
    ENDIF

```

Figura 6.5: Pseudocod pentru algoritm exercițiul 2)

- A. Algoritmul returnează $arr[1]+arr[n]$, cu complexitatea $O(1)$
 - B. Algoritmul returnează suma tuturor elementelor din x , cu complexitatea $O(\log n)$
 - C. Algoritmul returnează suma tuturor elementelor din x , cu complexitatea $O(n)$
 - D. Algoritmul returnează 0 și are complexitate $O(1)$
3. Care este numărul minim de înmulțiri care se pot efectua pentru a calcula x^{15} (inclusiv ridicare la pătrat)

- A. 4
- B. 5
- C. 7
- D. 9

4. Considerăm problema căutării unui element x într-un tablou de dimensiune n . Problema se poate rezolva în timp $O(\log n)$ dacă:

- 1) șirul este sortat;
- 2) șirul este sortat și rotit cu k poziții, $k \leq n$ dat;
- 3) șirul este sortat și rotit k poziții, $k \leq n$, k nu este precizat;
- 4) șirul nu este sortat;

Alege variantele corecte:

- A. 1)
- B. 1) și 2)
- C. 1), 2) și 3)
- D. toate

5. Fie un tablou unidimensional ordonat crescător de n numere întregi distincte. Să se determine un indice m , $0 \leq m \leq n - 1$, cu $vect[m] = m$, dacă este posibil. În cazul în care NU există un astfel de indice, atunci să se afișeze -1. Indicație: folosim ideea de la căutarea binară.

Indicație: Metoda aleasă se bazează pe faptul ca vectorul este deja sortat și folosește principiul unei căutari binare obișnuite, verificând dacă elementul median are proprietatea cerută. Dacă da, îl returnează, iar dacă nu, continuă căutarea după cum urmează:

- dacă elementul median este mai mare decât indicele său, căutarea va continua pe ramura dreaptă;
- dacă elementul median este mai mic decât indicele său, căutarea va continua pe ramura stângă.

Astfel, complexitatea algoritmului este următoarea:

- caz favorabil: Elementul median este egal cu indicele său $\Rightarrow O(1)$;
- caz nefavorabil: Nu există niciun punct cu proprietatea cerută $\Rightarrow$ algoritmul fiind unul de tipul căutării binare va avea complexitatea $O(\log n)$;

- caz mediu: pe caz mediu, complexitatea căutării binare este, de asemenea, $O(\log n)$.
6. Se dau $n - 1$ numere naturale distincte între 0 și $n - 1$. Scriind o funcție care implică metoda Divide et Impera, determinați numărul care lipsește.
 7. Se dă un șir de n elemente. Se poate ca anumite elemente să se repete (anumite elemente pot apărea de mai multe ori în șir). Găsiți o variantă de a șterge duplicatele eficient - complexitatea dorită pentru metoda de ștergere este $O(n * \log n)$.
 8. Se dă un vector de dimensiune foarte mare, vector care conține doar elemente de 0 și de 1. Vectorul îndeplinește proprietatea că toate elementele de 1 sunt la început, iar toate elementele de 0 la final. Care este numărul total de zerouri?
Indicație: Cum vectorul este sortat, se poate utiliza căutarea binară pentru a obține prima apariție a lui 0. O dată ce avem această poziție, *index*, numărul pe care îl căutăm este *dimensiune - index*. Complexitatea este $O(\log n)$.
 9. Se consideră o matrice $n \times n$ și un element dat, x . Știind că în matricea dată fiecare linie și fiecare coloana este sortată crescător, să se verifice dacă elementul x se află în matrice. Se cere complexitate liniară.

Intrarea este de forma:

$$matrix[4][4] = \begin{bmatrix} 10 & 20 & 30 & 40 \\ 15 & 25 & 35 & 45 \\ 27 & 29 & 37 & 48 \\ 32 & 33 & 39 & 50 \end{bmatrix} \quad (6.5)$$

$x = 29$

Ieșirea: Element găsit pe poziția (2, 1)

Indicație: Incercați să definiți o regulă de a parcurge matricea cu proprietățile propuse, de exemplu, parcurgere plecând dintr-un colț.

Capitolul 7

Algoritmi Greedy

7.1 Aspecte teoretice

O clasă de probleme frecvent întâlnită în practică este cea a problemelor de optimizare de tipul:

Să se determine $x \in X$ astfel încât:

- *x satisface anumite condiții (restricții);*
- *x optimizează (minimizează sau maximizează) un criteriu.*

O subclasă importantă în informatică o reprezintă cea în care X este o mulțime finită, problema de optimizare fiind numită în acest caz discretă sau combinatorială. De asemenea, dacă acel criteriu pe care dorim să îl optimizăm este o funcție liniară, atunci ne încadrăm într-o altă subclasă de problema, și anume probleme de programare liniară. Indiferent de situație, la o primă vedere problema pare foarte simplă: pentru primul caz fiind suficient să se parcurgă X și să se aleagă elementul care satisface restricțiile și optimizează criteriul (o astfel de abordare bazată pe metoda forței brute devine inefficientă atunci când numărul de elemente din X este foarte mare), iar pentru al doilea caz, există oricând posibilitatea aplicării principiilor care stau la baza algoritmilor de tip simplex¹.

Despre un algoritm Greedy este adevărat că:

- tehnica Greedy determină soluția unei probleme de optimizare pe baza alegerii unui optim local;
- tehnica conduce către algoritmi simpli, intuitivi și eficienți (de obicei operația dominantă este cea de selecție a elementelor);
- alegerea se face fără revenire, componentă cu componentă în soluția finală;

¹ https://en.wikipedia.org/wiki/Simplex_algorithm

- folosirea criteriului de alegere se face pe setul de date de intrare, eventual cu o pregătire prealabilă acestora;
- modul de rezolvare al problemelor este construcția soluției componentă cu componentă, plecându-se inițial de la mulțimea vidă;
- dacă o componentă satisface criteriul de alegere corespunzător configurației curente $\Rightarrow$ participă la soluția finală;
- pentru a alege o astfel de componentă trebuie să ne asigurăm că alegerea **optimului local** conduce la **optimul global**;
- tehnica nu conduce întotdeauna la o soluție optimă, plus că există situații în care soluția găsită este suboptimă (adică apropiată de valoarea optimă, dar nu exact valoarea optimă - de aceea, trebuie să ne asigurăm că o problema are proprietate de substructură optimă).

Atenție că există o serie de condiții pentru a aplica tehnica Greedy:

- trebuie să existe o mulțime de **candidați**;
- trebuie să avem definită o funcție care verifică dacă o anumită mulțime de candidați constituie o **soluție posibilă**, nu neapărat optimă, a problemei;
- trebuie să definim o funcție care verifică dacă o mulțime de candidați este **fezabilă**, adică dacă este posibil să completăm această mulțime astfel încât să obținem o soluție posibilă, nu neapărat optimă, a problemei;
- există o **funcție de selecție** care indică la orice moment care este cel mai promițător dintre candidații încă nefolosiți;
- există o **funcție obiectiv** care dă valoarea unei soluții, funcția pe care urmărim să o optimizăm (minimizăm/maximizăm).

Descrierea formală a unui algoritm Greedy este:

{C este mulțimea candidaților}

{S este mulțimea în care construim soluția}

subalgoritm greedy(C) este:

$S \leftarrow \emptyset$

while not soluție(S) and $C \neq \emptyset$ do

$x \leftarrow$ un element din C care maximizează/minimizează

select(x)

$C \leftarrow C - \{x\}$

if fezabil($S \cup x$) then $S \leftarrow S \cup x$

```
if soluție(S) then returnează S  
else returnează "nu există soluție"
```

În descrierea de mai sus este necesar să observăm următoarele: la fiecare pas se alege cel mai bun candidat de la momentul respectiv, fără a studia alternativele disponibile în momentul respectiv și viabilitatea acestora în timp. Dacă un candidat este inclus în soluție, rămâne acolo, fără a putea fi modificat, iar dacă este exclus din soluție, nu va mai putea fi niciodată selectat drept un potențial candidat. Funcția *select* este de obicei derivată din funcția obiectiv, iar uneori aceste două funcții sunt chiar identice.

Este foarte important să identificăm cazurile în care se poate aplica Greedy și cazurile în care este nevoie de altceva. Astfel, există probleme în care urmărind criteriul Greedy nu ajungem la soluția optimă. Sunt situații în care această soluție neoptimă este o aproximare suficientă pentru ce avem nevoie. Problemele NP-complete (nondeterministic polynomial time) necesita multă putere de calcul pentru a găsi optimul absolut. Pentru a optimiza aceste calcule mulți algoritmi folosesc decizii Greedy și găsesc un optim foarte aproape de cel absolut.

7.2 Algoritmi Greedy - Probleme rezolvate

Fie următoarele probleme:

1. Lucy are un program foarte încărcat pentru ziua următoare în care multe activități se desfășoară în același timp. Ea hotărăște să își aleagă cât mai multe activități care se desfășoară în momente diferite.

Ajutați-o pe Lucy să își organizeze activitățile astfel încât să poată face cât mai multe într-o zi.

Date de intrare: "n" - numărul total de intervale de timp, un vector cu "n" activități la care știm "s" și "f" (timpul de START și de FINISH), cu restricția: $s_i < f_i, \forall i \in 0, 1, \dots, n$

Date de ieșire: O serie maximă de activități care nu se desfășoară în același timp.

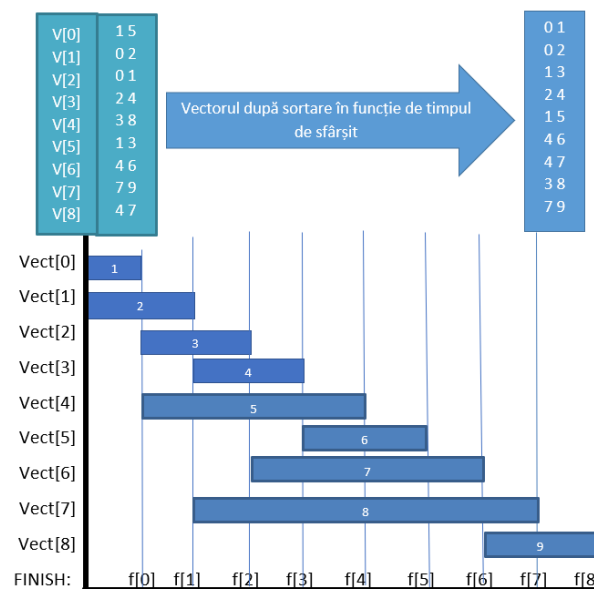


Figura 7.1: Exemplu problema cu activități

Exemplu: Se consideră activitățile din Figura 7.1. Maximul număr de activități care nu se suprapun este 4. Aceste activități pot fi: 1, 3, 6, 9 sau 1, 4, 6, 9 sau 2, 4, 6, 9.

Rezolvare: Primul pas este să ordonăm activitățile în ordine crescătoare în funcție de timpul de FINISH. Apoi, parcurgem vectorul și la fiecare pas verificăm dacă timpul de start al activității curente este mai mare decât timpul

de sfârșit al ultimei activități salvate: $\text{vect}[i].\text{start} \geq \text{vect}[i-1].\text{finish}$. Dacă da, atunci numărul de activități posibile crește și salvăm poziția i . Ultimul pas din rezolvare este să afișăm numărul maxim posibil și activitățile care nu se suprapun.

Complexitate: în orice caz, numărul de pași este același deoarece vom ordona activitățile $O(n \cdot \log n)$ și apoi vom parcurge vectorul $O(n)$. Așadar complexitatea depinde de tipul de ordonare și poate fi: în caz favorabil: $n \cdot \log n$ și în caz defavorabil: n^2 .

Algoritmul scris în C++ este 7.1. Semnalăm utilizarea unei structuri pentru a reține variabilele de tip "Activitate".

Secvență de cod 7.1: Cod C++ pentru problema 1

```

1
2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5
6 struct Activity
7 {
8     int start;
9     int finish;
10 };
11 bool comp(Activity nr1, Activity nr2)
12 {
13     return nr1.finish < nr2.finish;
14 }
15 void Read(std::vector<Activity>& array)
16 {
17     int number;
18     std::cout << "Number of activities=";
19     std::cin >> number;
20     std::cout << "Activities:" << "\n";
21
22     array.resize(number);
23     for (int i = 0; i < array.size(); i++)
24     {
25         std::cout << i + 1 << " ";
26         std::cin >> array[i].start >> array[i].finish;
27     }
28 }
29 void Number(std::vector<Activity>& array)
30 {
31     int count = 1;
32     std::vector<int> maxi;
33     maxi.resize(array.size());
34     maxi[0] = 1;
35
36     for (int i = 1; i < array.size(); i++)
37     {
38         if (array[i].start >= array[maxi[count - 1] - 1].finish)
39         {
40             maxi[count] = i + 1;
41             count++;
42         }
43     }
44     std::cout << "Maximum number of activities is " << count << ".\n";
45     std::cout << "The activities are:";
46     for (int i = 0; i < count; i++)
47     {
48         std::cout << "\n";
49         std::cout << array[maxi[i] - 1].start << " " << array[maxi[i] - 1].finish;
50     }
51 }
52 int main()
53 {
54     std::vector<Activity> array;
55     Read(array);
56
57     sort(array.begin(), array.end(), comp);

```

```
58     Number(array);  
59 }
```

2. David lucrează ca bucatăar într-un restaurant. În fiecare dimineață, el trebuie să conducă de acasă la restaurant pe un drum drept cu lungimea X . Drumul are exact două benzi numerotate 1 și 2, dar există N obstacole (numerotate de la 1 la N) pe drum. Pentru fiecare i , al i -lea obstacol blochează banda L_i la distanța X_i și nu blochează cealaltă bandă.

David nu poate trece printr-un obstacol. Pentru a evita obstacolul i , poate schimba banda la distanța x_{i-1} , iar în această poziție nu trebuie să existe un alt obstacol. Cu toate acestea, de fiecare dată când schimbă banda, nu o poate schimba din nou până când nu conduce pentru cel puțin o distanță D spre restaurant. David poate începe pe orice banda își dorește. Uneori, este imposibil să ajungă la restaurant.

Care este distanța maximă pe care David o poate parcurge înainte de a ajunge la un obstacol?

Dacă ajunge la restaurant atunci răspunsul este X .

Exemple:

- Distanța până la restaurant = 10, distanța pentru a schimba banda = 20, număr de obstacole = 2, distanța obstacolelor = 4, 7, poziția obstacolelor = 1, 2
- Distanța până la restaurant = 15, distanța pentru a schimba banda = 20, număr de obstacole = 4, distanța obstacolelor = 4, 6, 9, 13, poziția obstacolelor = 1, 2, 2, 1
- Distanța până la restaurant = 10, distanța pentru a schimba banda = 1, număr de obstacole = 5, distanța obstacolelor = 1, 3, 5, 7, 9, poziția obstacolelor = 1, 2, 1, 2, 1
- Distanța până la restaurant = 10, distanța pentru a schimba banda = 2, număr de obstacole = 2, distanța obstacolelor = 4, 5, poziția obstacolelor = 1, 2

Răspunsuri:

- 10 (Se începe pe banda 2 și schimbă banda înainte de al doilea obstacol pentru a ajunge la restaurant)
- 13 (Se începe pe banda 2, se schimbă banda înainte de al doilea obstacol, dar nu poate să evite ultimul obstacol)
- 10 (Se începe pe banda 2 și schimbă banda înainte de fiecare obstacol pentru a ajunge la restaurant)

- 5 (Se începe pe banda 2, dar nu poate evita al doilea obstacol deoarece nu este suficient spațiu între cele 2 obstacole)

Algoritmul scris în C++ este 7.2.

Secvență de cod 7.2: Cod C++ pentru problema 2

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void rezolvare ()
6  {
7      int numarObstacole, distantaRestaurant, distantaSchimbareBanda;
8      cout << "Distanța pana la restaurant: ";
9      cin >> distantaRestaurant;
10     cout << "Distanța pentru a putea schimba banda: ";
11     cin >> distantaSchimbareBanda;
12     cout << "Numarul de obstacole: ";
13     cin >> numarObstacole;
14     vector<int> distantaObstacol(numarObstacole);
15     for (int i = 0; i < numarObstacole; i++)
16     {
17         cin >> distantaObstacol[i];
18     }
19     vector<int> bandaObstacol(numarObstacole);
20     for (int i = 0; i < numarObstacole; i++)
21     {
22         cin >> bandaObstacol[i];
23     }
24     int distantaParcursaFinal = 0, bandaCurenta, distantaParcursaCurenta = 0,
25         ↪ distantaParcursaMinima = 0;
26     if (bandaObstacol[0] == 1)
27     {
28         bandaCurenta = 2;
29     }
30     else
31     {
32         bandaCurenta = 1;
33     }
34     bool accidentat = false;
35     for (int i = 0; i < numarObstacole; i++)
36     {
37         if (bandaCurenta == bandaObstacol[i])
38         {
39             if (distantaObstacol[i] <= distantaParcursaMinima)
40             {
41                 distantaParcursaFinal = distantaObstacol[i];
42                 accidentat = true;
43                 break;
44             }
45             distantaParcursaMinima = distantaObstacol[i];
46             distantaParcursaCurenta = distantaParcursaMinima;
47             distantaParcursaMinima += distantaSchimbareBanda;
48             if (bandaCurenta == 1)
49             {
50                 bandaCurenta = 2;
51             }
52             else
53             {
54                 bandaCurenta = 1;
55             }
56         }
57         else {
58             if (distantaParcursaMinima < distantaObstacol[i])
59             {
60                 distantaParcursaMinima = distantaObstacol[i] + 1;
61             }
62             distantaParcursaCurenta = distantaObstacol[i];
63         }
64     }
65     if (!accidentat)
66     {
67         distantaParcursaFinal = distantaRestaurant;
68     }
69     cout << "Distanța parcursă: " << distantaParcursaFinal << endl;
70 }
```

```

70
71 int main()
72 {
73     rezolvare();
74     return 0;
75 }

```

Complexitate: Care este operația cea mai costisitoare din varianta de rezolvare propusă? Cât este complexitate?

3. Ionuț vrea să facă o petrecere și încearcă să se decidă pe cine să cheme. El vrea să aleagă n persoane și a făcut o listă cu perechile de persoane care se cunosc. Vrea să cheme cât mai multe persoane, ținând cont de două restricții:

- (a) fiecare persoană să aibă minim 5 alte persoane pe care le cunoaște;
- (b) fiecare persoană să aibă 5 persoane pe care nu le cunoaște.

Într-un fișier text avem: lista de n persoane (nume și id) și lista de perechi de persoane care se cunosc (exemplu: (2,4), (2,7), etc). Să se scrie într-un fișier text de ieșire cea mai bună alegere pentru invitații la petrecere, pe baza restricțiilor date mai sus. Fișierul de ieșire trebuie să fie cât mai clar - să reiasă cât mai clar cine vine la petrecere

Rezolvarea problemei se bazează pe formarea unei matrice de adiacență, în care se reprezintă relațiile dintre posibili invitați. (1 - se cunosc, 0 - nu se cunosc). Pentru simplitate, id-ul persoanelor se alege de la $[0, n-1]$. De n - ori (nr de posibili invitați), se parcurge matricea pe fiecare linie (sau coloana - datorită simetriei lui "a cunoaște") și se verifică condițiile de a fi invitat (minim 5 cunoscuți, exact 5 necunoscuți). Dacă nu respectă aceste condiții, linia și coloana corespunzătoare posibilului invitat se vor face 0, "stergând" astfel toate relațiile în care se află. Reluarea de n -ori este necesară pentru a verifica la fiecare pas dacă invitații respectă condițiile, după eliminarea unui posibil invitat. Un exemplu de astfel de matrice de adiacență pentru care toți vor veni la petrecere este în Figura 7.2.

Algoritmul scris în C++ este 7.3.

Secvență de cod 7.3: Cod C++ pentru problema 3

```

1 #include <iostream>
2 #include <string>
3 #include <algorithm>
4 #include <vector>
5 #include <fstream>
6 #include <numeric>
7 using namespace std;
8
9 const int nrMinimCunoscuti = 5;
10 const int nrFixNeunoscuti = 5;
11
12 struct listaInvitati
13 {
14     string name;
15     unsigned short int id = 0;

```


	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	1	1	1	1	1	1	0	0	0	0
1	1	1	0	1	1	1	1	0	1	0	0	0
2	1	1	0	1	1	1	0	0	1	0	0	0
3	1	1	1	0	1	1	0	0	0	1	0	0
4	1	1	1	1	0	1	0	0	0	0	1	0
5	1	1	1	1	1	0	0	0	0	0	0	1
6	1	0	0	0	0	0	0	1	1	1	1	1
7	0	1	0	0	0	0	1	0	1	1	1	1
8	0	0	1	0	0	0	1	1	0	1	1	1
9	0	0	0	1	0	0	1	1	1	0	1	1
10	0	0	0	0	1	0	1	1	1	1	0	1
11	0	0	0	0	0	1	1	1	1	1	1	0

Figura 7.2: Exemplu problema cu petrecere și invitați

```

16 };
17
18 void readInviteesFromFile(vector<listaInvitati>& invitees, vector<pair<int, int>
    ↳ >>& relatii, string filePath)
19 {
20     ifstream myfile(filePath);
21     if (myfile.is_open())
22     {
23         unsigned int numberOfPersons;
24         myfile >> numberOfPersons;
25         invitees.resize(numberOfPersons);
26         for (unsigned int noInvitees = 0; noInvitees < numberOfPersons;
    ↳ noInvitees++)
27         {
28             myfile >> invitees[noInvitees].name >> invitees[noInvitees].id;
29         }
30         while (!myfile.eof())
31         {
32             pair<int, int> relatieCurenta;
33             myfile >> relatieCurenta.first >> relatieCurenta.second;
34             relatii.push_back(relatieCurenta);
35         }
36         myfile.close();
37     }
38     else
39         cout << "Unable to open file";
40 }
41
42 vector<vector<bool>> getAdjacentMatrix(vector<pair<int, int>> relatii, int
    ↳ numarinvitati)
43 {
44     vector<vector<bool>> matRelatii;
45     matRelatii.resize(numarinvitati);
46     for (int i = 0; i < numarinvitati; i++)
47     {
48         matRelatii.at(i).resize(numarinvitati);
49     }
50     for (unsigned int relatie = 0; relatie < relatii.size(); relatie++)
51     {
52         matRelatii[relatii[relatie].first][relatii[relatie].second] = 1;
53         matRelatii[relatii[relatie].second][relatii[relatie].first] = 1;
54     }
55     return matRelatii;
56 }
57
58 int getRemainingInvitees(vector<listaInvitati> invitees)
59 {
60     int remainingInvitees = 0;
61     for (unsigned int noInvite = 0; noInvite < invitees.size(); noInvite++)

```

```

62     {
63         if (!invitees.at(noInvite).name.empty())
64             remainingInvitees++;
65     }
66     return remainingInvitees;
67 }
68
69 void makeZero(vector<vector<bool>>& matRelatii, int rowAndColumnToRemove)
70 {
71     for (int i = 0; i < matRelatii.size(); i++)
72     {
73         matRelatii[rowAndColumnToRemove][i] = 0;
74         matRelatii[i][rowAndColumnToRemove] = 0;
75     }
76 }
77
78 void removeFromList(vector<vector<bool>>& matRelatii, vector<listaInvitati>&
    ↳ invitees)
79 {
80     for (unsigned int checkList = 0; checkList < matRelatii.size(); checkList++)
81     {
82         for (unsigned int currentLine = 0; currentLine < matRelatii.size();
            ↳ currentLine++)
83         {
84             int remainingInvitees = getRemainingInvitees(invitees);
85             int numarCunoscuti = accumulate(matRelatii.at(currentLine).begin(),
            ↳ matRelatii.at(currentLine).begin() + matRelatii.size(), 0);
86             int numarNecunoscuti = remainingInvitees - numarCunoscuti - 1;
87             bool isInvited = true;
88             if (invitees.at(currentLine).name.empty())
89                 isInvited = false;
90             if (isInvited && (numarCunoscuti < nrMinimCunoscuti ||
            ↳ numarNecunoscuti != nrFixNeunoscuti))
91             {
92                 invitees.at(currentLine).name.clear();
93                 makeZero(matRelatii, currentLine);
94             }
95         }
96     }
97 }
98
99 void printList(vector<listaInvitati> invitees, vector<vector<bool>> matRelatii,
    ↳ string filePath)
100 {
101     ofstream myfile(filePath);
102     if (myfile.is_open())
103     {
104         myfile << "La petrecere vor fi invitati: ";
105         for (unsigned int noInvite = 0; noInvite < invitees.size(); noInvite++)
106         {
107             if (accumulate(matRelatii.at(noInvite).begin(), matRelatii.at(
            ↳ noInvite).begin() + matRelatii.size(), 0))
108                 myfile << invitees.at(noInvite).name << ", ";
109         }
110         myfile.close();
111     }
112     else cout << "Unable to write to file.";
113 }
114
115 int main()
116 {
117     vector<listaInvitati> invitees;
118     vector<pair<int, int>> relatii;
119     string filePathRead = "fileIn.txt";
120     string filePathWrite = "fileOut.txt";
121     readInviteesFromFile(invitees, relatii, filePathRead);
122     vector<vector<bool>> matRelatii = getAdjacentMatrix(relatii, invitees.size())
    ↳ ;
123     removeFromList(matRelatii, invitees);
124     printList(invitees, matRelatii, filePathWrite);
125 }

```

7.3 Algoritmi Greedy - Probleme propuse

Problemele pe care le propunem în această secțiune sunt:

1. Se consideră o mulțime A cu n numere reale. Se cere o submulțime S a sa cu număr maxim de elemente, astfel încât suma elementelor sale să fie maximă.

Exemplu: Pentru numerele 1 -5 8 6 2 -2 4 răspunsul este 1 8 6 2 4 (suma este 21).

2. Un instructor de schi are la dispoziție n perechi de schiuri, $n < 100$, pe care trebuie să le distribuie la n elevi începători. Scrieți un program care să distribuie cele n perechi de schiuri astfel încât suma diferențelor absolute dintre înălțimea elevului și lungimea schiurilor atribuite să fie minimă. Se cere ca programul să afișeze maniera în care trebuie distribuite cele n schiuri, de forma "Schiul i este asociat elevului j ".
3. Se consideră n șiruri de numere ordonate crescător $S_1, S_2, \dots, S_n$, având lungimile $L_1, L_2, \dots, L_n$. Să se interclaseze șirurile date și să se obțină un nou șir cu lungimea $L_1 + L_2 + L_3 + \dots + L_n$, astfel încât numărul de comparații între elementele șirurilor date să fie minim.

Hint: Pentru a furniza o idee ajută toare, să luă un exemplu. Presupunem ca avem 3 șiruri S_1, S_2 și S_3 cu lungimile 40, 50 și 30. Pentru interclasarea lui S_1 cu S_2 numărul de comparații este $40+50=90$. Dacă interclasăm șirul obținut cu S_3 se vor mai face încă $90+30=120$ interclasări. În total $90+120=210$ comparații.

Pentru interclasarea lui S_1 cu S_3 numărul de comparații este $40+30=70$. Dacă interclasăm șirul obținut cu S_2 se vor mai face încă $70+50=120$ interclasări. În total $70+120=190$ comparații.

Pentru interclasarea lui S_2 cu S_3 numărul de comparații este $50+30=80$. Dacă interclasăm șirul obținut cu S_1 se vor mai face încă $80+40=120$ interclasări. În total $80+120=200$ comparații.

Pentru exemplul dat, interclasarea în ordinea S_1, S_3, S_2 se face cu minimum de comparații 190. Concluzia este că la fiecare pas se interclasasează cele mai scurte 2 șiruri până la final.

4. Se consideră n intervale închise $[a, b]$, a, b numere întregi, $n, a, b < 100000$. Să se determine reuniunea acestora.
5. Tatăl Claudiei este un cunoscut om de afaceri, remarcat prin firma de IT pe care o administrează singur. Acesta a obținut foarte multe proiecte la începutul acestui an, motiv pentru care dorește să angajeze un număr de k

persoane pentru a își mări echipa și pentru a se asigura că proiectele vor fi finalizate la timp. Anul 2020 a fost unul greu și pentru firma acestuia așa că el vrea să angajeze persoane care să muncească mult, iar suma totală pe care trebuie să o plătească acestora în fiecare lună să fie minimă.

Din lipsă de timp, bărbatul o roagă pe Claudia să îl ajute în alegerea pe care o are de făcut, menționând faptul că dorește eficiență maximă, suma totală minimă și că nu poate oferi unui angajat mai mult de o anumită sumă. Munca pe care o are de făcut Sonia nu este una ușoară, însă tatăl ei are încredere că va reuși. Ajut-o pe Claudia să ducă la bun sfârșit sarcina pe care i-a dat-o tatăl ei.

Date de intrare: Prima linie conține două numere întregi n și k ($1 \leq k \leq n \leq 105$) - numărul de persoane care participă la interviu și respectiv numărul de posturi. Fiecare dintre următoarele n linii conține două numere a_i și b_i care reprezintă munca oferită de persoana i și salariul pe care aceasta îl solicită.

Date de ieșire: Să se afișeze persoanele care constituie soluția optimă având în vedere cerințele care trebuie îndeplinite și suma pe care angajatorul trebuie să o plătească lunar noilor angajați. În cazul în care nu au fost ocupate toate posturile să se semnaleze acest fapt printr-un mesaj corespunzător.

6. Piratul Roby dorește să găsească locația unei comori. Acesta trebuie să parcurgă un traseu pe care se află diferite premii și restricții. Scopul său este să ajungă la poziția comorii colectând un număr cât mai mare de premii de pe drum. Traseul parcurs de Roby este reprezentat printr-o matrice de dimensiune $m \times n$. Pe fiecare celulă a matricii se află câte un premiu. El pornește de la poziția $(0,0)$ și comoara se află pe poziția (x,y) . Acesta nu are cum să se întoarcă înapoi deci singurele mutări pe care le poate face sunt în "jos" și în "dreapta". Ajutați-l pe Roby să ajungă la comoară adunând un număr cât mai mare de premii de pe drum.

Date de intrare: m - numărul de linii ale matricii, n - numărul de coloane ale matricii, a - o matrice de $m \times n$ celule care conține valoarea premiilor, (x,y) - coordonatele pe care se află comoara (linia și coloana)

Date de ieșire: Coordonatele fiecărei celule pe care Roby o parcurge și numărul de premii adunate până la comoară.

Exemplu:

Pozitie de start: $(0,0)$

Pozitie comoara: $(x,y)=(3,3)$

Figurile 7.3 și 7.4 exprimă clar faptul că valoarea totală a premiilor adunate este 32.

	0	1	2	3	
0	1	7	2	1	0
1	3	5	6	2	1
2	14	8	3	3	2
3	40	2	1	5	3

Figura 7.3: Matricea considerată exemplu pentru problema cu comoara

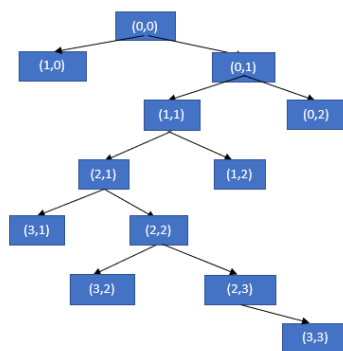


Figura 7.4: Maniera de alegere pentru problema cu comoara

Capitolul 8

Algoritmi de Programare Dinamică

8.0.1 Aspecte teoretice

Așa cum am văzut în capitolul referitor la "Algoritmi Divide et Impera", algoritmi de acest tip împart o problemă în subprobleme independente, rezolvă subproblemele și apoi combină soluțiile acestor subprobleme pentru a obține soluția problemei inițiale.

Într-o manieră oarecum asemanătoare, metoda programării dinamice rezolvă o problemă prin combinarea soluțiilor unor subprobleme, dar spre deosebire de Divide et Impera, această tip de tehnică se aplică atunci când subproblemele se suprapun. Spunem că subproblemele se suprapun, atunci când ele au subprobleme comune. În această situație, aplicarea unui algoritm de tip Divide et Impera ar face mai mult decât este necesar, rezolvând de mai multe ori subproblemele comune.

Particularitatea metodei programării dinamice constă în faptul că fiecare subproblemă este rezolvată o singură dată, iar soluția ei este păstrată într-o structură de date auxiliară. Astfel se evită recalcularea soluțiilor subproblemelor care se repetă și soluțiile subproblemelor pot fi folosite ulterior pentru a construi soluția problemei inițiale.

Metoda programării dinamice se aplică în general la rezolvarea problemelor de optimizare care au proprietatea de substructură optimă (ca și metoda Greedy), dar se poate aplica și unor probleme în care nu căutăm un optim, cum ar fi cazul problemelor de numărare.

Etapele rezolvării unei probleme cu metoda programării dinamice sunt:

1. Descrierea structurii unei soluții optime (adică a modului în care soluția problemei depinde de soluțiile subproblemelor). Această etapă presupune:
 - identificarea formei generale a problemei inițiale și a fiecărei subprobleme ;

- verificarea proprietății de substructură optimă;
- 2. Definirea unor relații de recurență care exprimă modul în care se obține soluția problemei din soluțiile subproblemelor. În general, în stabilirea acestei relații intervine criteriul de optimalitate;
- 3. Calculul valorii unei soluții optime prin rezolvarea relațiilor de recurență. De obicei relațiile sunt rezolvate în manieră ascendentă (bottom - up), astfel încât să completăm structura auxiliară cu soluțiile subproblemelor;
- 4. Construirea soluției optime - se face pe baza informațiilor determinate în etapa anterioară.

Etapele 1-3 reprezintă esența algoritmilor de programare dinamică. Dacă avem nevoie numai de valorile optime, nu și de soluția optimă în sine, atunci etapa 4 poate fi omisă. Dacă avem nevoie și de soluția optima, poate fi nevoie ca în etapa 3 să memorăm informații în plus, pentru a putea reconstrui ușor soluția optimă.

Trebuie menționat că dacă problema admite mai multe soluții optime, folosirea acestei metode ne permite să determinăm în general una dintre acele soluții, nu pe toate.

Rezolvarea relațiilor de recurență

În cazul problemelor de programare dinamică, odată identificată relația de recurență, este necesar să stabilim modul în care o vom rezolva. Există două abordări principale:

1. **Ascendentă (bottom - up)**: se porneste de la cazul/cazurile particulare și se determină noi valori pe baza relației de recurență și a valorilor existente;
2. **Descendentă (top-down)**: valoarea de calculat se exprimă în funcție de valori anterioare, care trebuie la rândul lor calculate. Aceasta abordare se implementează de cele mai multe ori recursiv, și poate conduce la algoritmi mai ineficienți.

Eficientizarea rezolvării recurențelor se poate realiza folosind tehnica memoizării.

Tehnica memoizării

- Motivație: Rezolvarea relațiilor de recurență folosind abordarea ascendentă presupune rezolvarea, o singură dată, a tuturor subproblemelor (inclusiv a celor ale căror soluții nu contribuie la soluția problemei date)
Soluționarea recurențelor folosind abordarea descendentă, va rezolva numai

subproblemele care contribuie la soluția problemei, însă acestea sunt rezolvate ori de câte ori apar.

De aceea a apărut ideea de a combina cele două tipuri de abordări

- Folosind tehnica memoizării rezolvăm o singură dată doar subproblemele ale căror soluții intervin în soluția problemei inițiale.

Exisă două etape în aplicarea acestei tehnici:

- Se inițializează structura de date cu o valoare care trebuie să fie diferită de orice valoare care s-ar obține prin rezolvarea relațiilor de recurență.
- Se calculează valoarea dorită în manieră recursivă, însă toate valorile intermediare se păstrează și se folosesc dacă este nevoie.

8.0.2 Algoritmi de Programare Dinamică - Probleme rezolvate

În această secțiune vă propunem o serie de probleme care urmăresc să vă familiarizeze cu mecanismul programării dinamice. Am încercat, în măsura posibilităților, să alegem probleme cât mai diverse și să oferim explicații cât mai detaliate. Lista de probleme rezolvate este însă departe de a fi exhaustivă. Cea mai bună cale de a vă obișnui cu această tehnică va fi să încercați să rezolvați cât mai multe probleme de acest tip.

1. Un sportiv are ca exercițiu de încălzire să urce o scară cu N trepte. El poate urca treptele fie una câte una, fie câte două deodată pentru a mări efortul făcut. Să se determine câte modalități de a urca treptele are sportivul.

În cazul acestei probleme avem:

- Date de intrare: numărul de trepte, care este de tip întreg;
- Date de ieșire: numărul de variante pe care le are sportivul pentru a urca treptele (problema este una de numărare);
- Algoritm de rezolvare:

Se poate observa ușor că dacă $N = 1$, atunci avem o singură variantă de a urca o treaptă. Dacă $N = 2$, atunci treptele pot fi urcate fie una câte una (o modalitate) sau ambele deodată (a doua modalitate).

Pentru a urca N trepte sportivul are posibilitatea de a urca $N - 1$ trepte și apoi încă una, sau să urce $N - 2$ trepte și apoi două trepte deodată. Dacă notăm cu $P[i]$ numărul de variante pe care le avem pentru a urca i trepte, atunci $N - 2$ trepte pot fi urcate în $P[N - 2]$ moduri, iar $N - 1$ trepte pot fi urcate în $P[N - 1]$ moduri. Aceasta înseamnă că N trepte pot fi urcate în

$$P[N] = P[N - 1] + P[N - 2]$$

moduri. Folosirea mecanismului recursivității în acest caz este extrem de ineficientă deoarece subprobleme identice se vor rezolva de mai multe ori. Variantele mai eficiente de rezolvare a recurenței ar fi de tip bottom-up sau top-down cu memoizare.

- Algoritmul scris în C++ este prezentat în continuare și are implementate metode pentru ambele tipuri de abordări.

Secvență de cod 8.1: Cod C++ pentru problema 1

```

1  #include <iostream>
2  #include <vector>
3
4  long int bottom_up_counting(int stairs)
5  {
6      //metoda rezolva recurenta in maniera bottom-up
7      std::vector<long int> possWays;
8      possWays.resize(stairs + 1);
9      for (int i = 0; i <= stairs; i++)
10     {
11         if (i <= 2)
12             possWays[i] = i;
13         else
14             possWays[i] = possWays[i - 1] + possWays[i - 2];
15     }
16     return possWays[stairs];
17 }
18
19 long int top_down_count(int stairs, std::vector<long int>& possWays)
20 {
21     //metoda recursiva este modificata astfel incat sa salveze rezultatele
22     //    ↪ in vectorul "possWays"
23     //metoda verifica mai intai daca problema este deja rezolvata. Daca da,
24     //    ↪ returneaza rezultatul.
25     //Daca nu, calculeaza valoarea corespunzatoare conform relatiei de
26     //    ↪ recurenta
27     if (stairs <= 2)
28     {
29         possWays[stairs] = stairs;
30         return possWays[stairs];
31     }
32     if (possWays[stairs] != 0)
33         return possWays[stairs];
34     else
35     {
36         possWays[stairs] = top_down_count(stairs - 1, possWays) +
37                             ↪ top_down_count(stairs - 2, possWays);
38         return possWays[stairs];
39     }
40 }
41
42 long int top_down_memoized_counting(int stairs)
43 {
44     std::vector<long int> possWays(stairs + 1, 0);
45     return top_down_count(stairs, possWays);
46 }
47
48 int main()
49 {
50     int stairs;
51     std::cout << "Introduceti numarul de trepte ";
52     std::cin >> stairs;
53     std::cout << "numarul de variante in care se pot urca ";
54     std::cout << stairs << " trepte este: ";
55     std::cout << bottom_up_counting(stairs) << " calculate bottom-up \n";
56     std::cout << "numarul de variante in care se pot urca ";
57     std::cout << stairs << " trepte este: ";
58     std::cout << top_down_memoized_counting(stairs) << " calculate top-
59     ↪ down";
60 }

```

- Analiza complexității. Din punct de vedere al complexității de timp,

algoritmul implică trecerea prin toate componentele vectorului P . De aici, rezultă complexitatea $O(N)$.

2. Se consideră un triunghi (dreptunghic) de numere naturale, format din n linii. Pe prima linie avem un număr, pe a doua linie două numere, ..., pe ultima linie n numere.

Presupunem că pornind de la un anumit element al triunghiului, se poate merge fie către elementul de pe următoarea linie situat sub el (pe aceeași coloană), fie către elementul de pe următoarea linie și următoarea coloană (în diagonală).

Să se determine care este șirul, aflat între elementul din vârful triunghiului și un element aflat pe ultima linie a triunghiului, care are cea mai mare sumă a elementelor.

Exemplu:

```

1
3 5
2 4 3
5 1 6 3

```

Sumele care se pot forma sunt:

$$\begin{aligned}
 s_1 &= 1 + 3 + 2 + 5 = 11; & s_2 &= 1 + 3 + 2 + 1 = 7; & s_3 &= 1 + 3 + 4 + 1 = 9; \\
 s_4 &= 1 + 3 + 4 + 6 = 14; & s_5 &= 1 + 5 + 4 + 1 = 11; & s_6 &= 1 + 5 + 4 + 6 = 16; \\
 s_7 &= 1 + 5 + 3 + 6 = 15; & s_8 &= 1 + 5 + 3 + 3 = 12
 \end{aligned}$$

și se poate observa că cea mai mare este s_6 .

În cazul acestei probleme avem:

- Date de intrare: triunghiul de numere naturale, memorat într-o matrice T (sub și pe diagonala principală) + dimensiunea acestuia;
- Date de ieșire: acele elemente ale triunghiului care formează suma maximă și valoarea acestei sume;
- Algoritm de rezolvare: Putem considera că o posibilă subproblemă a problemei date este reprezentată de calculul sumei maxime între vârful triunghiului și poziția (i, j) , $i \leq n$, $j \leq i$ din matrice. Cum subproblema este caracterizată prin indicii de poziție i și j , înseamnă că vom avea nevoie de o structură bidimensională (o matrice) S pentru a reține soluțiile subproblemelor. Matricea va avea aceleași dimensiuni ca și triunghiul, iar elementul $S[i][j]$ reprezintă suma maximă care se poate forma între vârf și poziția (i, j) din matrice. Verificăm principiul de substructură optimă pentru problema dată.

Presupunem că avem o soluție optimă, adică un șir de n numere $nr_1, nr_2, \dots, nr_n$ care respectă condițiile problemei și care formează suma maximă.

Atunci numerele de la $nr_1, nr_2, \dots, nr_i$ formează o sumă maximă în raport cu sumele care s-ar putea forma pornind de la nr_1 (situat pe poziția $(1, 1)$) și până la nr_i (situat pe linia i și coloana j). Dacă această sumă nu ar fi maximă, atunci ar exista o altă posibilă alegere a numerelor între poziția $(1, 1)$ și poziția (i, j) , care ar fi mai mare. Înlocuind în șirul inițial numerele $nr_1, nr_2, \dots, nr_i$ cu cele pentru care suma ar fi mai mare, ar rezulta o soluție cu sumă mai mare decât suma lui $nr_1, nr_2, \dots, nr_n$. Atunci am obține o contradicție cu ipoteza.

Relația de recurență: În condițiile mișcărilor permise de problemă, în poziția (i, j) , se poate ajunge din $(i - 1, j)$ și $(i - 1, j - 1)$ dacă ele există. În acest caz, suma maximă până în (i, j) , va fi egală cu elementul de pe poziția (i, j) adunat cu maximumul dintre sumele până în pozițiile care îl preced pe (i, j) . Relațiile de recurență pot fi scrise atunci sub forma:

$$S[i][j] = \begin{cases} T[0][0] & i = j = 0 \\ T[i][j] + S[i - 1][j] & j = 0, i \geq 1 \\ T[i][j] + S[i - 1][j - 1] & j = i, i \geq 1 \\ T[i][j] + \max(S[i - 1][j - 1], S[i - 1][j]) & \text{altfel} \end{cases}$$

Soluția problemei va fi dată atunci de elementul maxim de pe ultima linie a matricei S .

Pentru a avea posibilitatea de a afișa și numerele care formează soluția optimă, am avea nevoie de o matrice *Drum*, în care, pentru fiecare poziție (i, j) să reținem coloana în care se găsește predecesorul ei.

- Algoritmul scris în C++ este prezentat mai jos. Rezolvarea conține o metodă care se ocupă cu citirea input-ului problemei, o metodă care determină poziția elementului maxim al unui vector, una care construiește un vector de vectori de aceleași dimensiuni cu obiectul pasat ca și parametru și metoda care rezolvă efectiv problema și care întoarce elementele căutate într-o stivă. Conținutul stivei este afișat în metoda `main()`.

Secvență de cod 8.2: Cod C++ pentru problema 2

```

1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <algorithm>
5 #include <stack>
6 using namespace std;
7
8 void readMatrixFromFile(vector<vector<int>>& triangle, string filePath)
9 {

```

```

10     std::ifstream myFile(filePath);
11     if (myFile.is_open())
12     {
13         int size, element;
14         myFile >> size;
15         for (int i = 0; i < size; i++)
16         {
17             vector<int> linie;
18             for (int j = 0; j <= i; j++)
19             {
20                 myFile >> element;
21                 linie.push_back(element);
22             }
23             triangle.push_back(linie);
24         }
25         myFile.close();
26     }
27     else
28         std::cout << "unable to open file";
29 }
30 int positionOfMaxElement(vector<int> vect)
31 {
32     int max = INT_MIN, position = 0;
33     for (int i = 0; i < vect.size(); i++)
34     {
35         if (vect[i] > max)
36         {
37             max = vect[i];
38             position = i;
39         }
40     }
41     return position;
42 }
43 }
44
45 vector<vector<int>>> createEmptyMatrix(vector<vector<int>>> inputMatrix)
46 {
47     //metoda creaza o matrice de dimensiunile celei date ca parametru
48     vector<vector<int>>> matrix;
49     matrix.resize(inputMatrix.size());
50     for (int i = 0; i < matrix.size(); i++)
51         matrix[i].resize(inputMatrix[i].size());
52     return matrix;
53 }
54
55 void maximumSumAndPath(vector<vector<int>>> triangle, stack<int> &stackPath,
56     ↪ int &maxSum)
57 {
58     //matricea "sum" se initializeaza cu elementele din //
59     ↪ matricea "triangle"
60     // apoi la elementul de pe poz (i,j) se va adauga
61     //maximul dintre sumele pana in pozitiile care sunt predecesori ai
62     ↪ pozitiei curente
63     vector<vector<int>>> sum = triangle;
64     vector<vector<int>>> path = createEmptyMatrix(triangle);
65     path[0][0] = -1;
66     for (int i = 1; i < triangle.size(); i++)
67     {
68         for (int j = 0; j <= i; j++)
69         {
70             if (j == 0)
71             {
72                 sum[i][j] = sum[i-1][j] + sum[i-1][j];
73                 path[i][j] = j;
74             }
75             else
76             {
77                 if (j == i)
78                 {
79                     sum[i][j] = sum[i][j] + sum[i-1][j-1];
80                     path[i][j] = j-1;
81                 }
82                 else
83                 {
84                     sum[i][j] = sum[i][j] + max(sum[i-1][j], sum[i-1][j-1]);
85                     sum[i-1][j] > sum[i-1][j-1] ? path[i][j] = j : path[i][j] = j-1;
86                 }
87             }
88         }
89     }
90 }

```

```

87
88     }
89 }
90 //determinam coloana pe care se afla elementul maxim de pe ultima linie
91 int colMaximum=positionOfMaxElement(sum[sum.size()-1]);
92 maxSum = sum[sum.size()-1][colMaximum];
93
94 // creem o stiva in care vom avea drumul cu suma maxima
95 int j = colMaximum;
96 int i = path.size()-1;
97 while (i>=0)
98 {
99     stackPath.push(triangle[i][j]);
100    j = path[i][j];
101    i--;
102 }
103
104 }
105
106 int main()
107 {
108     vector<vector<int>>> triangle;
109     string filePath = "dateIn.txt";
110     readMatrixFromFile(triangle, filePath);
111     stack<int> stackPath;
112     int maximumSum;
113     maximumSumAndPath(triangle, stackPath, maximumSum);
114     cout << "suma maxima in triunghiul citit este: " << maximumSum << endl;
115     cout << "elementele de pe drumul cu suma maxima sunt: ";
116     while (!stackPath.empty())
117     {
118         cout << stackPath.top() << " ";
119         stackPath.pop();
120     }
121 }

```

- Analiza complexității: Se poate observa ușor că indicele j este dependent de i . În ciclul for interior (cel pentru j se va intra: de două ori pentru $i = 1$, de trei ori pentru $i = 2, \dots$, de n ori pentru $i = n - 1$. Asta înseamnă că numărul de operații este proporțional cu:

$$2 + 3 + \dots + n = \frac{(n+2)(n-1)}{2}$$

ceea ce înseamnă că algoritmul are complexitate de timp $O(n^2)$.

3. O companie de prelucrare a lemnului taie trunchiuri de copaci în bușteni de diferite lungimi (numere naturale), pe care apoi îi vinde mai departe. pentru fiecare buștean de lungime i se cunoaște prețul p_i cu care acesta poate fi vândut mai departe.

Compania vrea să știe cum să facă tăieturile pe fiecare copac de lungime n dată, astfel încât profitul obținut de pe urma vânzării buștenilor să fie maxim.

Exemplu: $n = 4$

Lungime	1m	2m	3m	4m	5m	6m	7m	8m	9m	10m
Preț	1	5	7	8	10	11	15	17	18	20

În cazul acestei probleme avem:

- Date de intrare: Vectorul care reține prețurile de vânzare pentru bușteni de diferite lungimi și lungimea n a copacului care urmează să fie tăiat;
- Date de ieșire: Profitul maxim care se poate obține de pe urma tăierii copacului și modalitatea de tăiere;
- Algoritm de rezolvare:

Numărul maxim de tăieturi care pot fi făcute într-un copac de lungime n este $n - 1$. Asta înseamnă că numărul total de variante de a tăia copacul (nu neaparat distincte ca lungimi finale) este 2^{n-1} , deoarece avem opțiunea de a tăia (care poate fi asimilată cu 1), sau de a nu tăia (echivalentă cu 0) la distanța i față de un capăt fixat (fie el cel stâng), pentru $i \in \{1, 2, \dots, n - 1\}$ (ceea ce ar însemna de fapt toate combinațiile de 0 și 1 pe $n - 1$ poziții). Variantele de a tăia un copac de lungime $n = 4$ și profitul corespunzător sunt prezentate mai jos:

Varianta v_i	profit p_i
1+1+1+1	4
1+1+2	7
1+2+1	7
2+1+1	7
1+3	8
3+1	8
2+2	10
4	8

Se poate observa că varianta optimă de a tăia bușteanul este în două bucăți de câte $2m$.

Fie acum un copac de lungime n oarecare. În cazul în care am face o tăietură la o distanță i de capătul fixat obținem două bucăți de lungime i și respectiv $n - i$, sau cu alte cuvinte două subprobleme ale problemei inițiale. Ceea ce ne dorim este să avem un profit maxim pentru fiecare dintre ele.

Fie b_n beneficiul maxim care se poate obține pentru un copac de lungime n . Valoarea maximă a lui b_n se poate obține dacă luăm în considerare toate variantele de a face o primă tăietură, care să conducă la obținerea unor bucăți de lungime i și respectiv $n - i$, bucăți pentru care presupunem că am cunoaște beneficiul maxim b_i și respectiv b_{n-i} obținut în urma tăierii lor ulterioare.

De asemenea trebuie luat în considerare și faptul că profitul obținut din vânzarea copacului întreg ar putea fi mai mare decât ceea ce am putea obține de pe urma tăierii lui. Astfel ajungem la o relație de recurență care ne permite determinarea beneficiului maxim b_n din posibilele ben-

eficii maxime ale părților sale, iar această relație este următoarea:

$$b_n = \max(p_n, b_1 + b_{n-1}, b_2 + b_{n-2}, \dots, b_{n-1} + b_1)$$

O relație în formă simplificată se poate obține dacă considerăm că tăiem copacul la distanță i de capătul fixat, obținem două bucăți de lungime i și respectiv $n - i$, dar de data aceasta numai bucata din dreapta, nu și cea din stânga poate fi tăiată mai departe. Astfel, relația de recurență va fi:

$$b_n = \max_{1 \leq i \leq n} (p_i + b_{n-i})$$

Dacă se dorește inclusiv modul de tăiere al copacului, atunci, de fiecare dată când se găsește o poziție de tăiere în care maximul este mai bun decât cel precedent, se va reține și poziția respectivă într-un vector.

- Algoritmul scris în C++ este prezentat mai jos. Rezolvarea conține o metodă care rezolvă recurența ascendent și determină inclusiv soluția optimă, și o metoda care rezolvă rezolvă recurența descendent cu memorizare.

Secvență de cod 8.3: Cod C++ pentru problema 3

```

1  #include <iostream>
2  #include <vector>
3  #include <fstream>
4  #include <algorithm>
5  using namespace std;
6
7  void readPriceData(vector<int> &price, string filePath)
8  {
9      ifstream myFile(filePath);
10     if (myFile.is_open())
11     {
12         int size, element;
13         myFile >> size;
14         price.push_back(0);
15         for (int i = 1; i < size + 1; i++) {
16             myFile >> element;
17             price.push_back(element);
18         }
19         myFile.close();
20     }
21     else
22         cout << "unable to open file";
23 }
24
25 int bottom_up_cutting_log(vector<int> price, int logSize, vector<int> &cut)
26 {
27     vector<int> benefit;
28     // vectorul cu beneficiul va avea dimensiunea copacului si va fi
29     //   ↗ initializat cu -1
30     benefit.resize(logSize + 1);
31     // vectorul cu taieturile va avea si el dimensiunea copacului
32     cut.resize(logSize + 1);
33     //incepem constructia vectorului de jos in sus
34     benefit[0] = 0;
35     for (int i = 1; i <= logSize; i++)
36     {
37         benefit[i] = -1;
38         for (int j = 1; j <= i; j++)
39         {
40             if (benefit[i] < price[j] + benefit[i - j])
41                 // cand se determina o valoarea mai buna a beneficiului

```

```

42         // se memoreaza si pozitia unde a fost facuta taietura
43         benefit[i] = price[j] + benefit[i - j];
44         cut[i] = j;
45     }
46 }
47 }
48 return benefit[logSize];
49 }
50
51 int top_down_cutting_log(vector<int> price, int size, vector<int> &benefit)
52 {
53     // metoda recursiva modificata astfel incat sa salveze rezultatele in
54     // vectorul "benefit"
55     // metoda va verifica mai intai daca problema este deja rezolvata. Daca
56     // da, va returna rezultatul.
57     // Daca nu, va calcula valoarea corespunzatoare in mod natural
58     if (benefit[size] >= 0)
59         return benefit[size];
60     if (size == 0)
61         benefit[size] = 0;
62     else
63     {
64         for (int i = 1; i <= size; i++)
65         {
66             benefit[size] = max(benefit[size], price[i] +
67                                 // top_down_cutting_log(price, size-i, benefit));
68         }
69         return benefit[size];
70     }
71
72 int top_down_memoized_cutting_log(vector<int> price, int logSize)
73 {
74     // vectorul cu beneficiul va avea dimensiunea copacului si va fi
75     // initializat cu -1
76     vector<int> benefit(logSize + 1, -1);
77     return top_down_cutting_log(price, logSize, benefit);
78 }
79
80 int main()
81 {
82     vector<int> price;
83     string filePath = "dataIn.txt";
84     readPriceData(price, filePath);
85     int logSize;
86     // vectorul cut va pastra solutia optima ( unde se fac taieturile)
87     vector<int> cut;
88     cout << "Introduceti lungimea copacului:";
89     cin >> logSize;
90     cout << "beneficiul MAXIM obtinut de pe urma copacului cu lungimea
91     // introdusa este: ";
92     cout << bottom_up_cutting_log(price, logSize, cut) << " calculat bottom
93     // -up \n";
94     cout << "taieturile se fac de: ";
95     int i = logSize;
96     while (i > 0)
97     {
98         cout << cut[i] << " metri; ";
99         i = i - cut[i];
100     }
101     cout << " \n beneficiul MAXIM obtinut de pe urma copacului cu lungimea
102     // introdusa este: ";
103     cout << top_down_memoized_cutting_log(price, logSize) << " calculat top
104     // -down \n";
105 }

```

- Analiza complexității: Discutăm complexitatea metodei care implementează recurența ascendent. Cum pentru fiecare copac de lungime i se iau în considerare toate variantele de a face prima tăiere la distanță j , $j \leq i$ de capătul fix, va rezulta că pentru un copac de lungime1 se face o comparație, un copac de lungime2 se fac 2 comparații, ..., un copac de lungimen se fac n comparații. Astfel numărul de operații va

fi:

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

ceea ce înseamnă că algoritmul are complexitate $O(n^2)$.

4. Problema discretă a rucsacului: O persoană are la dispoziție un rucsac cu care poate transporta o greutate maximă G și n obiecte. Pentru fiecare obiect se cunoaște greutatea g_i (număr natural) și câștigul c_i care de poate obține de pe urma transportului obiectului respectiv. Cum ar trebui făcută selecția obiectelor în vederea transportului lor, astfel încât greutatea totală a obiectelor selectate să fie mai mică sau egală cu G și profitul obținut să fie maxim?

Obiectele pot fi transportate doar întregi, nu se admite transportul unei părți din obiect.

Exemplu: $G = 10$, $n = 5$

Index	0	1	2	3	4
Greutate	1	4	2	10	5
Cost	2	4	8	15	7

În cazul acestei probleme avem:

- Date de intrare: un vector cu n elemente în care regăsim caracteristicile obiectelor (g_i, c_i) și greutatea maximă G ;
- Date de ieșire: valoarea maximă a profitului și obiectele care trebuie transportate;
- Algoritm de rezolvare: Problema generală este caracterizată de două lucruri: numărul n de obiecte din care putem selecta (și implicit caracteristicile lor) și greutatea maximă a rucsacului G . În cazul unei subprobleme, aceste caracteristici ar trebui să se micșoreze. Astfel, o posibilă subproblemă a problemei rucsacului ar presupune selecția din setul de i obiecte $\{o_0, o_1, \dots, o_{i-1}\}$, $i \leq n$, (primele i obiecte), pentru a umple în mod optim, un rucsac de capacitate j , $j \leq G$.

Pentru a reține câștigul maxim în cazul fiecărei subprobleme, vom avea nevoie de o matrice *Castig* de dimensiune $(n+1) \times (G+1)$. Dacă rucsacul are capacitate 0, evident nu vom putea selecta niciun obiect, și atunci câștigul este 0. Dacă rucsacul are capacitate j , dar avem posibilitatea de a alege dintre primele $i = 0$, obiecte, atunci câștigul este de asemenea 0.

Dacă avem de luat decizia de a include sau nu obiectul o_i , într-un rucsac cu capacitate j , atunci raționamentul este următorul:

- Obiectul poate fi inclus numai dacă $g_i \leq j$. În aceste condiții vom avea două posibilități:
- Dacă obiectul o_i este ales, atunci câștigul care se obține este suma dintre costul c_i al obiectului și câștigul maxim obținut alegând dintre obiectele $\{o_0, o_1, \dots, o_{i-1}\}$ astfel încât să umplem restul de $j - g_i$ din capacitate.
- Dacă obiectul o_i nu este ales, atunci câștigul maxim obținut pentru capacitatea j este câștigul maxim obținut alegând dintre obiectele $\{o_0, o_1, \dots, o_{i-1}\}$ pentru a umple rucsacul de capacitate j .

Evident, obiectul este ales sau nu în funcție de câștigul realizat în cele două situații menționate anterior, care trebuie să fie maxim. Astfel vom avea relațiile de recurență următoare:

$$Castig[i+1][j] = \begin{cases} 0 & i = 0 \text{ sau } j = 0 \\ c_i + Castig[i][j - g_i] & \text{daca } g_i \leq j, \text{ si} \\ & c_i + Castig[i][j - g_i] > Castig[i][j] \\ Castig[i][j] & \text{altfel} \end{cases}$$

Profitul maxim care se poate obține se va regăsi în $Castig[n][G]$

- Algoritmul scris în C++ este prezentat mai jos. Rezolvarea conține o metodă care se ocupă cu citirea input-ului problemei (obiectele sunt declarate ca structură), o metoda care determină matricea *Castig* și o returnează, și o altă metodă care pe baza matricei *Castig* reconstituie și afișează soluția optimă.

Secvență de cod 8.4: Cod C++ pentru problema 4

```

1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <algorithm>
5  using namespace std;
6
7  struct Obiect
8  {
9      int greutate, cost;
10 };
11
12 void readObjFromFile(vector<Obiect>& vectOb, int& capMax, string filePath)
13 {
14     ifstream myFile(filePath);
15     if (myFile.is_open())
16     {
17         int numarObiecte;
18         myFile >> numarObiecte >> capMax;
19         vectOb.resize(numarObiecte);
20         for (int i = 0; i < numarObiecte; i++) {
21             myFile >> vectOb[i].greutate >> vectOb[i].cost;
22         }
23         myFile.close();
24     }
25     else
26         cout << "unable to open file";
27 }
28
29 vector<vector<int>>> findOptimum(vector<Obiect> vectOb, int capMax)

```

```

30 {
31     // instantiem matricea castig
32     //de dimensiune (nrObiecte + 1) x (capMax + 1)
33     int nrOb = vectOb.size();
34     std::cout << nrOb;
35     vector<vector<int>> castig(nrOb + 1);
36     for (int i = 0; i <= nrOb; i++)
37     {
38         castig[i].resize(capMax + 1);
39     }
40
41     // calculam elementele din matricea castig conform relatie de recurenta
42     for (int i = 0; i < nrOb; i++)
43     {
44         // luam in considerare toate greutatile intermediare
45         //de la 0 la capacitatea maxima
46         for (int j = 0; j <= capMax; j++)
47         {
48             if (i == 0 || j == 0)
49             {
50                 castig[i][j] = 0;
51             }
52             // nu luam in discutie elementul "O_i" daca greutatea lui
53             //depaseste greutatea "j"
54             if (vectOb[i].greutate > j) {
55                 castig[i+1][j] = castig[i][j];
56             }
57             else {
58                 // altfel vom lua in considerare castigul maximul pe care
59                 // obtine prin includerea sau excluderea ob "O_i"
60                 if (vectOb[i].cost + castig[i][j-vectOb[i].greutate] > castig
61                     [i][j])
62                 {
63                     castig[i+1][j] = vectOb[i].cost + castig[i][j-vectOb[i].
64                         greutate];
65                 }
66                 else {
67                     castig[i+1][j] = castig[i][j];
68                 }
69             }
70         }
71     }
72     return castig;
73 }
74
75 void displaySolution(vector<Obiect> vectOb, vector<vector<int>> castig)
76 {
77     //solutia va fi retinuta in vectorul sol
78     // daca castig[i][j] != castig[i-1][j], inseamna ca
79     // obiectul "i" din lista, avand index i-1 a fost selectat
80     int nrOb = vectOb.size();
81     vector<bool> sol(nrOb, false);
82     int i = castig.size()-1;
83     int j = castig[0].size()-1;
84     while (castig[i][j] > 0)
85     {
86         if (castig[i][j] == castig[i-1][j])
87         {
88             i--;
89         }
90         else {
91             sol[i-1] = true;
92             j -= vectOb[i-1].greutate;
93             i--;
94         }
95     }
96     //afisarea solutiei. Indexarea este facuta de la 0!
97     cout << "\n obiectele cu urmatoarii indici sunt alese: ";
98     for (int i = 0; i < sol.size(); i++)
99     {
100         if (sol[i])
101         {
102             cout << i << " ";
103         }
104     }
105 }
106 }

```

```

107
108 int main()
109 {
110     vector<Obiect> vectOb;
111     int capMax;
112     std::string filePath = "dateIn.txt";
113     readObjFromFile(vectOb, capMax, filePath);
114     int nrOb = vectOb.size();
115     vector<vector<int>> castig = findOptimum(vectOb, capMax);
116     cout << "profitul maxim obtinut poate fi " << castig[nrOb][capMax];
117     displaySolution(vectOb, castig);
118 }

```

- Analiza complexității: Din punct de vedere al complexității de timp, algoritmul ascendent implică calcularea tuturor elementelor matricei *Castig*. De aici, rezultă complexitatea $O(nG)$.
 - încercați să rescrieți algoritmul folosind tehnica memoizării.
5. Se consideră un vector cu n elemente întregi. Se cere să se afișeze cel mai lung subșir crescător al vectorului (numerele nu trebuie să se afle pe poziții consecutive)
- Exemplu: Pentru $n = 5$ și $v = \{4, 1, 10, 6, 10\}$ se poate afișa $s = \{4, 10, 10\}$
- În cazul acestei probleme avem:

- Date de intrare: un vector cu n elemente întregi ;
- Date de ieșire: un subșir crescător de lungime maximă;
- Algoritm de rezolvare: Problema se poate rezolva încercând să calculăm, pentru fiecare element al vectorului în parte, lungimea celui mai lung subșir crescător care se poate forma începând cu el. La final vom selecta elementul din vector de la care se poate forma cel mai lung subșir crescător și acest subșir este afișat.

Se poate observa că determinarea subșirului crescător de lungime maximă care începe cu un anumit element este mai ușor de făcut pentru elementele de la sfârșitul vectorului:

- cel mai lung subșir care se poate forma începând cu ultimul element are lungime 1;
- cel mai lung subșir care se poate forma începând cu penultimul element are lungime 2 dacă $v[n-2] \leq v[n-1]$ sau 1 altfel;

Considerăm acum că o subproblemă a problemei date este determinarea cel mai lung subșir crescător pornind de la poziția k fixată și până la sfârșitul vectorului. Pentru a reține soluțiile subproblemelor vom avea nevoie de un vector L cu n componente, unde $L[k]$ are semnificația explicată mai sus.

În conformitate cu observațiile anterioare, putem scrie relațiile de recurență următoare:

$$L[k] = \begin{cases} 1 + \max\{L[i] | v[i] \geq v[k], i \in k+1, \dots, n-1\} \\ 1 + \max\{L[i] | v[i] \geq v[k]\}, k+1 \leq i \leq n-1 \end{cases}$$

Vectorul L se completează de la dreapta la stânga. Se calculează mai apoi maximul din vector, și acesta reprezintă dimensiunea celui mai lung subșir crescător care se poate forma.

Dacă vrem să și afișăm acest subșir, atunci pornim din poziția j a maximului și afișăm $v[j]$. Apoi se găsește primul element mai mare sau egal cu $v[j]$ și pentru care elementul corespunzător din vectorul L este cu 1 mai mic decât maximul. Algoritmul continuă până când se epuizează toate elementele subșirului.

- Algoritmul scris în C++ este prezentat mai jos. Rezolvarea conține câte o metodă pentru citirea și afișarea unui vector, o metodă care returnează poziția maximului într-un vector și metoda care construiește vectorul L pe baza căruia determină mai apoi subșirul crescător. Acest subșir va fi returnat de metodă.

Secvență de cod 8.5: Cod C++ pentru problema 5

```

1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <algorithm>
5  using namespace std;
6
7  void readVectFromFile(vector<int>& vect, string filePath)
8  {
9      ifstream myFile(filePath);
10     int elem;
11     if (myFile.is_open())
12     {
13         while (!myFile.eof())
14         {
15             myFile >> elem;
16             vect.push_back(elem);
17         }
18         myFile.close();
19     }
20     else
21         std::cout << "unable to open file";
22 }
23
24 void writeVect(vector<int>& vect)
25 {
26     for (int i = 0; i < vect.size(); i++)
27         cout << vect[i] << " ";
28 }
29
30 int posMaxVector(vector<int> vect)
31 {
32     int max = vect[0];
33     int position = 0;
34     for (int i = 1; i < vect.size(); i++)
35     {
36         if (max < vect[i])
37         {
38             max = vect[i];
39             position = i;
40         }
41     }
42     return position;
43 }
44
45 /*metoda determina lungimea maxima a unui subsir
46 crescator si returneaza ac subsir*/
47 vector<int> longestIncrSequence(vector<int> vect)
48 {
49     /*construim sirul cu lungimile subsirurilor si il
50     initializam cu 1 pt ca aceasta este lungimea minima
51     posibila a unui subsir */
52     vector<int> maxSubLength(vect.size(), 1);

```

```

53     int size = maxSubLength.size();
54     maxSubLength[size - 1] = 1;
55     for (int k = size - 2; k >= 0; k--)
56     {
57         int max = 0;
58         for (int i = k + 1; i < size; i++)
59         {
60             if (vect[i] >= vect[k] && max < maxSubLength[i])
61             {
62                 max = maxSubLength[i];
63                 maxSubLength[k] = 1 + max;
64             }
65         }
66     }
67
68     // determinam maximul din sirul "maxSubLength"
69     // si pozitia corespunzatoare
70     int position = posMaxVector(maxSubLength);
71     int maxLength = maxSubLength[position];
72     //construim subsirul cresc de lungime maxima
73     vector<int> subsequence;
74     subsequence.push_back(vect[position]);
75     for (int i = position + 1; i < size; i++)
76     {
77         if (vect[i] >= vect[position] && maxSubLength[i] == maxLength - 1)
78         {
79             maxLength--;
80             subsequence.push_back(vect[i]);
81         }
82     }
83     return subsequence;
84 }
85
86 int main()
87 {
88     vector<int> vect;
89     string filePath = "dateIn.txt";
90     readVectFromFile(vect, filePath);
91     vector<int> subsequence = longestIncrSequence(vect);
92     cout << "Lungimea maxima a unui subsir cresc este ";
93     cout << subsequence.size() << " \n";
94     cout << "Elementele subsirului sunt: ";
95     writeVect(subsequence);
96 }

```

- Analiza complexității: Cum pentru fiecare element al sirului se determină un maxim, această operație presupune parcurgerea șirului până în capăt. Pentru elementul de pe poziția $n - 2$ se face o comparație, pentru cel de pe poziția $n - 3$ se fac 2 comparații, ..., pentru elementul de pe poziția 0 se fac $n - 1$ comparații. Astfel numărul de operații va fi:

$$1 + 2 + \dots + n - 1 = \frac{n(n - 1)}{2}$$

ceea ce înseamnă că algoritmul are complexitate $O(n^2)$.

6. Se consideră două șiruri. Să se determine subsecvența de lungime maximă care se regăsește în ambele șiruri. O subsecvență se presupune că are elementele la rând (pe poziții consecutive în șir)
Exemplu: Șirurile "ABADBC" și "BABAD" au în comun secvența de dimensiune maximă 4 următoare "ABAD".
Alte subsecvente comune ar fi : "A", "B", "D", "AB", "AD", "BA", etc.
O astfel de problemă are aplicabilitate în soft-urile pentru detectarea plagiatului și în bioinformatică. În cazul acestei probleme avem:

- Date de intrare: două șiruri de caractere $X[0, \dots, n-1]$ de dimensiune n și respectiv $Y[0, \dots, m-1]$ de dimensiune m ;
- Date de ieșire: un alt șir care reprezintă subsecvența maximă comună;
- Algoritm de rezolvare: Ideea algoritmului este aceea de a determina dimensiunea celui mai lung sufix comun (terminație a cuvântului), pentru toate subsecvențele ambelor șiruri.

Pentru două subsecvențe date, dacă ultimele litere nu coincid, atunci cel mai lung sufix comun are dimensiunea 0. Dacă ultimele litere coincid, atunci dimensiunea celui mai lung sufix comun va fi 1 (pentru litera care coincide) la care se adună dimensiunea celui mai lung sufix comun al subsecvențelor date din care am șters practic ultima literă. Pentru a formaliza raționamentul anterior, considerăm că o subproblemă a problemei date este determinarea celei mai lungi subsecvențe comune a șirurilor $X[0, \dots, i-1]$ de lungime i și $Y[0, \dots, j-1]$ de lungime j . Rezultatul subproblemelor va fi păstrat într-un tablou bidimensional *LCSuffix*.

În conformitate cu observațiile anterioare, putem scrie relațiile de recurență următoare:

$$LCSuffix[i][j] = \begin{cases} 0 & i = 0 \text{ sau } j = 0 \\ 0 & \text{dacă } X[i-1] \neq Y[j-1] \\ 1 + LCSuffix[i-1][j-1] & \text{altfel} \end{cases}$$

Dimensiunea celei mai lungi subsecvențe comune va fi dată de elementul maxim din matricea *LCSuffix*. Dacă vrem să afișăm această subsecvență trebuie să memorăm într-o variabilă poziția de final (relativ la unul dintre șiruri) a sufixului de dimensiune maximă.

- Algoritmul scris în C++ este prezentat mai jos. Rezolvarea conține metoda care construiește matricea *LCSuffix*, și determină poziția de final a sufixului de dimensiune maximă. Acest sufix va fi returnat de metodă.

Secvență de cod 8.6: Cod C++ pentru problema 6

```

1  #include <iostream>
2  using namespace std;
3
4  //metoda cauta cea mai lunga subsecventa comuna si o returneaza
5  string LongestCommonSubstring(string& firstString, string& secondString)
6  {
7      int size1 = firstString.size();
8      int size2 = secondString.size();
9
10     // alocam spatiu pentru matricea LComSuffix
11     int** LComSuffix = new int* [size1+1];
12     for (int i = 0; i < size1 + 1; i++)
13         LComSuffix[i] = new int [size2+1];

```

```

14
15 // variabila maxLength va pastra dim celei mai lungi subsecvente comune
16 // var position pastreaza ultima pozitie din sufixul maxim comun
17 // (pozitia este considerata relativ la primul sir)
18 int maxLength = 0;
19 int position = -1;
20
21 // Construim matricea LComSuffix
22 for (int i = 0; i < size1 + 1; i++)
23 {
24     for (int j = 0; j < size2 + 1; j++)
25     {
26         if (i == 0 || j == 0)
27             LComSuffix[i][j] = 0;
28         else
29             if (firstString[i - 1] == secondString[j - 1])
30             {
31                 LComSuffix[i][j] = 1 + LComSuffix[i - 1][j - 1];
32                 if (maxLength < LComSuffix[i][j])
33                 {
34                     maxLength = LComSuffix[i][j];
35                     position = i - 1;
36                 }
37             }
38             else
39                 LComSuffix[i][j] = 0;
40     }
41 }
42
43 // dealocam memoria unde a fost pastrata matricea LComSuffix
44 for (int i = 0; i < size1; i++)
45     delete [] LComSuffix[i];
46 delete [] LComSuffix;
47
48 // din primul sir selectam subsecventa comuna si o returnam
49 return firstString.substr(position - maxLength + 1, maxLength);
50 }
51
52 int main()
53 {
54     string firstString = "abababac aa bbb";
55     string secondString = "caadabababac aaa bbd";
56     string result = LongestCommonSubstring(firstString, secondString);
57
58     cout << "lungime maxima " << result.size() << "\n";
59     if (result.size())
60     {
61         cout << "subsirul comun este:" << result;
62     }
63 }

```

- Analiza complexității: Rezolvarea problemei presupune construcția matricei *LCSuffix*, ceea ce înseamnă că algoritmul are complexitate $O(n*m)$. Menționăm că problema poate fi rezolvată și în complexitate $O(n+m)$, dacă se folosește un arbore de sufix. Un arbore de sufix este o structură de date în care sufixe pentru două (sau mai multe) șiruri distincte sunt stocate. Ei sunt folosiți cu precădere în problemele care presupun procesarea șirurilor de caractere.
7. Se consideră un text care are maxim 100 de litere. Textul poate fi codificat înlocuind apariții consecutive ale subsecvențelor sale, cu subsecvența respectivă urmată de numărul ei de apariții.
- Exemplu: Textul $T = \text{"daaabbbaabbccxyzxyz"}$ ar putea fi codificat în varianta $\text{"d1a3b2a3b2c2xyz2"}$ dar și în varianta "d1aaabb2c2xyz2" . După cum se poate observa, al doilea mod de codificare este mai scurt, deci ar fi mai convenabil.

Să se scrie un program care să codifice optimal un text (codul rezultat să fie de lungime minimă).

În cazul acestei probleme avem:

- Date de intrare: Șirul de caractere T , având lungimea n ;
- Date de ieșire: un alt șir care reprezintă codificarea optimală a lui T ;
- Algoritm de rezolvare:

Se poate observa pe exemplul ales că șirul conține mai multe subșiruri care pot fi restrânse folosind o codificare optimală. De exemplu, subșirul "aaabbaaabb" care începe de la poziția 2 până la poziția 11 inclusiv, sau subșirul "xyzxyz" care începe de la poziția 14 până la poziția 19 inclusiv. Asta ne poate sugera faptul că o posibilă subproblemă problemei date ar fi determinarea unei codificări de lungime minimă pentru toate caracterele din text, începând de la poziția i și până la poziția j adică pentru $T[i, \dots, j]$, $0 \leq i \leq j < n$. Subproblemele evident se suprapun.

Pentru a memora soluțiile subproblemelor vom avea nevoie de o matrice *Lung* pătratică, de ordin n , în care, în mod evident $Lung[i][j]$ reprezintă lungimea codificării optime pentru $T[i, \dots, j]$, $0 \leq i \leq j < n$. Se poate observa din relația între indici că va fi folosită numai jumătatea de deasupra diagonalei principale.

Pentru a stabili relația de recurență observăm mai întâi că orice caracter c se codifică pe două poziții $c1$. Pentru șiruri $T[i, \dots, j]$ mai lungi trebuie să luăm în calcul următoarele posibilități:

- Nu există repetiții în șir, ceea ce înseamnă că lungimea codificării este egală cu lungimea șirului plus încă 1 (adică $j - i + 2$);
- Șirul constă în repetarea de k ori a aceluiași grup de caractere $T[i, \dots, j] = ss\dots s$, și atunci codificarea este sk ;
- Dacă nu suntem în niciuna dintre situațiile precedente, atunci codificarea optimală a lui $T[i, \dots, j]$ se bazează pe codificările optime ale lui $T[i, \dots, k]$ și $T[k + 1, \dots, j]$, unde k este în intervalul $i \leq k \leq j - 1$ și este ales în mod optim.

În conformitate cu observațiile anterioare, putem scrie relațiile de recurență următoare:

$$Lung[i][j] = \begin{cases} 2 & i = j \\ \min\{j - i + 2, \min1, \min2\} & altfel \end{cases}$$

unde $\min1$ și respectiv $\min2$ sunt lungimile minime în ultimele două situații enumerate mai sus și care sunt date de relațiile:

$$\min1 = \min_s \{strlen(s) + NrCifre(k)\}$$

cu s subsir al lui $T[i, \dots, j]$,

$$\min2 = \min_{i \leq k \leq j-1} \{Lung[i][k] + Lung[k+1][j]\}$$

Lungimea optimă a codificării şirului va fi reprezentată de elementul $Lung[0][n-1]$. Dacă vrem să obţinem şi codificarea optimă, atunci vom folosi partea de sub diagonală a matricei pentru a reţine informaţii care să ne spună în care dintre cele trei situaţii enumerate mai sus a fost obţinut minimul. Astfel vom avea:

$$Lung[j][i] = \begin{cases} 0 & \text{dacă } Lung[i][i] = j - i + 1 \\ k & k = \text{lung subsir } s \\ -k & k = \text{poz de splitare} \end{cases}$$

- Algoritmul scris în C++ este prezentat mai jos. Rezolvarea conţine o metodă care determină numărul de cifre al unui număr dat, o metodă care verifică dacă un şir este format prin concatenarea unui subsir, o metodă care determină matricea *Lung* şi metoda recurentă care afişează codificarea optimă.

Secvenţă de cod 8.7: Cod C++ pentru problema 7

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int numarCifre(int numar)
6  {
7      if (numar == 0)
8          return 1;
9      int contor = 0;
10     while (numar != 0) {
11         contor++;
12         numar /= 10;
13     }
14     return contor;
15 }
16 // metoda verifica daca sirul T[i..j]="sss..ss"
17 // unde s este subsir de dim k a lui T[i..j] s=T[i..k-1]
18 bool esteSubsir(string T, int i, int j, int k)
19 {
20     string s = T.substr(i, k);
21     for (int p = i + k; p <= j; p += k)
22     {
23         string s1 = T.substr(p, k);
24         if (s.compare(s1) != 0)
25             return false;
26     }
27     return true;
28 }
29
30 vector<vector<int>>> calculLungimiCodif(string T)
31 {
32     int nrCar = T.size();
33     //matricea va fi creata de dimensiunea dorita
34     vector<vector<int>>> lung;
35     lung = vector<vector<int>>>(nrCar, vector<int>(nrCar));
36     for (int i = 0; i < nrCar; i++)
37     {
38         lung[i][i] = 2;
39     }
40     // codificam acum sirurile de dim >=2
41     for (int dim = 2; dim <= nrCar; dim++)

```

```

42     {
43         // sirurile incep de la poz "i"
44         for (int i = 0; i <= nrCar - dim; i++)
45         {
46             //determinam poz de final j
47             int j = i + dim - 1;
48             // trecem prin cele 3 situatii posibile
49             // situatia in care codificare este tot sirul
50             lung[i][j] = dim + 1;
51             lung[j][i] = 0;
52             // situatia in care sirul contine subsir care se repeta
53             for (int k=1; k<=dim/2; k++)
54                 if (dim % k == 0)
55                 {
56                     if (esteSubsir(T, i, j, k))
57                     {
58                         lung[i][j] = k + numarCifre(dim / k);
59                         // retinem lungime subsirului
60                         lung[j][i] = k;
61                         break;
62                     }
63                 }
64             // situatia in care sirul este codificat pe baza
65             // codificarii subsirurilor sale
66             for (int k = i; k < j; k++)
67             {
68                 if (lung[i][j] > lung[i][k] + lung[k + 1][j])
69                 {
70                     lung[i][j] = lung[i][k] + lung[k + 1][j];
71                     // retinem poz optima de splitare
72                     //semnul "-" este pentru a se face diferenta
73                     // intre situatia 2 si situatia 3
74                     lung[j][i] = -k;
75                 }
76             }
77         }
78     }
79     return lung;
80 }
81
82 // metoda recursiva pentru afisarea codificarii optime
83 void afisareCodificare(string T, vector<vector<int>> Lung, int i, int j)
84 {
85     if (i == j)
86         cout << T[i] << 1;
87     else
88     {
89         int k = Lung[j][i];
90         //cazul 1
91         if (k == 0)
92         {
93             string s = T.substr(i, j-i+1);
94             cout << s << 1;
95         }
96         else
97             //cazul 2
98             if (k > 0)
99             {
100                 string s = T.substr(i, k);
101                 cout << s << (j-i+1)/k;
102             }
103             else
104             {
105                 afisareCodificare(T, Lung, i, abs(k));
106                 afisareCodificare(T, Lung, abs(k)+1, j);
107             }
108         }
109     }
110
111 int main()
112 {
113     string myString = "aaabbaaabbaabbcxyzxyz";
114     vector<vector<int>> Length = calculLungimiCodif(myString);
115     cout << "o posibila codificarea optimala pentru text este: ";
116     afisareCodificare(myString, Length, 0, myString.size() - 1);
117 }

```

- Complexitatea este exprimată în notația O și ordinul depinde de lungimea

șirului de caractere procesat, astfel avem $O(dim^2)$. .

8.0.3 Algoritmi de Programare Dinamică - Probleme propuse

Problemele pe care le propunem în această secțiune sunt următoarele:

1. Andrei este provocat de Vlad la un joc. Având la dispoziție o bucată de carton dreptunghiulară, Vlad a împărțit-o în $n \times m$ celule și în fiecare celulă a trecut niște numere naturale. Acum el îi cere lui Andrei să găsească cea mai bună cale de a acumula un punctaj maxim. Pentru a face jocul mai interesant el pune însă niște condiții. Îl obligă pe Andrei să pornească din colțul stînga sus și să se oprească în colțul dreapta jos, și îi permite să meargă doar în jos sau la dreapta.

Exemplu: Pentru numerele următoare scrise pe bucata de carton

$$Joc[3][4] = \begin{bmatrix} 2 & 1 & 3 & 4 \\ 5 & 7 & 6 & 9 \\ 6 & 8 & 7 & 2 \end{bmatrix} \quad (8.1)$$

Andrei ar trebui să ajungă la punctajul maxim de 31 de puncte.

2. Se consideră că literele alfabetului sunt codificate folosind numere naturale astfel: $(1, 'A'), (2, 'B'), \dots, (26, 'Z')$. Dat fiind un număr natural cu n cifre, determinați numărul total de decodificări posibile și afișați-le pe acestea. Împărțirea numărului se poate face în secvențe valide cu câte o cifră sau cu două.

Exemplu: Pentru numărul 4126, variantele de decodificări sunt 3:

$$\{ "DABF", "DAZ", "DLF" \}$$

. Pentru numărul 2113, variantele de decodificări sunt 5:

$$\{ "BAAC", "BKC", "BAM", "UAC", "UM" \}$$

Indicație: Din exemplele de mai sus se poate observa că o singură cifră între 1 – 9 poate fi decodificată în mod unic. Două cifre între 10 – 26 pot fi decodificate în literele $J - Z$. Numărul de decodificări pentru un șir de n cifre poate fi calculat ca izolând prima cifră și calculând numărul de decodificări pentru secvența de $n - 1$ cifre rămase, sau izolând primele două cifre dacă ele sunt între 10 – 26 și calculând numărul de decodificări pentru secvența de $n - 2$ cifre rămase.

3. Fie X și Y două șiruri de lungime n și respectiv m . Să se determine un subșir comun $v = \{v[i_1], v[i_2], \dots, v[i_k]\}$, $i_1 < i_2, \dots < i_k$ de lungime maximală. Dacă există mai multe soluții se poate afișa oricare dintre ele.

Exemplu: Pentru $X = \text{"ABCBADB"}$ și $Y = \text{"DACBDDDB"}$ se afișează $s = \text{"CBDB"}$

Indicație: O posibilă subproblemă problemei date s-ar obține dacă am vrea să determinăm cel mai lung subșir comun pentru șirurile $X[0..i]$, respectiv $Y[0..j]$ cu $i \leq n - 1$ și $j \leq m - 1$. Dacă ultimele două elemente coincid, $X[i] = Y[j]$, atunci problema se reduce la a găsi cel mai lung subșir comun al șirurilor $X[0..i - 1]$ și $Y[0..j - 1]$. Dacă $X[i] \neq Y[j]$, atunci subșirul de lungime maximă este cel mai lung dintre subșirurile comune al perechilor $X[0..i]$, și $Y[0..j - 1]$, respectiv $X[0..i - 1]$, $Y[0..j]$.

4. Se dă un șir de caractere S . Să se determine cea mai lungă subsecvență contiguă (elemente pe poziții consecutive) care este palindrom. Dacă există mai multe soluții se poate afișa oricare dintre ele.

Exemplu: Pentru șirul $S = \text{"abcbdadbcddda"}$, se va afișa "cbdadbcb" .

Indicație: Pe exemplu dat se poate observa că palindromul nu începe de la prima poziție și nu se oprește la ultima poziție. Ceea ce ar sugera să considerăm subsecvențe de forma $s[i..j]$, $i \leq j$, pentru a verifica dacă sunt palindroame. Rezultatele ar trebui atunci memorate într-o matrice P de variabile booleene, pentru care:

$$P[i][j] = \begin{cases} true & \text{dacă } S[i..j] \text{ palindrom} \\ false & \text{altfel} \end{cases}$$

5. Să se rezolve problema discretă a rucsacului, dacă avem la dispoziție un rucsac de capacitate G și n tipuri de obiecte, în condițiile în care dispunem de un număr nelimitat de obiecte de orice tip.

Pentru fiecare tip de obiect O_i se cunosc greutatea și câștigul care poate fi obținut de pe urma transportării obiectului (g_i, c_i)

Indicație: Algoritmul implementat poate fi similar cu cel de la problema clasică a rucsacului, dar într-o formă simplificată.

Matricea $Castig$ va fi un vector cu $G + 1$ componente. Vectorul se parcurge o dată pentru tipul de obiect O_0 , apoi pentru tipul de obiect $O_2, \dots$, iar ultima dată pentru tipul de obiect O_{n-1} . La fiecare parcurgere se urmărește să se încarce optim greutatea $1, 2, \dots, G$.

Spre deosebire de problema rucsacului la alegerea obiectului O_i rămâne să

se actualizeze încărcarea rucsacului cu greutate j , pornind de la greutatea $j - g_i$, și folosind optim obiectele $O_0, O_1, \dots, O_{i-1}$, în cazul acestei probleme, se actualizează încărcarea rucsacului cu greutate $j - g_i$, folosind obiectele $O_0, O_1, \dots, O_i$. Acest lucru face posibilă selecția mai multor produse de același tip.

6. O țeavă cu o anumită lungime L trebuie confecționată din n bucăți de țeavă pe care le avem la dispoziție, fiecare având o lungime l_i dată. Nu avem la dispoziție niciun instrument cu care să tăiem țevile. Se cere să se găsească, dacă există, o variantă de a obține țeava de lungime L prin sudarea anumitor bucăți de țeavă din cele n date.

Exemplu: Considerăm că avem bucățile de țeavă cu lungimile $\{2, 5, 7, 8, 11, 14\}$. O țeavă de lungime $L = 20$ se poate obține fie prin sudarea bucăților de lungimi $\{2, 7, 11\}$, fie prin ale celor cu lungimi de $\{5, 7, 8\}$. Dacă însă $L = 6$, atunci nu pot obține țeava de lungime dorită din bucățile pe care le avem.

7. La un atelier de croitorie, croitoreasa a găsit o bucată de material de dimensiune $m \times n$. Materialul are o serie de imperfecțiuni pe care croitoreasa le-a marcat pe pânză. Ea își dorește să taie din bucata de material o parte, de formă pătratică, de dimensiune cât mai mare posibil. Cum poate face acest lucru?

Exemplu: Dacă pe material zonele cu imperfecțiuni sunt marcate cu 0, iar restul cu 1, atunci, pentru un material de forma:

$$Material = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & 0 & 1 \\ 1 & 0 & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 & 1 \\ 1 & 1 & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 & 1 \\ 0 & 1 & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & 0 & 0 \end{bmatrix} \quad (8.2)$$

bucata care ar putea fi tăiată este cea marcată cu roșu.

Indicație: Căutăm o submatrice pătratică de dimensiune cât mai mare care să nu conțină zerouri. Fie poziția (i, j) din matrice care conține 1; o considerăm ca fiind de exemplu colțul dreapta jos al matricei și ne uităm la vecini, ca să vedem cât de mult putem merge în sus, la stânga și pe diagonală spre stânga, cu elemente numai de 1. Atunci minimul dintre aceste raspunsuri posibile plus 1 reprezintă dimensiunea maximă a matricei care are numai 1 și are colțul dreapta jos în (i, j) .

8. Se citește n , un număr natural de la tastatură și o permutare a numerelor $\{1, 2, \dots, n\}$. Se cere să se determine numărul de ordine al permutării, dacă ele sunt considerate în ordine lexicografică.

Exemplu: Pentru mulțimea $\{1, 2, 3\}$ permutările în ordine lexicografică și numerele de ordine corespundă toare sunt:

Nr. ordine	Permutare
1	123
2	132
3	213
4	231
5	312
6	321

Indicație: Se poate observa că oricare număr i dintre $\{1, 2, \dots, n\}$, apare de $(n-1)!$ ori ca prim element în lista cu toate permutările. Dacă $a_1 a_2 \dots a_n$ este permutarea inițială și $Nr(P(n))$ reprezintă numărul de ordine al permutării $P(n)$ de n elemente $\{1, 2, \dots, n\}$, atunci:

$$Nr(P(n)) = \begin{cases} 1 & n = 1 \\ (n-1)!(a_1 - 1) + Nr(P(n-1)) & n \neq 1 \end{cases}$$

Atenție că permutarea $P(n-1)$ este permutarea $a_2 \dots a_n$, și este formată din elementele $\{1, 2, \dots, n\}$, din care lipsește unul. Din ea trebuie să se obțină permutarea cu același număr de ordine lexicografică, dar a mulțimii $\{1, 2, \dots, n-1\}$ pentru ca formula anterioară să fie valabilă. Acest lucru se poate face dacă din orice element mai mare decât cifra eliminată se scade 1.

Exemplu: Permutarea $(2, 1, 3, 4)$ este a șaptea în ordine lexicografică a permutărilor mulțimii $\{1, 2, 3, 4\}$ cu $n = 4$ elemente.

$$\begin{aligned} Nr((2, 1, 3, 4)) &= 3!(2-1) + Nr((1, 3, 4)) = 6 + Nr((1, 2, 3)) = \\ &= 6 + 2!(1-1) + Nr((2, 3)) = 6 + Nr((1, 2)) = 6 + 1!(1-1) + Nr((2)) = \\ &= 6 + Nr((1)) = 6 + 1 = 7 \end{aligned}$$

9. Se dă o matrice binară (având elemente de 0 și 1) pătratică. Se cere să se determine dimensiunea celei mai mari selecții de celule de 1 care formează semnul + (plus). Elementele selectate trebuie să se afle pe o singură linie și pe o singură coloană, iar semnul trebuie să fie simetric

Exemplu: În matricea

$$B = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & \mathbf{1} & 0 & 1 & 1 \\ 0 & 1 & \mathbf{1} & 1 & 1 & 0 \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 \\ 1 & 1 & \mathbf{1} & 0 & 1 & 1 \\ 1 & 0 & \mathbf{1} & 1 & 0 & 0 \end{bmatrix} \quad (8.3)$$

cel mai mare semn plus care se poate forma este din celulele colorate în roșu.

Indicație: Se pot folosi patru matrice auxiliare: St, Dr, Sus, Jos , în care $St[i][j], Dr[i][j], Sus[i][j], Jos[i][j]$ păstrează valorile de 1 consecutive aflate la stânga, dreapta, sus sau jos față de poziția (i, j) , inclusiv acea celulă.

10. Autocorectorul unui telefon are un dicționar de K cuvinte. Acesta înlocuiește orice cuvânt scris de utilizator cu forma cea mai apropiată din dicționarul propriu.

Două cuvinte sunt considerate apropiate dacă numărul diferențelor dintre acestea este minim. Se consideră diferență orice literă care lipsește, este în plus, sau este modificată față de cuvântul original.

Se citesc, de la tastatură sau din fișier, un cuvânt A și cele K cuvinte din dicționar. Să se afișeze care este cuvântul cu care autocorectorul îl va înlocui pe cel original. Dacă sunt mai multe cuvinte cu același număr de diferențe, să se afișeze toate.

Indicație: Pentru fiecare cuvânt C din dicționar se poate crea o matrice Dif în care, $Dif[i][j]$ reprezintă numărul minim de modificări necesare pentru a forma subșirul $A[0...i-1]$ din subșirul $C[0...j-1]$

11. În bucătăria unui restaurant se află un teanc cu N farfurii. Noul ajutor de bucătar are la dispoziție un timp T să le spele. El știe cât durează spălarea fiecărei farfurii în parte (timpii pot fi diferiți de la o farfurie la alta). El poate să renunțe la spălarea ei și să decidă să o spargă, dar spartul și curățatul cioburilor durează exact 1 minut pentru orice farfurie.

Farfuriile sunt numerotate de la 1 la N și ajutorul de bucătar începe întotdeauna din vârful teancului spre bază (el nu poate lua decât farfuria din vârf). El poate începe cu farfuria 1, pe care o poate spăla sau sparge, apoi cu farfuria 2, etc. De asemenea, în teanc pot să rămână farfurii. Este o soluție de preferat în loc să le spargă.

Să se determine numărul maxim de farfurii pe care le poate spăla ajutorul de bucătar în timpul T dat. Pentru acest număr de farfurii să se determine și timpul minim necesar spălării lor.

Exemplu: Pentru $N = 4$, $T = 6$ și timpii de spălare $\{1, 3, 2, 2\}$ se spală farfuriile $\{1, 3, 4\}$ și se sparge 2. Timpul minim necesar spălării vaselor este

$$T_{min} = 1 + 1 + 2 + 2 = 6.$$

Pentru $N = 7$, $T = 12$ și timpii de spălare $\{9, 1, 3, 2, 6, 2, 4\}$ se spala farfuriile $\{2, 3, 4, 6\}$ și se sparg 1 și 5. Timpul minim necesar spălării vaselor este $T_{min} = 1 + 2 + 3 + 2 + 1 + 2 = 11$.

12. Ana are o grădină de formă dreptunghiulară împărțită în $N \times M$ zone separate prin gard. Care este numărul de variante pe care le are Ana să ajungă din colțul NV (nord-vest) al grădinii în colțul SE(sud-est) dacă ea poate merge numai în jos sau la dreapta, și poate face pe drum exact K schimbări ale direcției de mers.

Se consideră schimbare a direcției de mers o mișcare în jos urmată de una la dreapta, sau o mișcare la dreapta urmată de una în jos.

Exemplu: Pentru un teren cu 3×3 parcele, etichetate ca mai jos:

$$Teren = \begin{bmatrix} (0,0) & (0,1) & (0,2) \\ (1,0) & (1,1) & (1,2) \\ (2,0) & (2,1) & (2,2) \end{bmatrix} \quad (8.4)$$

și $K = 2$ avem două posibilități de drum:

$$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2)$$

$$(0,0) \rightarrow (1,0) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (2,2)$$

Capitolul 9

Diverse probleme practice

Acest capitol însumează o serie de probleme recapitulative, probleme tradiționale din toate capitolele tratate în această carte. Încurajăm cititorul să rezolve întâi toate exercițiile grile și tradiționale propuse specific pentru fiecare capitol (în plus, recomandăm parcurgerea problemelor rezolvate de la fiecare capitol), iar apoi propunem rezolvarea următoarelor probleme:

1. Într-un acvariu sunt n piranha cu dimensiunile $a_1, a_2, \dots, a_n$. Peștii sunt numerotați de la stânga la dreapta în ordinea poziției din acvariu. Studenții de la Facultatea de Biologie vor să vadă dacă există un piranha dominant în acvariu. Un peste este numit dominant dacă acesta poate mânca toți ceilalți pești din acvariu. Ceilalți pești nu se vor opune când acesta îi va mânca. Deoarece acvariul este destul de îngust și lung, un peste poate mânca doar peștii adiacenți acestuia într-o mișcare. Peștii pot face oricâte mișcări vor. Mai precis:

- peștele i poate mânca peștele $i-1$ dacă peștele $i-1$ există și $a_{i-1} < a_i$
- peștele i poate mânca peștele $i+1$ dacă peștele $i+1$ există și $a_{i+1} < a_i$

Când peștele i mănâncă un piranha, dimensiunea lui crește cu 1. Treaba ta este să vezi dacă există un astfel de piranha dominant în acvariu. Dacă găsești un astfel de peste nu trebuie să găsești alți pești dominanți.

De exemplu, dacă $a = [3, 3, 5^*, 4, 5]$, atunci cel de-al treilea peste reprezintă peștele dominant.

Date de intrare: Prima linie se da numărul peștilor din acvariu. (Numărul trebuie să fie ≥ 2)

Date de ieșire: Pentru fiecare test, se va afișa răspunsul: -1 dacă nu există piranha dominanți în acvariu sau indexul oricărui peste dominant dacă există. Dacă există mai multe răspunsuri corecte se poate alege oricare dintre acestea.

Exemplu:

input 5

5 3 4 4 5

3

1 1 1

5

4 4 3 4 4

5

5 5 4 3 2

3

1 1 2

5

5 4 3 5 5

output 1

-1

2

3

3

1

2. Un bloc este format din m etaje (numerotate de la 1 la m) și există un lift care este folosit pentru a se face deplasarea între diferite etaje. Ascensorul este conectat la un computer care înregistrează mișcarea într-o secvență a . Ori de câte ori ascensorul se mută la un etaj diferit, computerul adaugă noul număr de etaj la secvența a . În prezent, secvența a are n elemente. Din păcate, computerul este infectat cu un virus care a înlocuit unele elemente ale lui a cu -1 . Andrei, matematicianul blocului vrea acum să știe care ar putea fi numărul minim de ori în care liftul și-a schimbat direcția. Adică, de câte ori ascensorul, urcând, începe să coboare și invers.

Ajutați-l pe Andrei răspunzând întrebării sale sau stabiliți că secvența a este invalidă.

Date de intrare: Prima linie conține un singur număr întreg T care indică numărul de cazuri de testare. Urmează descrierea cazurilor de test T . Prima linie a fiecărui caz de testare conține două numere întregi n și m . A doua linie conține n numere întregi separate prin spațiu $a_1, a_2, \dots, a_n$.

Date de ieșire: Pentru fiecare caz de testare, tipăriți o singură linie care conține un număr întreg - numărul minim de ori când ascensorul și-a schimbat direcția sau -1 dacă secvența a dată este nevalidă.

Exemplu:

4 5

2 3 4 3

Se afișează 1 pentru ca liftul își schimbă sensul o singură dată.

4 5

2 -1 4 3

Se afișează 1 pentru ca liftul își schimbă sensul o singură dată, înlocuind -1 cu 3. Dacă -1 s-ar fi înlocuit cu altceva, rezultatul nu ar mai fi minim.

5 4

-1 -1 3 -1 -1

Se afișează 1 pentru ca liftul își schimbă sensul o singură dată, secvența poate fi înlocuită cu 1 2 3 4 3, astfel liftul își schimbă poziția o singură dată.

5 5

-1 -1 3 -1 -1

Se afișează 0 pentru că înlocuind secvența cu 1 2 3 4 5, liftul doar urcă.

3. Sorin este membru al Armatei Române și este responsabil de lansarea rachetelor. O dronă a descoperit locația bazei inamice, și se pare că este dotată cu tancuri pregătite pentru război, deși au promis că nu vor începe alt război. Sorin s-a hotărât să distrugă tancurile pentru a preveni războiul. Totuși, bugetul Armatei Române este unul destul de restrâns, așa că Sorin se gândește să facă economie de rachete, distrugând mai multe tancuri cu o singură rachetă, dacă este posibil. Câte rachete lansează Sorin?

Locația tancurilor este reprezentată de un vector de puncte citite din fișier, iar la rularea programului se cere diametrul exploziei rachetei. Se va afișa numărul de rachete necesare pentru distrugerea tuturor tancurilor.

Exemplu:

Pentru punctele (2, 3), (22, 8), (8, 13), (20, 15), (5, 1), (12, 10), (3, 4) și diametrul exploziei rachetei 100. Se va afișa 4. Pentru aceleași puncte și diametrul exploziei rachetei 1. Se va afișa 7.

4. Persia este atacată de către spartani. Spartanii sunt aliniați unul lângă altul; o formațiune greu de spart. Tacticianul Persiei propune să îl omoare pe generalul Spartei prima dată pentru a îi dezorganiza, altfel o confruntare

frontală îi va aduce înfrângerea. El trimite câte un soldat pentru a găsi locația generalului, știind că soldații sunt așezați în funcție de rang în modul următor: $11 \dots 22 \dots 33 \dots n-1 \ n-1 \dots n$ (generalul) $\dots n-1 \ n-1 \dots 33 \dots 22 \dots 11$. Odată ce generalul este găsit, va concentra întreaga armată pe locația generalului. Să se afle locația generalului.

5. Marcel a primit moștenire de la bunicul lui niște bănuți de diferite valori. Acesta a început să se joace cu bănuții și să formeze diferite șiruri din ei. La fiecare șir el dorește să afle o sumă maximă astfel: dacă alege un ban de pe poziția i , cel de pe poziția $i+1$ nu mai poate fi ales. Scrieți un program cu privire la această problemă.

Regula principală: se poate da "skip" și la mai multe valori, nu doar la una, dar NU se pot alege două valori consecutive.

Exemple:

$14, 3, 27, 4, 5, 15, 1 \Rightarrow \text{valoarea } 56 = 14 + 27 + 15$

$16, 22, 4, 31, 55, 56, 11, 7 \Rightarrow \text{valoarea } 109 = 56 + 31 + 22$

$18, 19, 66, 54, 22, 13, 89 \Rightarrow \text{valoarea } 177 = 89 + 66 + 22$

6. Într-un depozit există un raft cu $n+1$ spații de depozitare, numerotate de la 1 la $n+1$. Primele n spații de depozitare sunt ocupate cu n pachete numerotate cu valori între 1 și n , iar spațiul de depozitare $n+1$ este gol. Administratorul depozitului decide mutarea pachetelor, astfel încât pentru orice i , pachetul numerotat cu i să se afle în spațiul de depozitare i . Pentru aceasta se va folosi spațiul de depozitare suplimentar, $n+1$, singura manevră validă fiind mutarea unui pachet dintr-un spațiu de depozitare în altul, cu condiția ca acesta să fie gol.

Determinați o succesiune de manevre prin care fiecare pachet să fie în spațiul corect.

Discutați complexitatea obținută pentru rezolvarea problemei, iar, în plus, să se precizeze și justifice metoda de programare aleasă.

7. Ana este o cântăreță încăpătoare. Ea știe deja să cânte n piese, și pentru fiecare piesă se cunoaște durata, exprimată în minute și secunde. Ana va participa la o emisiune de televiziune, unde va putea cânta timp de T secunde. El vrea să cânte cât mai multe piese, pentru a-și demonstra talentul deosebit.

Cum trebuie Ana să aleagă piesele pentru emisiune?

Discutați complexitatea obținută pentru rezolvarea problemei, iar, în plus, să se precizeze și justifice metoda de programare aleasă.

8. Să se scrie subprograme pentru:

- Verificare dacă un CNP (șir de caractere sau de numere de 13 componente) dat este al unei femei sau al unui barbat. Funcția verifică acest aspect, dar afișează și anul nașterii persoanei cu CNP-ul dat;
- Exprimarea unui număr din vbaza 10 într-o bază dată (baza dată este maxim 10).

Pentru fiecare subprogram se precizează și maniera de apelare (se elaborează un `main()` corespunzător)

9. Un vânător și-a luat arcul și toba lui cu săgeți și a plecat în pădure la vânătoare. În pădure a dat de o grămadă de animale, iar el putea să își dea seama ce greutate aveau acestea.

- Se citește de la tastatură numărul de săgeți pe care acesta le-a luat când a plecat de acasă și se citește din fișier o listă cu greutățile animalelor pe care le-a văzut el în pădure în ordine crescătoare. Să se determine câte animale au sub 60 de kg și dacă ar avea suficiente săgeți vânătorul pentru aceste animale (luând în considerare că pentru fiecare animal are nevoie fix de o săgeată).

Exemplu:

numărul de săgeți: 6

greutăți: 25, 30, 35, 40, 45, 50, 55, 65, 75, 90.

Numărul de animale sub 60 de kg este 7 și se afișează faptul că vânătorul nu are suficiente săgeți pentru fiecare animal.

Să se determine complexitatea algoritmului utilizat și să se justifice metoda/tehnica de programare abordată.

- Pentru fiecare animal de la subpunctul anterior se citesc din nou din fișier numărul de săgeți necesare pentru ca vânătorul să le doboare. Pe baza acestor criterii (greutate și numărul de săgeți necesare) să se determine care este greutatea maximă de vânat pe care poate vânătorul să o aducă acasă pe baza numărului de săgeți disponibile citit anterior.

Exemplu:

Se pastrează informațiile precedente și se citesc următoarele numere de săgeți necesare: 2, 1, 3, 2, 2, 3, 4, 5, 2, 1.

Pe baza greutăților de la subpunctul anterior și pe baza numărului de săgeți disponibile, rezultatul final este: 240 de kilograme (1 pentru 90 + 2 pentru 75 + 2 pentru 45 + 1 pentru 30)

Să se determine complexitatea algoritmului utilizat și să se justifice metoda/tehnica de programare abordată.

10. Se dă un tablou cu $n + 1$ elemente, în care fiecare element este un număr întreg din intervalul $[1, n]$. Prin urmare, există cel puțin un element care se repetă. Este posibil ca toate elementele să fie egale.

Să se găsească unul dintre elementele care se repetă (prin 2 variante diferite, cu complexitate comentată).

11. Scrieți un program care citește de la tastatură un număr natural nenul, S , având maximum 9 cifre, și printr-o metodă eficientă din punct de vedere al timpului de executare, determină trei valori naturale a căror sumă este egală cu S , și al căror produs este maxim. Cele trei valori vor fi afișate în ordine crescătoare separate prin câte un spațiu.
12. Se citește un număr natural, "number" și apoi "number" numere naturale nenule cu cel mult 5 cifre fiecare. Afișați fracția ireductibilă minimă care se poate construi folosind numere dintre cele citite.

Exemplu: number = 5, numerele 7 9 8 3 6, Output: 1/3 (obținută din simplificarea fracției 3/9)

Observație: Se dorește o rezolvare folosind obiecte de tip *Fractie*.

13. Directorul unei școli organizează o excursie la final de an. A stabilit că elevii vor fi cazați într-un hotel care dispune de B locuri în camere doar pentru băieți și F locuri în camere doar pentru fete. Aceste locuri trebuie să fie ocupate în totalitate. În plus, numărul de locuri nu poate fi suplimentat. Școala are N clase. Pentru fiecare dintre ele se cunoaște numărul de băieți și numărul de fete. În fiecare clasă există cel puțin un elev dar pot fi clase cu niciun băiat sau nicio fată. Determinați o modalitate de alegere a unor clase, pentru a putea organiza excursia în condițiile anterioare. Odată aleasă o clasă pentru a merge în excursie, trebuie să participe la această ieșire toți elevii din clasă.
14. Se dă o matrice cu m linii și n coloane, având elemente întregi. Să se determine o submatrice a sa, de formă dreptunghiulară care să aibă suma elementelor maximă.

Exemplu: În matricea:

$$M = \begin{bmatrix} 0 & 2 & 0 & 0 & 1 & 0 & 0 & 4 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & \mathbf{2} & \mathbf{3} & \mathbf{2} & \mathbf{1} & 0 & 5 \\ -1 & 0 & \mathbf{3} & \mathbf{2} & \mathbf{2} & \mathbf{1} & 2 & 1 \\ 1 & 1 & \mathbf{3} & \mathbf{3} & \mathbf{2} & \mathbf{1} & 1 & 2 \end{bmatrix} \quad (9.1)$$

submatricea colorată în roșu este cea cu suma maximă.

15. Se dă un vector care reprezintă valorile unor bancnote, din care dispunem de un număr nelimitat. Se citește o sumă S de la tastatură. Se cere să se determine numărul minim de bancnote din care poate fi plătită suma S pe baza vectorului inițial.

Exemplu: Se citește vectorul 10, 6, 1 și suma 18. Se va afișa 3.

16. Se dă un șir de caractere și un vector(dicționar) de șiruri de caractere. Să se determine dacă primul șir de caractere poate fi segmentat într-o secvență de succesiuni care se regăsesc în dicționar.

Exemplu: Se citește șirul: "problemădespărțireîncuvinte" și vectorul de string-uri: "cu", "va", "da", "nu", "problemă", "despărțire", "d", "e", "s", "p", "a", "rțire", "de", "des", "despă", "ire", "încuvinte", "în", "cuvinte" . Șirul de caractere se poate segmenta în mult mai multe moduri, dar o variantă poate fi: "Problemă despă rțire încuvinte" deci răspunsul este afirmativ.

Bibliografie selectivă

- [1] Algorithms Hardcover, Sedgewick R., ISBN-10032157351X, 2011
- [2] Algorithms in a Nutshell, Heineman G.T., ISBN-10059651624X, 2008
- [3] Algorithms dynamic programming, Resursă online:
<https://www.codesdope.com/course/algorithms-dynamic-programming/>
- [4] Greedy algorithms, Resursă online: www.geeksforgeeks.org/greedy-algorithms/
- [5] Introduction to Algorithms, Cormen T.H., 3rd Edition (The MIT Press) 3rd Edition, 2009
- [6] Introduction to the Design and Analysis of Algorithms, Levitin A., ISBN-100132316811, 2011
- [7] Greedy algorithms, Stanford classes, Resursă online
- [8] MIT Courses, Resursă online
- [9] MIT Courses, Resursă online
- [10] Templates in C++, <https://en.cppreference.com/w/cpp/language/templates>, Resursă online
- [11] The C++ Programming Language (4th Edition), Stroustrup B., Addison-Wesley ISBN 978-0321563842, 2013
- [12] The Design and Evolution of C++, Stroustrup B., Addison-Wesley, ISBN 0-201-54330-3, 1994